

目录

二、在洛达 JIRA 沟通技术问题或获取资料	3
三、在联发科 MTK 下载洛达的技术资料	9
五、安装 1561/62/63 的编译环境及编译工程	10
六、安装 1565/68 的编译环境及编译工程	12
七、可视化开发环境 Xtensa Xplorer 安装	17
八、编译 AB1561、62、63 项目的过程剖析	17
九、分开编译 1565 或 1568 的 dsp 和 mcu	22
十、文档 AB1561_AB1562_AB1563_Get_Started_Guide.pdf 内容学习笔记	27
十一、将固件程序烧录到耳机	31
十二、LED 灯效配置及调用	36
十三、项目 CHP200 的原理图理解	42
十四、添加按键事件和关联 LED	44
十五、将 patch 补丁文件打入 SDK	46
十六、软件版本管理 TortoiseHg	49
十七、Tile 功能测试	53
十八、工程配置 cfg 的打开方式	54
十九、修改定制按键 UI 配置	56
二十、提示音 (VP) 文件路径	63
二十一、如何生成 FOTA 烧录文件	64
二十二、手机 app 上进入 Debug 模式和在线 log	66
二十三、修改蓝牙 Audio 和蓝牙 BLE 的名称	68
二十四、发送 RACE Command 触发 Event	69
二十五、NVR 文件并入 FW	70
二十六、手机 Airoha_SDK_UT 升级固件	76
二十七、开机后程序开始执行的 Main 函数	77
二十八、硬件 GPIO 与 EPT 工具配置对应关系	80
二十九、Shell UI 消息分发管理机制	94
三十、打开或关闭 HAL 硬件模块	102
三十一、SourceInsight 查找工程文件和代码	103
三十二、AB1561M 的 EVK 板外设驱动文件	108
三十三、使用 Logging tool 监测系统运行事件信息	109
三十四、ANC 相关参数	122
三十五、BCompare 比较文件代码差异	122
三十六、抓工程的 log 追踪旋转编码器执行	123
三十七、修改 NTC 参数	137
三十八、查看 ANC 模式和切换方法	137
三十九、开启双麦通话功能	140
四十、在线测试 ANC 功能同时抓 log	142
四十一、用编码器调节振动大小	145
四十二、sh 文件与 makefile 文件与 MK 文件的关系	147
四十三、电量和音量等级设置	148

四十四、修改 custom EQ 的设置	149
四十五、DSP 声音设置	152
四十六、找出修改的 NVkey 参数	153
四十七、苹果手机上 SDK DUT 软件更新方法.....	154

二、在洛达 JIRA 沟通技术问题或获取资料

点击项目名称

Welcome to Airoha eService Customer Support Portal

What do you need help with?

Which of the following best describes what you need help with?

Skullcandy Crusher 2

Skullcandy Riff2

Back

点击 Firmware 进入软件问题

Airoha eService

Skullcandy Crusher 2

Welcome to Airoha eService Customer Support Portal for your project. You can search solutions for your issue, or raise support request to Airoha support team.

Our eService portal created for your project is **only for reporting the issues/questions specific to project itself**. For question/inquiry about AIROHA new chipsets, feature roadmap, price info...etc. which are nothing to do with this on-going project itself, please send your inquiry or question by email to your AIROHA sales representative or agent instead of raising an issue ticket in this eService system

To improve our support efficiency, ticket will be **closed automatically if pending w/o customer feedback for over 10 working days**.

All sensitive information is protected by [Privacy Policy and Legal Statement](#). Please always keep the list up-to-date by acknowledging Airoha in time if changed.

For project mgmt/file mgmt tickets, please go Requests/All Requests and search for reqs type "Project Mgmt" and "File Mgmt" accordingly.

For eService guide, please refer to [Customer Easy Guide](#)

For document repository, please access [eServiceDocs](#)

What do you need help with?

Hardware Related

Software Related

Firmware

The result of the software application or product does not meet with the expectations

BQB Certification

Products can't pass the certification about protocol conformance, profile conformance or profile interoperability

编辑问题描述，有的项是可选填的

Airoha eService / Skullcandy Crusher 2

Firmware

Summary

(Required) Precise and clear as possible when you give a summary (Title) to the bug report. Don't report more than one issue in the same report.

Description

(Required) This includes the abstract of the issue. Describes your environment. Try and figure out exactly what causes the problem.

Customer Priority

None

(Required) Priority levels are designed to characterize the impact to business as reported by the customer and serve as a means for escalation. Issues are answered on a first come, first served basis

Immediate: An Issue that results in a high

在 Summary 里输入问题或搜索关键词时，右侧会显示可能与此问题相关的解决方法

Summary

AB1568实现呼吸灯

Description

请问AB1568如何配置呼吸灯？谢谢。

Customer Priority

Low

Expected Date

2022/02/14

Reproducibility (optional)

None

Reproduce Procedure (optional)

We've found solutions that could save you time

0002_如何配置为呼吸灯mode

<1>led_control_internal.c 默认配置如下： 20190731170236.png 20190731170153.png <2> 将 LED_INNER_LED_MODE 改成 LED_MODE_BREATH即可。

002_Breathe_LED_Behavior

Background: 查看Configure tool的LED的手册，发现是无法配置LED呼吸功能的，而实际项目用呼吸LED做指示灯效果更为常见。先抛一个结论：如果要实现呼吸灯效果，需要Configure tool + source code 结合修改，才有办法实现完整的功能效果。目前LED的线路图如下，是由155x IC里面集成的PMU模块MT6388来控制的，所以，所有的寄存器配置参数都是围绕MT6388 PMU模块来设定。image2019-11-22_14-15-21.png 与呼吸灯相关的参数 Tr1, Tr2, Ton, Toff, Tf1, Tf2

[AB156X] 如何使LED Breath模式

并根据led pattern的index来区分呼吸灯闪烁模式。下面例子是将led_pattern_index >= 20的LED模式变为呼吸模式，并在音乐播放的时候使用呼吸灯。1. 根据需要的呼吸效果的

点击解决方法，可以看到详细说明

0002_如何配置为呼吸灯mode

<1>led_control_internal.c 默认配置如下：

```

#define LED_INNER_DEBUG_EN
#define LED_MODE_PWM 0
#define LED_MODE_BREATH 1
#define LED_INNER_LED_MODE LED_MODE_PWM

static bsp_led_config_t led_style_parameter_convert(one_led_style_t *p_style)
{
    static bsp_led_config_t config;
    memset(&config, 0, sizeof(bsp_led_config_t));
    config.mode = LED_INNER_LED_MODE;
    config.on_time = p_style->st1 * p_style->time_unit;
    config.off_time = p_style->st2 * p_style->time_unit;
    config.brightness = p_style->brightness;
    config.repeat_times = p_style->repeat_tit;
    log_iner_info("[%d][style] on time(%d),off time(%d),tot1 repeats(%d)\r\n", 3, config.on_time, config.off_time, config.repeat_times);
    return &config;
}

```

<2> 将LED_INNER_LED_MODE 改成 LED_MODE_BREATH即可。

Summary

AB1568实现呼吸灯

We've found solutions that could

[AB156X] 如何使LED Breath模式 与 Blink模式 共存

下面以SDK2.7.1为例。

SDK中自带两种LED模式，breath 呼吸模式 与 blink闪烁模式。

两种模式都是操作PMU，使其对LED进行控制。

单纯切换两种模式通过 mcu\project\ab1565_ab1568_evk\apps\earbuds_ref_design\src\apps\led\led_control_internal.c 文件中 APP_LED_MODE 宏定义来实现。

APP_LED_MODE 有两个可选值，例如下面设置为blink模式：

```
#define APP_LED_MODE BSP_LED_BLINK_MODE //BSP_LED_BLINK_MODE or BSP_LED_BREATH_MODE
```

此时如果需要额外设置1个或几个呼吸模式的LED样式，需要在led_style_enable(uint8_t ledx, one_led_style_t *config, uint32_t syn_tm_ms, uint32_t led_pattern_index) 中添加led pattern index 变量的传入。并根据led pattern的index来区分呼吸和闪烁模式。

下面例子是将led_pattern_index >= 20 的LED模式变为呼吸模式，并在音乐播放的时候使用呼吸灯。

1. 根据需要的呼吸效果的led pattern index来为其设置呼吸模式。

```

bsp_led_status_t led_style_enable(uint8_t ledx, one_led_style_t *config, uint32_t syn_tm_ms, uint32_t led_pattern_index)
{
    bsp_led_config_t bsp_cfg;

    if (config == NULL) {
        return APP_LED_STATUS_ERROR;
    }

    /* Init bsp led. */
    // if (bsp_led_status_ok != bsp_led_init(ledx, APP_LED_MODE)) {
    //     return APP_LED_STATUS_ERROR;
    // }

    if (led_pattern_index >= 20) // select your pattern index which need breath mode
    {
        if (bsp_led_status_ok != bsp_led_init(ledx, BSP_LED_BREATH_MODE)) {
            return APP_LED_STATUS_ERROR;
        }
    }
    else
    {
        // select your pattern index which need blink mode
    }
}

```

也可以添加附件，然后 Creat 即可创建问题

Attachment (optional)

Drag and drop files, paste screenshots, or

browse

Description about Attachments (optional)

Share with CHP200_Grandsun_CHP200

Create Cancel

Airoha eService / Skullcandy Crusher 2 / SKDYCH2-37

SDK 2.7.1打入patch后编译问题 WORKING

Committed Date: N/A

Comment on this request...

Activity

2021/10/28 10:12

LATEST

err log build SDK V2.7.1 and patch by (481 KB)

Details 2021/10/28 10:12

Description

Hi Eugene Yang,

SDK 2.7.1原文档编译通过，将patch文件打入SDK后，在编译CM4时出现一个FAIL，详情请查看附件，谢谢。

Customer Priority

Immediate

Expected Date

2021/10/28

SDK SW Version

Other

SDK SW Version / Other

V2.7.1

Don't notify me

Close

Escalate

Shared with

wuchaopeng

Creator

CHP200_Bragi

Remove

CHP200_Grandsun_CHP200

Remove

CHP200_Skullcandy

Remove

Share with Customer

Update Your Contact Information

In order to speed up support process, please leave your contact information. Especially for your cellular phone number. Thank you!

Your Contact Information:

查看以往沟通的问题

Airoha eService Requests

可以看到以往沟通的问题

Any status

CHP200_Grandsun_CHP200

Any request type

Search for requests

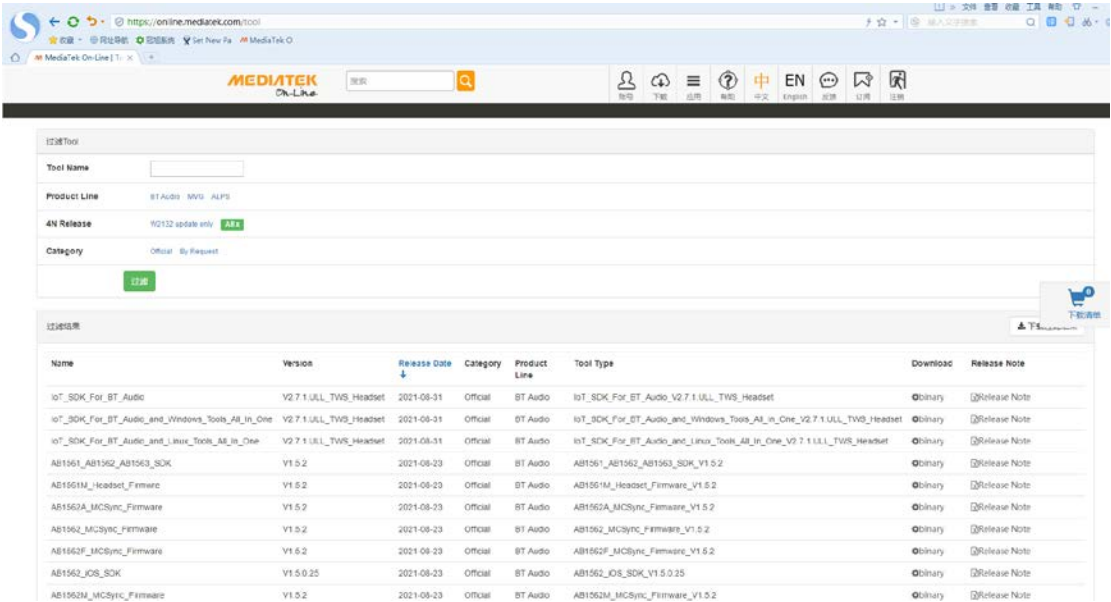
Type	Reference	Summary	Service desk	Status	Requester
	SKDYRIF2-14	通话过程中，对方听不到EVK版MIC的声音	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com
	SKDYRIF2-12	Key Event 里的哪个事件是用来清除POL列表？	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com
	SKDYRIF2-7	请问 ISINK0 ISINK1 两个LED指示灯在代码中哪个位置进行初始化？	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com
	SKDYRIF2-11	ATK 能否将软件配置成大于3台机的multiple连接	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com
	SKDYRIF2-9	请问哪个 Key Event能实现手动回连PDL表中的最后一个设备？	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com
	SKDYRIF2-10	0x001E KeyEvent事件都不能在按键触发下，让EVK从开机状态直接切换到配对状态？	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com
	SKDYRIF2-8	请问代码如何修改实现组合按键触发事件	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com
	SKDYCH2-14	CHP200 schematic review	Skullcandy Crusher 2	CLOSED	Reed Peng
	SKDYRIF2-K	AB1561_AB1562_AB1563_SDK_V1.4.0 编译下载到EVK EVK无反应	Skullcandy Riff2	CLOSED	yu.hu@grandsun.com

洛达开发文档及视频

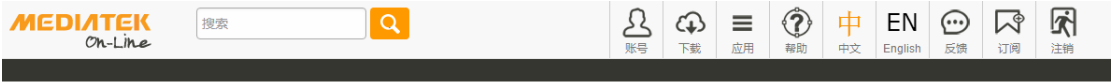
<https://eservicedocs.airoha.com.tw/display/DC/Document+Centre>

三、在联发科 MTK 下载洛达的技术资料

<https://online.mediatek.com/tool/download> 下载资料网站



点击 Downloads 列对应的 binary，会收到下载信息，然后等待状态变成 Ready 即可下载

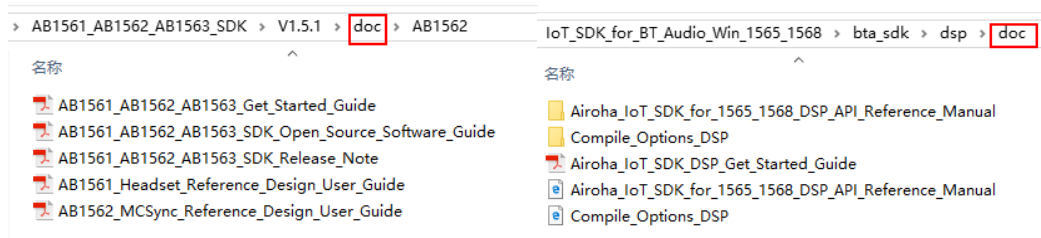


Download List				
Download ID	Status	Received Time	Expiration Time	Download
QUEUE39550474	Ready	2021-10-20 20:06	2021-10-23 20:14	Download
QUEUE39089450	Ready	2021-09-29 15:00	2021-10-02 15:08	Download
QUEUE39089157	Ready	2021-09-29 14:28	2021-10-02 14:35	Download
QUEUE39089109	Ready	2021-09-29 14:21	2021-10-02 14:29	Download

Action	Size	Transfer ID
	820.4MB	162833049586

点击下载

所有安装包均可以从 MOL（MTK 客户支持网（MediaTek On-Line）上获取，注意资料的更新（提示：相关参考文档都在工程文件夹下的 doc 文件夹里，SDK 开发的文档在 SDK 安装路径下的 doc 文件夹）



https://eservicedocs.airoha.com.tw/display/DC/00_AB156x+Hardware+Document+List

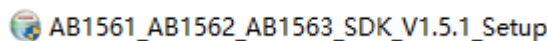
五、安装 1561/62/63 的编译环境及编译工程

先安装 Xtensa 工具链（用于编译源文件和构建目标文件，一种方法是命令脚本安装，另一种方法是 Xplorer IDE 安装），

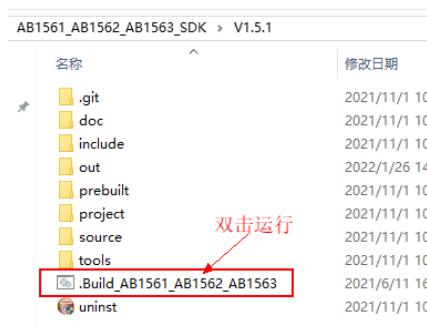
在压缩包 AB1562_Toolchain_Setup_V1.1.0_exe_1.1.0（需要联网，运行 install_cadence）



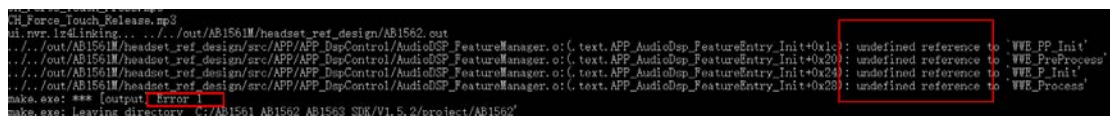
再安装 SDK 包，AB1561_AB1562_AB1563_SDK_V1.5.1_Setup.exe



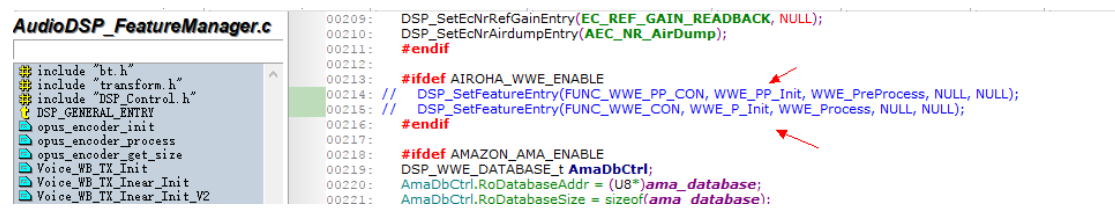
接着运行 1561、1562、1563 的 SDK 下的批处理文件



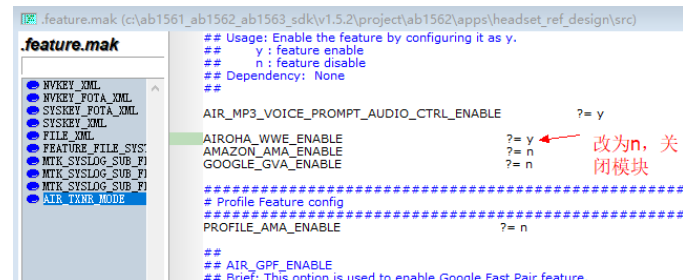
编译出错



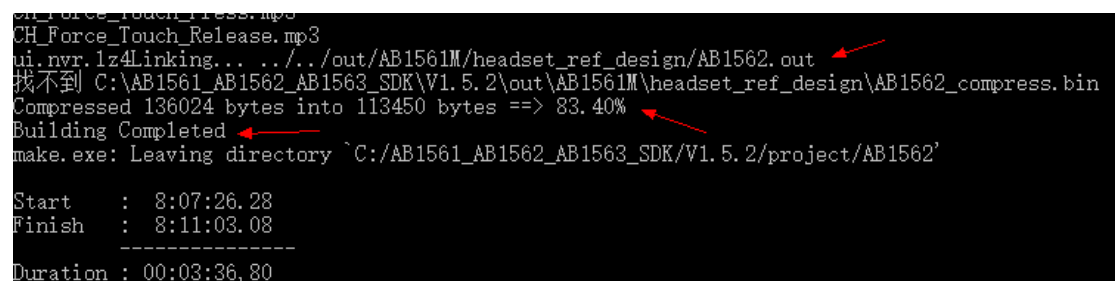
把对应的代码注释掉



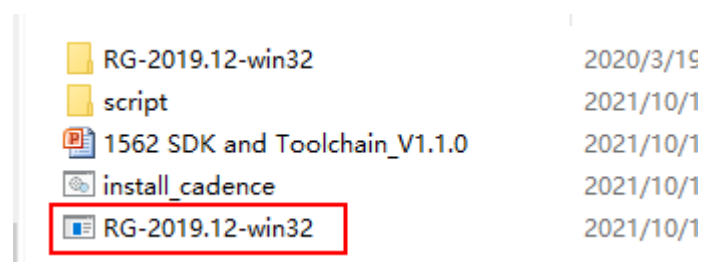
或者在\project\AB1562\apps\headset_ref_design\src\feature.mak 文件里
把 AIROHA_WWE_ENABLE 这 define 改成 n



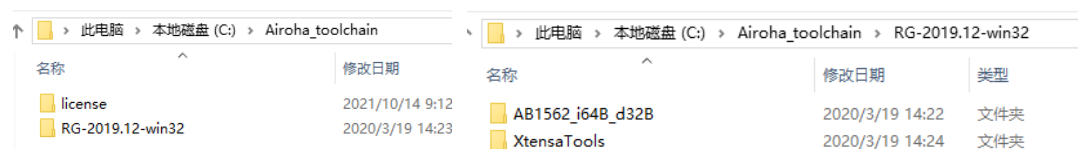
再次编译，成功了



弹出 FLEXnet license Finder 窗口要求指定许可服务器，尝试连接网络后再运行批处理文件，问题仍未解决。换一台电脑就可以正常编译了，没弹出 FLEXnet license Finder，我在安装工具链时运行，再运行 RG-2019.12-win32。所以我也在我的电脑上尝试运行 RG-2019.12-win32，结果竟然能编译成功了



安装完成后，安装目录下有



六、安装 1565/68 的编译环境及编译工程

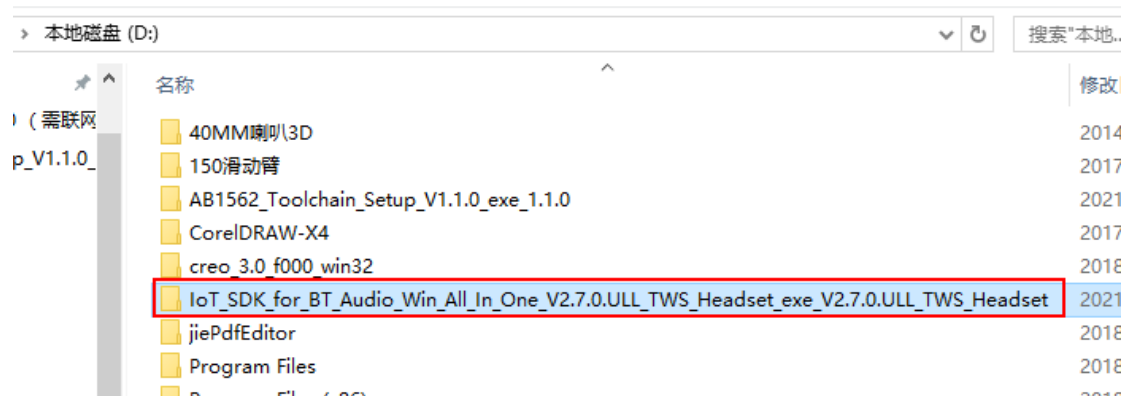
(用 msys 工具 (执行 ./install.sh 命令时需要联网)):

(参考文档 [Airoha_IoT_SDK_for_BT_Audio_Build_Environment_Guide.pdf](#) 或观看安装视频 [AB155x_Windows_Dev_Env_Setup.mp4](#).):

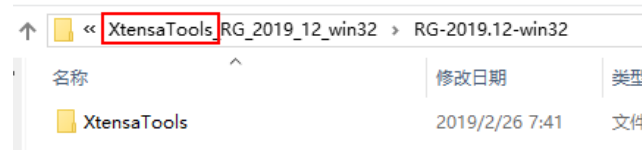
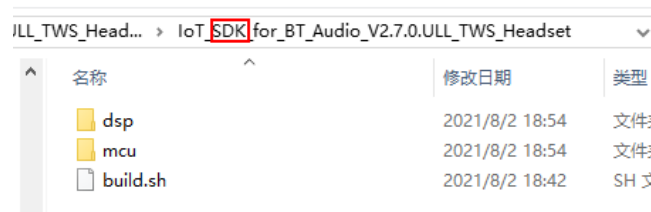
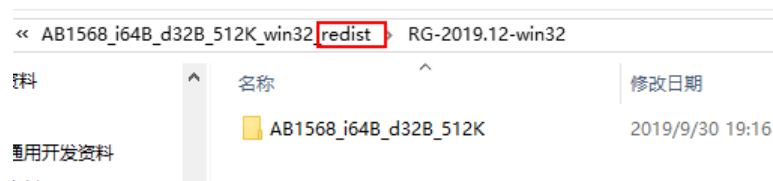
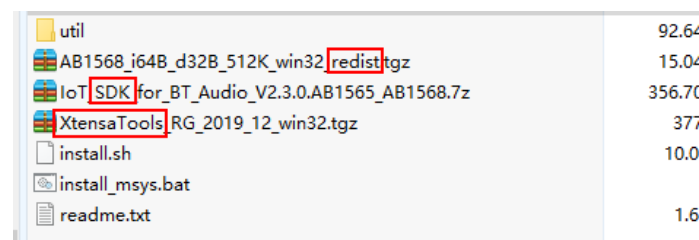
拷贝压缩包 (包含 SDK 和工具包)

IoT_SDK_for_BT_Audio_Win_All_In_One_V2.7.1.ULL_TWS_Headset_exe_V2.7.1.ULL_TWS_Headset

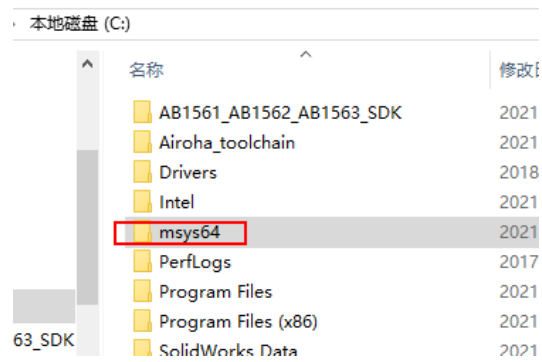
解压文件夹到 D 盘的根目录下



打开文件夹, 运行批处理文件 install_msys.bat 安装 msys, 默认设置



安装完成后, C 盘出现 msys64 文件夹



运行 msys64 文件夹里的



msys 相关命令的使用方法查询:

```
MSYS ~
$ help
GNU bash, 版本 4.4.23(1)-release (x86_64-pc-msys)
这些 shell 命令是内部定义的，请输入 'help' 以获取一个列表。
输入 'help 名称' 以得到有关函数 '名称' 的更多信息。
使用 'info bash' 来获取关于 shell 的更多一般性信息。
使用 'man -k' 或 'info' 来获取不在列表中的命令的更多信息。

名称旁边的星号(*)表示该命令被禁用。

job_spec [&]
(( 表达式 ))
. 文件名 [参数]
:
[ 参数... ]
[[ 表达式 ]]
alias [-p] [名称[=值] ... ]
bg [任务声明 ...]
bind [-lpsPSVK] [-m 键映射] [-f 文>
break [n]
builtin [shell 内建 [参数 ...]]
caller [表达式]
case 词 in [模式 [模式]...] 命令 ;;
cd [-L][-P [-e]] [-@]] [目录]
command [-pVv] 命令 [参数 ...]
compgen [-abcdefgjkuv] [-o 选项] [->
complete [-abcdefgjkuv] [-pr] [-DE] >
comptopt [-o] [-o 选项] [-DE] [名称 ...>
continue [n]
coproc [名称] 命令 [重定向]
declare [-aAfFgiInrtux] [-p] [名称[=>
dirs [-clpv] [+N] [-N]
disown [-h] [-ar] [jobspec ... | pid >
echo [-neE] [参数 ...]
enable [-a] [-dnps] [-f 文件名] [名称>
eval [参数 ...]
exec [-cl] [-a 名称] [命令 [参数 ...]>
exit [n]
export [-fn] [名称[=值] ...] 或 expor>
false
fc [-e 编辑器名] [-lnr] [起始] [终结]>
fg [任务声明]
for 名称 [in 词语 ... ] ; do 命令; do>
for (( 表达式1; 表达式2; 表达式3 )); >
function 名称 { 命令 ; } 或 name O {>
getopts 选项字符串 名称 [参数]
hash [-lr] [-p 路径名] [-dt] [名称 ...>
help [-dms] [模式 ...]
history [-c] [-d 偏移量] [n] 或 hist>
if 命令; then 命令; [ elif 命令; the>
jobs [-lnprs] [任务声明 ...] 或 jobs>
kill [-s 信号声明 | -n 信号编号 | >
let 参数 [参数 ...]
local [option] 名称[=值] ...
logout [n]
mapfile [-d 分隔符] [-n 计数] [-O >
popd [-n] [+N | -N]
printf [-v var] 格式 [参数]
pushd [-n] [+N | -N | 目录]
pwd [-LPW]
read [-ers] [-a 数组] [-d 分隔符] [->
readarray [-n 计数] [-O 起始序号] [->
readonly [-aAf] [名称[=值] ...] 或 r>
return [n]
select NAME [in 词语 ... ;] do 命令;>
set [--abefhkmnptuvxBCHP] [-o 选项名>
shift [n]
shopt [-pqsu] [-o] [选项名 ...]
source 文件名 [参数]
suspend [-f]
test [表达式]
time [-p] 管道
times
trap [-lp] [[参数] 信号声明 ...]
true
type [-afptP] 名称 [名称 ...]
typeset [-aAfFgiInrtux] [-p] 名称[=>
ulimit [-SHabcdefiklmnpqrstuvxPT] >
umask [-p] [-S] [模式]
unalias [-a] 名称 [名称 ...]
unset [-f] [-v] [-n] [名称 ...]
until 命令; do 命令; done
variables - 一些 shell 变量的名称>
wait [-n] [编号 ...]
while 命令; do 命令; done
{ 命令 ; }
```

进入到文件夹

IoT_SDK_for_BT_Audio_Win_All_In_One_V2.7.0.ULL_TWS_Headset_exe_V2.7.0.ULL_TWS_Headset 的路径下（因为路径下有文件 install.sh 文件），
输入 cd d:命令，回车进入 D 盘

```
GS-35514@GS-PC-0446 MSYS ~
$ cd d:

GS-35514@GS-PC-0446 MSYS /d
$ |
```

列出当前路径下的文件夹和文件

```
GS-35514@GS-PC-0446 MSYS /d
$ ls
'$RECYCLE.BIN'
'04_Current_Production.wmv'
150清动臂
40MM喇叭3D
7030头戴夹持力测试治具3D.dwg
'AB1561_AB1562_AB1563_Airoha_To
'AB1561M_GS(Riff2 wireless)_V1.
'AB1561M_GS(Riff2 wireless)_V1.
```

输入 cd ./文件名开头部分，按 Tab 键可以补全，回车进入路径

```
GS-35514@GS-PC-0446 MSYS /d
$ cd ./IoT_SDK_for_BT_Audio_Win_All_In_One_V2.7.1.ULL_TWS_Headset_exe_V2.7.1.ULL_TWS_Headset/
```

输入 `./install.sh` 命令开始安装

```
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_Win_All_In_One_V2.7.0.ULL_TWS_Headset_exe_V2.7.0.ULL_TWS_Headset
$ ./install.sh
```

安装完成后，生成了工程文件夹 `bta_sdk`

（里面就是 `IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset` 解压出来的文件）

此电脑 > 本地磁盘 (D:) > IoT_SDK_for_BT_Audio_Win_1565_1568 > bta_sdk			
名称	修改日期	类型	大小
dsp	2021/8/2 18:54	文件夹	
mcu	2021/8/2 18:54	文件夹	
build.sh	2021/8/2 18:42	SH 文件	
mapping_disk	2021/10/14 16:14	文件	

打印提示编译路径和命令

```
Path = IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset.7z
Type = 7z
Physical Size = 376960195
Headers Size = 170050
Method = LZMA:24 BCJ
Solid = +
Blocks = 2

Everything is Ok

Folders: 1612
Files: 17132
Size: 1249502122
Compressed: 376960195
/home/GS-35514/airoha_sdk_toolchain/xtensa/RG-2019.12-win32/ is installed.
Dsp package was installed.
Installation done. Please change directory to /d/IoT_SDK_for_BT_Audio_Win_1565_1568/bta_sdk to start the first build.
Example:
cd /d/IoT_SDK_for_BT_Audio_Win_1565_1568/bta_sdk
./build.sh ab1565_evk earbuds_ref_design
```

(D:) > IoT_SDK_for_BT_Audio_W

名称
bta_sdk
util
155x_node
AB1568_i64B_d32B_512K_1
install.sh
install_msys
IoT_SDK_for_BT_Audio_V2.
readme
XtensaTools_RG_2019_12_1

输入 `cd bta_sdk` 命令进入 `bta_sdk` 目录

```
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_Win_1565_1568/bta_sdk
$ cd bta_sdk/
```

再输入命令进行编译工程（`dsp` 和 `mcu` 都会编译）

```
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_Win_All_I
$ ./build.sh ab1565_evk earbuds_ref_design
```

编译命令相关说明在文档

The screenshot displays the 'headset_ref_design' directory structure and build instructions. The directory listing shows files like config_bin, ept_config, GCC, inc, src, and readme. The build instructions show how to run the application, build the headset_ref_design application, and download the application binary.

Directory Structure:

名称	修改日期	类型
config_bin	2021/11/24 21:52	文件夹
ept_config	2021/11/24 21:52	文件夹
GCC	2021/12/9 8:55	文件夹
inc	2021/11/24 21:52	文件夹
src	2021/11/24 21:53	文件夹
readme	2021/8/2 18:41	文本文档

Build Instructions:

```

#par Run the application
- How to build the headset_ref_design application
- GCC version
- make command ".\build.sh ab1565 ab1568 evk headset_ref_design -f=feature <board>.mk"
  under the SDK\mcu root folder.
- make command ".\build.sh ab1565 ab1568 evk headset_ref_design -fm=feature <board>.mk"
  f80=feature <board>.mk" under the SDK root folder.
- How to download the headset_ref_design application
- Install and use Flash Tool ("sdk_root\tools\PC_tool_Win.zip") to download the
  application binary headset_ref_design.bin.
  
```

Project Configuration Files:

```

- \b inc/apps: Header files of demo APPs.
- \b src: Common files for system start.
- \b src/boards: Common source files for board.
- \b src/apps: Source code of demo APPs
- Project configuration files using GCC
- \b GCC/feature.mk: The feature configuration file.
- \b GCC/feature "board".mk: The feature configuration file for a different board.
- \b Makefile: The Makefile.
- \b GCC/ab156x_flash.ld: The linker script.
- \b GCC/syscalls.c: The syscalls for GCC.
- \b GCC/startup_ab156x.s: The startup file for GCC.
  
```

编译顺序 (DSP0- > CM4(bootloader -> cm4)), DSP 表示数字信号处理, 做音频处理的, CM4 表示 Cortex-M4, 是跑 Freertos 系统的单片机。
直到出现以下信息, return code 0 表示编译成功

```

TOTAL BUILD: PASS
=====
End CM4 Build
=====

cd H:/
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/earbuds_ref_de
sign_efi.elf out/abi1565_ekv/earbuds_ref_design/debug
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/earbuds_ref_de
sign_dis out/abi1565_ekv/earbuds_ref_design/debug
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/earbuds_ref_de
sign.map out/abi1565_ekv/earbuds_ref_design/debug
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/cm4_log_str.bi
n out/abi1565_ekv/earbuds_ref_design/debug
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/Flash_download
.cfg out/abi1565_ekv/earbuds_ref_design/download
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/*_bootloader.b
in out/abi1565_ekv/earbuds_ref_design/download
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/earbuds_ref_de
sign.bin out/abi1565_ekv/earbuds_ref_design/download
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/partition_tabl
e.bin out/abi1565_ekv/earbuds_ref_design/download
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/*_patch_hdr.bi
n out/abi1565_ekv/earbuds_ref_design/download
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/nvmdm.bin out/a
bi1565_ekv/earbuds_ref_design/download
cp -f mcu/out/abi1565_ekv/earbuds_ref_design/feature_65_ekv/filesystem.bi
n out/abi1565_ekv/earbuds_ref_design/download
=====
Summary CO-BUILD
=====
./build.sh abi1565_ekv earbuds_ref_design
Start CO-Build: 2021年10月14日 16:14:12
End CO-Build: 2021年10月14日 16:21:19
OSPO BUILD: PASS
CM4 BUILD: PASS
TOTAL CO-BUILD: PASS (return code 0)

G5-35514MG5-PC-0446 MYSV /d/Tot_SDK_for_BT_AudioWin_All_In_One_V2.7.0.ULL_TWS_H
edset_exe_V2.7.0.ULL_TWS_Headset/bta_sdk

```

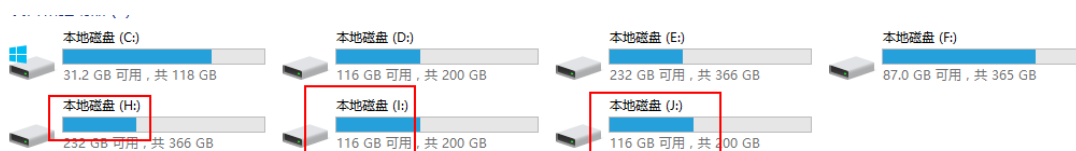
生成 out 文件（编译结果文件）

在 download 文件夹里

« bta_sdk » out » ab1565_evk » earbuds_ref_design » download		
名称	修改日期	类型
ab1565_bootloader.bin	2021/10/14 16:21	BIN 文件
dsp0_headset_ref_design.bin	2021/10/14 16:15	BIN 文件
earbuds_ref_design.bin	2021/10/14 16:21	BIN 文件
filesystem.bin	2021/10/14 16:21	BIN 文件
flash_download.cfg	2021/10/14 16:21	CFG 文件
nvdm.bin	2021/10/14 16:21	BIN 文件
partition_table.bin	2021/10/14 16:21	BIN 文件

如果再有新 SDK 发布，不需要再安装编译环境，直接拷贝 SDK 到目录 bta_sdk 下即可。用同样的方法进行编译。

在有 SDK 的 X 磁盘里编译工程时，在我的电脑里都会多出一个大小一样的 X 盘（里面只有 SDK 文件），重启电脑后消失



七、可视化开发环境 Xtensa Xplorer 安装

If the customer wants to debug through IDE, please first contact Cadence to obtain a license and Cadence Xtensa Explorer.

八、编译 AB1561、62、63 项目的过程剖析

如果耳机芯片型号是 AB1562。

工程文件目录在

本地磁盘 (C:) » AB1561_AB1562_AB1563_SDK » V1.5.1 » project » AB1562 » apps		
名称	修改日期	类型
headset_ref_design	2021/10/14 15:55	文件夹
mcsync_ref_design	2021/10/14 15:55	文件夹

运行批处理文件

[C:] > AB1561_AB1562_AB1563_SDK > V1.5.1		
名称		修改日期
.git		2021/10/14
doc		2021/10/14
include		2021/10/14
out		2021/10/15
prebuilt		2021/10/14
project		2021/10/14
source		2021/10/14
tools		2021/10/14
.Build_AB1561_AB1562_AB1563		2021/6/11
uninst		2021/10/14

DOS 窗口打印信息与批处理文件内容对应

编译环境AB1561_AB1562_AB1563

AB1562 Auto Build Script
1.Build Debug
2.Clean
99.Exit

Select:

.Build_AB1561_AB1562_AB1563 - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```

set shainfo111=0

set /a "x = 0"
:while1
    echo AB1562 Auto Build Script
    echo 1.Build Debug
    echo 2.Clean
    echo 99.Exit
    echo.
    set /p select1="Select: "
:while2
    IF "%select1%"=="1" (
        set mode1=MODE=DEBUG
        set action=all
    )
:while3
    echo Choose Target Board
    echo 1. AB1562
    echo 2. AB1562M

```

编译环境AB1561_AB1562_AB1563

AB1562 Auto Build Script
1.Build Debug
2.Clean
99.Exit

Select: 1
Choose Target Board
1. AB1562
2. AB1562M
3. AB1562A
4. AB1562AM
5. AB1562F
6. AB1561M
7. AB1562U
8. AB1563
9. AB1563A
99.Exit

Select:

.Build_AB1561_AB1562_AB1563 - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```

set mode1=MODE=DEBUG
set action=all
:while3
    echo Choose Target Board
    echo 1. AB1562
    echo 2. AB1562M
    echo 3. AB1562A
    echo 4. AB1562AM
    echo 5. AB1562F
    echo 6. AB1561M
    echo 7. AB1562U
    echo 8. AB1563
    echo 9. AB1563A
    echo 99.Exit
    echo.
    set select2=0
    set /p select2="Select: "

```

```
编译环境AB1561_AB1562_AB1563
AB1562 Auto Build Script
1. Build Debug
2. Clean
99. Exit

Select: 1
Choose Target Board
1. AB1562
2. AB1562M
3. AB1562A
4. AB1562AM
5. AB1562F
6. AB1561M
7. AB1562U
8. AB1563
9. AB1563A
99. Exit

Select: 1
Choose Application
1. awsmce_ref_design
2. headset_ref_design
99. Exit

Select:
```

```
.Build_AB1561_AB1562_AB1563 - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

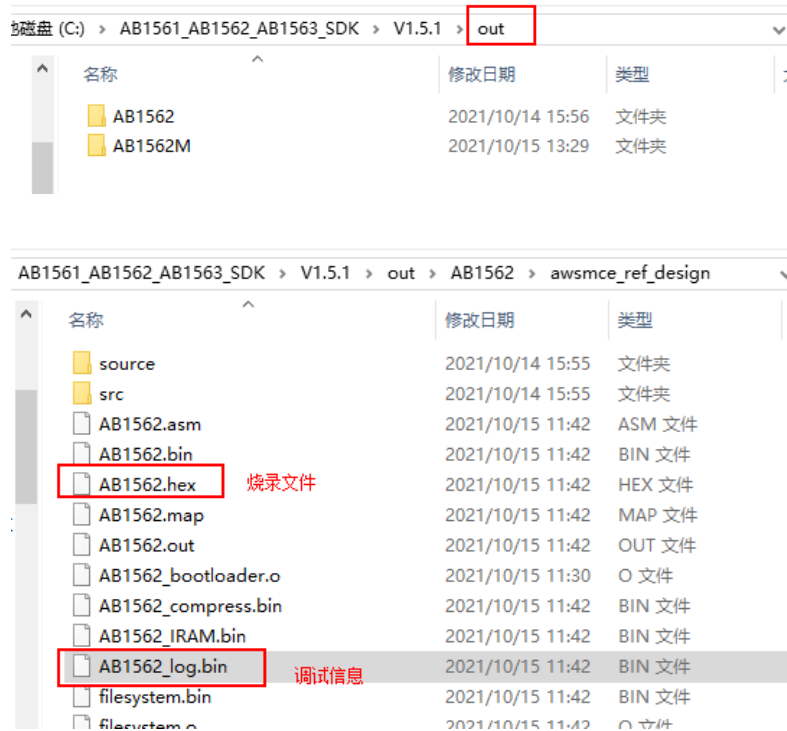
) ELSE (
  echo Invalid Select
  goto :while2
)
goto :while3

:while3
  echo Choose Application
  echo 1. awsmce_ref_design
  echo 2. headset_ref_design
  echo 99. Exit
  echo.
  set select3=0
  set /p select3="Select: "
  SET STARTTIME=%TIME%
  goto fun2

:fun2
  rem generate sha number
  IF "%shainfo111%"=="1"
  goto fun6
)

set /p ref_head=<.git/
SET _result=%ref_head:
set /p sha_number=<.gi
echo const char head_t
echo const char sha_u
```

编译结果保存的文件路径



如果芯片是 AB1561M，那么编译时要选对芯片

```

AB1562 Auto Build Script
1. Build Debug
2. Clean
99. Exit

Select: 1
Choose Target Board
1. AB1562
2. AB1562M
3. AB1562A
4. AB1562AM
5. AB1562F
6. AB1561M
7. AB1562U
8. AB1563
9. AB1563A
99. Exit

Select: 6

Choose Application
1. awsme_ref_design
2. headset_ref_design
99. Exit

Select: 2
make.exe: Entering directory `C:/AB1561_AB1562_AB

```

编译成功

```

ui.nvr.lz4Linking...../out/AB1561M/headset_ref_design/AB1562.out
找不到 C:\AB1561_AB1562_AB1563_SDK\V1.5.1\out\AB1561M\headset_ref_design\AB1562_compress.bin
Compressed 135560 bytes into 113041 bytes ==> 83.39%
Building Completed
make.exe: Leaving directory `C:/AB1561_AB1562_AB1563_SDK/V1.5.1/project/AB1562'

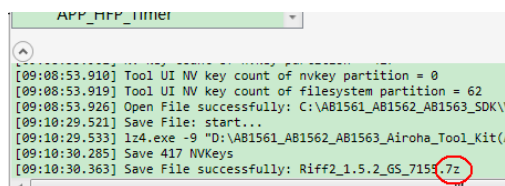
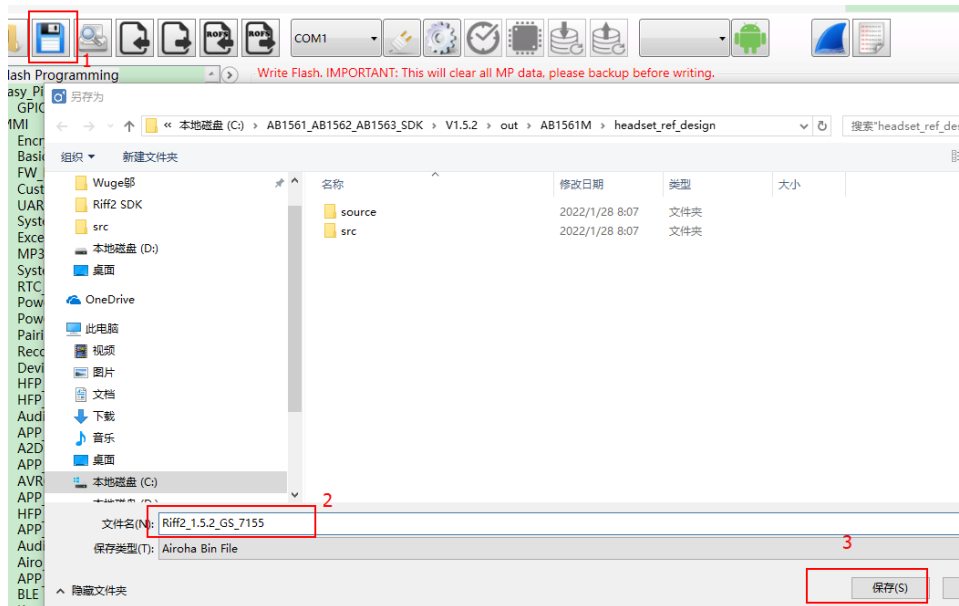
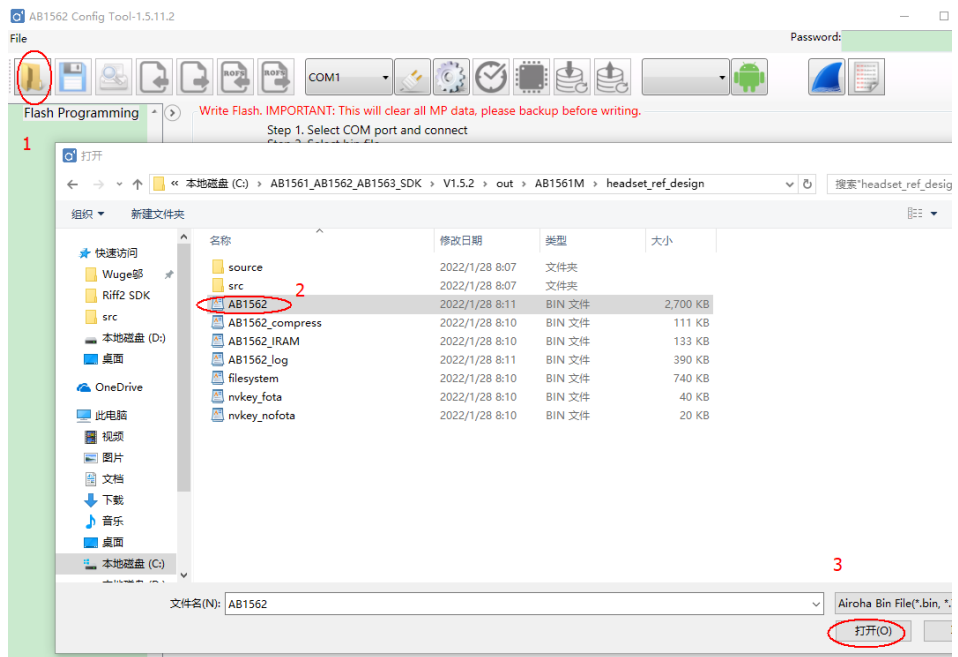
Start   : 14:14:07.55
Finish  : 14:17:48.55
-----
Duration : 00:03:41.00

```

生成 out 文件夹，AB562.bin 是固件，AB1562_log.bin 是抓 log 用的文件

AB1561_AB1562_AB1563_SDK > V1.5.2 > out > AB1561M > headset_ref_design				
名称	修改日期	类型	大小	
source	2022/1/28 8:07	文件夹		
src	2022/1/28 8:07	文件夹		
AB1562.asm	2022/1/28 8:10	ASM 文件	20,193 KB	
AB1562	2022/1/28 8:11	BIN 文件	2,700 KB	
AB1562.hex	2022/1/28 8:11	HEX 文件	9,064 KB	
AB1562.map	2022/1/28 8:11	MAP 文件	5,434 KB	
AB1562.out	2022/1/28 8:11	OUT 文件	26,155 KB	
AB1562_bootloader.o	2022/1/28 8:10	O 文件	33 KB	
AB1562_compress	2022/1/28 8:10	BIN 文件	111 KB	
AB1562_IRAM	2022/1/28 8:10	BIN 文件	133 KB	
AB1562_log	2022/1/28 8:11	BIN 文件	390 KB	
filesystem	2022/1/28 8:10	BIN 文件	740 KB	
filesystem.o	2022/1/28 8:10	O 文件	741 KB	
iram_compress.o	2022/1/28 8:10	O 文件	112 KB	
nvkey_fota	2022/1/28 8:10	BIN 文件	40 KB	
nvkey_fota_bin.o	2022/1/28 8:10	O 文件	41 KB	
nvkey_nofota	2022/1/28 8:10	BIN 文件	20 KB	
nvkey_nofota_bin.o	2022/1/28 8:10	O 文件	21 KB	

将 bin 文件保存成 7z 文件，在 config tool 里打开 bin 文件，然后加入相应配置和提示音，再点击保存，输入文件名，7z 格式（默认格式）



[AB156x]耳机EMP 操作流程

由 Guitao Xie (谢贵涛)创建, 最后修改于十月 25, 2021

耳机更新支持EMP的软件后，如何连接两支手机呢？

可按照以下步骤连接两支手机：

1. 耳机开机触发可见模式，连接第一支手机，随后关闭手机蓝牙；
2. 耳机再次触发进入可见配对模式，连接第二支手机；
3. 第一支手机重新打开蓝牙，手动点击与耳机进行连线；自此完成耳机同时与两支手机连接的功能；
4. 后续耳机重开会主动回连一支手机，第二支手机需要在蓝牙界面上手动点击重新连接。

EMP 操作流程

九、分开编译 1565 或 1568 的 dsp 和 mcu

此电脑 > 本地磁盘 (D:) > IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset		
名称	修改日期	类型
dsp	2021/8/2 18:54	文件夹
mcu	2021/8/2 18:54	文件夹
build.sh	2021/8/2 18:42	SH 文件

SDK 包含 dsp 和 mcu

此电脑 > 本地磁盘 (D:) > IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset > dsp > project			
名称	修改日期	类型	大小
ab1565_ab1568_evk	2021/8/2 18:41	文件夹	

dsp

此电脑 > 本地磁盘 (D:) > IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset > mcu > project			
名称	修改日期	类型	大小
ab1565_ab1568_evk	2021/8/2 18:41	文件夹	
readme	2021/8/2 18:41	文本文档	5 KB

mcu

在 msys 上运行编译命令

```
/d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
燕秀在win10上安装无法使用的解决办法.rar
有道词典
主动加密文件夹(gs-pc-0446.szgs.com)'
GS-35514@GS-PC-0446 MSYS /d
$ cd IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset/
IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset/
IoT_SDK_for_BT_Audio_Win_1565_1568/
GS-35514@GS-PC-0446 MSYS /d
$ cd IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset/
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ls
build.sh dsp mcu
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ./build.sh dsp/
.config build.sh driver/ prebuilt/ tools/
.config.mk config/ kernel/ project/
.rule.mk doc/ middleware/ readme.txt
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ./build.sh dsp/project/ab1565_ab1568_evk/
```

提示使用命令 ./build.sh list

```
/d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
-mk
-fd0=<feature makefile>
Replace feature.mk with other makefile for dsp0. For example,
the feature_example.mk is under project folder, -fd0=feature_example.
mk
will replace feature.mk with feature_example.mk.
-fd1=<feature makefile>
Replace feature.mk with other makefile for dsp1. For example,
the feature_example.mk is under project folder, -fd1=feature_example.
mk
will replace feature.mk with feature_example.mk.
-mcu
Build MCU only. (Use default dsp bin at 'dsp/prebuilt/dsp0/' instead
of build dsp bin).
=====
List Available Example Projects
=====
Usage: ./build.sh list
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$
```

列出了所有可以编译工程的项

```
/d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset

=====
List Available Example Projects
=====
Usage: ./build.sh list

GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ./build.sh list
Mapping sdk folder to disk H:
$subst_disk : H
$subst_dir : /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$subst_dir_without_slash : d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$working_dir_disk : d
=====
Available Build Projects:
=====
ab1552_asia
  earbuds_ref_design
    board_share: ab155x_evk
    cm4: earbuds_ref_design
    dsp0: dsp0_headset_ref_design
    fm: feature_52_asia.mk
    fd0: feature_52_asia.mk
```

board_share 为对应的板子, fm 是编译时调用的 mk 文件
指定编译某个项的命令 ./build.sh <板名> <工程名> <参数>

./build.sh <board> <project> [clean] <argument>

参数有四种格式

```
=====
Build Project
=====
Usage: ./build.sh <board> <project> [clean] <argument>

Example:
./build.sh ab155x_evk earbuds_ref_design
./build.sh ab155x_evk earbuds_ref_design -fm=feature_ab1552_evk.mk
./build.sh clean (clean folder: out)
./build.sh ab155x_evk clean (clean folder: out/ab155x_evk)
./build.sh ab155x_evk earbuds_ref_design clean (clean folder: out/ab155x_evk/earbuds_ref_design)

Argument:
-fm=<feature makefile>
  Replace feature.mk with other makefile for [mcu]. For example,
  the feature_example.mk is under project folder, -fm=feature_example.mk
  will replace feature.mk with feature_example.mk.

-fd0=<feature makefile>
  Replace feature.mk with other makefile for [dsp0]. For example,
  the feature_example.mk is under project folder, -fd0=feature_example.mk
  will replace feature.mk with feature_example.mk.

-fd1=<feature makefile>
  Replace feature.mk with other makefile for [dsp1]. For example,
  the feature_example.mk is under project folder, -fd1=feature_example.mk
  will replace feature.mk with feature_example.mk.

-mcu
  Build MCU only. (Use default dsp bin at 'dsp/prebuilt/dsp0/' instead of build dsp bin).
```

执行命令 ./build.sh ab1565_evk earbuds_ref_design

不同的命令参数会调用不同的 mk 文件进行编译

```
/d/IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset

cm4: dongle_ref_design
dsp0: dsp0_headset_ref_design
fm: feature_65_evb.mk
fd0: feature_65_evb_ull_dongle.mk
ab1565_evk 硬件
  earbuds_ref_design 应用
    board_share: ab1565_ab1568_evk
    cm4: earbuds_ref_design
    dsp0: dsp0_headset_ref_design
    fm: feature_65_evk.mk
    fd0: feature_65_evb.mk
ab1565_evk
  earbuds_ref_design_lwire
    board_share: ab1565_ab1568_evk
    cm4: earbuds_ref_design
    dsp0: dsp0_headset_ref_design
    fm: feature_65_evk_lwire.mk
    fd0: feature_65_evb.mk
ab1565_evk
  earbuds_ref_design_emp
    board_share: ab1565_ab1568_evk
    cm4: earbuds_ref_design
    dsp0: dsp0_headset_ref_design
    fm: feature_65_evk_emp.mk
```

`./build.sh ab1565_evk earbuds_ref_design`

```

ab1565_evk
  earbuds_ref_design
    board_share: ab1565_ab1568_evk
    cm4: earbuds_ref_design
    dsp0: dsp0_headset_ref_design
    fm: feature_65_evk.mk
    fd0: feature_65_evb.mk
        
```

`./build.sh ab1568_evk headset_ref_design`

```

ab1568_evk
  headset_ref_design
    board_share: ab1565_ab1568_evk
    cm4: headset_ref_design
    dsp0: dsp0_headset_ref_design
    fm: feature_68_evk.mk
    fd0: feature_68_evb.mk
        
```

ab1565_ab1568_evk > apps > earbuds_ref_design > GCC

名称	修改日期	类型
feature_65_8m_cell_v3	2021/8/2 18:41	MK 文件
feature_65_8m_cell_v3_ull	2021/8/2 18:41	MK 文件
feature_65_8m_evk	2021/8/2 18:41	MK 文件
feature_65_8m_evk_emp_share_1wire	2021/8/2 18:41	MK 文件
feature_65_8m_evk_leaudio	2021/8/2 18:41	MK 文件
feature_65_8m_evk_leaudio_1wire	2021/8/2 18:41	MK 文件
feature_65_8m_evk_ull	2021/8/2 18:41	MK 文件
feature_65_8m_evk_xiaoai	2021/8/2 18:41	MK 文件
feature_65_cell_v2	2021/8/2 18:41	MK 文件
feature_65_evb	2021/8/2 18:41	MK 文件
feature_65_evb_1wire	2021/8/2 18:41	MK 文件
feature_65_evk	2021/8/2 18:41	MK 文件
feature_65_evk_1wire	2021/8/2 18:41	MK 文件
feature_65_evk_emp	2021/8/2 18:41	MK 文件

ab1565_ab1568_evk > apps > headset_ref_design > GCC

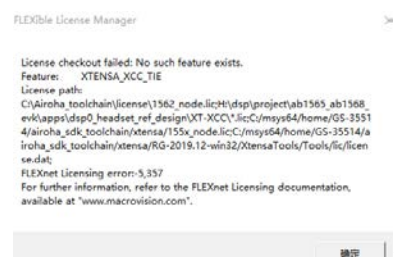
名称	修改日期	类型
ab156x_evb.cmm	2021/8/2 18:41	CMM 文件
ab156x_flash.ld	2021/8/2 18:41	LD 文件
ab156x_flash_8m.ld	2021/8/2 18:41	LD 文件
ab156x_flash_8m_fota_test.ld	2021/8/2 18:41	LD 文件
ab156x_sysram.ld	2021/8/2 18:41	LD 文件
feature	2021/8/2 18:41	MK 文件
feature_65_8m_evk	2021/8/2 18:41	MK 文件
feature_65_evb	2021/8/2 18:41	MK 文件
feature_65_evb_emp	2021/8/2 18:41	MK 文件
feature_65_evk	2021/8/2 18:41	MK 文件
feature_65_evk_emp	2021/8/2 18:41	MK 文件
feature_68_evb_ull	2021/8/2 18:41	MK 文件
feature_68_evk	2021/12/1 19:56	MK 文件
feature_68_evk_emp	2021/8/2 18:41	MK 文件
feature_68_evk_ull	2021/8/2 18:41	MK 文件
feature_68_evk_ull_a2dp	2021/8/2 18:41	MK 文件

```

MSYS /d/IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset
$ ./build.sh ab1565_evk earbuds_ref_design
Mapping sdk folder to disk H:
$subst_disk : H
$subst_dir : /d/IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset
$subst_dir_without_slash : dIoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset
$working_dir_disk : d
board from mapping_proj.cfg: ab1565_ab1568_evk

cd H:/dsp
Start DSP0 Build
=====
output folder change to: /h/dsp/out/ab1565_ab1568_evk/dsp0_headset_ref_design/feature_65_evb
        
```

弹出以下窗口说明编译环境出问题，需要重新安装编译环境（联网状态下）。



提示没有 feature，按 Ctrl+C 强制退出编译

单独编译 mcu

`./build.sh ab1565_evk earbuds_ref_design -mcu`

```
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ./build.sh ab1565_evk earbuds_ref_design -mcu
```

```
MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
Build... aes_alt.o PASS
Build... sha1_alt.o
Build... des_alt.o PASS
Build... sha256_alt.o
Build... md5_alt.o PASS
Build... sha512_alt.o
Build... sha1_alt.o PASS
Build... gcm_alt.o
Build... sha256_alt.o PASS
Build... rsa_internal.o
Build... sha512_alt.o PASS
Build... gcm_alt.o PASS
Build... ar1.o
Build... chachapoly.o
Build... rsa_internal.o PASS
Build... poly1305.o
Build... ar1.o PASS
Build... hmac.o
Build... chachapoly.o PASS
Build... poly1305.o PASS
Build... nist_kw.o
Build... hmac.o PASS
Build... syscalls.o
Build... ssl_msg.o PASS
Build... ssl_tls13_keys.o
Build... nist_kw.o PASS
Build... syscalls.o
Build... ssl_msg.o PASS
Build... bl_dbgprint.o
Build... ssl_tls13_keys.o PASS
Build... custom_bldconf.o
```

```
Build... verno.o PASS
Linking...
Done
   text    data    bss    dec    hex filename
2037388    8664   308048 2354100 23ebb4 I:/mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/earbuds_ref_design.elf
append XML file: ../../../../config/chip/ab156x/nvkey.xml
7 NV added.
parse 1 xml done.
append XML file: ../../../../project/ab1565_ab1568_evk/apps/earbuds_ref_design/config_bin/nvkey.xml
4 NV added.
parse 1 xml done.
append XML file: ../../../../project/ab1565_ab1568_evk/apps/earbuds_ref_design/config_bin/ab1565_evk/nvkey.xml
240 NV added.
parse 1 xml done.
Generate NVDM image.
NVDM config: 400, 4096, 1024, 16, 32
total avail space = 44820

Get NVDM image.
Successfully merge 251 NVKEYs and generate a NVDM image of 64 KBytes
# For generate the language model binary file and copy the binary file to the output path.
copy.firmware.sh...
BOARD=ab1565_ab1568_evk
ab1565_evk
cp /i/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/GCC/ab156x_evk.cmm to /i/mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/
make: 离开目录"/i/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/GCC"
Build CM4 Firmware...Done

=====
./build.sh ab1565_ab1568_evk earbuds_ref_design -f=feature_65_evk.mk -o=OUT=out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk bl -cobuild
Start Build: 2021年10月25日 14:30:07
BOOTLOADER BUILD : PASS
End Build: 2021年10月25日 14:36:06
MCU warnings: 14
MCU Build Log: /i/mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/log/err.log
TOTAL BUILD: PASS

=====
End CM4 Build

=====

cd I:/
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/earbuds_ref_design.elf out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/earbuds_ref_design.dfs out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/earbuds_ref_design.map out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/cm4_log_str.bin out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/Flash_download.cfg out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/^_bootloader.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/earbuds_ref_design.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/partition_table.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/^_patch_hdr.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/nvdm.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature_65_evk/filesystem.bin out/ab1565_evk/earbuds_ref_design/download

=====
Summary CO-BUILD
=====
./build.sh ab1565_evk earbuds_ref_design -mcu
Start CO-Builds: 2021年10月25日 14:29:58
End CO-Build: 2021年10月25日 14:36:07
CM4 BUILD: PASS
TOTAL CO-BUILD: PASS (return code 0)
```

编译成功

生成 out 文件夹，里面是编译结果文件

out > ab1565_evk > earbuds_ref_design > download			
out > ab1565_evk > earbuds_ref_design > debug		名称	修改日期
名称	修改日期	ab1565_bootloader.bin	2021/10/25 14:
		earbuds_ref_design.bin	2021/10/25 14:
cm4_log_str.bin	2021/10/25	filesystem.bin	2021/10/25 14:
earbuds_ref_design.elf	2021/10/25	flash_download.cfg	2021/10/25 14:
earbuds_ref_design.map	2021/10/25	nvdm.bin	2021/10/25 14:
		partition_table.bin	2021/10/25 14:

单独编译 dsp，feature makefile 文件所在路径是 XT-XCC 文件夹

dsp	>	project	>	ab1565_ab1568_evk	>	apps	>	dsp0_headset_ref_design	>	XT-XCC
名称		修改日期		类型		大小				
 65_68_switch		2021/8/2 18:41		MK 文件		1 KB				
 dsp0_flash.lcf		2021/8/2 18:41		LCF 文件		12 KB				
 dsp0_flash_3rd_nr.lcf		2021/8/2 18:41		LCF 文件		12 KB				
 dsp0_flash_8m.lcf		2021/8/2 18:41		LCF 文件		12 KB				
 dsp0_flash_cell.lcf		2021/8/2 18:41		LCF 文件		12 KB				
 dsp0_flash_leaudio_dongle.lcf		2021/8/2 18:41		LCF 文件		12 KB				
 dsp0_flash_no_lm.lcf		2021/8/2 18:41		LCF 文件		12 KB				
 dsp0_iram.lcf		2021/8/2 18:41		LCF 文件		10 KB				
 feature		2021/8/2 18:41		MK 文件		4 KB				
 feature_65_8m_cell		2021/8/2 18:41		MK 文件		9 KB				
 feature_65_8m_cell_leaudio		2021/8/2 18:41		MK 文件		8 KB				
 feature_65_8m_cell_ull		2021/8/2 18:41		MK 文件		10 KB				
 feature_65_8m_evb		2021/8/2 18:41		MK 文件		8 KB				
 feature_65_8m_evb_headset		2021/8/2 18:41		MK 文件		8 KB				
 feature_65_8m_evb_leaudio		2021/8/2 18:41		MK 文件		8 KB				
 feature_65_8m_evb_leaudio_src		2021/8/2 18:41		MK 文件		8 KB				
 feature_65_8m_evb_ull_leaudio		2021/8/2 18:41		MK 文件		10 KB				

./build.sh ab1565_evk earbuds_ref_design -fd0=feature_ab1565_evk.mk

```

GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ./build.sh ab1565_evk earbuds_ref_design -fd0=feature_ab1565_evk.mk
Mapping sdk folder to disk I:
$subst_disk : I
$subst_dir : /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$subst_dir_without_slash : dIoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$working_dir_disk : d
board from mapping_proj.cfg: ab1565_ab1568_evk

cd I:/dsp
=====
Start DSP0 Build
=====
output folder change to: /i/dsp/out/ab1565_ab1568_evk/dsp0_headset_ref_design/Feature_ab1565_evk
Error: cannot find feature_ab1565_evk.mk under project/ab1565_ab1568_evk/apps/dsp0_headset_ref_design/XT-XCC.
=====
End DSP0 Build
=====

```

编译失败，因为 mk 文件不存在

```

cd I:/
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/earbuds_ref_design.elf out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/earbuds_ref_design.dis out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/earbuds_ref_design.map out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/cm4_log_str.bin out/ab1565_evk/earbuds_ref_design/debug
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/flash_download.cfg out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/*_bootloader.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/earbuds_ref_design.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/partition_table.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/*_patch_hdr.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/nvdm.bin out/ab1565_evk/earbuds_ref_design/download
cp -f mcu/out/ab1565_ab1568_evk/earbuds_ref_design/feature/filesystem.bin out/ab1565_evk/earbuds_ref_design/download
=====
Summary CO-BUILD
=====
./build.sh ab1565_evk earbuds_ref_design -fd0=feature_ab1565_evk.mk
Start CO-Build: 2021年10月25日 14:51:07
End CO-Build: 2021年10月25日 14:57:08
DSP0 BUILD: FAIL
CM4 BUILD: PASS
TOTAL CO-BUILD: FAIL (return code 1)

```

编译错误 log 记录可以在 mcu 指定路径下看相关文件

\\mcu\\out\\ab1565_ab1568_evk\\headset_ref_design\\feature_68_evk\\log\\err.log

```

In file included from h:\mcu\middleware\atk\mcu\inc\mcukey.h:39:0,
                 from ../../../../../../project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/events/apps_race_cmd_event.c:45:
h:\mcu\middleware\atk\mcu\inc\mcukey.h:232:15: note: expected 'const uint8_t*' but argument is of type 'int32_t*'
mcu_status_t mcu_write_data_item(const char *group_name,
../../../../project/ab1565_ab1568_evk/apps/headset_ref_design/src/task_def.c:143:6: error: redefinition of 'test_task'
void test_task(void *param)
                                        ^
../../../../project/ab1565_ab1568_evk/apps/headset_ref_design/src/task_def.c:124:6: note: previous definition of 'test_task' was here
void test_task(void *param)
                                        ^
arm-none-eabi-gcc.exe: error: ../../../../../../out/ab1565_ab1568_evk/headset_ref_design/feature_68_evk/obj/project/ab1565_ab1568_evk/apps/headset_ref_design/src/task_def.o: No such file or directory
make: *** [Makefile:608: h:\mcu\out\ab1565_ab1568_evk/headset_ref_design/feature_68_evk/headset_ref_design.elf] 错误 1

```

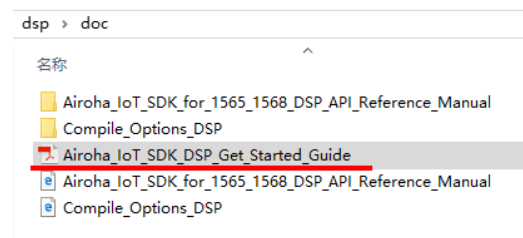
打印以下表示编译成功

```

=====
Summary CO-BUILD
=====
./build.sh ab1568_evk headset_ref_design
Start CO-Build: 2021年10月28日 10:37:55
End CO-Build: 2021年10月28日 10:45:53
DSP0 BUILD: PASS
CM4 BUILD: PASS
TOTAL CO-BUILD: PASS (return code 0)

```

参考说明文档



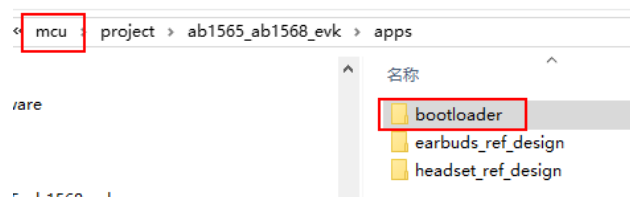
清除工程命令 `./build.sh clean`，SDK 下的 out 文件夹会被清除

```
GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ls
build.sh dsp mapping_disk mcu out

GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ./build.sh clean
Mapping sdk folder to disk I:
$subst_disk : I
$subst_dir : /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$subst_dir_without_slash : dIoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$working_dir_disk : d
rm -rf I:/mcu/out
rm -rf I:/dsp/out
rm -rf /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset/out

GS-35514@GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset
$ ls
build.sh dsp mapping_disk mcu
```

1565 或 1568 上电最先运行的程序是 mcu 目录下的 bootloader



十、文档 AB1561_AB1562_AB1563_Get_Started_Guide.pdf

内容学习笔记

Software Development Kit(SDK) 洛达的开发板统称 EVK

平台架构布局 Architecture of the platform

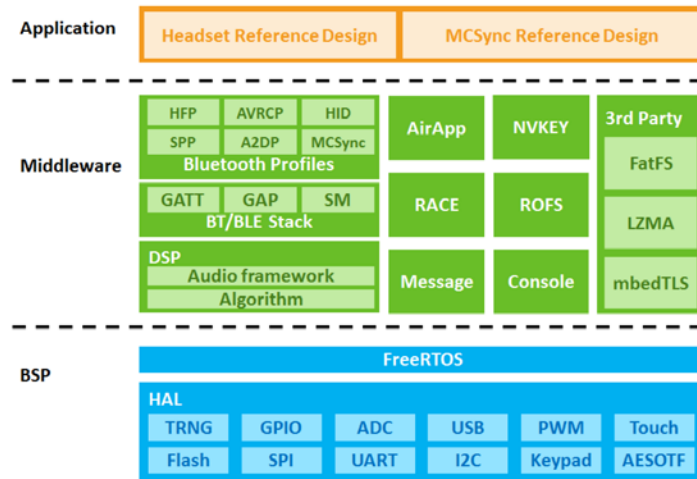


Figure 1. Architecture layout of the platform

蓝牙配置:

headset 耳机

MCSync (MultiCast Synchronization) 多路广播同步

SDK 提供可参考的工程应用程序 (SDK 路径/SDK 版本/工程/AB1562/应用程序/工程名)

project > AB1562 > apps > headset_ref_design	
名称	修改日
ept_config	2021/1
inc	2021/1
src	2021/1

ept_config 工程 GPIO 配置文件

inc 工程通用头文件

src 工程源文件, 头文件和讲述工程配置文件

开发环境的下载和安装方法:

To install the SDK, please get **AB1561_AB1562_AB1563_SDK_Vx.x.x** from MOL

Xtensa 工具链下载和安装:

方法 1: 使用脚本命令行自动安装

方法 2: 使用界面安装 Xporler IDE

Get latest Airoha_AB1561_AB1562_AB1563_Toolchain_Setup from MOL.

Execute install_cadence.bat.

将图像文件(bin 文件)下到 EVK 板:

Please contact Airoha AE to get Airoha.Tool.Kit.exe tool. Use the Airoha.Tool.Kit.exe application and select **Config Tool** to flash the image for AB1561_AB1562_AB1563 EVK.

打开所需的 bin 文件

使用 USB 线将 EVK 板连接到 PC 机

在 Airoha Tool Kit 软件里的 Config Tool 工具选择正确的 COM 端口并点击连接

点击 Write DA+Write Flash 准备自动下载

按一下 EVK 板子上的 RST 键，再拨板子上的 Regen 开关，开始自动下载

运行 EVK 板子上的程序：

把适配器接到板子

将电源开关拨到 1 模式

按下 RST 键和长按 REGEN 键 5 秒重启板子

判断板子是否启动的三种方法（D5 灯闪三下、音频输出口有声音、用 Config Tool 查看 log）

查看系统 log 的方法（使用 Airoha Tool Kit 软件中的 Logging Tool）

Windows 系统上在线调试 EVK 板子（用 JTAG 和集成开发环境 Xtensa Xplorer，类似 Keil 调试）

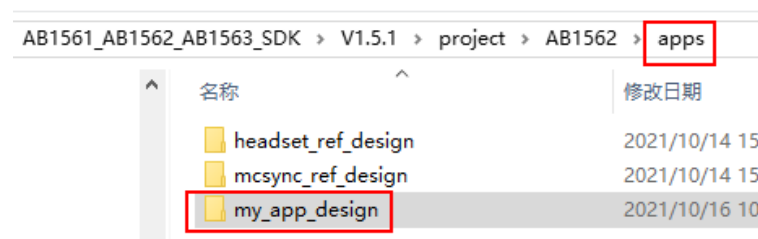
使用 SDK 构建工程：

双击运行 SDK 目录下的 bat 批处理文件（如果没有使用默认路径(C:\usr\xtensa)安装 Xtensa Xplorer，则需要设置该路径和 XTENSA_SYSTEM 路径到这里）

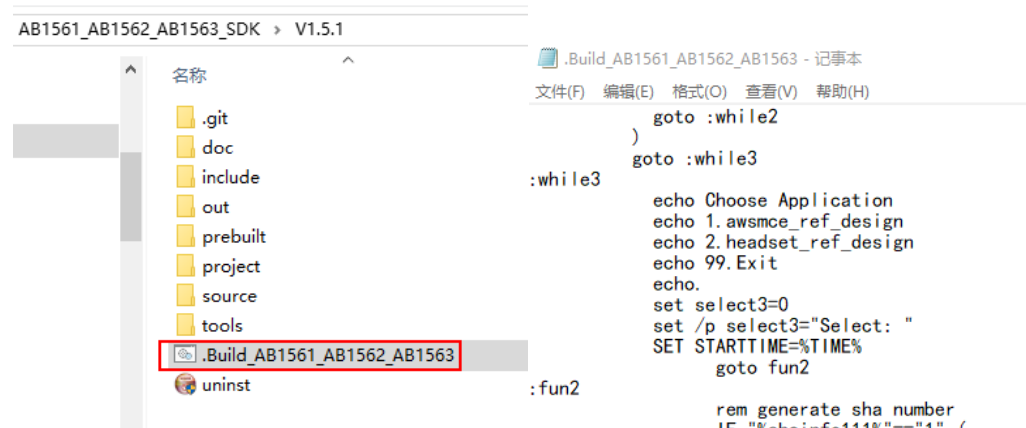
根据提示选择需要编译的工程

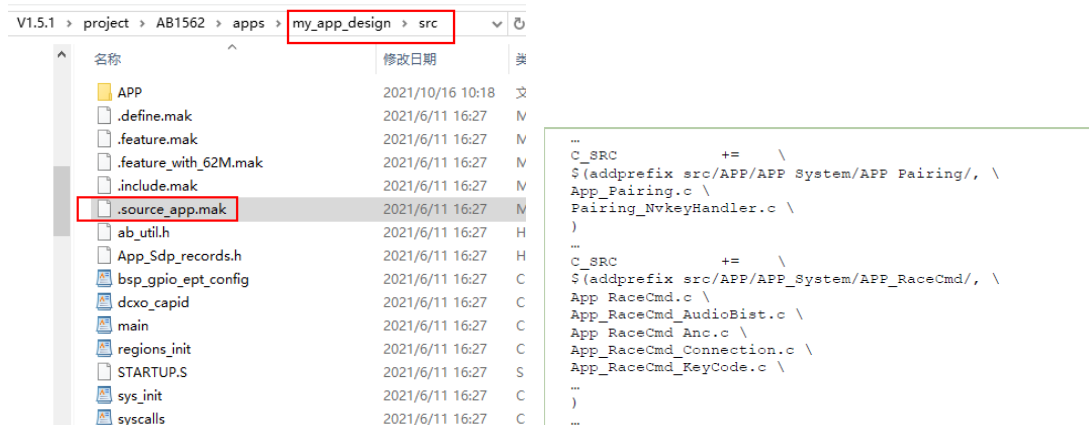
用一个现有的工程来创建自己的工程：

在 SDK 安装路径的 ..\project\AB1562\apps 文件夹下复制其中一个现有工程，粘贴并重命名 my_app_design（可以对该文件夹的源代码进行任意改动，如添加源文件和头文件）

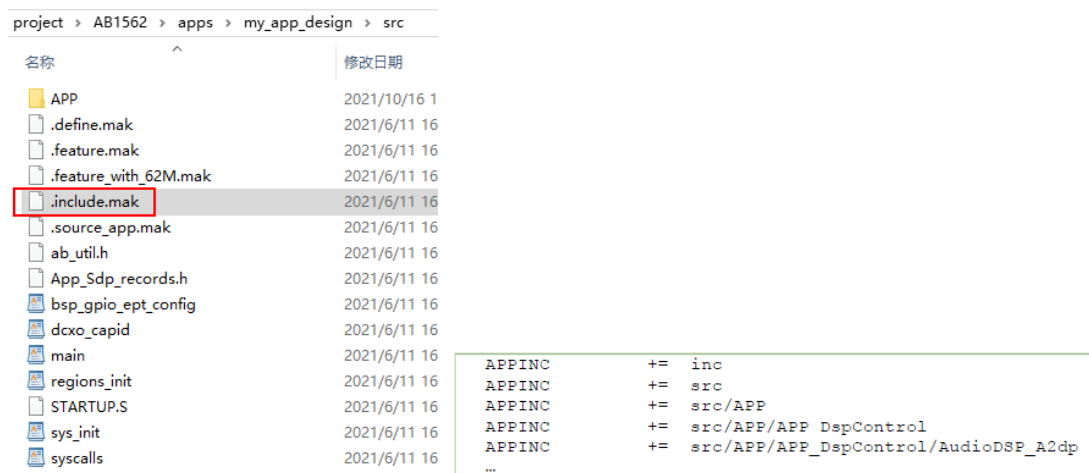


用文本方式打开 bat 批处理文件并做必要的改动





要编译所添加头文件，需要修改include.mak 文件



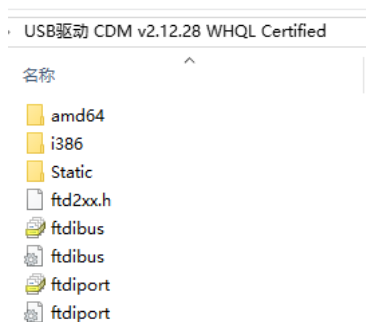
FOTA 是用于 EVK 升级的，用 SPP 无线升级，耳机电量低于 70%不能升级。

AB1568 有多种电源模式（Active、Idle、Sleep、RTC、Off），可参考 SDK 目录下的文件
Airoha_IoT_SDK_for_BT_Audio_Power_Mode_Developers_Guide.pdf

十一、将固件程序烧录到耳机

1、烧录固件到 AB1562:

用 USB 转串口线连接耳机到电脑，安装 USB 驱动程序包 CDM v2.12.28 WHQL Certified

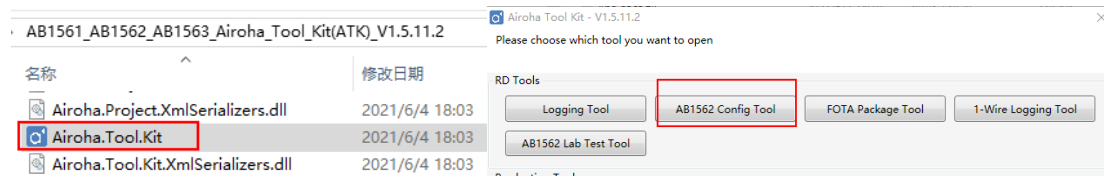


在设备管理器中指定到该驱动文件夹，安装两次即可

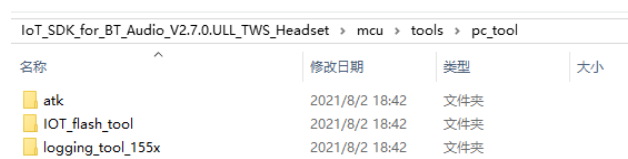


安装成功，出现端口号

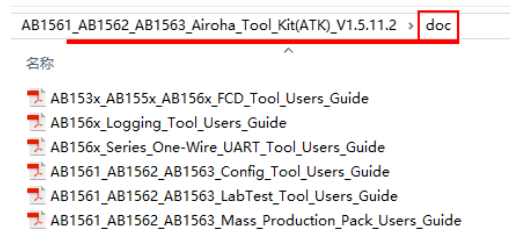
打开烧录工具 Airoha.Tool.Kit 的 AB1562 Config Tool



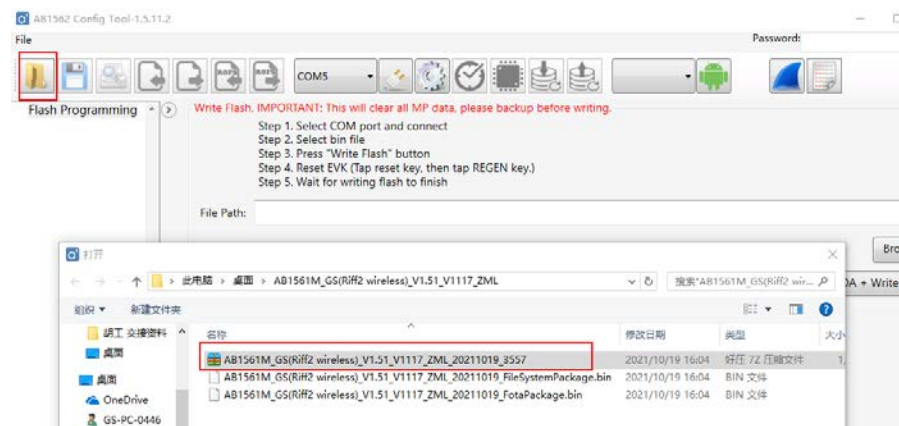
在以下路径也可以找到工具



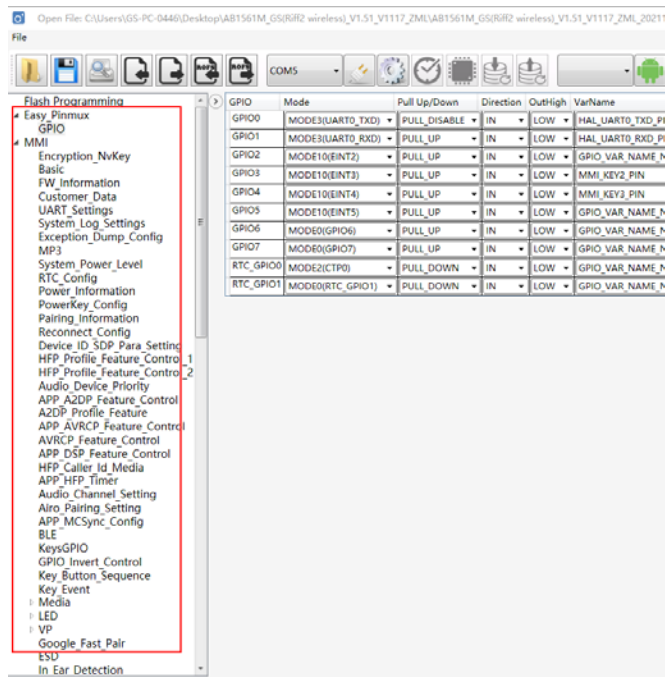
工具的使用方法可以参考 Airoha_Tool_Kit(ATK)目录下的 doc 文档



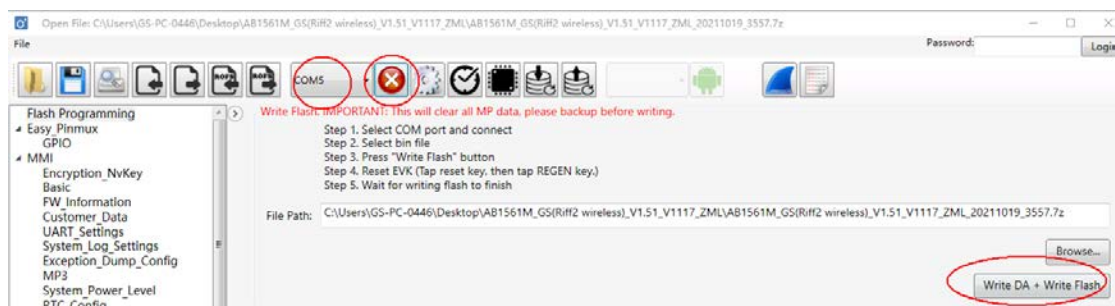
打开程序文件（7z 文件，路径不能有中文）



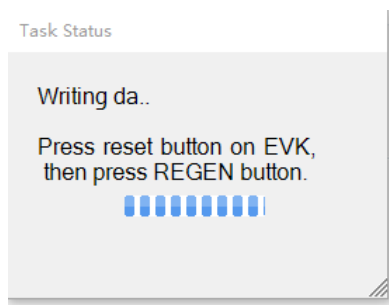
左边列表就是程序文件中的配置参数，可以查看参数



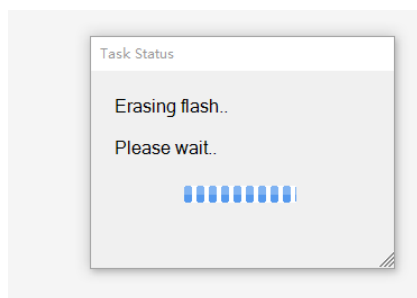
选择对应端口并连接，再点击烧录按钮 Write DA + Write Flash



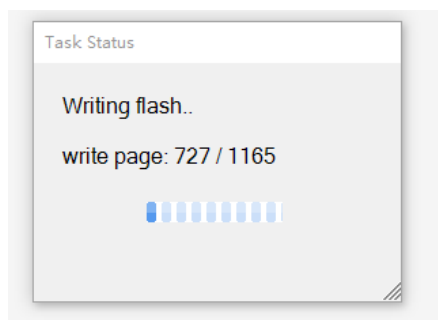
出现以下提示（只有几秒钟）时，拔插 USB 重连耳机，就开始自动下载程序了



如果没弹出提示，说明对应端口未打开



擦除固件



正在烧录中

```
[17:16:48.585] Please enable comport
[17:19:18.211] Write da start
[17:19:26.481] Write da fail
[17:19:32.915] Write da start
[17:19:39.175] Write da ok
[17:19:39.175] Write flash start
[17:20:08.256] Write flash ok (35600.1596 ms)
```

烧录成功

如果修改源代码，则保存后用 SDK 目录下 bat 批处理文件进行编译。

2、烧录固件到 AB1568:

用 Source Insight 软件编辑源代码，用命令进行编译，用 Flash Tool 软件

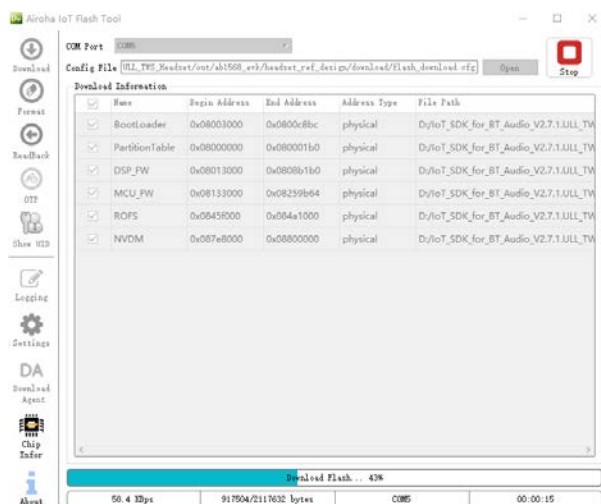
(IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset\mcu\tools\pc_tool\IOT_flash_tool\win) 烧录到耳机

IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset > mcu > tools > pc_tool > IOT_flash_tool > win			
名称	修改日期	类型	大小
IoT_Flash_Tool_Windows_V2.9.3_20200515	2021/8/2 18:42	好压 RAR 压缩文件	17,596 KB

打开工程编译结果文件里的.cfg 文件

IoT_SDK_for_BT_Audio_V2.7.0.ULL_TWS_Headset > out > ab1565_evk > earbuds_ref_design > download			
名称	修改日期	类型	大小
ab1565_bootloader.bin	2021/10/25 17:19	BIN 文件	40 KB
earbuds_ref_design.bin	2021/10/25 17:19	BIN 文件	1,218 KB
filesystem.bin	2021/10/25 17:19	BIN 文件	200 KB
flash_download.cfg	2021/10/25 17:19	CFG 文件	2 KB
nvdm.bin	2021/10/25 17:19	BIN 文件	64 KB
partition_table.bin	2021/10/25 17:19	BIN 文件	1 KB

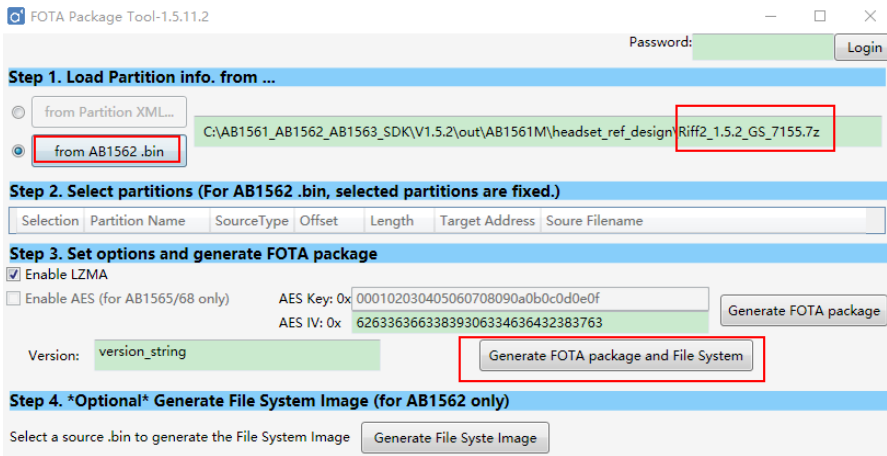
可以选择烧录哪个 bin 文件，选择端口号，点击 start，按住电源键插 USB 线，硬件复位开始上传。



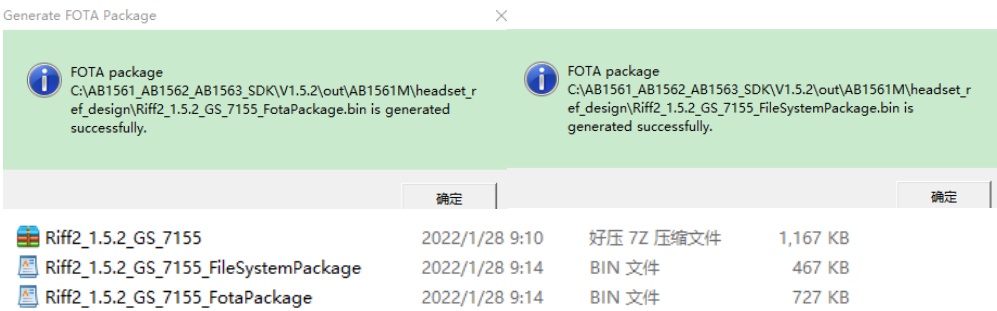
用 Config Tool 软件修改 NV Key 并烧录到耳机

3、用 FOTA 升级 AB1562 的固件

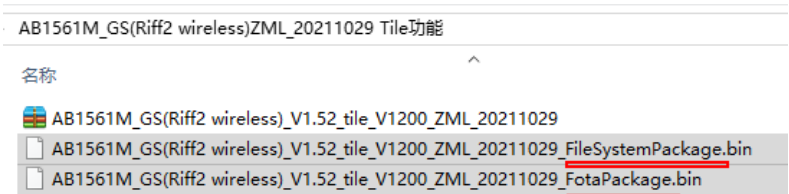
生成 FOTA 文件



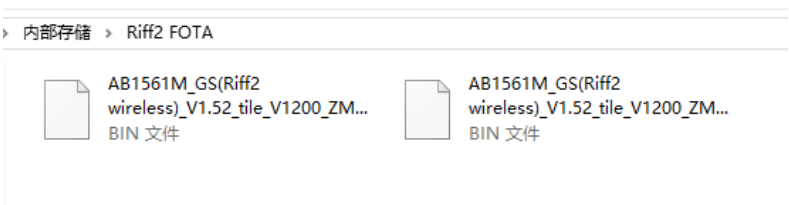
生成两个包的 bin 文件



将两个固件 bin 文件拷贝到手机上自定义的文件夹



手机上创建任意命名的文件夹 Riff2 FOTA



打开手机上用 Airoha_SDK_UT 软件进行升级

点击 SPP，选择对应的蓝牙名称，点击 CONNECT

Step1 点击 REQUEST DFU

Step2 FOTA bin 选 xxx_FotaPackage.bin

Step3 点击 START FOTA 开始升级，升级成功会重启重连

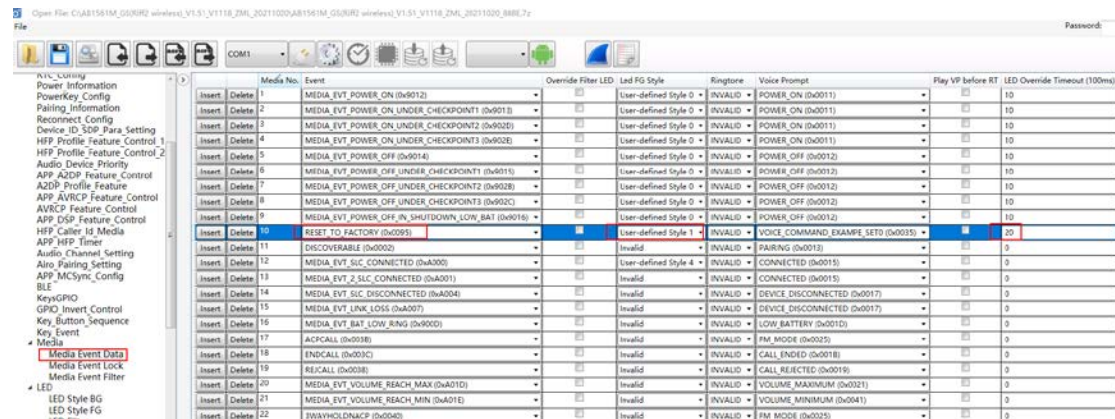
Step2 ROFS package 选 xxx_FileSystemPackage.bin

Step3 点击 START ROFS UPDATE 开始升级，升级成功会重启重连

十二、LED 灯效配置及调用

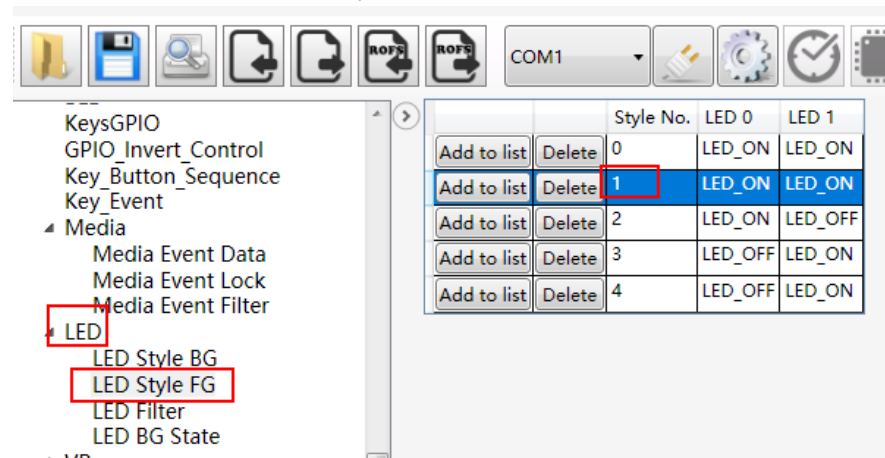
使用 Airoha Tool Kit 配置 LED

配置恢复出厂设置事件的指示灯效

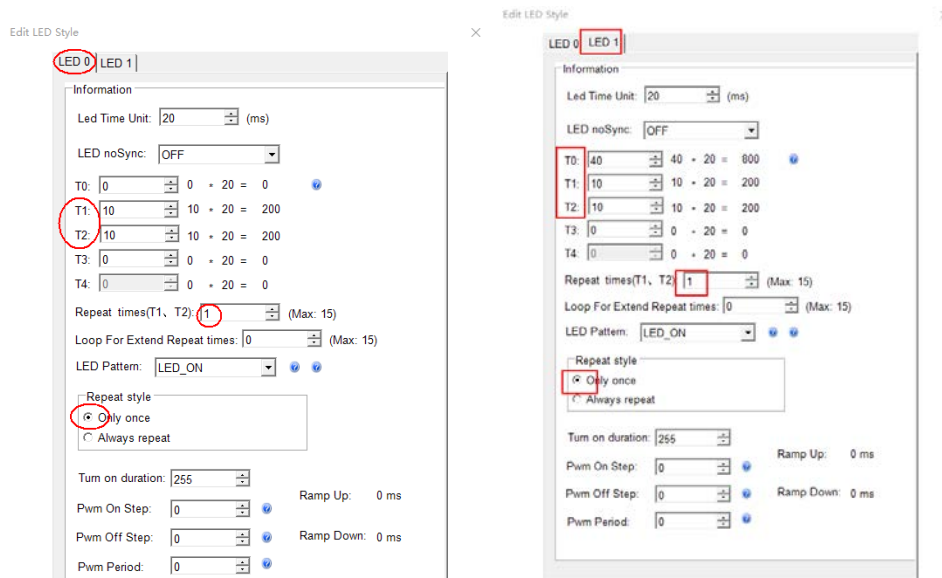


MMI --- Media/Media Event Data --- RESET_TO_FACTORY(Event)---Style 1(Led FG Style)使用前景格式 1

再看 Style 1(Led FG Style)的配置 (LED Style BG 表示后端事件发生时灯效, 比如从连接状态变为未连接状态而产生, LED Style FG 表示前端事件发生时灯效, 比如因按键动作发生而产生)

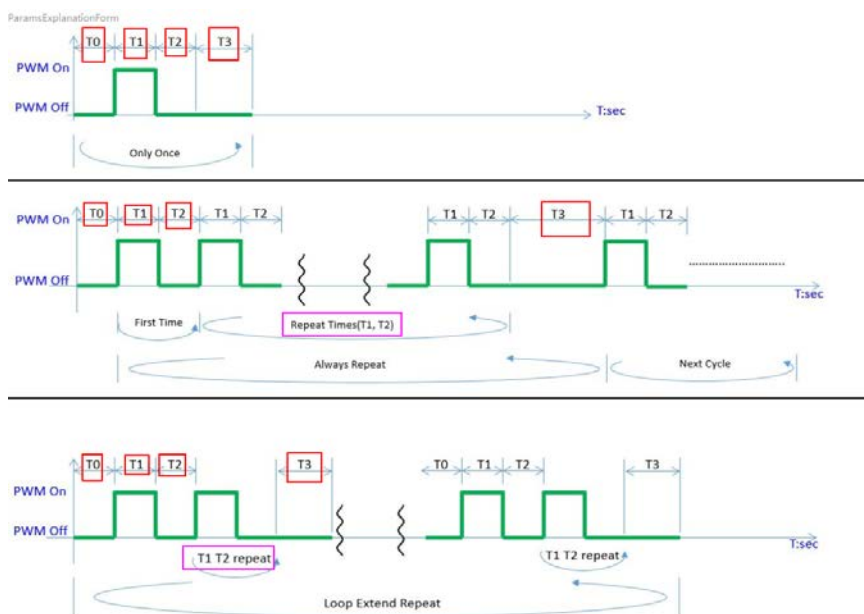


LED0 表示红灯, LED1 表示蓝灯, 双击该格式

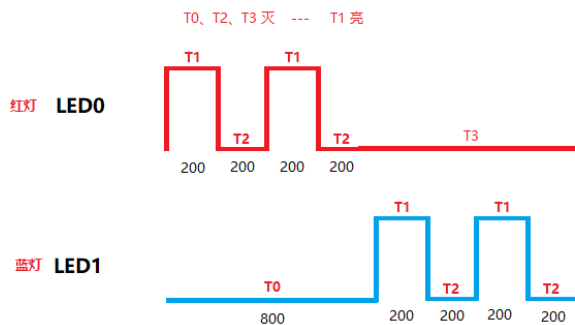


点击蓝色问号可以看到脉冲说明

T0 表示 LED 灯延时启动时间, T1 表示 LED 灯亮的时间, T2 表示 LED 灯灭的时间, T3 表示循环下次启动的间隔时间, Repeat 表示循环的次数, Look For Extend Repeat 扩展循环次数控制, 如果 Repeat 和 Look For Extend Repeat 均为 0, 表示无限循环 T0-T1-T2

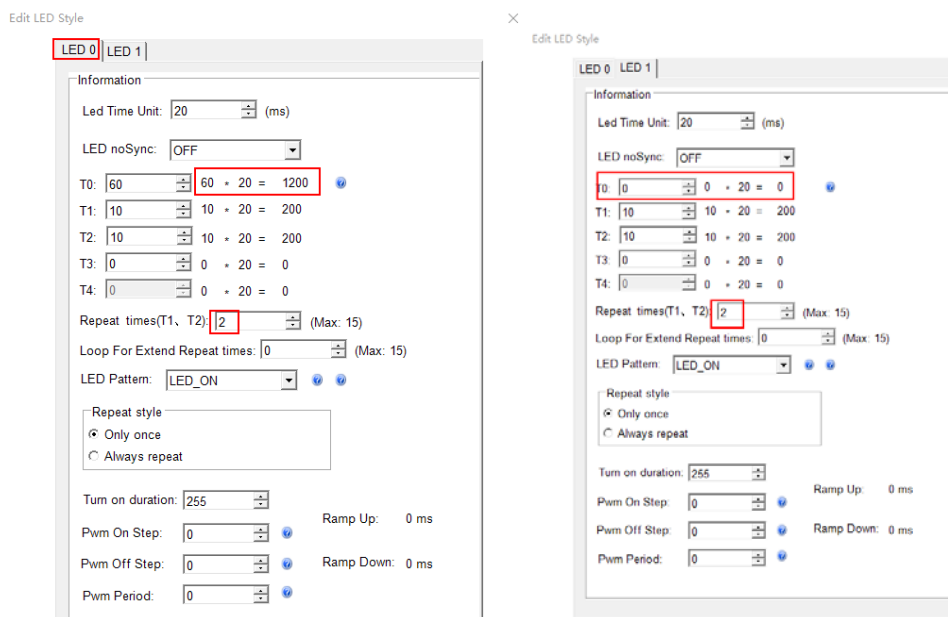


T0、T2、T3 是关, T1 是开, T0 也是亮之前的停顿时间, 上图 LED0 和 LED1 的配置对应脉冲图如下 (T1、T2 只循环一次)

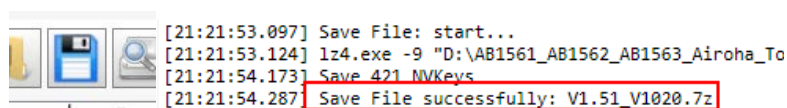


灯效：先是红灯闪2下，再是蓝灯闪2下

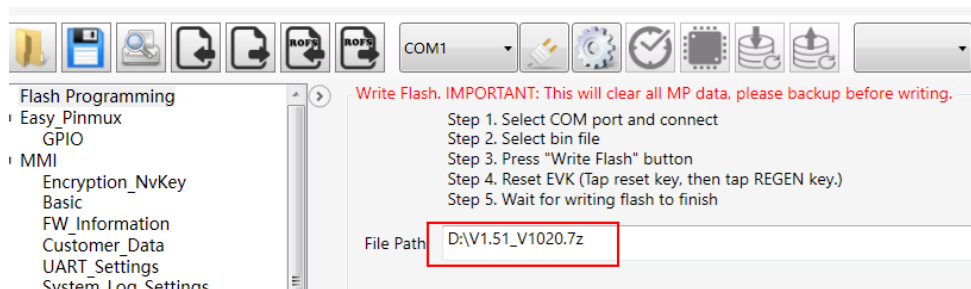
改成灯效：显示蓝灯闪3下，再是红灯闪3下



点击保存（默认是.7z 文件）



按照烧录方法把刚才保存的文件烧录到耳机



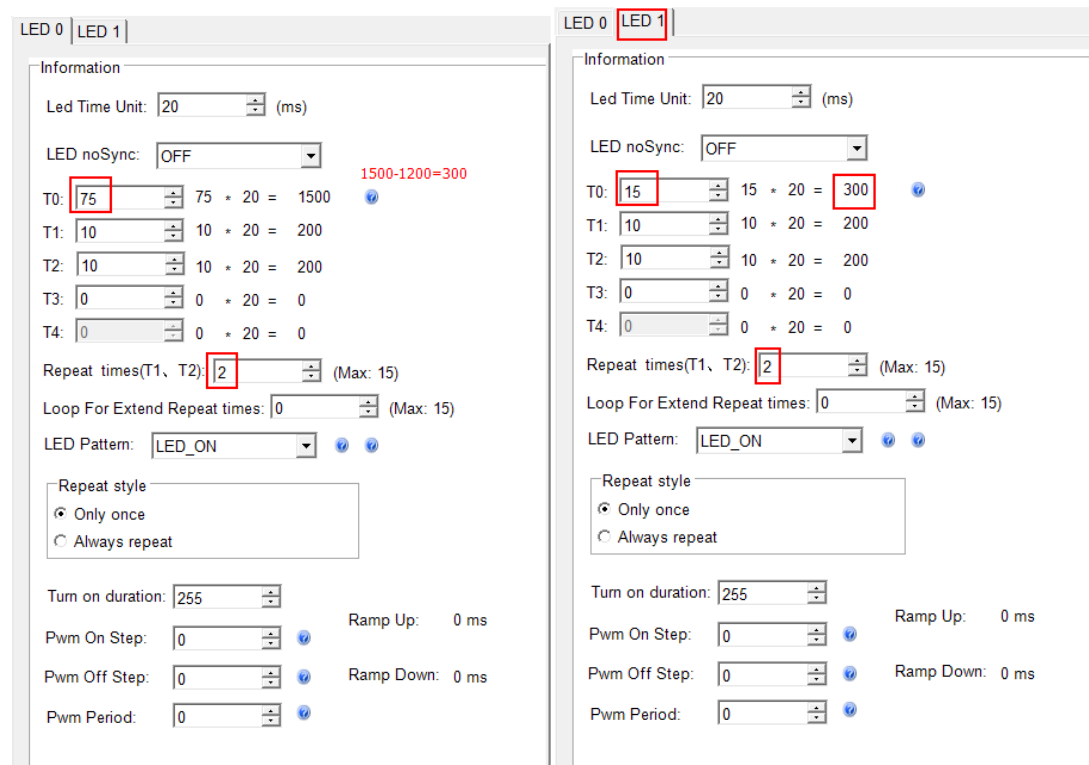
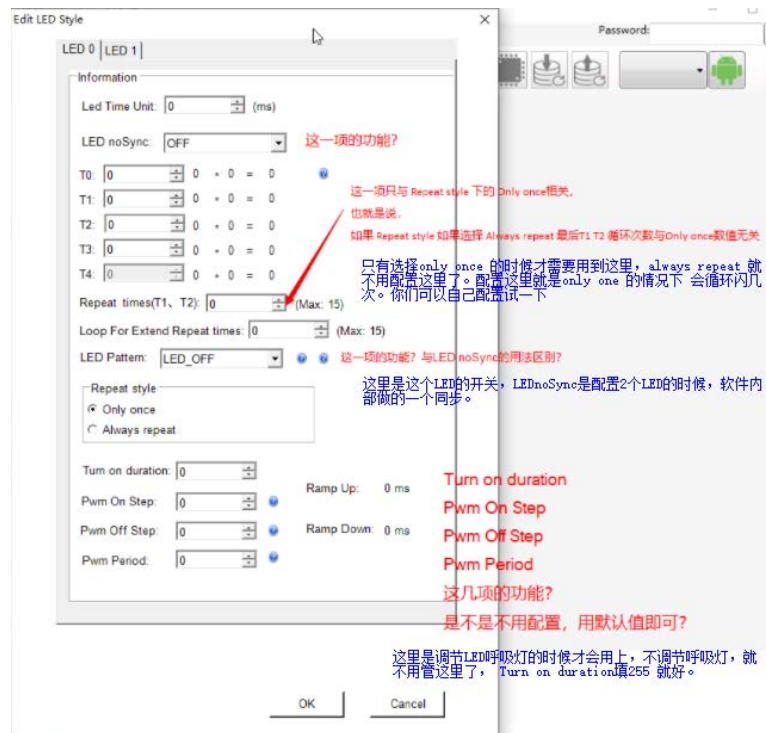
```

[21:21:54.175] Save 421 NVKEYs
[21:21:54.287] Save File successfully: V1.51_V1020.7z
[21:27:26.707] Please enable comport
[21:27:33.365] Write da start
[21:27:40.628] Write da ok
[21:27:40.628] Write flash start
[21:28:09.693] Write flash ok (36620.5635 ms)

```

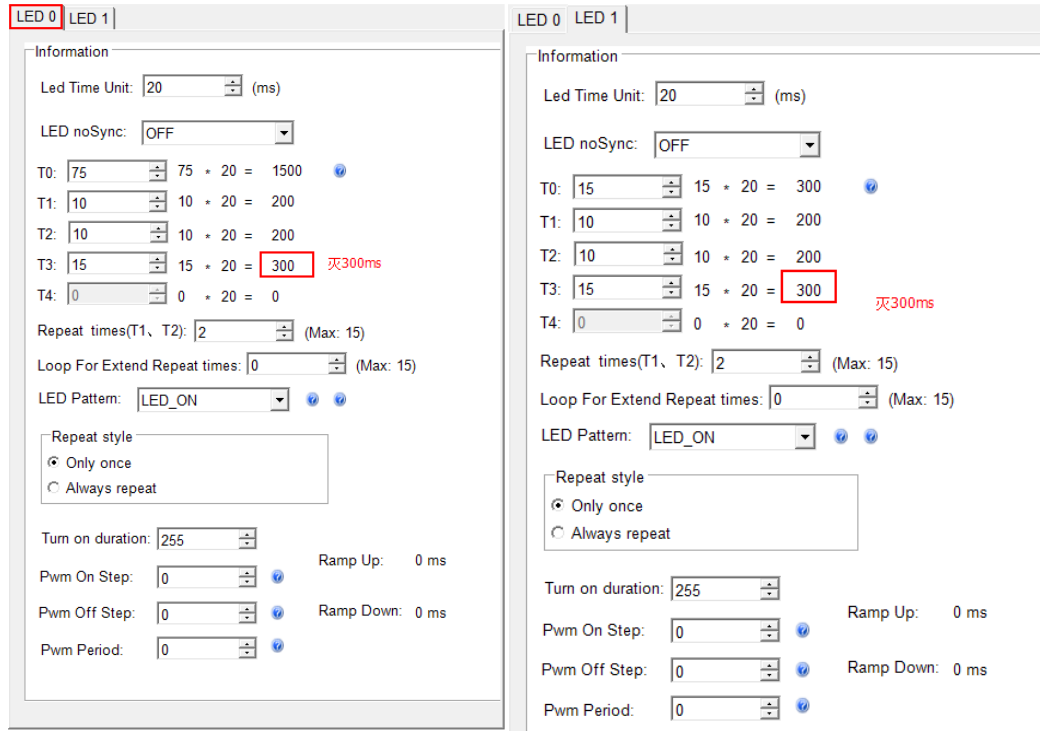
测试，灯效的结果与配置一致：蓝灯闪3下，再到红灯闪3下

改成：灭300ms后，蓝灯闪3下，再到红灯闪3下



保存后下载到耳机，测试，灯效与配置一致

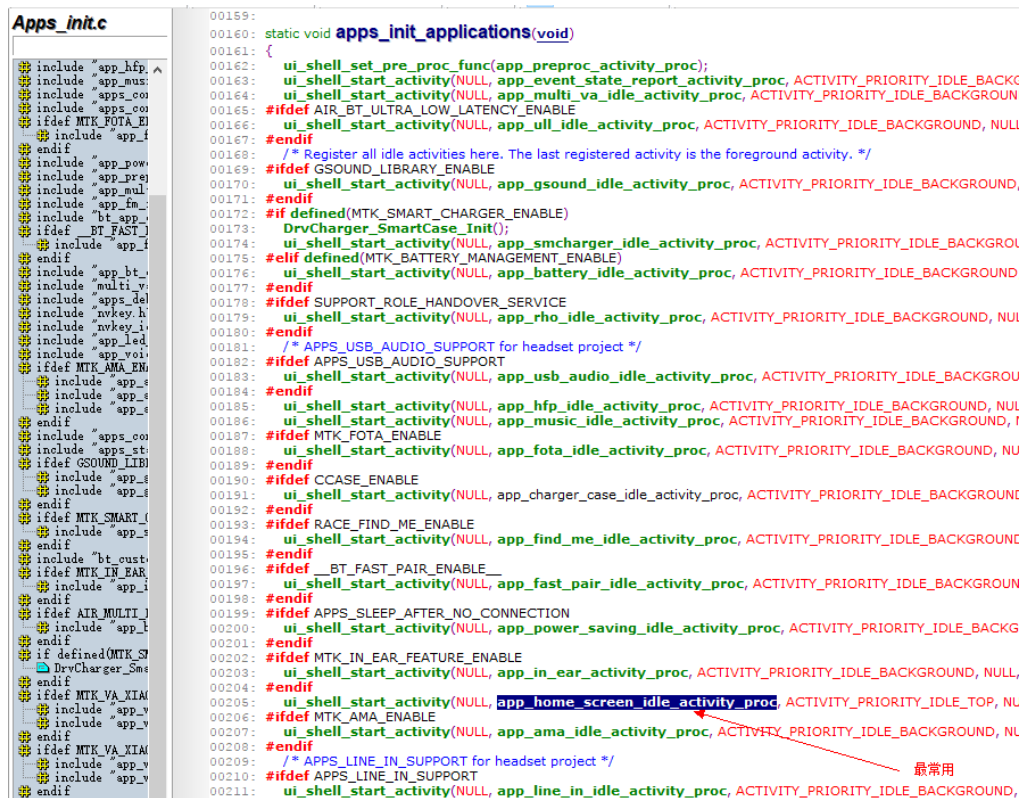
改成：灭 300ms 后，蓝灯闪 3 下，再到红灯闪 3 下，灭 300ms



保存后下载到耳机，测试，灯效与配置不完全一致：最后的灭 300ms 没出现

代码中调用函数控制灯效：

比如主屏幕（home screen）灯效，最常用，在文件 Apps_init.c



在函数 `_proc_key_event_group` 中体现, `app_home_screen_idle_activity.c` 中说明各种事件处理 `_proc_ui_shell_group` 和 `_proc_key_event_group` 和 `_app_interaction_event_proc`

```
App_home_screen_idle_activity.c
#include "app_home_screen_idle_activity.h"
#include "support_role_handover_service.h"
#include "app_rho_idle_activity.h"
#include "app_bt_conn_component_in_home.h"
#include "apps_config_led_manager.h"
#include "apps_config_led_index_list.h"
#include "apps_config_vp_manager.h"
#include "apps_config_vp_index_list.h"
#include "apps_config_key_remapper.h"
#include "apps_config_features_dynamic.h"
#include "app_bt_state_service.h"
#include "app_all_idle_activity.h"
#include "battery_management.h"
#include "apps_aws_sync_event.h"
#include "multi_va_manager.h"
#include "project_config.h"
#include "apps_customer_config.h"
#include "FreeRTOS.h"
#include "timers.h"
#include "ui_shell_manager.h"
#include "apps_atcmd.h"
#include "apps_events_event_group.h"
#include "apps_events_key_event.h"
#include "apps_events_bt_event.h"
#include "apps_config_event_list.h"
#include "app_bt_state_service.h"
#include "bt_customer_config.h"
#include "nvm_config_factory_reset.h"
#include "at_command_bt.h"
#include "bt_sink_srv.h"
#include "bt_app_common.h"
#include "bt_power_on_config.h"
#include "apps_events_interaction_event.h"

00452: } ? _end_proc_ui_shell_group ?
00453:
00454: static bool _proc_key_event_group(ui_shell_activity_t *self,
00455:                                   uint32_t event_id,
00456:                                   void *extra_data,
00457:                                   size_t data_len)
00458: {
00459:     bool ret = false;
00460:     home_screen_local_context_type_t *local_context = (home_screen_lo
00461:
00462:     uint8_t key_id;
00463:     airo_key_event_t key_event;
00464:     app_event_key_event_decode(&key_id, &key_event, event_id);
00465:
00466:     /* Do not power off before key released. */
00467:     if (local_context->power_off_waiting_release_key == key_id && key_ev
00468:         local_context->power_off_waiting_release_key = DEVICE_KEY_NONE;
00469:     app_home_screen_check_and_do_power_off(local_context);
00470:
00471:
00472:     /* Ignore other key event if Homescreen APP will power off. */
00473:     if (APP_HOME_SCREEN_STATE_POWERING_OFF == local_context->state
00474:         return true;
00475:
00476:
00477:     apps_config_key_action_t action;
00478:     if (extra_data) {
00479:         action = *(uint16_t *)extra_data;
00480:     } else {
00481:         action = apps_config_key_event_remapper_map_action(key_id
00482:
00483:
00484:     switch (action) {
00485:     case KEY_BT_OFF: {
00486:         APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", Send
00487:         ui_shell_send_event(false, EVENT_PRIORITY_HIGHEST,

```

不同 action 下的灯效和提示音

```
App_home_screen_idle_activity.c
#include "app_home_screen_idle_activity.h"
#include "support_role_handover_service.h"
#include "app_rho_idle_activity.h"
#include "app_bt_conn_component_in_home.h"
#include "apps_config_led_manager.h"
#include "apps_config_led_index_list.h"
#include "apps_config_vp_manager.h"
#include "apps_config_vp_index_list.h"
#include "apps_config_key_remapper.h"
#include "apps_config_features_dynamic.h"
#include "app_bt_state_service.h"
#include "app_all_idle_activity.h"
#include "battery_management.h"
#include "apps_aws_sync_event.h"
#include "multi_va_manager.h"
#include "project_config.h"
#include "apps_customer_config.h"
#include "FreeRTOS.h"
#include "timers.h"
#include "ui_shell_manager.h"
#include "apps_atcmd.h"
#include "apps_events_event_group.h"
#include "apps_events_key_event.h"
#include "apps_events_bt_event.h"
#include "apps_config_event_list.h"
#include "app_bt_state_service.h"
#include "bt_customer_config.h"
#include "nvm_config_factory_reset.h"
#include "at_command_bt.h"
#include "bt_sink_srv.h"
#include "bt_app_common.h"
#include "bt_power_on_config.h"
#include "apps_events_interaction_event.h"
#include "bt_sink_srv_ami.h"
#include "bt_device_manager.h"
#include "bt_device_manager_test_mode.h"
#include "bt_connection_manager.h"
#include "bt_connection_manager_intern.h"
#include "nvm.h"
#include "nvm_id_list.h"
#include "nvkey.h"
#include "nvkey_id_list.h"
#include "mtk_anc_enable.h"
#include "app_anc_service.h"
#include "mtk_anc_enable.h"
#include "xiaohi.h"
#include "hal_rtc.h"
#include "hal_vdt.h"
#include "mtk_aws_mce_enable.h"
#include "bt_aws_mce_report.h"
#include "bt_aws_mce_srv.h"
#include "fota.h"
#include "nvm_config.h"

00494: switch (action) {
00495: case KEY_BT_OFF: {
00496:     APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", Send REQUEST_ON_OFF_BT", 0);
00497:     ui_shell_send_event(false, EVENT_PRIORITY_HIGHEST,
00498:         EVENT_GROUP_UI_SHELL_APP_INTERACTION,
00499:         APPS_EVENTS_INTERACTION_REQUEST_ON_OFF_BT,
00500:         (void *)false, 0,
00501:         NULL, 0);
00502:
00503:     ret = true;
00504:     break;
00505: }
00506: case KEY_POWER_OFF: {
00507:     /* Apply "power off" VP and foreground LED pattern. */
00508:     apps_config_set_foreground_led_pattern(LED_INDEX_POWER_OFF, 30, false);
00509:     if (battery_management_get_battery_property(BATTERY_PROPERTY_CHARGER_EXIST)
00510:         #if defined(AIR_APP_SYSTEM_ON_BY_LINE_IN_ENABLE)
00511:         || app_line_in_is_plug_in()
00512:     ) {
00513:         apps_config_set_vp(VP_INDEX_POWER_OFF, false, 0, VOICE_PROMPT_PRIO_HIGH, false, NU
00514:         APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", Send REQUEST_ON_OFF_BT whe
00515:         ui_shell_send_event(false, EVENT_PRIORITY_HIGHEST,
00516:             EVENT_GROUP_UI_SHELL_APP_INTERACTION,
00517:             APPS_EVENTS_INTERACTION_REQUEST_ON_OFF_BT,
00518:             (void *)false, 0,
00519:             NULL, 0);
00520:     } else {
00521:         ui_shell_send_event(false, EVENT_PRIORITY_HIGH, EVENT_GROUP_UI_SHELL_APP_INTERAC
00522:         APPS_EVENTS_INTERACTION_POWER_OFF, NULL, 0, NULL, 0);
00523:         apps_config_set_vp(VP_INDEX_POWER_OFF, false, 0, VOICE_PROMPT_PRIO_EXTREME, true
00524:         trigger_power_off_flow(self, true, true);
00525:     }
00526:     #if defined(AB1565)
00527:     #else
00528:     /* When use long press to power off, must wait the key release. */
00529:     if (key_event == AIRO_KEY_LONG_PRESS_1 && key_event <= AIRO_KEY_SLONG) {
00530:         local_context->power_off_waiting_release_key = key_id;
00531:     }
00532:     #endif
00533:
00534:     ret = true;
00535:     break;
00536: }
00537: case KEY_POWER_ON: {
00538:     /* Power on BT if if current bt_power_state is disabled. */
00539:     if (local_context->bt_power_state == APP_HOME_SCREEN_BT_POWER_STATE_DISABLED) {
00540:         app_bt_state_service_set_bt_on_off(true, false, false, false);
00541:         apps_config_set_vp(VP_INDEX_POWER_ON, false, 0, VOICE_PROMPT_PRIO_HIGH, false, NU
00542:         apps_config_set_foreground_led_pattern(LED_INDEX_POWER_ON, 30, false);
00543:     }
00544:     break;
00545: }
00546: case KEY_SYSTEM_REBOOT: {
00547:     trigger_reboot_flow(self, 0);
00548:     ret = true;
00549:     break;
00550: }
00551: case KEY_DISCOVERABLE: {
00552:     #ifdef MTK_AWS_MCE_ENABLE
00553:
00554:     }
00555:     case APPS_EVENTS_INTERACTION_UPDATE_LED_BG_PATTERN:
00556:         ret = true;
00557:         APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", Connection state = %x, is_bt_vii
00558:         if (local_ctx->is_bt_visible) {
00559:             apps_config_set_background_led_pattern(LED_INDEX_CONNECTABLE, true, APPS_CO
00560:         } else {
00561:             apps_config_state_t app_state = homescreen_app_get_mmi_state(self);
00562:             switch (app_state) {
00563:             case APP_BT_OFF:
00564:                 apps_config_set_background_led_pattern(LED_INDEX_IDLE, false, APPS_CONFI
00565:                 break;
00566:             case APP_DISCONNECTED:
00567:                 apps_config_set_background_led_pattern(LED_INDEX_DISCONNECTED, true, AF
00568:                 break;
00569:             case APP_CONNECTED:
00570:                 #ifdef MTK_AWS_MCE_ENABLE
00571:                 if (role == BT_AWS_MCE_ROLE_PARTNER || role == BT_AWS_MCE_ROLE_CLINET) {
00572:                     if (local_ctx->aws_connected) {
00573:                         apps_config_set_background_led_pattern(LED_INDEX_IDLE, false, APPS_C
00574:                     } else {
00575:                         apps_config_set_background_led_pattern(LED_INDEX_DISCONNECTED, trn
00576:                     }
00577:                 } else {
00578:                 }
00579:             }
00580:             #endif
00581:             {
00582:                 apps_config_set_background_led_pattern(LED_INDEX_IDLE, false, APPS_CON
00583:             }
00584:         }
00585:         break;
00586:     }
00587:     #endif
00588:     #endif
00589:     #endif
00590:     #endif
00591:     #endif
00592:     #endif
00593:     #endif
00594:     #endif
00595:     #endif
00596:     #endif
00597:     #endif
00598:     #endif
00599:     #endif
00600:     #endif
00601:     #endif
00602:     #endif
00603:     #endif
00604:     #endif
00605:     #endif
00606:     #endif
00607:     #endif
00608:     #endif
00609:     #endif
00610:     #endif
00611:     #endif
00612:     #endif
00613:     #endif
00614:     #endif
00615:     #endif
00616:     #endif
00617:     #endif
00618:     #endif
00619:     #endif
00620:     #endif
00621:     #endif
00622:     #endif
00623:     #endif
00624:     #endif
00625:     #endif
00626:     #endif
00627:     #endif
00628:     #endif
00629:     #endif
00630:     #endif
00631:     #endif
00632:     #endif
00633:     #endif
00634:     #endif
00635:     #endif
00636:     #endif
00637:     #endif
00638:     #endif
00639:     #endif
00640:     #endif
00641:     #endif
00642:     #endif
00643:     #endif
00644:     #endif
00645:     #endif
00646:     #endif
00647:     #endif
00648:     #endif
00649:     #endif
00650:     #endif
00651:     #endif
00652:     #endif
00653:     #endif
00654:     #endif
00655:     #endif
00656:     #endif
00657:     #endif
00658:     #endif
00659:     #endif
00660:     #endif
00661:     #endif
00662:     #endif
00663:     #endif
00664:     #endif
00665:     #endif
00666:     #endif
00667:     #endif
00668:     #endif
00669:     #endif
00670:     #endif
00671:     #endif
00672:     #endif
00673:     #endif
00674:     #endif
00675:     #endif
00676:     #endif
00677:     #endif
00678:     #endif
00679:     #endif
00680:     #endif
00681:     #endif
00682:     #endif
00683:     #endif
00684:     #endif
00685:     #endif
00686:     #endif
00687:     #endif
00688:     #endif
00689:     #endif
00690:     #endif
00691:     #endif
00692:     #endif
00693:     #endif
00694:     #endif
00695:     #endif
00696:     #endif
00697:     #endif
00698:     #endif
00699:     #endif
00700:     #endif
00701:     #endif
00702:     #endif
00703:     #endif
00704:     #endif
00705:     #endif
00706:     #endif
00707:     #endif
00708:     #endif
00709:     #endif
00710:     #endif
00711:     #endif
00712:     #endif
00713:     #endif
00714:     #endif
00715:     #endif
00716:     #endif
00717:     #endif
00718:     #endif
00719:     #endif
00720:     #endif
00721:     #endif
00722:     #endif
00723:     #endif
00724:     #endif
00725:     #endif
00726:     #endif
00727:     #endif
00728:     #endif
00729:     #endif
00730:     #endif
00731:     #endif
00732:     #endif
00733:     #endif
00734:     #endif
00735:     #endif
00736:     #endif
00737:     #endif
00738:     #endif
00739:     #endif
00740:     #endif
00741:     #endif
00742:     #endif
00743:     #endif
00744:     #endif
00745:     #endif
00746:     #endif
00747:     #endif
00748:     #endif
00749:     #endif
00750:     #endif
00751:     #endif
00752:     #endif
00753:     #endif
00754:     #endif
00755:     #endif
00756:     #endif
00757:     #endif
00758:     #endif
00759:     #endif
00760:     #endif
00761:     #endif
00762:     #endif
00763:     #endif
00764:     #endif
00765:     #endif
00766:     #endif
00767:     #endif
00768:     #endif
00769:     #endif
00770:     #endif
00771:     #endif
00772:     #endif
00773:     #endif
00774:     #endif
00775:     #endif
00776:     #endif
00777:     #endif
00778:     #endif
00779:     #endif
00780:     #endif
00781:     #endif
00782:     #endif
00783:     #endif
00784:     #endif
00785:     #endif
00786:     #endif
00787:     #endif
00788:     #endif
00789:     #endif
00790:     #endif
00791:     #endif
00792:     #endif
00793:     #endif
00794:     #endif
00795:     #endif
00796:     #endif
00797:     #endif
00798:     #endif
00799:     #endif
00800:     #endif
00801:     #endif
00802:     #endif
00803:     #endif
00804:     #endif
00805:     #endif
00806:     #endif
00807:     #endif
00808:     #endif
00809:     #endif
00810:     #endif
00811:     #endif
00812:     #endif
00813:     #endif
00814:     #endif
00815:     #endif
00816:     #endif
00817:     #endif
00818:     #endif
00819:     #endif
00820:     #endif
00821:     #endif
00822:     #endif
00823:     #endif
00824:     #endif
00825:     #endif
00826:     #endif
00827:     #endif
00828:     #endif
00829:     #endif
00830:     #endif
00831:     #endif
00832:     #endif
00833:     #endif
00834:     #endif
00835:     #endif
00836:     #endif
00837:     #endif
00838:     #endif
00839:     #endif
00840:     #endif
00841:     #endif
00842:     #endif
00843:     #endif
00844:     #endif
00845:     #endif
00846:     #endif
00847:     #endif
00848:     #endif
00849:     #endif
00850:     #endif
00851:     #endif
00852:     #endif
00853:     #endif
00854:     #endif
00855:     #endif
00856:     #endif
00857:     #endif
00858:     #endif
00859:     #endif
00860:     #endif
00861:     #endif
00862:     #endif
00863:     #endif
00864:     #endif
00865:     #endif
00866:     #endif
00867:     #endif
00868:     #endif
00869:     #endif
00870:     #endif
00871:     #endif
00872:     #endif
00873:     #endif
00874:     #endif
00875:     #endif
00876:     #endif
00877:     #endif
00878:     #endif
00879:     #endif
00880:     #endif
00881:     #endif
00882:     #endif
00883:     #endif
00884:     #endif
00885:     #endif
00886:     #endif
00887:     #endif
00888:     #endif
00889:     #endif
00890:     #endif
00891:     #endif
00892:     #endif
00893:     #endif
00894:     #endif
00895:     #endif
00896:     #endif
00897:     #endif
00898:     #endif
00899:     #endif
00900:     #endif
00901:     #endif
00902:     #endif
00903:     #endif
00904:     #endif
00905:     #endif
00906:     #endif
00907:     #endif
00908:     #endif
00909:     #endif
00910:     #endif
00911:     #endif
00912:     #endif
00913:     #endif
00914:     #endif
00915:     #endif
00916:     #endif
00917:     #endif
00918:     #endif
00919:     #endif
00920:     #endif
00921:     #endif
00922:     #endif
00923:     #endif
00924:     #endif
00925:     #endif
00926:     #endif
00927:     #endif
00928:     #endif
00929:     #endif
00930:     #endif
00931:     #endif
00932:     #endif
00933:     #endif
00934:     #endif
00935:     #endif
00936:     #endif
00937:     #endif
00938:     #endif
00939:     #endif
00940:     #endif
00941:     #endif
00942:     #endif
00943:     #endif
00944:     #endif
00945:     #endif
00946:     #endif
00947:     #endif
00948:     #endif
00949:     #endif
00950:     #endif
00951:     #endif
00952:     #endif
00953:     #endif
00954:     #endif
00955:     #endif
00956:     #endif
00957:     #endif
00958:     #endif
00959:     #endif
00960:     #endif
00961:     #endif
00962:     #endif
00963:     #endif
00964:     #endif
00965:     #endif
00966:     #endif
00967:     #endif
00968:     #endif
00969:     #endif
00970:     #endif
00971:     #endif
00972:     #endif
00973:     #endif
00974:     #endif
00975:     #endif
00976:     #endif
00977:     #endif
00978:     #endif
00979:     #endif
00980:     #endif
00981:     #endif
00982:     #endif
00983:     #endif
00984:     #endif
00985:     #endif
00986:     #endif
00987:     #endif
00988:     #endif
00989:     #endif
00990:     #endif
00991:     #endif
00992:     #endif
00993:     #endif
00994:     #endif
00995:     #endif
00996:     #endif
00997:     #endif
00998:     #endif
00999:     #endif
01000:     #endif
01001:     #endif
01002:     #endif
01003:     #endif
01004:     #endif
01005:     #endif
01006:     #endif
01007:     #endif
01008:     #endif
01009:     #endif
01010:     #endif
01011:     #endif
01012:     #endif
01013:     #endif
01014:     #endif
01015:     #endif
01016:     #endif
01017:     #endif
01018:     #endif
01019:     #endif
01020:     #endif
01021:     #endif
01022:     #endif
01023:     #endif
01024:     #endif
01025:     #endif
01026:     #endif
01027:     #endif
01028:     #endif
01029:     #endif
01030:     #endif
01031:     #endif
01032:     #endif
01033:     #endif
01034:     #endif
01035:     #endif
01036:     #endif
01037:     #endif
01038:     #endif
01039:     #endif
01040:     #endif
01041:     #endif
01042:     #endif
01043:     #endif
01044:     #endif
01045:     #endif
01046:     #endif
01047:     #endif
01048:     #endif
01049:     #endif
01050:     #endif
01051:     #endif
01052:     #endif
01053:     #endif
01054:     #endif
01055:     #endif
01056:     #endif
01057:     #endif
01058:     #endif
01059:     #endif
01060:     #endif
01061:     #endif
01062:     #endif
01063:     #endif
01064:     #endif
01065:     #endif
01066:     #endif
01067:     #endif
01068:     #endif
01069:     #endif
01070:     #endif
01071:     #endif
01072:     #endif
01073:     #endif
01074:     #endif
01075:     #endif
01076:     #endif
01077:     #endif
01078:     #endif
01079:     #endif
01080:     #endif
01081:     #endif
01082:     #endif
01083:     #endif
01084:     #endif
01085:     #endif
01086:     #endif
01087:     #endif
01088:     #endif
01089:     #endif
01090:     #endif
01091:     #endif
01092:     #endif
01093:     #endif
01094:     #endif
01095:     #endif
01096:     #endif
01097:     #endif
01098:     #endif
01099:     #endif
01100:     #endif
01101:     #endif
01102:     #endif
01103:     #endif
01104:     #endif
01105:     #endif
01106:     #endif
01107:     #endif
01108:     #endif
01109:     #endif
01110:     #endif
01111:     #endif
01112:     #endif
01113:     #endif
01114:     #endif
01115:     #endif
01116:     #endif
01117:     #endif
01118:     #endif
01119:     #endif
01120:     #endif
01121:     #endif
01122:     #endif
01123:     #endif
01124:     #endif
01125:     #endif
01126:     #endif
01127:     #endif
01128:     #endif
01129:     #endif
01130:     #endif
01131:     #endif
01132:     #endif
01133:     #endif
01134:     #endif
01135:     #endif
01136:     #endif
01137:     #endif
01138:     #endif
01139:     #endif
01140:     #endif
01141:     #endif
01142:     #endif
01143:     #endif
01144:     #endif
01145:     #endif
01146:     #endif
01147:     #endif
01148:     #endif
01149:     #endif
01150:     #endif
01151:     #endif
01152:     #endif
01153:     #endif
01154:     #endif
01155:     #endif
01156:     #endif
01157:     #endif
01158:     #endif
01159:     #endif
01160:     #endif
01161:     #endif
01162:     #endif
01163:     #endif
01164:     #endif
01165:     #endif
01166:     #endif
01167:     #endif
01168:     #endif
01169:     #endif
01170:     #endif
01171:     #endif
01172:     #endif
01173:     #endif
01174:     #endif
01175:     #endif
01176:     #endif
01177:     #endif
01178:     #endif
01179:     #endif
01180:     #endif
01181:     #endif
01182:     #endif
01183:     #endif
01184:     #endif
01185:     #endif
01186:     #endif
01187:     #endif
01188:     #endif
01189:     #endif
01190:     #endif
01191:     #endif
01192:     #endif
01193:     #endif
01194:     #endif
01195:     #endif
01196:     #endif
01197:     #endif
01198:     #endif
01199:     #endif
01200:     #endif
01201:     #endif
01202:     #endif
01203:     #endif
01204:     #endif
01205:     #endif
01206:     #endif
01207:     #endif
01208:     #endif
01209:     #endif
01210:     #endif
01211:     #endif
01212:     #endif
01213:     #endif
01214:     #endif
01215:     #endif
01216:     #endif
01217:     #endif
01218:     #endif
01219:     #endif
01220:     #endif
01221:     #endif
01222:     #endif
01223:     #endif
01224:     #endif
01225:     #endif
01226:     #endif
01227:     #endif
01228:     #endif
01229:     #endif
01230:     #endif
01231:     #endif
01232:     #endif
01233:     #endif
01234:     #endif
01235:     #endif
01236:     #endif
01237:     #endif
01238:     #endif
01239:     #endif
01240:     #endif
01241:     #endif
01242:     #endif
01243:     #endif
01244:     #endif
01245:     #endif
01246:     #endif
01247:     #endif
01248:     #endif
01249:     #endif
01250:     #endif
01251:     #endif
01252:     #endif
01253:     #endif
01254:     #endif
01255:     #endif
01256:     #endif
01257:     #endif
01258:     #endif
01259:     #endif
01260:     #endif
01261:     #endif
01262:     #endif
01263:     #endif
01264:     #endif
01265:     #endif
01266:     #endif
01267:     #endif
01268:     #endif
01269:     #endif
01270:     #endif
01271:     #endif
01272:     #endif
01273:     #endif
01274:     #endif
01275:     #endif
01276:     #endif
01277:     #endif
01278:     #endif
01279:     #endif
01280:     #endif
01281:     #endif
01282:     #endif
01283:     #endif
01284:     #endif
01285:     #endif
01286:     #endif
01287:     #endif
01288:     #endif
01289:     #endif
01290:     #endif
01291:     #endif
01292:     #endif
01293:     #endif
01294:     #endif
01295:     #endif
01296:     #endif
01297:     #endif
01298:     #endif
01299:     #endif
01300:     #endif
01301:     #endif
01302:     #endif
01303:     #endif
01304:     #endif
01305:     #endif
01306:     #endif
01307:     #endif
01308:     #endif
01309:     #endif
01310:     #endif
01311:     #endif
01312:     #endif
01313:     #endif
01314:     #endif
01315:     #endif
01316:     #endif
01317:     #endif
01318:     #endif
01319:     #endif
01320:     #endif
01321:     #endif
01322:     #endif
01323:     #endif
01324:     #endif
01325:     #endif
01326:     #endif
01327:     #endif
01328:     #endif
01329:     #endif
01330:     #endif
01331:     #endif
01332:     #endif
01333:     #endif
01334:     #endif
01335:     #endif
01336:     #endif
01337:     #endif
01338:     #endif
01339:     #endif
01340:     #endif
01341:     #endif
01342:     #endif
01343:     #endif
01344:     #endif
01345:     #endif
01346:     #endif
01347:     #endif
01348:     #endif
01349:     #endif
01350:     #endif
01351:     #endif
01352:     #endif
01353:     #endif
01354:     #endif
01355:     #endif
01356:     #endif
01357:     #endif
01358:     #endif
01359:     #endif
01360:     #endif
01361:     #endif
01362:     #endif
01363:     #endif
01364:     #endif
01365:     #endif
01366:     #endif
01367:     #endif
01368:     #endif
01369:     #endif
01370:     #endif
01371:     #endif
01372:     #endif
01373:     #endif
01374:     #endif
01375:     #endif
01376:     #endif
01377:     #endif
01378:     #endif
01379:     #endif
01380:     #endif
01381:     #endif
01382:     #endif
01383:     #endif
01384:     #endif
01385:     #endif
01386:     #endif
01387:     #endif
01388:     #endif
01389:     #endif
01390:     #endif
01391:     #endif
01392:     #endif
01393:     #endif
01394:     #endif
01395:     #endif
01396:     #endif
01397:     #endif
01398:     #endif
01399:     #endif
01400:     #endif
01401:     #endif
01402:     #endif
01403:     #endif
01404:     #endif
01405:     #endif
01406:     #endif
01407:     #endif
01408:     #endif
01409:     #endif
01410:     #endif
01411:     #endif
01412:     #endif
01413:     #endif
01414:     #endif
01415:     #endif
01416:     #endif
01417:     #endif
01418:     #endif
01419:     #endif
01420:     #endif
01421:     #endif
01422:     #endif
01423:     #endif
01424:     #endif
01425:     #endif
01426:     #endif
01427:     #endif
01428:     #endif
01429:     #endif
01430:     #endif
01431:     #endif
01432:     #endif
01433:     #endif
01434:     #endif
01435:     #endif
01436:     #endif
01437:     #endif
01438:     #endif
01439:     #endif
01440:     #endif
01441:     #endif
01442:     #endif
01443:     #endif
01444:     #endif
01445:     #endif
01446:     #endif
01447:     #endif
01448:     #endif
01449:     #endif
01450:     #endif
01451:     #endif
01452:     #endif
01453:     #endif
01454:     #endif
01455:     #endif
01456:     #endif
01457:     #endif
01458:     #endif
01459:     #endif
01460:     #endif
01461:     #endif
01462:     #endif
01463:     #endif
01464:     #endif
01465:     #endif
01466:     #endif
01467:     #endif
01468:     #endif
01469:     #endif
01470:     #endif
01471:     #endif
01472:     #endif
01473:     #endif
01474:     #endif
01475:     #endif
01476:     #endif
01477:     #endif
01478:     #endif
01479:     #endif
01480:     #endif
01481:     #endif
01482:     #endif
01483:     #endif
01484:     #endif
01485:     #endif
01486:     #endif
01487:     #endif
01488:     #endif
01489:     #endif
01490:     #endif
01491:     #endif
01492:     #endif
01493:     #endif
01494:     #endif
01495:     #endif
01496:     #endif
01497:     #endif
01498:     #endif
01499:     #endif
01500:     #endif
01501:     #endif
01502:     #endif
01503:     #endif
01504:     #endif
01505:     #endif
01506:     #endif
01507:     #endif
01508:     #endif
01509:     #endif
01510:     #endif
01511:     #endif
01512:     #endif
01513:     #endif
01514:     #endif
01515:     #endif
01516:     #endif
01517:     #endif
01518:     #endif
01519:     #endif
01520:     #endif
01
```

设置提示音函数

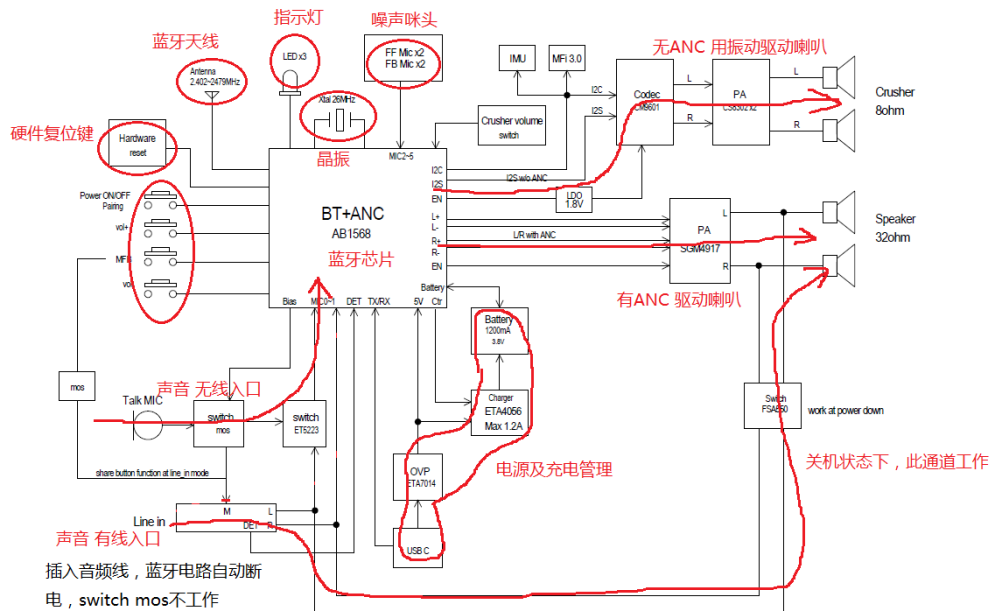
```
apps_config_set_vp(VP_INDEX_POWER_OFF, false, 0, VOICE_PROMPT_PRIO_HIGH, false, NULL);
```

十三、项目 CHP200 的原理图理解

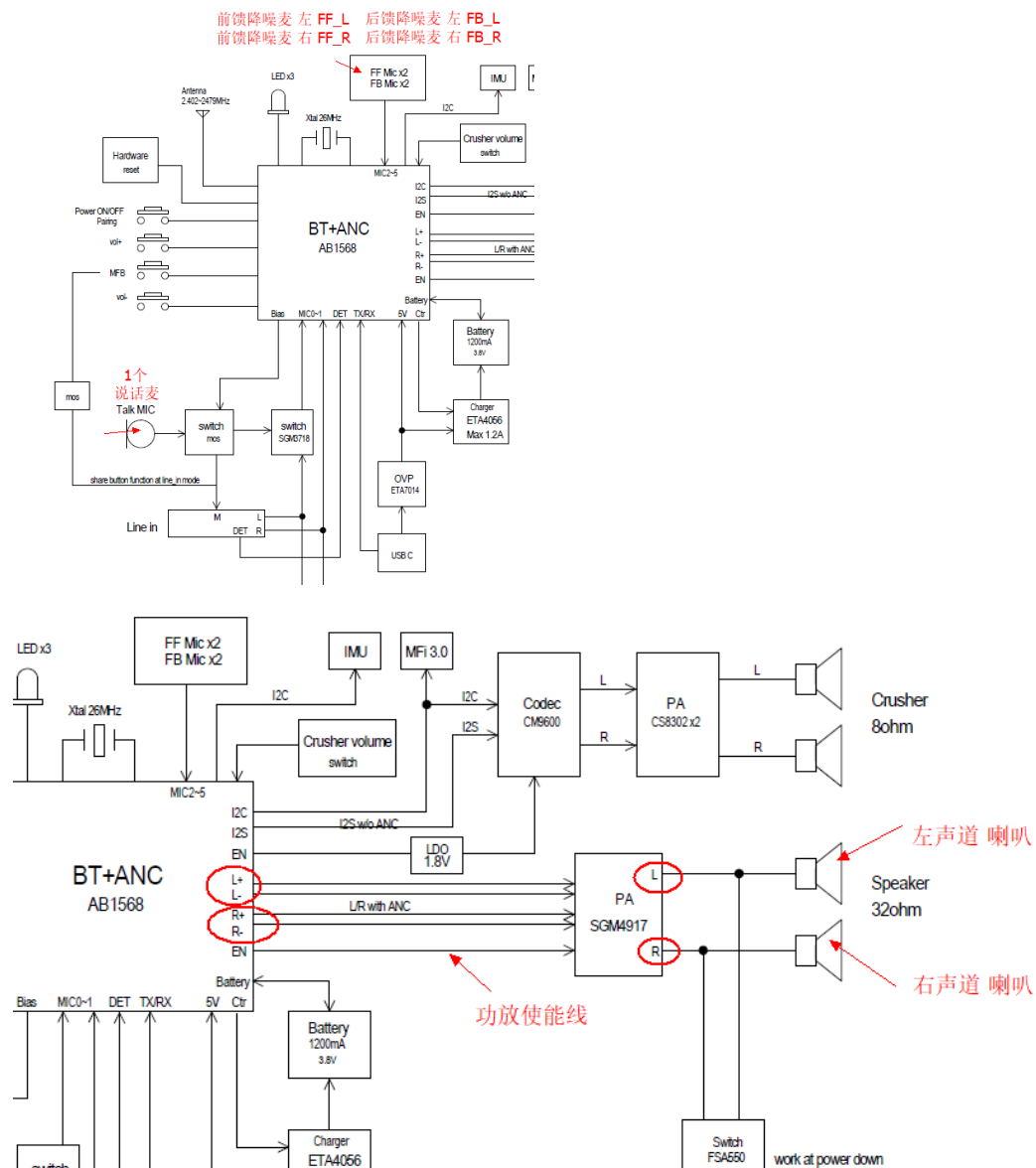
CHP200 耳机有三个工作模式：

- 1、 关机状态（保留 BLE 功能，直到电池没电为止）下，Line-in 接口直通到喇叭，可用有线方式听手机音乐等，蓝牙电路不起作用；
- 2、 开机状态下，Line-in 有音频线插入，MIC 咪头自动关闭，声音只从音频线进来
- 3、 开机状态下，Line-in 没有音频线插入，MIC 咪头自动打开，声音只从 MIC 咪头进来

主框图



CM9601 芯片是驱动振动喇叭的



Talk Mic 与 line in 的 LIN_L 复用了 AU_VIN0_P
line in 的 LIN_R 用了 AU_VIN1_P

Key Long Press Action

长按 Key Event Key Index S0 S1 S2 S3 S4 S5 S6 S7 S8 S9 S10

Insert	Delete	MIC_TO_SPK_DISABLE (0x0084)	KEY1	☒	☒	☒											
Insert	Delete	DISCOVERABLE (0x0002)	KEYNONE	☒	☒	☒											
Insert	Delete	AVRCP_FORWARD (0x005A)	KEY3			☒											
Insert	Delete	AVRCP_BACKWARD (0x005B)	KEY2			☒											
Insert	Delete	RESET_TO_FACTORY (0x0095)	KEY_COMB_23		☒	☒	☒										
Insert	Delete	MIC_TO_SPK_ENABLE (0x0083)	KEY_COMB_13	☒	☒	☒											

Key Long Press Release Action

Key Event Key Index S0 S1 S2 S3 S4 S5 S6 S7 S8 S9 S10

Insert	Delete		KEYNONE														
--------	--------	--	---------	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Key Double Action

Key Event Key Index S0 S1 S2 S3 S4 S5 S6 S7 S8 S9 S10

Insert	Delete	SET_VOICE_RECOGNITION (0x0033)	KEY1			☒											
--------	--------	--------------------------------	------	--	--	-------------------------------------	--	--	--	--	--	--	--	--	--	--	--

Key Triple Action

Key Event Key Index S0 S1 S2 S3 S4 S5 S6 S7 S8 S9 S10

Insert	Delete	ENABLE_LED (0x0011)	KEY1			☒											
--------	--------	---------------------	------	--	--	-------------------------------------	--	--	--	--	--	--	--	--	--	--	--

Key Long Long Press Action

Key Event Key Index S0 S1 S2 S3 S4 S5 S6 S7 S8 S9 S10

Insert	Delete	POWER_OFF (0x0018)	KEY1		☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒
--------	--------	--------------------	------	--	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------	-------------------------------------

KEY1 是主键，KEY2 是音量减键，KEY3 是音量加键

这个配置生效后在 S1、S2、S3 状态下，同时长按主键和音量加键，进入模式，长按主键，会退出模式，接着才关机（因为关机需要更长按才触发）

S0=KEY_MAP_APP_OFF
S1=KEY_MAP_APP_CONDISCABLE
S2=KEY_MAP_APP_CONNECTED
S3=KEY_MAP_APP_CONNECTED
S4=KEY_MAP_APP_HFP_INCOMING
S5=KEY_MAP_APP_HFP_OUTGOING
S6=KEY_MAP_APP_HFP_CALLACTIVE
S7=KEY_MAP_APP_HFP_CALLACTIVE_WITHOUT_SCO
S8=KEY_MAP_APP_HFP_CAIMG
S9=KEY_MAP_APP_HFP_CAOGG
S10=KEY_MAP_APP_HFP_CAMULTY
S11=KEY_MAP_APP_FAKEON
S12=KEY_MAP_APP_FAKEOFF
S13=KEY_MAP_APP_DETACHING_LINK
S14=KEY_MAP_APP_TEST_MODE
S15=KEY_MAP_APP_FM
S16=KEY_MAP_APP_LINE_IN
S17=KEY_MAP_APP_EEPROM_UPDATING
S18=KEY_MAP_APP_EEPROM_UPDATED_OK
S19=KEY_MAP_APP_EEPROM_UPDATED_FAIL
S20=KEY_MAP_APP_VOICE_PROMPT_LANG_SELECT
S21=KEY_MAP_APP_MCSYNC_PAIRING
S22=KEY_MAP_APP_INQUIRY
S23=KEY_MAP_APP_MP_STATE
S24=KEY_MAP_APP_MP3
S25=KEY_MAP_APP_STATE_RESERVED2
S26=KEY_MAP_APP_STATE_RESERVED3
S27=KEY_MAP_APP_STATE_RESERVED4
S28=KEY_MAP_APP_STATE_RESERVED5
S29=KEY_MAP_APP_STATE_RESERVED6
S30=KEY_MAP_APP_STATE_RESERVED7

进入此模式，让蓝灯常亮。在 MMI > LED > LED Style FG 下修改 Style 5 为红灯灭，蓝灯常亮

LED

LED Style BG

LED Style FG

LED Filter

LED BG State

VP

Google_Fast_Pair

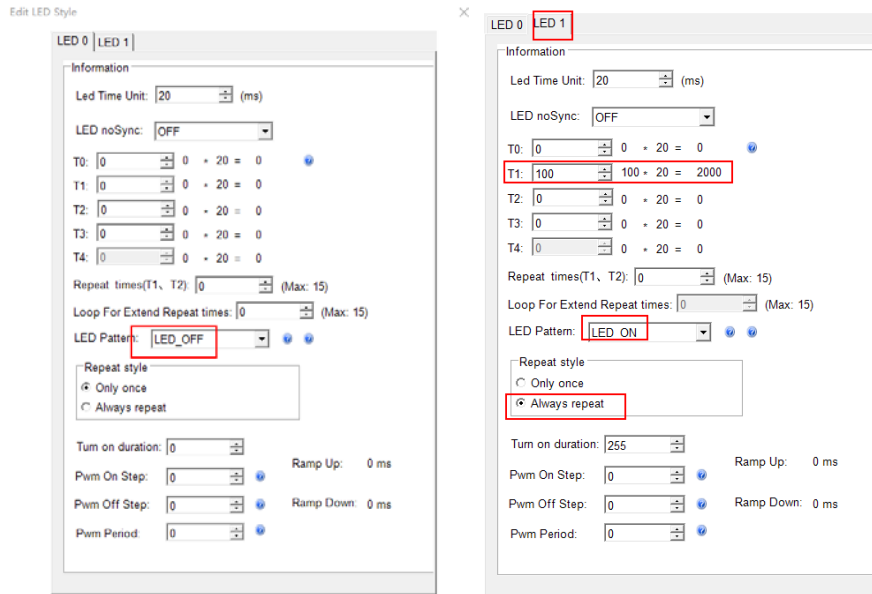
ESD

In_Ear_Detection

XIAOA1_setting

iAP2_Settings

Style No.	LED 0	LED 1
Add to list Delete 0	LED_ON	LED_ON
Add to list Delete 1	LED_OFF	LED_ON
Add to list Delete 2	LED_ON	LED_OFF
Add to list Delete 3	LED_OFF	LED_ON
Add to list Delete 4	LED_OFF	LED_ON
Add to list Delete 5	LED_OFF	LED_ON



按键事件与 LED 关联，在 MMI > Media > Media Event Data 下添加事件关联 LED FG 样式

Media	Insert	Delete	30	MEDIA_EVT_XIAOAI_EVT1 (0xA047)		Invalid	INVALID	CALLER_ID_SUFFIX
Media Event Data	Insert	Delete	31	MEDIA_EVT_XIAOAI_EVT2 (0xA048)		Invalid	INVALID	CALLER_ID_PREFIX
Media Event Lock	Insert	Delete	32	MEDIA_EVT_A2DP_ENABLE (0xA024)		User-defined Style 4	INVALID	INVALID (0x00FF)
Media Event Filter	Insert	Delete	33	MEDIA_EVT_A2DP_DISABLE (0xA025)		User-defined Style 4	INVALID	INVALID (0x00FF)
LED	Insert	Delete	34	MIC_TO_SPK_ENABLE (0x0083)		User-defined Style 5	INVALID	INVALID (0x00FF)
LED Style BG								
LED Style FG								

十五、将 patch 补丁文件打入 SDK

patch 文件（补丁）是在 Git 软件上用 diff 命令生成的，也可用 Git 的相关命令打入工程 diff 的功能就是用来比较两个文件的不同，然后记录下来（就是所谓的补丁）

```

GS-35514@GS-PC-0446 MSYS ~
$ diff --help
用法: diff [选项]... 文件列表
逐行比较<各文件>。

长选项的必需参数也是相应短选项的必需参数。
--normal          以正常的 diff 方式输出（默认）
-q, --brief       只有在文件不同时报告
-s, --report-identical-files 当两个一样时仍然显示结果
-c, -C NUM, --context[=NUM] 同时输出 NUM 行（默认为 3 行）的复制上下文内容
-u, -U 数量, --unified[=数量] 输出 <数量>（默认为 3）行一致化上下文
-e, --ed          以 ed script 方式输出
-n, --rcs        以 RCS diff 格式输出
-y, --side-by-side 以两栏的格式输出
-W, --width=数量 每行显示最多 <数量>（默认 130）个字符
--left-column    当有两行相同时只显示左边栏的一行
--suppress-common-lines 当有两行相同时不显示

-p, --show-c-function 显示每个变更位于哪个 C 函数中
-F, --show-function-line=正则 显示匹配给定<正则>表达式的最近一行
--label 标签      使用给定<标签>替代文件名和时间戳（可以重复）

-t, --expand-tabs 将输出中的 tab 转换成空格
-T, --initial-tab 每行先加上 tab 字符，使 tab 字符可以对齐
--tabsize=数字    TAB 格的宽度，默认为 8 个打印列宽
--suppress-blank-empty 在空的输出行之前去除空白或制表符
-l, --paginate    将输出送至“pr”指令来分页
  
```

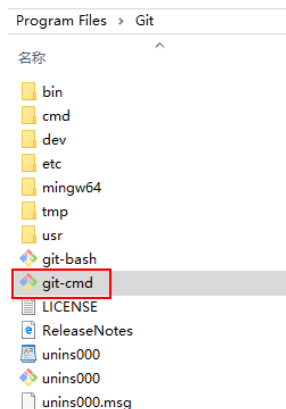
patch 文件如下


```
5
6 ---
7 ....apps/config/apps_config_led_index_list.h...|..1.+
8 ....inc/apps/led/led_control_internal.h...|..2.+
9 ....src/apps/app_music/app_music_activity.c...|..7.+++++
10 ....src/apps/led/app_led_control.c...|..8.+++++
11 ....src/apps/led/led_control_internal.c...|..20.+++++
12 5 files changed, 29 insertions(+), 9 deletions(-)
13
14 diff --git a/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/config/apps_config_led_index_list.h b/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/config/apps_config_led_index_list.h
15 index a25933b..67405d2 100755
16 --- a/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/config/apps_config_led_index_list.h
17 +++ b/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/config/apps_config_led_index_list.h
18 @@ -57,6 +57,7 @@ enum {
19     LED_INDEX_AIR_PAIRING_SUCCESS = 17,
20     LED_INDEX_AIR_PAIRING_FAIL = 18,
21     LED_INDEX_TRIGGER_RHO = 19,
22 +    LED_INDEX_PLAYING = 20,
23 };
24
25 #endif /* __APPS_CONFIG_APPS_CONFIG_LED_INDEX_LIST_H__ */
26 diff --git a/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/led/led_control_internal.h b/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/led/led_control_internal.h
27 index 85f31b7..d9c978b 100755
28 --- a/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/led/led_control_internal.h
29 +++ b/mcu/project/ab1565_ab1568_evk/apps/earbuds_ref_design/inc/apps/led/led_control_internal.h
30 @@ -68,7 +68,7 @@ void led_style_init();
31 /* @param[in] syn_tm_ms is the sync time for dual device using.
32  * @return #APP_LED_STATUS_OK, if the operation completed successfully.
33  */
34 -app_led_status_t led_style_enable(uint8_t ledx, one_led_style_t *config, uint32_t syn_tm_ms);
35 +app_led_status_t led_style_enable(uint8_t ledx, one_led_style_t *config, uint32_t syn_tm_ms, uint8_t led_pattern_index);
36
37 /**
38  * @brief This function disable led.
```

用 Git 把补丁打入 SDK

Git-2.32.0-64-bit

安装 Git 软件（默认设置即可）



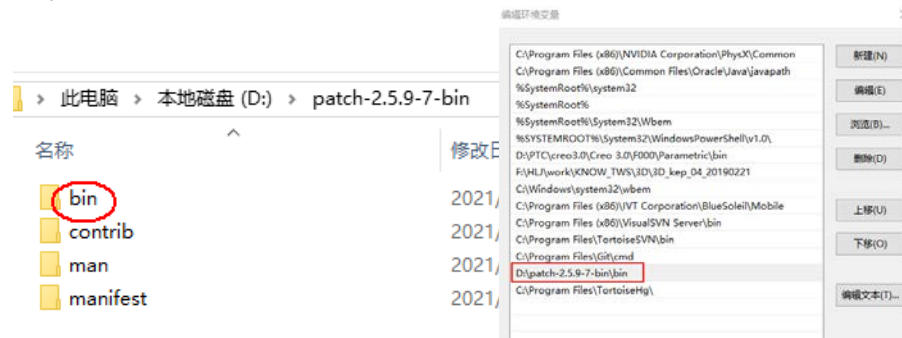
SDK 原始文件

名称	修改日期	类型
dsp	2021/10/27 17:00	文件夹
mcu	2021/8/31 2:34	文件夹
build	2021/8/31 2:14	SH 文件
mapping_disk	2021/10/27 17:00	文件

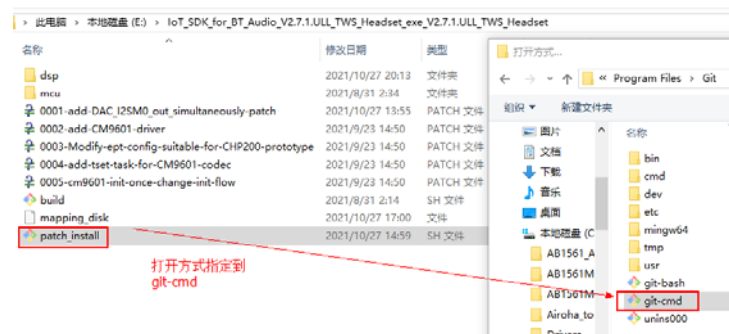
把 patch 补丁文件和脚本文件拷贝到 SDK 目录下

名称	修改日期	类型
dsp	2021/10/27 20:13	文件夹
mcu	2021/8/31 2:34	文件夹
0001-add-DAC_I2SM0_out_simultaneously-patch	2021/10/27 13:55	PATCH 文件
0002-add-CM9601-driver	2021/9/23 14:50	PATCH 文件
0003-Modify-ept-config-suitable-for-CHP200-prototype	2021/9/23 14:50	PATCH 文件
0004-add-tset-task-for-CM9601-codec	2021/9/23 14:50	PATCH 文件
0005-cm9601-init-once-change-init-flow	2021/9/23 14:50	PATCH 文件
build	2021/8/31 2:14	SH 文件
mapping_disk	2021/10/27 17:00	文件
patch_install	2021/10/27 14:59	SH 文件

把 patch 安装器 bin 路径添加到系统环境变量



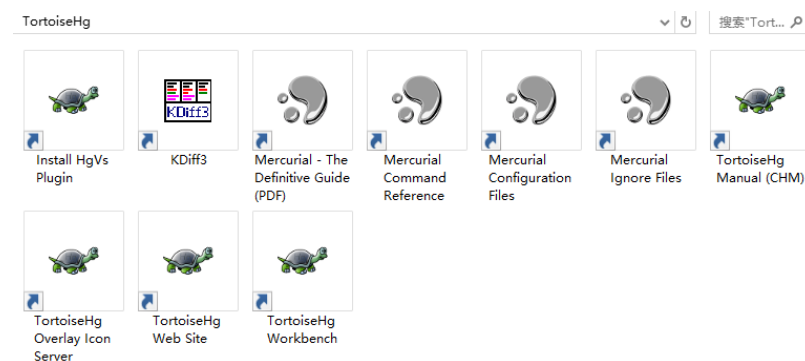
将脚本文件用 Git 运行即可把补丁添加到 SDK



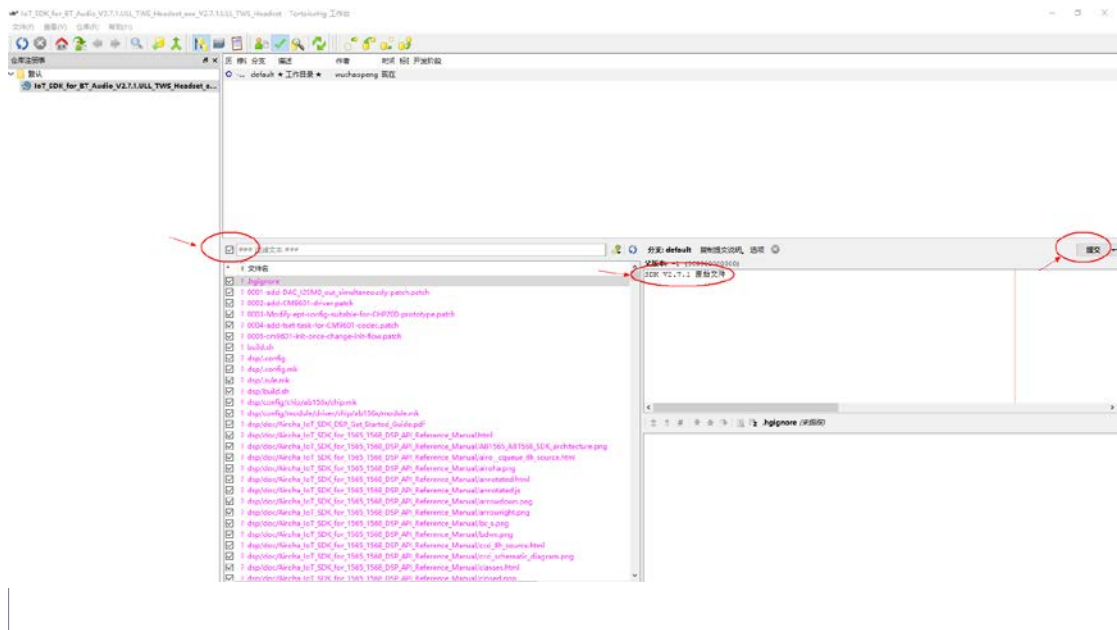
十六、软件版本管理 TortoiseHg

为了方便查看补丁被打到哪些文件上，可以在运行脚本文件之前把 SDK 原始文件放到版本管理工具（离线版 TortoiseHg，在线版 SVN）上

 tortoisehg-4.9.0-x64

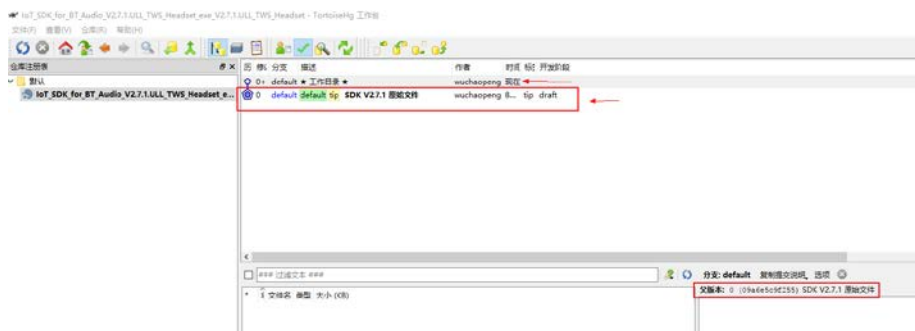


右击 SDK 文件夹 > Hg Workbench，文件 > 新建仓库 > 创建选中全部文件，输入父版本说明，点击提交

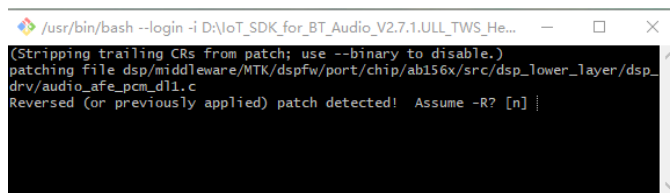


左下角显示提交进度

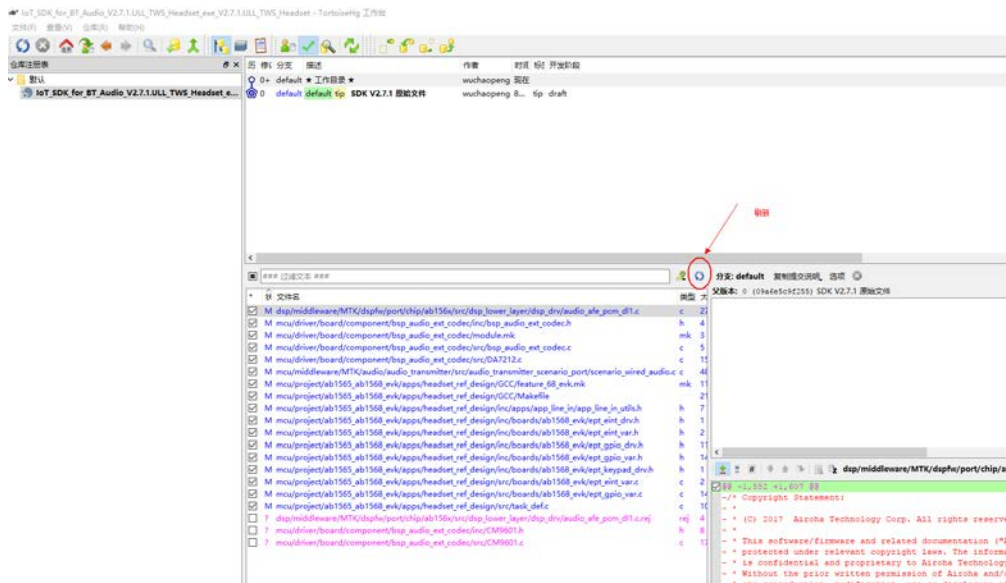
提交完成后出现一个父版本和当前版本（每提交一次就会新建一个版本）



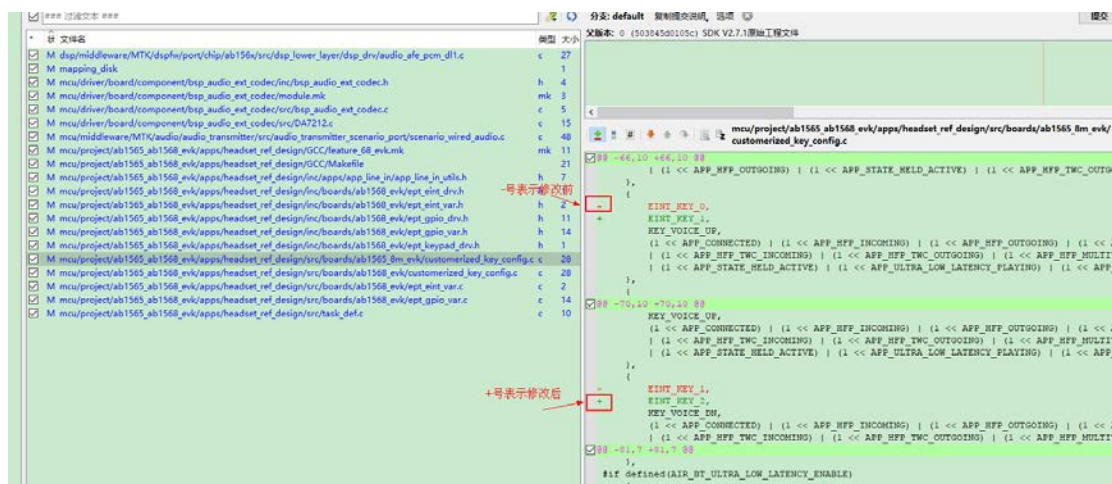
运行打补丁的脚本文件，双击运行，第一次弹出的窗口会自动关闭，说明打补丁完成，再次运行显示以下窗口。



在版本管理工具 TortoiseHg 中刷新现版本，可以看到被跟踪的文件（被补丁修改的）

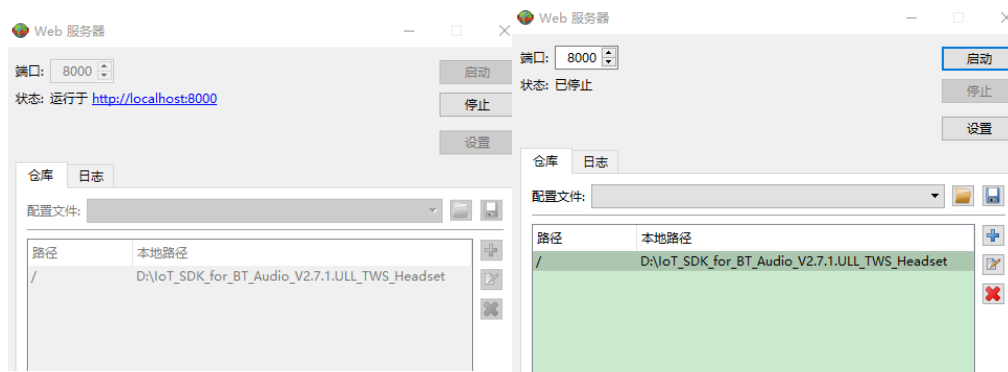


选择不显示未跟踪（未被修改）的文件

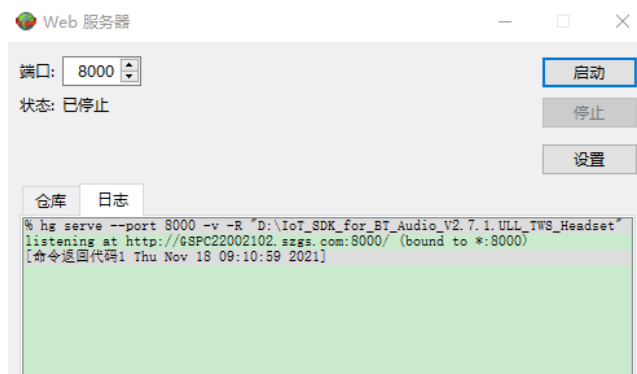


方便多人开发同一个工程，把工程同步到服务器上：

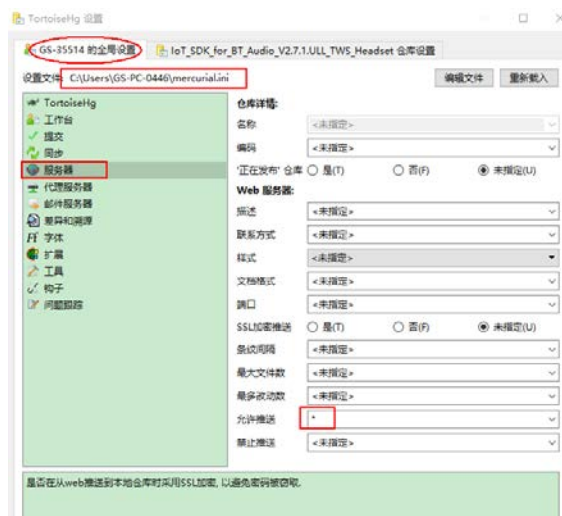
右击工程文件夹打开 TortoiseHg，在左列表选仓库，在菜单 仓库》Web 服务器
点击停止，可以看到仓库的路径



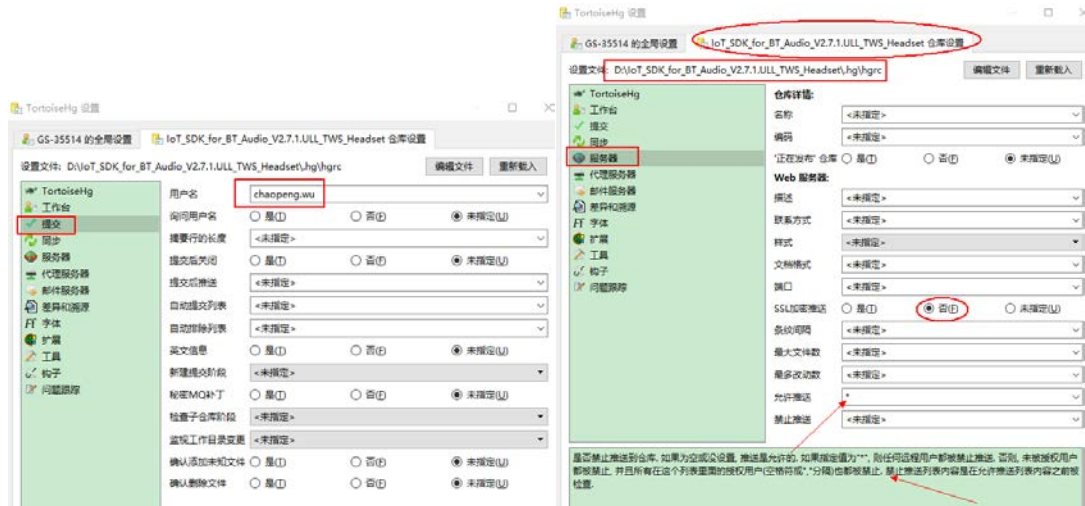
点日志，可以看到被监听的服务器端口



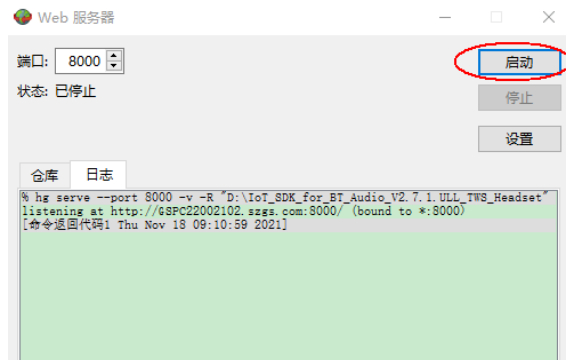
再点设置



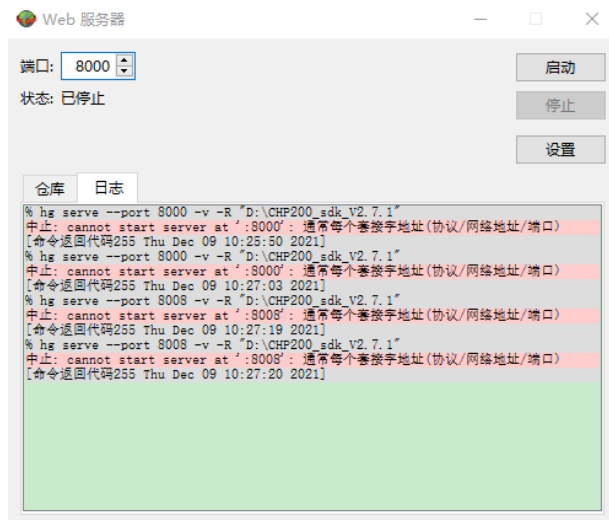
再进行仓库设置，写上提交的用户名，不加密推送，允许推送



设置好之后，点击启动



若出现无法启动，说明端口被占用，此时可以修改端口后再启动



十七、Tile 功能测试

首先耳机的固件程序有防丢功能（Tile）

苹果手机可以直接在 App Store 上下载安装 Tile 软件

安装手机则需要百度下载 XAPK Installer.apk 和 tile.xapk 安装包，先安装 XAPK Installer，再在 XAPK Installer 里选择 tile.xapk 即可

耳机开机，进入配对模式

绑定和激活 tile 功能时，手机先不要连接耳机蓝牙，打开 tile 软件

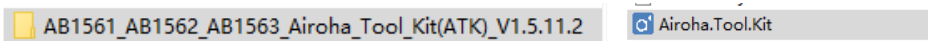
点击 Add a Tile，开启定位，选择客户无线耳机，选择 Crusher ANC/Crusher EVO，点击 Yes,activate Tile，提示 Click to setup，短按耳机主键，激活 tile 功能，点击钥匙图标，点 Next，连接上后，点击 Find，耳机会发出嗡嗡声。可以在钥匙界面选 More Options，最底部 Reset 即可清除耳机与本手机的防丢连接。

耳机关机后，Tile 功能仍保持运行，直到电池没电。

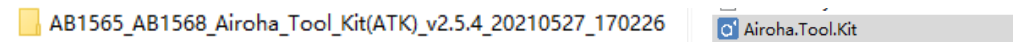
十八、工程配置 cfg 的打开方式

AB1562 与 AB1568 的配置查看工具也不同

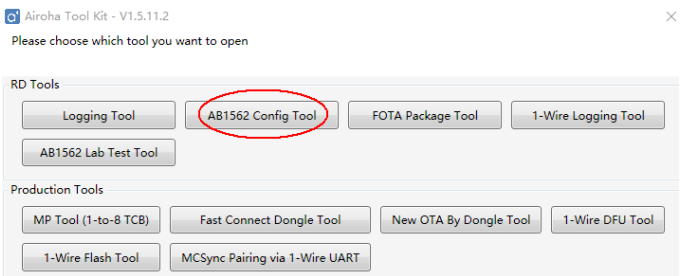
AB1562 的



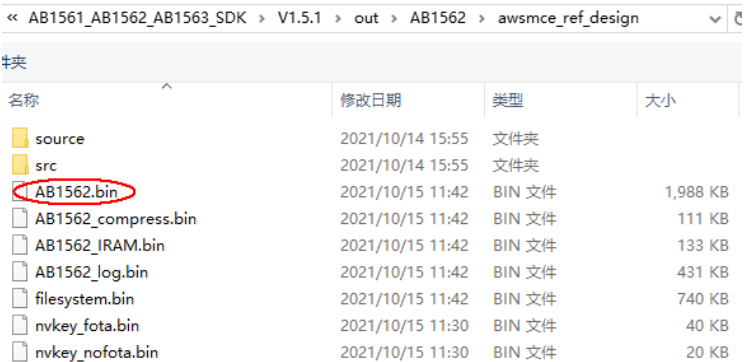
AB1568 的



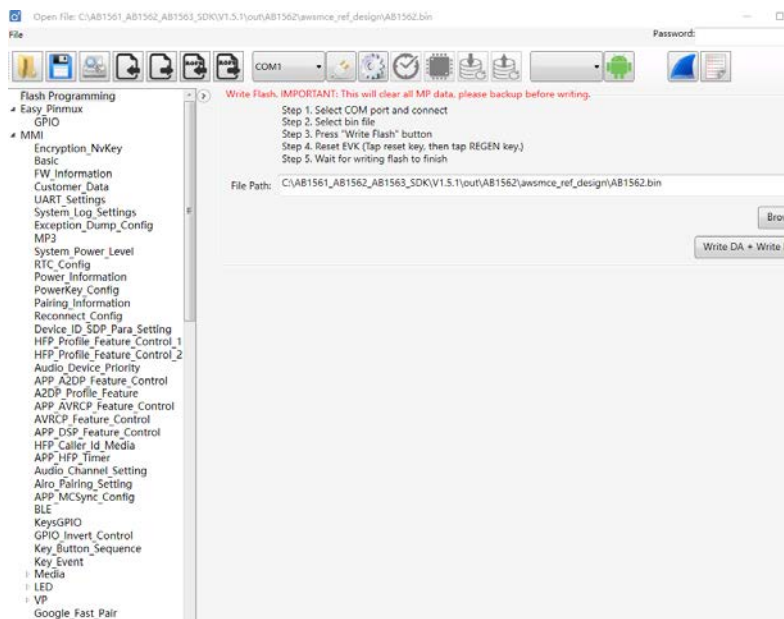
查看 AB1562 工程的配置，运行 ATK 软件，打开 AB1562 Config Tool



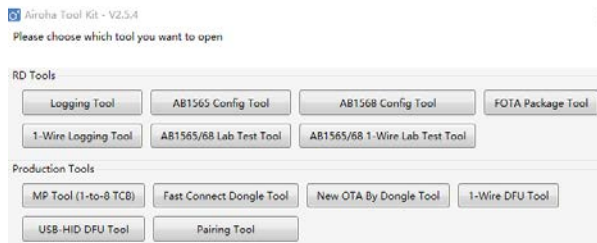
选择编译成功的 SDK 下的 out 文件夹中的 AB1562.bin 文件



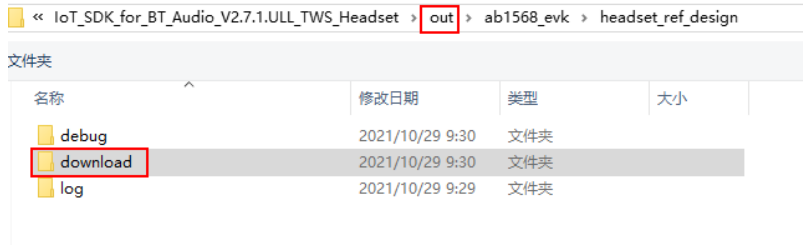
可以查看和修改配置参数



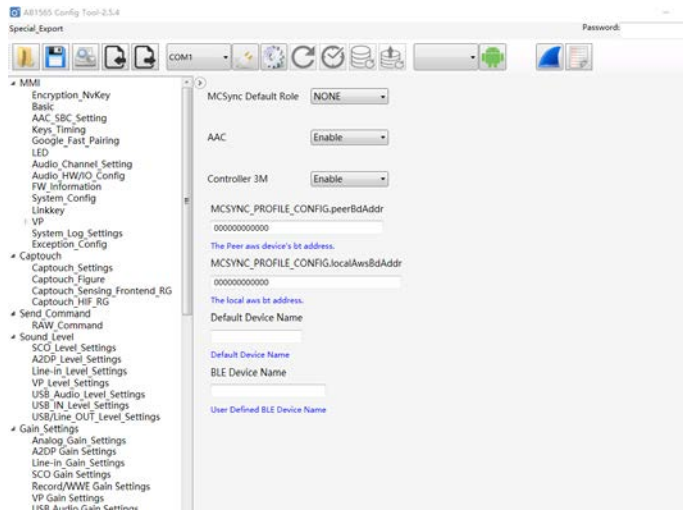
查看 AB1568 的配置，运行对应的 ATK 软件，打开 AB1565 Config Tool



选择编译成功的工程下的 out 文件夹中的 download 文件夹



可以查看和修改配置参数

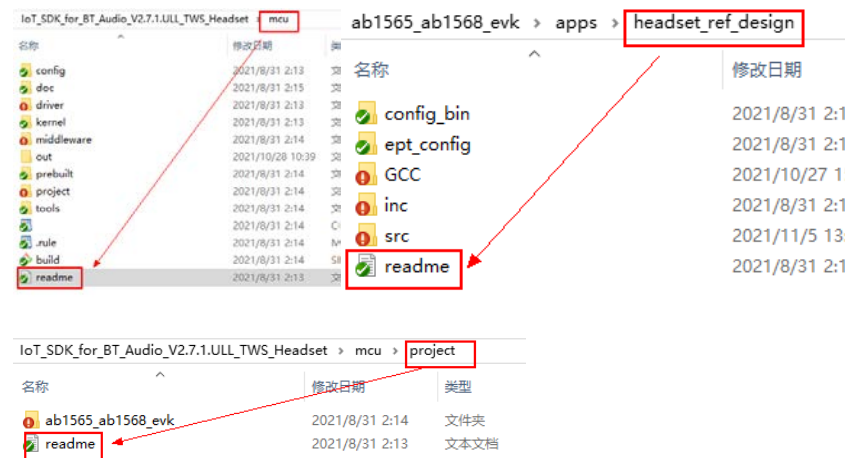


十九、修改定制按键 UI 配置

只需关注三个 C 文件即可

mcu > project > ab1565_ab1568_evk > apps > headset_ref_design > src > boards > ab1568_evk				
名称	修改日期	类型	大小	
customerized_key_config	2021/11/8 11:00	C 文件	28 KB	
ept_eint_var	2021/11/8 21:40	C 文件	2 KB	
ept_gpio_var	2021/11/8 21:57	C 文件	14 KB	
ept_eint_var.c.rej	2021/10/27 15:59	REJ 文件	3 KB	
ept_gpio_var.c.rej	2021/10/27 15:59	REJ 文件	14 KB	

在应用层修改功能，只需关注和弄清楚 mcu 文件夹下的 project 目录中的代码文件即可，每个目录下的 readme 有说明文件内容。



在 SDK 路径下找到文件 customized_key_config.c

路径：.\mcu\project\ab1565_ab1568_evk\apps\headset_ref_design\src\boards\ab1565_8m_evk

打开文件，可以看到双击的按键事件有哪些

```

const apps_config_key_event_map_t temp_key_double_click_configs[] = {
{
    DEVICE_KEY_POWER, ← 按键对象：电源键
    KEY_REJCALL, ← 行为类型：双击
    (1 << APP_HFP_INCOMING) ← 按键响应：拒接
},
{
    DEVICE_KEY_POWER,
    KEY_ONHOLD_CALL, ← 情景状态：电话拨入中
    (1 << APP_HFP_CALLACTIVE) | (1 << APP_HFP_CALLACTIVE_WITHOUT_SCO) | (1 << APP_HFP_TWC_OUTGOING) | (1 << APP_STATE_HELD_ACTIVE)
},
{
    DEVICE_KEY_POWER,
    KEY_REJCALL_SECOND_PHONE,
    (1 << APP_HFP_TWC_INCOMING)
},
{
    DEVICE_KEY_POWER,
    KEY_DISCOVERABLE,
    #if defined(AIR_MULTI_POINT_ENABLE) || defined(AIR_LE_AUDIO_ENABLE) || defined(AIR_BT_ULTRA_LOW_LATENCY_ENABLE)
    (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) | (1 << APP_CONNECTED) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING)
    #else
    (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE)
    #endif
},
#ifdef AIR_MCSYNC_SHARE_ENABLE
{
    EINT_KEY_0,
    KEY_SHARE_MODE_SWITCH,
    (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING)
},
{
    EINT_KEY_1,
    KEY_SHARE_MODE_FOLLOWER_SWITCH,
    (1 << APP_CONNECTABLE) | (1 << APP_DISCONNECTED) | (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING)
},
#else
{
    EINT_KEY_1,
    KEY_SWITCH_ANC_AND_PASSTHROUGH,
    (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) | (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING)
    | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
#endif
};

const apps_config_key_event_map_t temp_key_triple_click_configs[] = {
#ifdef MTK_AWS_MCE_ENABLE
{
    DEVICE_KEY_POWER, 电源键
    KEY_AIR_PAIRING, 进入配对
    (1 << APP_DISCONNECTED) 未连接状态
},
#endif
};

```

编译工程命令

```

GS-355140GS-PC-0446 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.1,ULL_TWS_Headset
$ ./build.sh ab1568_evk headset_ref_design

```

对应的工程源代码在

ab1565_ab1568_evk > apps > headset_ref_design			
名称	修改日期	类型	
config_bin	2021/8/31 2:14	文件夹	
ept_config	2021/8/31 2:14	文件夹	
GCC	2021/10/27 15:59	文件夹	
inc	2021/8/31 2:14	文件夹	
src	2021/10/28 10:46	文件夹	
readme	2021/8/31 2:14	文本文档	

只需查看和修改几个文件即可

apps_config_key_remapper.h

ab1565_ab1568_evk > apps > headset_ref_design > inc > apps > config				
名称	修改日期	类型	大小	
apps_config_event_list	2021/8/31 2:14	H 文件	11	
apps_config_features_dynamic_setting	2021/8/31 2:14	H 文件	4	
apps_config_key_remapper	2021/8/31 2:14	H 文件	7	
apps_config_led_index_list	2021/8/31 2:14	H 文件	4	

apps_config_key_remapper.c

ab1565_ab1568_evk > apps > headset_ref_design > src > apps > config			
名称	修改日期	类型	
apps_config_features_dynamic_setting	2021/8/31 2:14	C 文件	
apps_config_key_remapper	2021/8/31 2:14	C 文件	
apps_config_led_manager	2021/8/31 2:14	C 文件	
apps_config_vp_manager	2021/8/31 2:14	C 文件	

customerized_key_config.c

ab1565_ab1568_evk > apps > headset_ref_design > src > boards > ab1568_evk			
名称	修改日期	类型	大小
 customerized_key_config	2021/11/3 14:14	C 文件	
 ept_eint_var	2021/10/27 15:57	C 文件	

解读 customerized_key_config.c 文件，客户按键配置

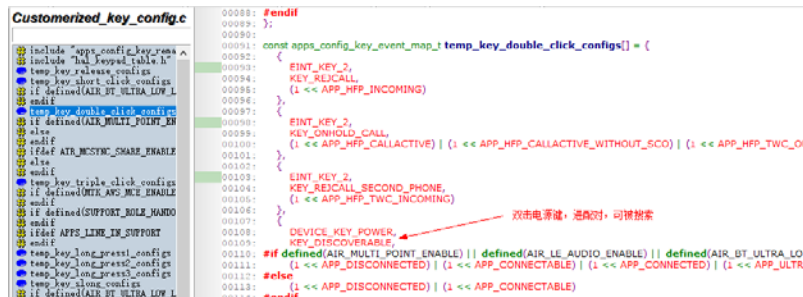
```

#include "apps_config_key_remap.h"
#include "hal_keypad_table.h"

const apps_config_key_event_map_t temp_key_release_configs[] = {
    {
        EINT_KEY_0,
        KEY_AVRCP_FAST_FORWARD_RELEASE,
        (1 << APP_A2DP_PLAYING)
    },
    {
        EINT_KEY_1,
        KEY_AVRCP_FAST_REWIND_RELEASE,
        (1 << APP_A2DP_PLAYING)
    },
};

const apps_config_key_event_map_t temp_key_short_click_configs[] = {

```



头戴耳机：如果没有配置左右 audio_channel，那么 default_configurable_table 将作为默认的按键行为配置，左右耳相同。

TWS 耳机：如果配置了左右 audio_channel，那么左耳将使用 left_configurable_table 作为按键行为配置

右耳将使用 default_configurable_table 作为按键行为配置

可以看到在什么状态下按了哪个键会出现什么响应

```

// For right side or default configurable mapping table
const static apps_config_configurable_table_t default_configurable_table[] = {
    {
        APPS_CONFIG_KEY_SHORT_CLICK, 短按
        DEVICE_KEY_POWER, 电源键
        KEY_AVRCP_PLAY, 音乐播放
        (1 << APP_CONNECTED) | (1 << APP_BT_OFF) | (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) 状态
    },
    {
        APPS_CONFIG_KEY_SHORT_CLICK, 短按
        DEVICE_KEY_POWER, 电源键
        KEY_AVRCP_PAUSE, 音乐暂停
        (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY) 状态
    },
    {
        APPS_CONFIG_KEY_DOUBLE_CLICK, 双击
        DEVICE_KEY_POWER, 电源键
        KEY_AVRCP_FORWARD, 上一曲
        (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY) 状态
    },
    {
        APPS_CONFIG_KEY_TRIPLE_CLICK, 双击
        DEVICE_KEY_POWER, 电源键
        KEY_AVRCP_BACKWARD, 下一曲
        (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
    },
    {
        APPS_CONFIG_KEY_LONG_PRESS_RELEASE_2, 长按后释放
        DEVICE_KEY_POWER, 电源键
        KEY_WAKE_UP_VOICE_ASSISTANT_CONFIRM, 语音助手确认
        (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
    },
    {
        APPS_CONFIG_KEY_LONG_PRESS_2, 长按不放
        DEVICE_KEY_POWER, 电源键
        KEY_WAKE_UP_VOICE_ASSISTANT_NOTIFY, /* To support play a "press" VP to notify user long press time, but confirm the action after user release the key */
        (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY) 状态
    },
    {
        APPS_CONFIG_KEY_LONG_PRESS_3, 长按不放
        DEVICE_KEY_POWER, 电源键
        KEY_POWER_OFF, 关机
        (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) | (1 << APP_CONNECTED) | (1 << APP_HFP_INCOMING)
        | (1 << APP_HFP_OUTGOING) | (1 << APP_HFP_CALLACTIVE) | (1 << APP_HFP_CALLACTIVE_WITHOUT_SCO) | (1 << APP_HFP_TWC_INCOMING)
        | (1 << APP_HFP_TWC_OUTGOING) | (1 << APP_HFP_MULTIPART_CALL) | (1 << APP_A2DP_PLAYING) | (1 << APP_STATE_HELD_ACTIVE)
        | (1 << APP_STATE_FIND_ME) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
    },
};

```

```

const static apps_config_configurable_table_t left_configurable_table[] = { 左声道配置
{
    APPS_CONFIG_KEY_SHORT_CLICK,
    DEVICE_KEY_POWER,
    KEY_AVRCP_PLAY,
    (1 << APP_CONNECTED)
},
{
    APPS_CONFIG_KEY_SHORT_CLICK,
    DEVICE_KEY_POWER,
    KEY_AVRCP_PAUSE,
    (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
#ifdef AIR_MULTI_POINT_ENABLE
{
    APPS_CONFIG_KEY_DOUBLE_CLICK,
    DEVICE_KEY_POWER,
    KEY_SWITCH_ANC_AND_PASSTHROUGH,
    (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) | (1 << APP_CONNECTED) | (1 << APP_HFP_CALLACTIVE)
    | (1 << APP_HFP_MULTIPART_CALL) | (1 << APP_A2DP_PLAYING) | (1 << APP_STATE_HELD_ACTIVE) | (1 << APP_HFP_INCOMING)
    | (1 << APP_HFP_OUTGOING) | (1 << APP_HFP_CALLACTIVE_WITHOUT_SCO) | (1 << APP_HFP_TWC_INCOMING) | (1 << APP_HFP_TWC_OUTGOING)
    | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
#else
{
    APPS_CONFIG_KEY_DOUBLE_CLICK,
    DEVICE_KEY_POWER,
    KEY_SWITCH_ANC_AND_PASSTHROUGH,
    (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) | (1 << APP_CONNECTED) | (1 << APP_HFP_CALLACTIVE)
    | (1 << APP_HFP_MULTIPART_CALL) | (1 << APP_A2DP_PLAYING) | (1 << APP_STATE_HELD_ACTIVE) | (1 << APP_HFP_CALLACTIVE_WITHOUT_SCO)
    | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
#endif
{
    APPS_CONFIG_KEY_TRIPLE_CLICK,
    DEVICE_KEY_POWER,
    KEY_AVRCP_BACKWARD,
    (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
{
    APPS_CONFIG_KEY_LONG_PRESS_RELEASE_2,
    DEVICE_KEY_POWER,
    KEY_WAKE_UP_VOICE_ASSISTANT_CONFIRM,
    (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
{
    APPS_CONFIG_KEY_LONG_PRESS_2,
    DEVICE_KEY_POWER,
    KEY_WAKE_UP_VOICE_ASSISTANT_NOTIFY, /* To support play a "press" VP to notify user long press time, but confirm the action after user release the key
    (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
{
    APPS_CONFIG_KEY_LONG_PRESS_3,
    DEVICE_KEY_POWER,
    KEY_POWER_OFF,
    (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) | (1 << APP_CONNECTED) | (1 << APP_HFP_INCOMING)
    | (1 << APP_HFP_OUTGOING) | (1 << APP_HFP_CALLACTIVE) | (1 << APP_HFP_CALLACTIVE_WITHOUT_SCO) | (1 << APP_HFP_TWC_INCOMING)
    | (1 << APP_HFP_TWC_OUTGOING) | (1 << APP_HFP_MULTIPART_CALL) | (1 << APP_A2DP_PLAYING) | (1 << APP_STATE_HELD_ACTIVE)
    | (1 << APP_STATE_FIND_ME) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_WIRED_MUSIC_PLAY)
},
};

```

结构体 apps_config_configurable_table_t 在头文件 apps_config_key_remapper.h 中定义

```

// apps_config_key_remapper.h
#ifndef APPS_CONFIG_KEY_REMAPPER_H
#define APPS_CONFIG_KEY_REMAPPER_H

#include "apps_config_event_list.h"
#include "apps_config_state_list.h"
#include "multi_va_manager.h"
#include "apps_config_key_event_map_t"
#include "apps_config_configurable_table_t"
#include "apps_config_key_action_state_map_t"
#include "apps_config_key_event_remapper.h"
#include "apps_config_key_set_mmi_state"
#include "apps_config_key_get_mmi_state"
#include "apps_config_key_remapper_init.h"
#include "MULTI_VA_SUPPORT_COMPETITION"

#endif

// apps_config_key.h
#ifndef APPS_CONFIG_KEY_H
#define APPS_CONFIG_KEY_H

#include "apps_config.h"
#include "apps_config_key.h"
#include "apps_config_key.h"
#include "apps_config_key.h"
#include "apps_config_key.h"
#include "apps_config_key.h"

#endif

```

按键动作定义在 app_config_key_remapper.c 中

```

// For right side or default configurable mapping table
const static apps_config_configurable_table_t default_configurable_table[] = {
#ifdef MULTI_VA_SUPPORT_COMPETITION
{
    APPS_CONFIG_KEY_DOUBLE_CLICK, 按键动作（单击、双击、长按之类）
    DEVICE_KEY_POWER, 按键编号，一般用power key，根据硬件做选择
    KEY_AVRCP_FORWARD, 按键行为，例如播放暂停，接打电话，音量调节
    (1 << APP_A2DP_PLAYING) 按键在什么情况下可以使用，例如这个表示只在音
    乐播放的时候使用
},

```

按键行为的宏定义在 apps_config_event_list.h 文件里查看


```

Customized_key_config.c
00264: // For right side or default configurable mapping table
00265: const static apps_config_configurable_table_t default_configurable_table[] = {
00266:     {
00267:         APPS_CONFIG_KEY_SHORT_CLICK,
00268:         DEVICE_KEY_POWER,
00269:         KEY_AVRCP_PLAY,
00270:         (1 << APP_CONNECTED) | (1 << APP_BT_OFF) | (1 << APP_DISCONNECTED) | (1 << APP_
00271:     },
00272:     {
00273:         APPS_CONFIG_KEY_SHORT_CLICK,
00274:         DEVICE_KEY_POWER,
00275:         KEY_AVRCP_PAUSE,
00276:         (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_W
00277:     },
00278:     {
00279:         APPS_CONFIG_KEY_DOUBLE_CLICK,
00280:         DEVICE_KEY_POWER,
00281:         KEY_AVRCP_FORWARD,
00282:         (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_W
00283:     },
00284:     {
00285:         APPS_CONFIG_KEY_TRIPLE_CLICK,
00286:         DEVICE_KEY_POWER,
00287:         KEY_AVRCP_BACKWARD,
00288:         (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP_W
00289:     },
00290:     {
00291:         APPS_CONFIG_KEY_LONG_PRESS_RELEASE_1,
00292:         DEVICE_KEY_POWER,
00293:         KEY_WAKE_UP_VOICE_ASSISTANT_CONFIRM,
00294:         (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENC
00295:     },
00296:     {
00297:         APPS_CONFIG_KEY_LONG_PRESS_2,
00298:         KEY_WAKE_UP_VOICE_ASSISTANT_NOTIFY, /* To support play a 'press' VP to notify user
00299:         (1 << APP_CONNECTED) | (1 << APP_A2DP_PLAYING) | (1 << APP_ULTRA_LOW_LATENC
00300:     },
00301:     {
00302:         APPS_CONFIG_KEY_LONG_PRESS_3,
00303:         DEVICE_KEY_POWER,
00304:         KEY_POWER_OFF,
00305:         (1 << APP_DISCONNECTED) | (1 << APP_CONNECTABLE) | (1 << APP_CONNECTED) | (1
00306:         (1 << APP_HFP_OUTGOING) | (1 << APP_HFP_CALL_ACTIVE) | (1 << APP_HFP_CALL_CT
00307:         (1 << APP_HFP_TWC_OUTGOING) | (1 << APP_HFP_MULTIPART_CALL) | (1 << APP_A
00308:         (1 << APP_STATE_FIND_ME) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (1 << APP
00309:     },
00310: },
00311: const static apps_config_configurable_table_t left_configurable_table[] = {
00312:     {
00313:         APPS_CONFIG_KEY_SHORT_CLICK,
00314:         DEVICE_KEY_POWER,
00315:         KEY_AVRCP_PLAY,
00316:         (1 << APP_CONNECTED)
00317:     },
00318: };

```

递进关系，通过 ui_shell_start_activity 注册一个 proc 后

```
00209: ui_shell_start_activity(NULL, app_home_screen_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_TOP, NULL, 0);
```

会在按键触发的时候收到 event_group:EVENT_GROUP_UI_SHELL_KEY，然后在这个 case 下定义处理 key event group 的函数

```

App_home_screen_idle_activity.c
01536: }
01537:
01538: bool app_home_screen_idle_activity_proc(ui_shell_activity_t *self,
01539:     uint32_t event_group,
01540:     uint32_t event_id,
01541:     void *extra_data,
01542:     size_t data_len)
01543: {
01544:     bool ret = false;
01545:
01546:     switch (event_group) {
01547:         /* UI Shell internal events. */
01548:         case EVENT_GROUP_UI_SHELL_SYSTEM: {
01549:             ret = _proc_ui_shell_group(self, event_id, extra_data, data_len);
01550:             break;
01551:         }
01552:         /* UI Shell key events. */
01553:         case EVENT_GROUP_UI_SHELL_KEY: {
01554:             ret = _proc_key_event_group(self, event_id, extra_data, data_len);
01555:             break;
01556:         }
01557:     }
01558:     return ret;
01559: }

```

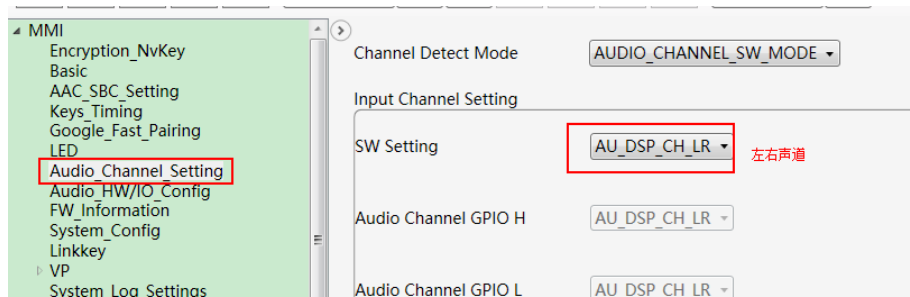
然后再在里面处理 key_event_id，具体的 key_event_id 定义在 app_config_event_list.h 中通过 case 选择不同 key_id 的处理方式即可。

```

App_config_key_
00239: apps_config_key_action_t apps_config_key_event_remapper_map_action_in_temp_state(
00240:     uint8_t key_id,
00241:     airo_key_event_t key_event,
00242:     apps_config_state_t temp_mmi_state)
00243: {
00244:     apps_config_key_action_t action_id = KEY_ACTION_INVALID;
00245:     size_t i;
00246:     apps_config_serialized_event_t serialized_event_id = APPS_CONFIG_KEY_INVALID;
00247:     const apps_config_key_event_map_list_t *mapped_list = NULL;
00248:     audio_channel_t channel = 0;
00249:     #ifdef MTK_AWS_MCE_ENABLE
00250:         channel = ami_get_audio_channel();
00251:     #endif
00252:
00253:     /* Convert the key event from airo_key_event_t to apps_config_serialized_event_t. */
00254:     switch (key_event) {
00255:         case AIRO_KEY_RELEASE:
00256:             serialized_event_id = APPS_CONFIG_KEY_RELEASE;
00257:             break;
00258:         case AIRO_KEY_SHORT_CLICK:
00259:             serialized_event_id = APPS_CONFIG_KEY_SHORT_CLICK;
00260:             break;
00261:         case AIRO_KEY_DOUBLE_CLICK:
00262:             serialized_event_id = APPS_CONFIG_KEY_DOUBLE_CLICK;
00263:             break;
00264:         case AIRO_KEY_TRIPLE_CLICK:
00265:             serialized_event_id = APPS_CONFIG_KEY_TRIPLE_CLICK;
00266:             break;
00267:         case AIRO_KEY_LONG_PRESS_1:
00268:             serialized_event_id = APPS_CONFIG_KEY_LONG_PRESS_1;
00269:             break;
00270:         case AIRO_KEY_LONG_PRESS_2:
00271:             serialized_event_id = APPS_CONFIG_KEY_LONG_PRESS_2;
00272:             break;
00273:         case AIRO_KEY_LONG_PRESS_3:
00274:             serialized_event_id = APPS_CONFIG_KEY_LONG_PRESS_3;
00275:             break;
00276:     }
00277: }

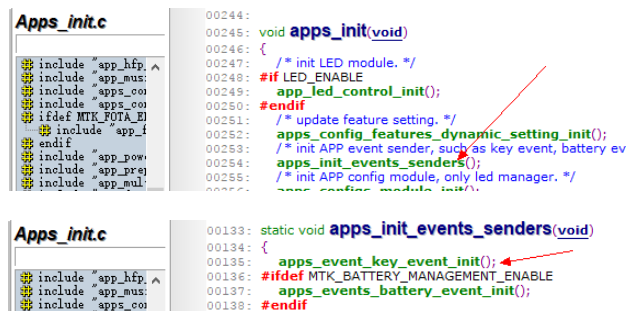
```

音频通道配置在 config tool 中可查看

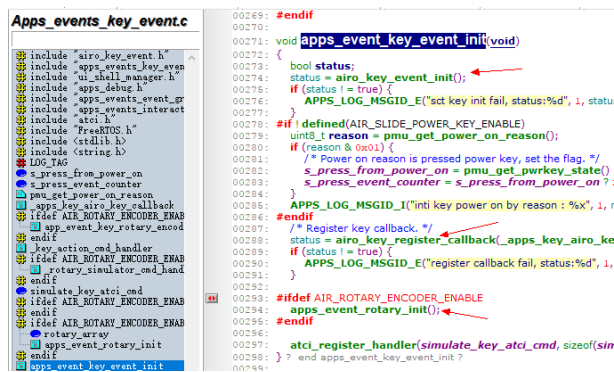


按钮响应事件执行过程:

开机先执行 apps_init.c 中的 apps_init_events_senders 函数



针对某个模块初始化 (比如回调)



某个操作失败才会打印

No.	Time	Ticks	CPU	Protocol	Length	Info
4163	2021-11-22 21:01:57.936531	126593	CM4	MSG_LOG	77	[M:BT_CM C:info F: L:]: [BT_CM][POWER][I] Standby reason is 0x0!
8814	2021-11-22 21:07:12.138186	448791	CM4	MSG_LOG	77	[M:BT_CM C:info F: L:]: [BT_CM][POWER][I] Standby reason is 0x0!
10569	2021-11-22 21:13:24.088232	141	CM4	MSG_LOG	54	[M:hal C:info F: L:]: Power off reason: 0
10960	2021-11-22 21:13:24.327381	344	CM4	MSG_LOG	67	[M:apps C:info F: L:]: Init key power on by reason: 4
11442	2021-11-22 21:13:24.700482	724	CM4	MSG_LOG	66	[M:apps C:info F: L:]: [app_battery] pmu_reason = 0x4

```

00139: } ? endairo_key_event_init ?
00140:
00141: boolairo_key_register_callback(airo_key_callback_t callback)
00142: {
00143:     if (callback == NULL) {
00144:         return false;
00145:     }
00146:     airo_key_event_context.callback = callback;
00147:     airo_key_event_context.user_data = user_data;
00148:
00149:     #ifdef AIRO_KEY_FEATURE_POWERKEY
00150:         hal_eint_mask(HAL_EINT_PMU);
00151:         airo_powerkey_init();
00152:     #endif
00153:     #ifdef AIRO_KEY_FEATURE_POWERKEY_POWERON_PRESS_EVENT
00154:         airo_powerkey_poweron_press_event_simulation();
00155:     #endif
00156:     hal_eint_unmask(HAL_EINT_PMU);
00157: #endif
00158:
00159:     #ifdef MTK_KEYPAD_ENABLE
00160:         airo_keypad_init();
00161:     #endif
00162:
00163:     #ifdef MTK_EINT_KEY_ENABLE
00164:         airo_eint_key_init();
00165:     #endif
00166:     #ifdef AIRO_KEY_FEATURE_EINTKEY_POWERON_PRESS_EVENT
00167:         airo_eint_key_poweron_press_event_simulation();
00168:     #endif
00169:
00170:     #ifdef HAL_CAPTOUCH_MODULE_ENABLED
00171:         airo_captouch_init();
00172:     #endif
00173:
00174:     #ifdef MTK_GSENSOR_KEY_ENABLE
00175:         airo_key_gsensor_init();
00176:     #endif
00177:     #ifdef AIR_PSENSOR_KEY_ENABLE
00178:         airo_key_psensor_init();
00179:     #endif
00180:     #ifdef AIR_BSP_INEAR_ENABLE
00181:         bsp_inear_init();
00182:     #endif
00183:     LOG_MSGD_I(common, "[airo_key] register callback done! %i\n", 0);
00184:     return true;
00185: } ? endairo_key_register_callback ?

```

二十、提示音（VP）文件路径

用 ATK 的 Config Tool 打开固件.7z 文件，在 VP > VP Event Table 可以看到固件中有的提示音

Open File: D:\GS\RR2 wireless\V1.52_file_V1201_ZML_20211101\A81561M_GS\RR2 wireless\V1.52_file_V1201_ZML_20211101.7z

File

COM1

en-US

VP

VP Event Table

VP No.	Event	VP Files
1	POWER_ON (0x0011)	030_SND_ON.mp3
2	POWER_OFF (0x0012)	031_SND_OFF.mp3
3	PAIRING (0x0013)	002_PAIRING.mp3
4	CONNECTED (0x0015)	003_CONNECTED.mp3
5	DEVICE_DISCONNECTED (0x0017)	004_DEVICE_DISCONNECTED.mp3, 002_PAIRING.mp3
6	CALL_REJECTED (0x0019)	035_SND_REJECT.mp3
7	CALL_ENDED (0x0018)	035_SND_REJECT.mp3
8	LOW_BATTERY (0x001D)	008_LOW_BATTERY.mp3
9	VOLUME_MAXIMUM (0x0021)	040_SND_VOLUME_MAX.mp3
10	FM_MODE (0x0025)	034_SND_ACCEPT.mp3
11	VOICE_COMMAND_EXAMPLE_SET0 (0x0035)	036_SND_HEAD_YES.mp3
12	VOICE_COMMAND_EXAMPLE_SET1 (0x0036)	037_SND_HEAD_NO.mp3
13	VOICE_COMMAND_EXAMPLE_SET2 (0x0037)	078_MUSIC_MODE.mp3
14	VOLUME_MINIMUM (0x0041)	041_SND_VOLUME_MIN.mp3
15	INCOMING_CALL (0x0018)	042_SND_RINGTONE.mp3
16	SEEK_UP (0x0026)	GAMING_MODE_ON.mp3
17	SEEK_DOWN (0x0027)	GAMING_MODE_OFF.mp3
18	CALLER_ID_PREFIX (0x003F)	079_MOVIE_MODE.mp3
19	CALLER_ID_SUFFIX (0x0040)	080_PODCAST_MODE.mp3
20	LINE_IN_MODE (0x0023)	043_SND_FIND.mp3

同时，在 ATK 目录下生成实时文件夹 working+时间（Voice Prompt 原文件，可以打开试听）

> AB1561_AB1562_AB1563_Airoha_Tool_Kit(ATK)_V1.5.11.2			
名称	修改日期	类型	
OtaTool	2021/6/4 18:03	文件夹	
setting	2021/6/4 18:03	文件夹	
wav	2021/6/4 18:03	文件夹	
Wireshark_Release	2021/4/22 9:20	文件夹	
working20211019170212	2021/10/19 17:02	文件夹	
working20211019170230	2021/10/19 17:02	文件夹	
working20211019171318	2021/10/19 17:13	文件夹	
working20211020161800	2021/10/20 16:18	文件夹	
working20211020173356	2021/10/20 17:33	文件夹	
working20211020201827	2021/10/20 21:21	文件夹	
working20211020212431	2021/10/20 21:24	文件夹	
working20211021075311	2021/10/21 7:53	文件夹	
working20211021091933	2021/10/21 9:21	文件夹	
working20211021095030	2021/10/21 9:55	文件夹	
working20211022132859	2021/10/22 13:28	文件夹	
working20211022133007	2021/10/22 13:30	文件夹	
working20211022134332	2021/10/22 13:43	文件夹	
working20211030104500	2021/10/30 10:45	文件夹	
working20211030105042	2021/10/30 10:50	文件夹	
working20211101083800	2021/11/1 8:38	文件夹	
working20211101133802	2021/11/1 13:38	文件夹	
Airoha.Console	2021/6/4 18:03	应用程序	
Airoha.FastConnectDongleA2DP ANC	2021/6/3 14:19	Windows 批处理	

AB1561_AB1562_AB1563_Airoha_Tool_Kit(ATK)_V1.5.11.2			
	working20211101133802	> FileSystem	> en-US
名称	#	标题	参与创作
000_POWER_ON			
001_POWER_OFF			
002_PAIRING			
003_CONNECTED			
004_DEVICE_DISCONNECTED			
008_LOW_BATTERY			
030_SND_ON			
031_SND_OFF			
034_SND_ACCEPT			
035_SND_REJECT			
036_SND_HEAD_YES			
037_SND_HEAD_NO			
040_SND_VOLUME_MAX			
041_SND_VOLUME_MIN			
042_SND_RINGTONE			
043_SND_FIND			
078_MUSIC_MODE			
079_MOVIE_MODE			
080_PODCAST_MODE			
GAMING_MODE_OFF			
GAMING_MODE_ON			

二十一、如何生成 FOTA 烧录文件

编译 SDK 生成 AB1562.bin 文件

V1.5.2_SDK_with_tile > out > AB1562M > headset_ref_design				
名称	修改日期	类型	大小	
source	2021/11/1 15:00	文件夹		
src	2021/11/1 15:01	文件夹		
AB1562.asm	2021/11/1 15:06	ASM 文件	20,847 KB	
AB1562.bin	2021/11/1 15:06	BIN 文件	2,712 KB	
AB1562.hex	2021/11/1 15:06	HEX 文件	9,105 KB	
AB1562.map	2021/11/1 15:06	MAP 文件	5,584 KB	
AB1562.out	2021/11/1 15:06	OUT 文件	26,302 KB	
AB1562_bootloader.o	2021/11/1 15:05	O 文件	33 KB	
AB1562_compress.bin	2021/11/1 15:06	BIN 文件	111 KB	
AB1562_IRAM.bin	2021/11/1 15:06	BIN 文件	133 KB	
AB1562_log.bin	2021/11/1 15:06	BIN 文件	392 KB	
filesystem.bin	2021/11/1 15:05	BIN 文件	750 KB	
filesystem.o	2021/11/1 15:05	O 文件	751 KB	
iram_compress.o	2021/11/1 15:06	O 文件	112 KB	
nvkey_fota.bin	2021/11/1 15:05	BIN 文件	40 KB	
nvkey_fota_bin.o	2021/11/1 15:05	O 文件	41 KB	
nvkey_nofota.bin	2021/11/1 15:05	BIN 文件	20 KB	
nvkey_nofota_bin.o	2021/11/1 15:05	O 文件	21 KB	

用 ATK 软件里的 FOTA Package Tool 加载 AB1562.bin 文件（或 7z 文件），点击 Generate FOTA package and File System

FOTA Package Tool-1.5.11.2

Password: ***** Login

Step 1. Load Partition info. from ...

☐ from Partition XML...

☒ from AB1562 .bin

D:\V1.5.2_SDK_with_tile\out\AB1562M\headset_ref_design\AB1562.bin

Step 2. Select partitions (For AB1562 .bin, selected partitions are fixed.)

Selection	Partition Name	Source Type	Offset	Length	Target Address	Source Filename

Step 3. Set options and generate FOTA package

☒ Enable LZMA

☐ Enable AES (for AB1565/68 only)

AES Key: 0x 000102030405060708090a0b0c0d0e0f

AES IV: 0x 62633636633839306334636432383763

Version: version_string

Generate FOTA package

Generate FOTA package and File System

Step 4. *Optional* Generate File System Image (for AB1562 only)

Select a source .bin to generate the File System Image

Generate File System Image

在 AB1562.bin 的同一个目录下生成两个 FOTA 文件，把这两个文件拷贝到手机上，用 app 去升级

V1.5.2_SDK_with_tile > out > AB1562M > headset_ref_design				
名称	修改日期	类型	大小	
source	2021/11/1 15:00	文件夹		
src	2021/11/1 15:01	文件夹		
AB1562.asm	2021/11/1 15:06	ASM 文件	20,847 B	
AB1562.bin	2021/11/1 15:06	BIN 文件	2,712 B	
AB1562.hex	2021/11/1 15:06	HEX 文件	9,105 B	
AB1562.map	2021/11/1 15:06	MAP 文件	5,584 B	
AB1562.out	2021/11/1 15:06	OUT 文件	26,302 B	
AB1562_bootloader.o	2021/11/1 15:05	O 文件	33 B	
AB1562_compress.bin	2021/11/1 15:06	BIN 文件	111 B	
AB1562_FileSystemPackage.bin	2021/11/1 15:30	BIN 文件	462 B	
AB1562_FotaPackage.bin	2021/11/1 15:30	BIN 文件	759 B	
AB1562_IRAM.bin	2021/11/1 15:06	BIN 文件	133 B	
AB1562_log.bin	2021/11/1 15:06	BIN 文件	392 B	
filesystem.bin	2021/11/1 15:05	BIN 文件	750 B	
filesystem.o	2021/11/1 15:05	O 文件	751 B	
iram_compress.o	2021/11/1 15:06	O 文件	112 B	
nvkey_fota.bin	2021/11/1 15:05	BIN 文件	40 B	
nvkey_fota_bin.o	2021/11/1 15:05	O 文件	41 B	
nvkey_nofota.bin	2021/11/1 15:05	BIN 文件	20 B	
nvkey_nofota_bin.o	2021/11/1 15:05	O 文件	21 B	

如果用 ATK 软件里的 Config Tool 更改配置后，可以保存成 7z 文件，再用 ATK 软件里的 FOTA Package Tool 加载 7z 文件来生成 FOTA 文件

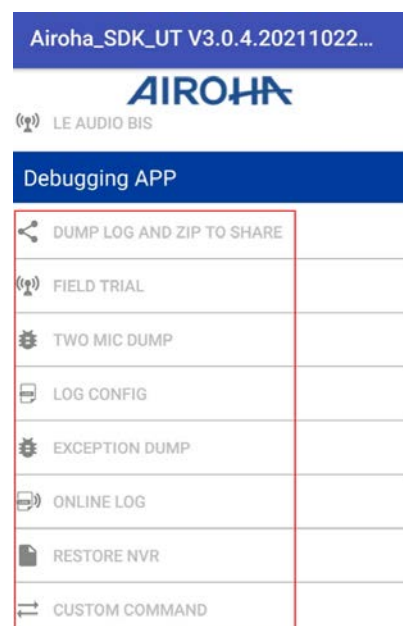
二十二、手机 app 上进入 Debug 模式和在线 log

打开手机的开发者模式（在设置里的关于手机那里，多点击几次版本号，直到出现进入开发者模式的提示）

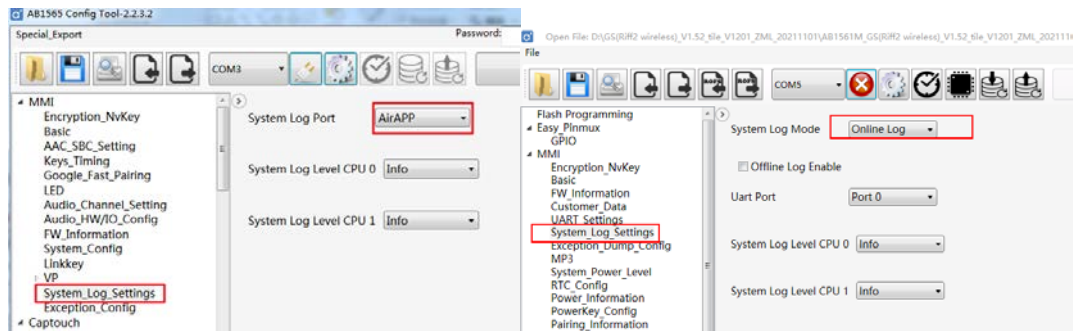
手机上打开 UT 软件，双击标题文字即可打开调试模式



蓝牙连接成功后，可以使用相关 Debug 功能

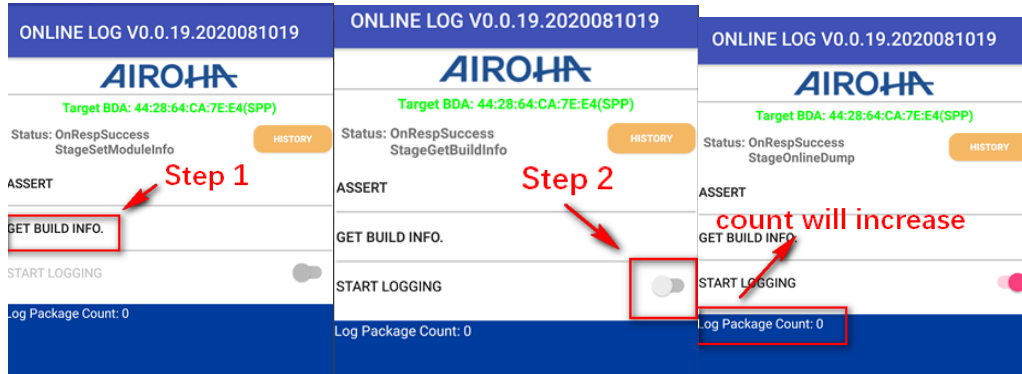


使用 config tool，修改 System_Log_Setting 里面的 log port 为 AirApp 或 Online log。



在耳机连接上手机之后，使用 UT_APP 选择对应的设备连接上 SPP。

在 UP_APP 中选择 online log 的选项。

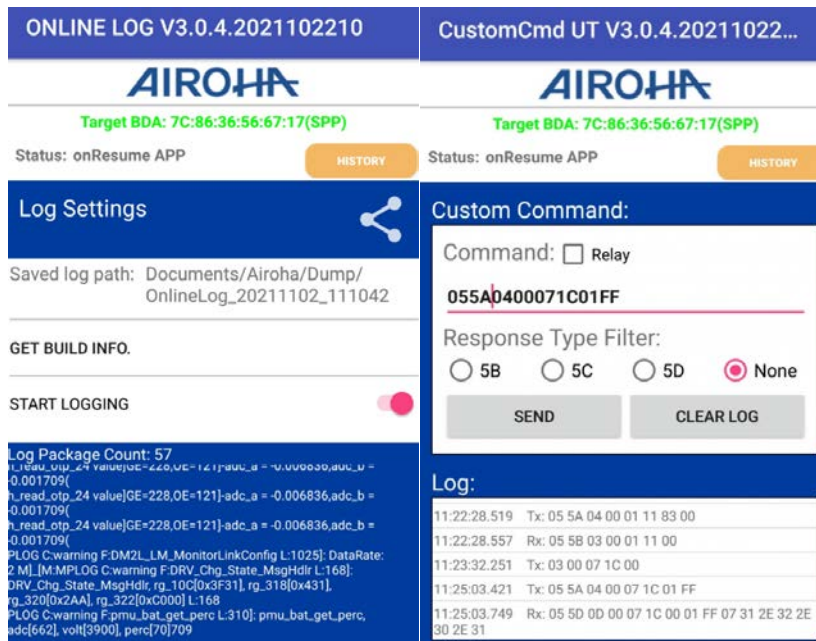


Online log 只支持看 agent log，不支持 partner log

不支持 BT HCI log

由于 BT 带宽限制，目前的 log leve 是 warning。（log throughput must < 10kB/s）

看 log，发 RACE Command

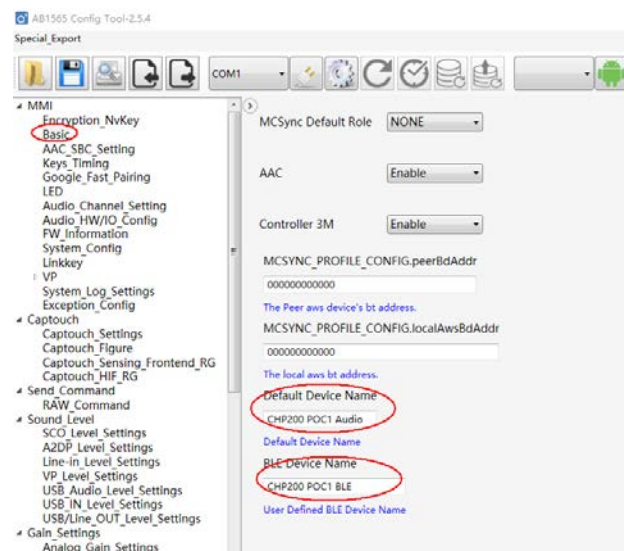


二十三、修改蓝牙 Audio 和蓝牙 BLE 的名称

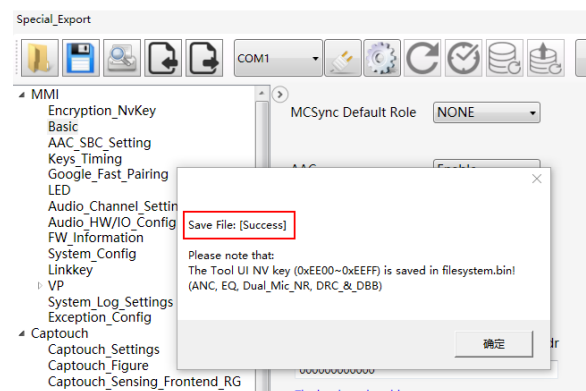
耳机有两个蓝牙地址和名称（音频 Audio 和透传 BLE）

改 AB1568 的

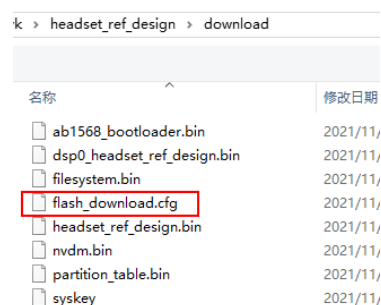
用 Config Tool 打开 download 文件夹，在 Basic 项输入两个名称，如果为空，每次烧录程序都会给一个随机名称



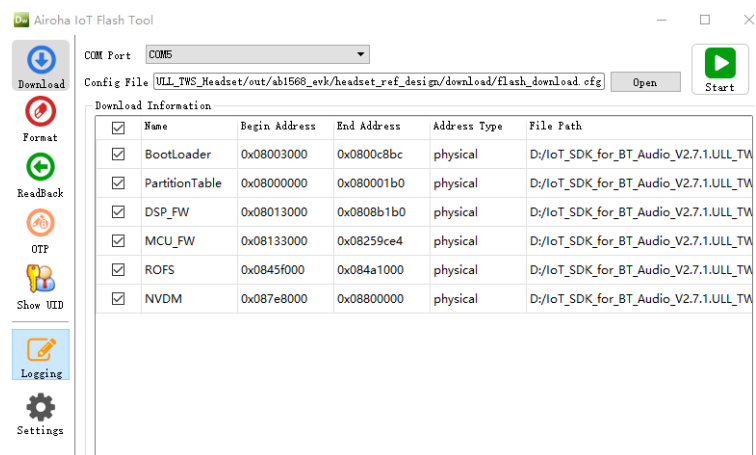
点击 Save，不用输入名称，直接保存到 download 目录即可



再用 Flash Tool 打开 download 文件夹下的 cfg 文件



点击 Start 进行下载



手机上的蓝牙搜索列表会显示刚改的名称
或者输入蓝牙名称后，先点击 **Write NV**，然后点击 **reset**



二十四、发送 RACE Command 触发 Event

目标：开启和关闭咪头直通喇叭的功能

查看指令说明 [AB156x_Series_Mass_Production_RACE_Application_Note.pdf](#)

[Airoha_RACE_Command_Specification.HTML](#)

2.1. RCMD Packet Format

RCMD Command Format (send to AB156x)

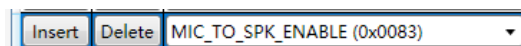
Command				
Channel	Type	Length	ID	Payload
1 byte	1 byte	2 bytes	2 bytes	Varied
0x05	0x5A or 0x5C	#1	RACE Command ID	#2

Table 2-1.RACE command format

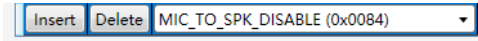
Type List

Type ID	Description
0x5A	Command Needs Response 需要响应的指令
0x5B	Response 响应
0x5C	Command Doesn't Need Response
0x5D	Notification

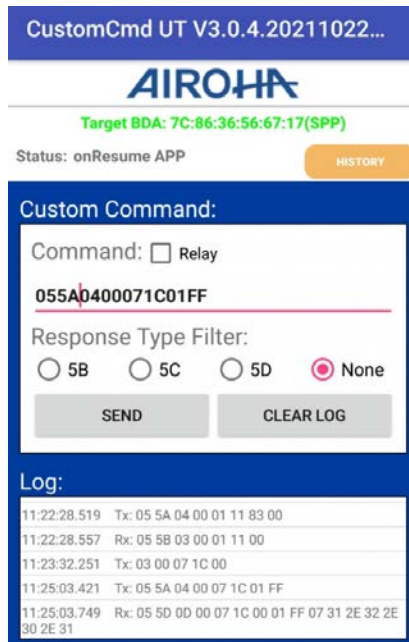
Table 2-3.RACE type list



那么打开 MIC_TO_SPK_ENABLE 模式的完整指令是 05 5A 04 00 01 11 83 00



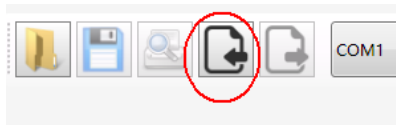
关闭 MIC_TO_SPK_DISABLE 模式的完整指令是 05 5A 04 00 01 11 84 00



二十五、NVR 文件并入 FW

方法一：单独 Import NVR

运行 config tool 后，直接点击 Import NVR，选择要导入的 NVR 文件，但是只能查看，不能保存

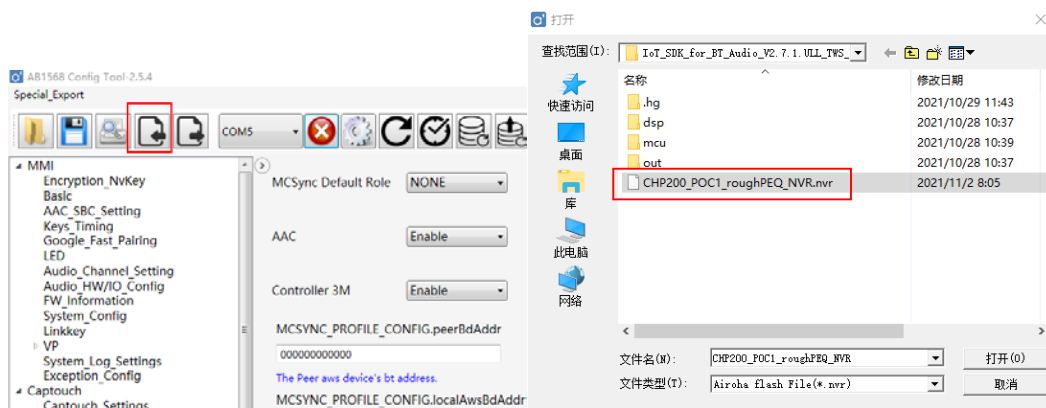


方法二：在 download 文件夹加载后（打开 bin 文件），再 Import NVR

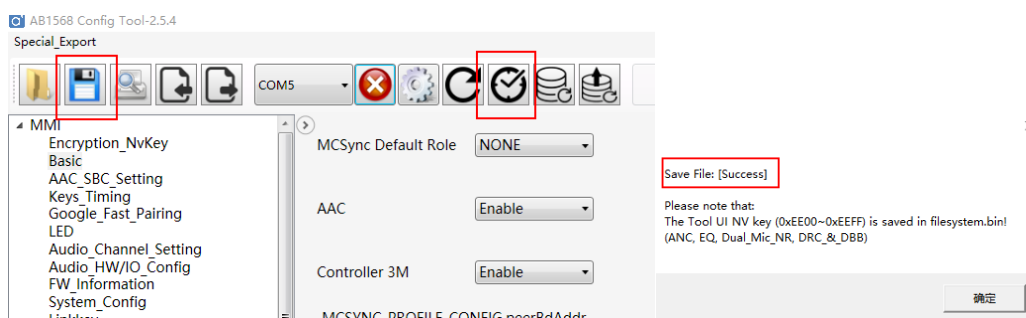
先打开工程的当前配置（bin 文件，含有 filesystem.bin 和 nvdm.bin 的 download 文件夹）

_MACOSX	2021/12/4 8:10	文件夹	
ab1568_bootloader.bin	2021/12/3 16:28	BIN 文件	39 KB
bos_headset_ref_design.bin	2021/12/3 16:28	BIN 文件	1,252 KB
dsp0_headset_ref_design.bin	2021/12/3 16:28	BIN 文件	472 KB
filesystem.bin	2021/12/3 16:28	BIN 文件	264 KB
flash_download.cfg	2021/12/3 16:28	CFG 文件	2 KB
nvdm.bin	2021/12/3 16:28	BIN 文件	96 KB
partition_table.bin	2021/12/3 16:28	BIN 文件	1 KB

然后 Import 导入 NVR 文件（其他人提供的文件），这个功能只是打开一个新的 NVR 文件，不是在现有的 NVR 文件上加载新 NVR 文件



点击 Write NV 把配置写入耳机（这种操作会把现有配置覆盖掉），此功能只是把当前界面的 NVkey 写入

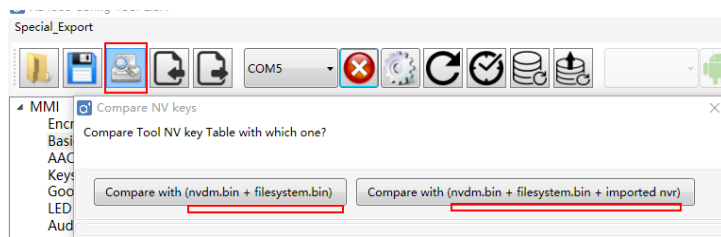


点击保存可以保存到 download 文件夹中，此时得到新 NVR 的 FW 固件（主要保存为 filesystem.bin 和 nvdm.bin）

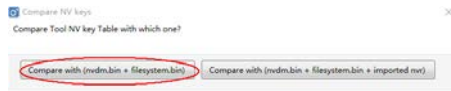


```
[17:47:20.700] Save File: Cancelled
[17:48:12.940] Save File: Start...
[17:48:16.992] Save File successfully: filesystem.bin
[17:48:23.868] Save File successfully: nvdm.bin
[17:48:23.876] Save File: Done
```

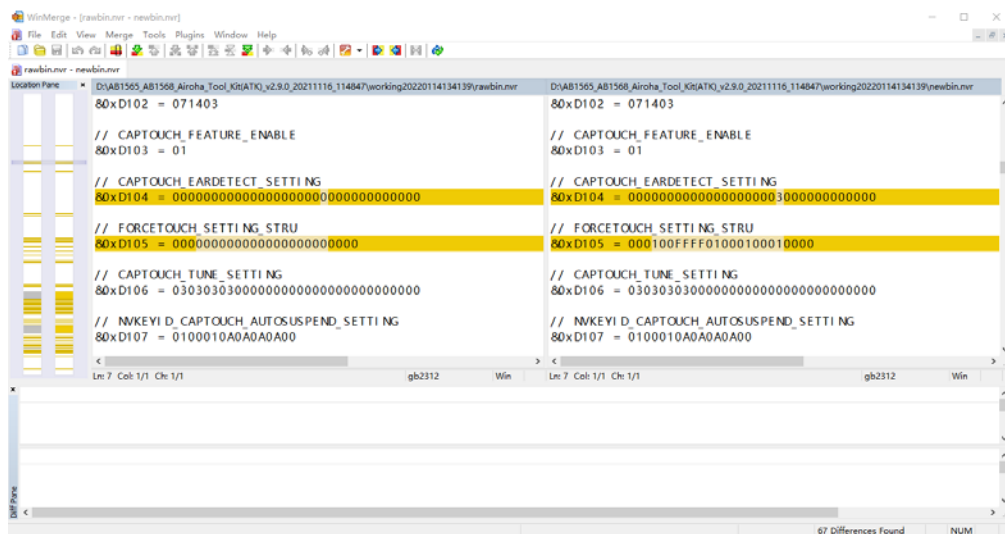
如果只把改变的参数变更到当前的 FW 里，那么需要单独找出被修改的 NVkey 值，然后更新到代码里即可。方法是：使用到 Compare NV 功能
对比更改前后的配置，用来比较刚打开的配置和修改后的配置差别在哪里



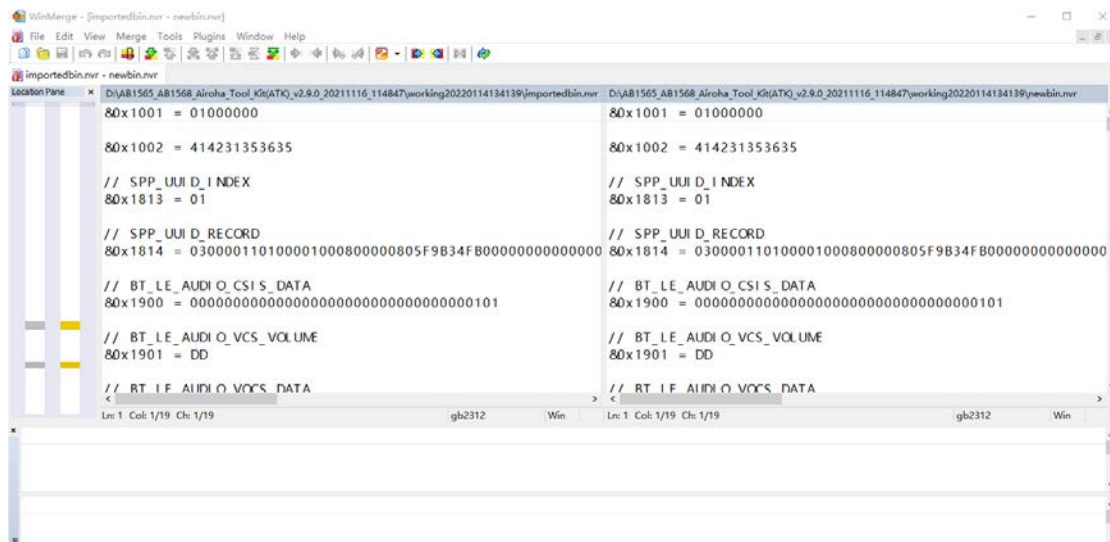
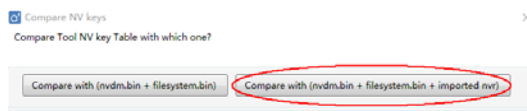
- 1、先用 Import NVR 导入 CHP200_WS2_Filter_PCBA+4dB_PulseEQ_20220105_Dump_All.nvr
- 2、然后点击 Compare NV，点击 Compare with (nvdm.bin +filesystem.bin)，比较 bin 文件的不同，这样就知道修改了哪些参数



rawbin.nvr 是原始文件，newbin.nvr 是修改文件，黄色表示未更改，浅黄色是被更改的



点击 Compare with (nvdm.bin+filesystem.bin+import nvr)，



newbin.bin 文件

查找 config tool 设置的 ANC 参数对应到工程什么文件里。

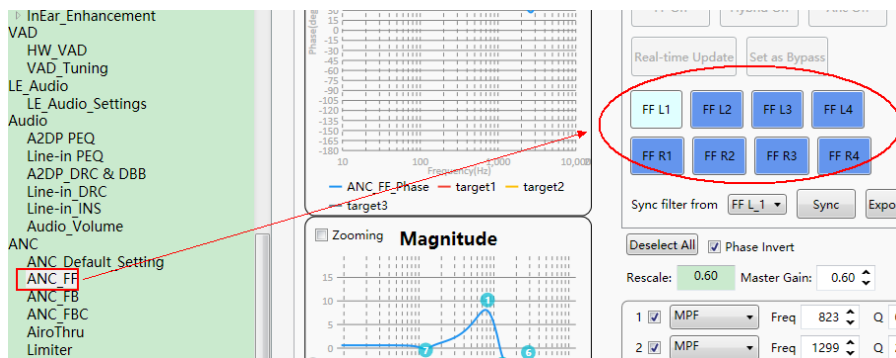
Tool UI 的参数在文件 Nvkey.xml、Nvkey_id_list.h、Nvkey_dspalg.h 里没找到对应地址。

但是 ANC_FILTER、PASSTHRU_FILTER 的 KeyID 可以找到对应的



与洛达那边沟通后才知道对应关系。

在 config tool 上调节的 ANC 参数值会保存 download 文件夹下的 Syskey.xml 文件

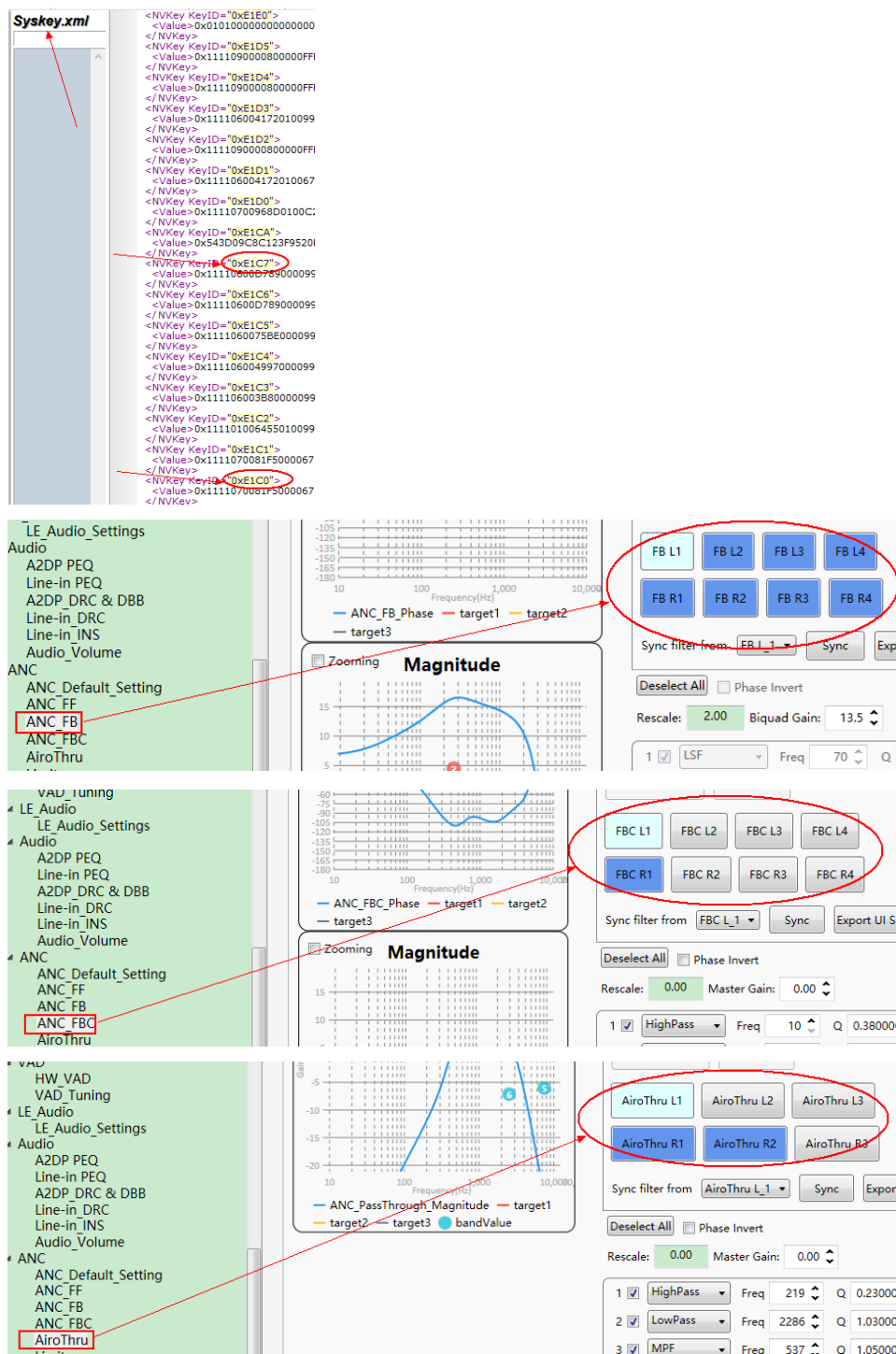


比如选中 FF L1, 点击  Write NV, 下面会打印出被修改的地址

```
[15:47:40.819] Write NV to device: Start...
[15:47:40.841] Write NV to device: No modified NV needs to update.
[15:47:48.838] Write NV to device: Start...
[15:47:48.939] 0xE1C0 = 1111070081F5000671D1D08EE203E2F656A64D1C9A29480ECB10C7E7FFFFF7F78F1A81AC81137EB5084881C2D4B67DFFFFF7F78B00F8050C0E07F
[15:47:48.978] Write NV to device: Done.
```

选中 FF R4, 点击  Write NV

```
[15:55:12.266] Write NV to device: Start...
[15:55:12.335] 0xE1C7 = 111106000789000099E2E22FDAE
[15:55:12.374] Write NV to device: Done.
```



测试得知参数的以下对应关系：

FF L1、FB L1、FBC L1 任一个的值都会覆盖地址 0xE1C0 的值

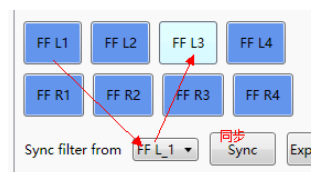
FF R1、FB R1、FBC R1 任一个的值都会覆盖地址 0xE1C1 的值

从图中概括出 ANC 共有 L1、R1、L2、R2、L3、R3、L4、R4 这 8 个滤波器

AiroThru 共有 L1、R1、L2、R2、L3、R3 这 6 个滤波器

download文件夹下的Syskey.xml内的地址NVKey KeyID	ANC_FF	ANC_FB	ANC_FBC
0xE1C0	FF L1	FB L1	FBC L1
0xE1C1	FF R1	FB R1	FBC R1
0xE1C2	FF L2	FB L2	FBC L2
0xE1C3	FF R2	FB R2	FBC R2
0xE1C4	FF L3	FB L3	FBC L3
0xE1C5	FF R3	FB R3	FBC R3
0xE1C6	FF L4	FB L4	FBC L4
0xE1C7	FF R4	FB R4	FBC R4
	AiroThru		
0xE1D0	AiroThru L1		
0xE1D1	AiroThru R1		
0xE1D2	AiroThru L2		
0xE1D3	AiroThru R2		
0xE1D4	AiroThru L3		
0xE1D5	AiroThru R3		

下图是把 FF L1 的设置复制给 FF L3



而导出的 NVR 文件中 Tool UI 的值是保存了 Config tool 上的设置，与代码无关

config tool配置

NVR 文件

ANC

ANC Default Setting

ANC FF

ANC FB

ANC FBC

AiroThru

Limiter

ANC FB Distortion Predictor

ANC Noise Floor Prediction

EE40 EE42 EE44 EE46

FF L1 FF L2 FF L3 FF L4

EE41 EE43 EE45 EE47

FB L1 FB L2 FB L3 FB L4

EE48 EE4A EE4C EE4E

FB R1 FB R2 FB R3 FB R4

EE49 EE4B EE4D EE4F

FBC L1 FBC L2 FBC L3 FBC L4

EE50 EE52 EE54 EE56

FBC R1 FBC R2 FBC R3 FBC R4

EE51 EE53 EE55 EE57

AiroThru L1 AiroThru L2 AiroThru L3

EE58 EE5A EE5C

AiroThru R1 AiroThru R2 AiroThru R3

EE59 EE5B EE5D

```
// Tool UI - ANC_FF_L_1
&0xEE40 = 003C000100370
// Tool UI - ANC_FF_R_1
&0xEE41 = 003C000100370
// Tool UI - ANC_FF_L_2
&0xEE42 = 00010000038C0
// Tool UI - ANC_FF_R_2
&0xEE43 = 00010000038C0
// Tool UI - ANC_FF_L_3
&0xEE44 = 00010000038C0
// Tool UI - ANC_FF_R_3
&0xEE45 = 00010000038C0
// Tool UI - ANC_FF_L_4
&0xEE46 = 00010000038C0
// Tool UI - ANC_FF_R_4
&0xEE47 = 00010000038C0
// Tool UI - ANC_FB_L_1
&0xEE48 = 00C8000103460
// Tool UI - ANC_FB_R_1
&0xEE49 = 00C8000103460
// Tool UI - ANC_FB_L_2
&0xEE4A = 0000000103A10
// Tool UI - ANC_FB_R_2
&0xEE4B = 0000000103A10
// Tool UI - ANC_FB_L_3
&0xEE4C = 00E8030003850
// Tool UI - ANC_FB_R_3
&0xEE4D = 00E8030003850
// Tool UI - ANC_FB_L_4
&0xEE4E = 00E8030003850
// Tool UI - ANC_FB_R_4
&0xEE4F = 00E8030003850
// Tool UI - AiroThru_L_1
&0xEE58 = 0000000102DB007
// Tool UI - AiroThru_R_1
&0xEE59 = 0000000102B400C
// Tool UI - AiroThru_L_2
&0xEE5A = 0100000000C8002
// Tool UI - AiroThru_R_2
&0xEE5B = 0000000000C8002
// Tool UI - AiroThru_L_3
&0xEE5C = 0100000000C8002
// Tool UI - AiroThru_R_3
&0xEE5D = 0100000000C8002
```

NVR 文件、download 文件夹下的 Syskey.xml 文件和工程中的 Nvkey.xml 文件有共同的滤波器。那么把 Syskey.xml 变更的参数更新（复制粘贴）到 Nvkey.xml 文件即可

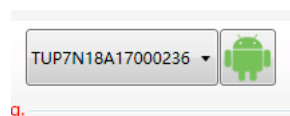


二十六、手机 Airoha_SDK_UT 升级固件

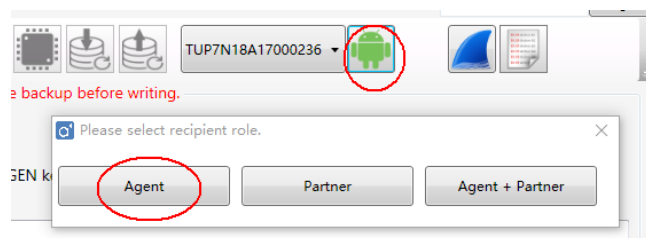
在手机的设置》系统》关于手机，不断点击版本号，就会进入开发者模式
 在手机的设置》系统》开发人员选项，打开 USB 调试
 把手机用 USB 线连接到电脑，设备管理器会出现连接手机接口



打开 config tool 软件，可以看到手机设备



点击小绿人，选 Agent



此时手机上提示安装 AirohaRelay，允许安装即可
 手机与耳机进行蓝牙配对连接，然后点击小绿人，点击 Agent

耳机开机后，系统先运行 bootloader 里的 main 函数，再执行 CM4（即 mcu）的 main 函数
 ...\\mcu\\project\\ab1565_ab1568_evk\\apps\\bootloader\\src 下的 main.c 文件

mcu > project > ab1565_ab1568_evk > apps > bootloader > src			
名称	修改日期	类型	大
 bl_dbgprint	2021/8/31 2:14	C 文件	
 bl_fota	2021/8/31 2:14	C 文件	
 bl_fota_flash_ctrl	2021/8/31 2:14	C 文件	
 bl_fota_upgrade	2021/8/31 2:14	C 文件	
 bl_fota_util	2021/8/31 2:14	C 文件	
 bl_rom_parameter	2021/8/31 2:14	C 文件	
 custom_blconfig	2021/8/31 2:14	C 文件	
 custom_fota	2021/8/31 2:14	C 文件	
 main	2021/8/31 2:14	C 文件	

```

00316: int main()
00317: {
00318:     bl_hardware_init();
00319:
00320:     #ifdef BL_FOTA_ENABLE
00321:         bl_fota_process();
00322:     #endif
00323:
00324:     bl_start_user_code();
00325:
00326:     return 0;
00327: }
00328:

```

...\mcu\project\ab1565_ab1568_evk\apps\headset_ref_design\src (headset_ref_design 是工程) 下的 main.c 文件

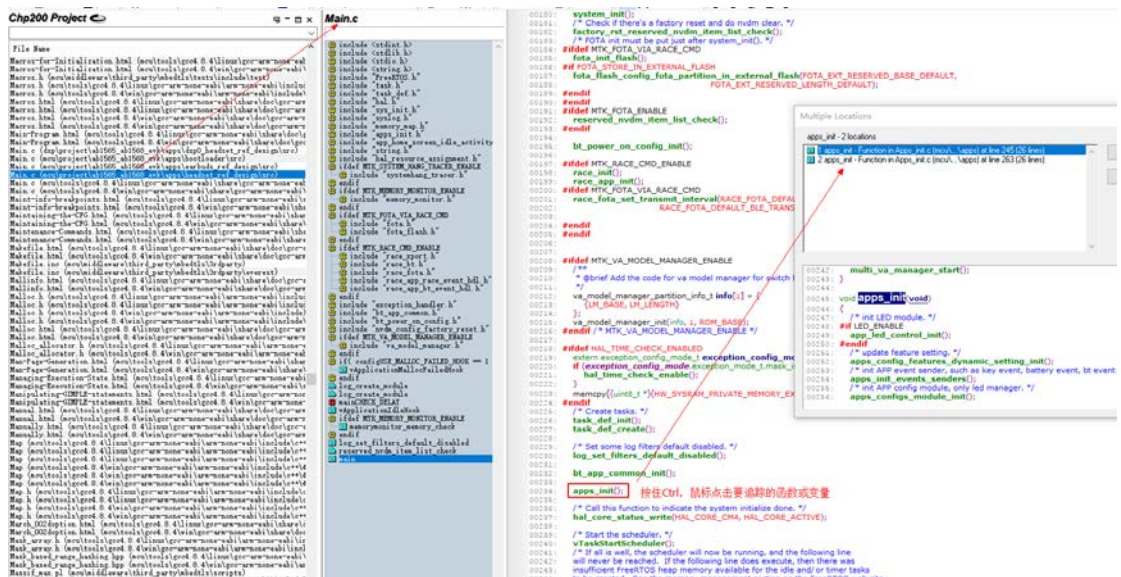
IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset > mcu > project > ab1565_ab1568_evk > apps > headset_ref_design > src			
名称	修改日期	类型	大小
 apps	2021/8/31 2:14	文件夹	
 boards	2021/8/31 2:14	文件夹	
 bt_app_utility	2021/8/31 2:14	文件夹	
 iap2	2021/8/31 2:14	文件夹	
 race	2021/8/31 2:14	文件夹	
 bt_init	2021/8/31 2:14	C 文件	21 KB
 dcxo_capid	2021/8/31 2:14	C 文件	4 KB
 main	2021/8/31 2:14	C 文件	9 KB
 nvdm_config	2021/8/31 2:14	C 文件	9 KB
 nvdm_config_factory_reset	2021/8/31 2:14	C 文件	9 KB
 regions_init	2021/8/31 2:14	C 文件	6 KB
 sys_init	2021/8/31 2:14	C 文件	13 KB
 system_ab156x	2021/8/31 2:14	C 文件	7 KB
 task_def	2021/10/28 10:33	C 文件	10 KB
 task_def.c.orig	2021/10/27 15:59	ORIG 文件	10 KB
 task_def.c.rej	2021/10/27 15:59	REJ 文件	1 KB

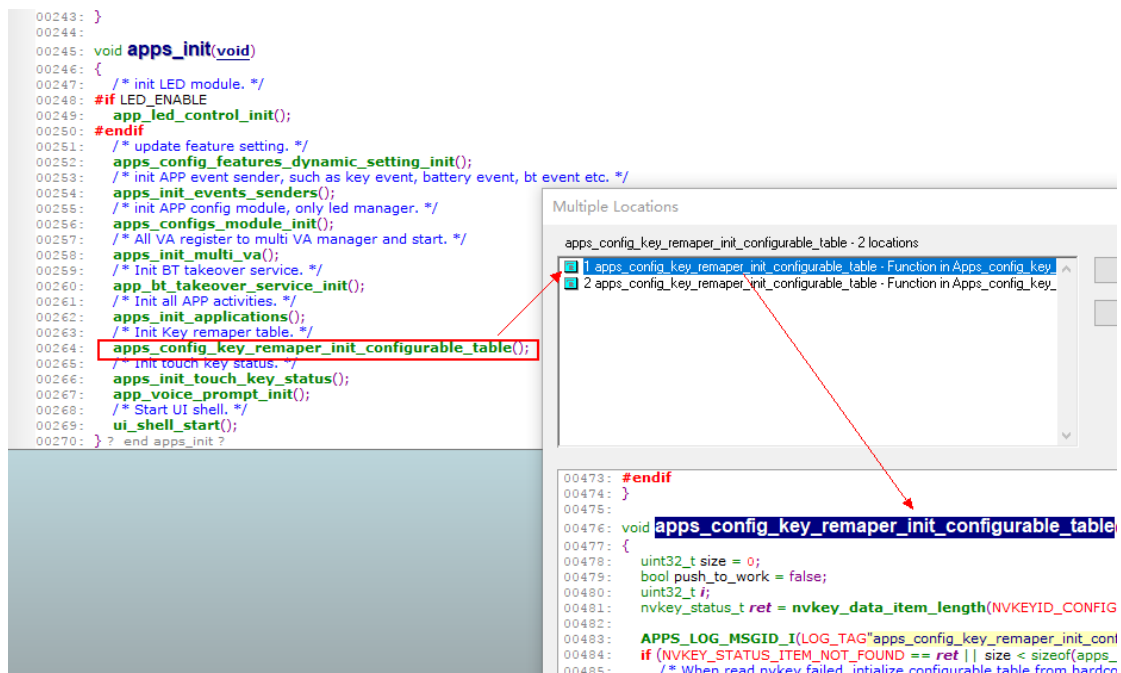
```

00172: */
00173: int main(void)
00174: {
00175:     #ifndef REBOOT_AFTER_EXCEPTION
00176:     exception_dump_config(DISABLE_WHILELOOP_MAGIC);
00177:     #endif
00178:
00179:     /* Do system initialization, eg: hardware, nvdm. */
00180:     system_init(); 系统初始化
00181:     /* Check if there's a factory reset and do nvdm clear. */
00182:     factory_rst_reserved_nvdm_item_list_check(); 检查需要保留不被复位的NVDM项
00183:     /* FOTA init must be put just after system_init(). */
00184:     #ifdef MTK_FOTA_VIA_RACE_CMD
00185:     fota_init_flash(); 初始化OTA的Flash配置
00186:     #if FOTA_STORE_IN_EXTERNAL_FLASH
00187:     fota_flash_config_fota_partition_in_external_flash(FOTA_EXT_RESERVED_BASE_DEFAULT,
00188:     FOTA_EXT_RESERVED_LENGTH_DEFAULT);
00189:     #endif
00190:     #endif
00191:     #ifdef MTK_FOTA_ENABLE
00192:     reserved_nvdm_item_list_check(); 检查FOTA需要保留不被升级的NVDM配置选项
00193:     #endif
00194:
00195:     bt_power_on_config_init(); 配置蓝牙模块工作模式，主要有DUT模式和正常模式
00196:
00197:     #ifdef MTK_RACE_CMD_ENABLE
00198:     race_init();
00199:     race_app_init(); UART RACE,BT TACE CMD模块初始化
00200:     #ifdef MTK_FOTA_VIA_RACE_CMD
00201:     race_fota_set_transmit_interval(RACE_FOTA_DEFAULT_SPP_TRANSMIT_INTERVAL_IN_MS,
00202:     RACE_FOTA_DEFAULT_BLE_TRANSMIT_INTERVAL_IN_MS);
00203:     #endif
00204:     #endif
00205:     #endif
00206:
00207:     #ifdef MTK_VA_MODEL_MANAGER_ENABLE
00208:     /**
00209:     * @brief Add the code for va model manager for switch language model.
00210:     */
00211:     va_model_manager_partition_info_t info[1] = {
00212:     {LM_BASE, LM_LENGTH}
00213:     };
00214:     va_model_manager_init(info, 1, ROM_BASE); 语音指令模块初始化
00215:     #endif /* MTK_VA_MODEL_MANAGER_ENABLE */
00216:
00217:     #ifdef HAL_TIME_CHECK_ENABLED
00218:     extern exception_config_mode_t exception_config_mode;
00219:     if (exception_config_mode.exception_mode_t.mask_irq_check_assert) {
00220:     hal_time_check_enable();
00221:     }
00222:     memcopy((uint8_t *) (HW_SYSRAM_PRIVATE_MEMORY_EXCEPTION_START + HW_SYSRAM_PRIVATE_MEMORY_EXCEPTION_LEN - sizeof
00223:     #endif
00224:     /* Create tasks. */
00225:     task_def_init();
00226:     task_def_create(); 系统模式task创建及初始化配置
00227:
00228:     /* Set some log filters default disabled. */
00229:     log_set_filters_default_disabled();
00230:
00231:     bt_app_common_init(); BLE, SPP,BT DEVICE管理等蓝牙app应用层配置
00232:
00233:     apps_init(); UI, KEY,LED,VP等应用模块初始化
00234:
00235:     /* Call this function to indicate the system initialize done. */
00236:     hal_core_status_write(HAL_CORE_CM4, HAL_CORE_ACTIVE);
00237:
00238:     /* Start the scheduler. */
00239:     vTaskStartScheduler(); 启动FREETROS系统任务管理器，开始各个任务调度运行
00240:     /* If all is well, the scheduler will now be running, and the following line
00241:     will never be reached. If the following line does execute, then there was
00242:

```

用 source insight 工程打开源代码，可以很方便地追踪代码

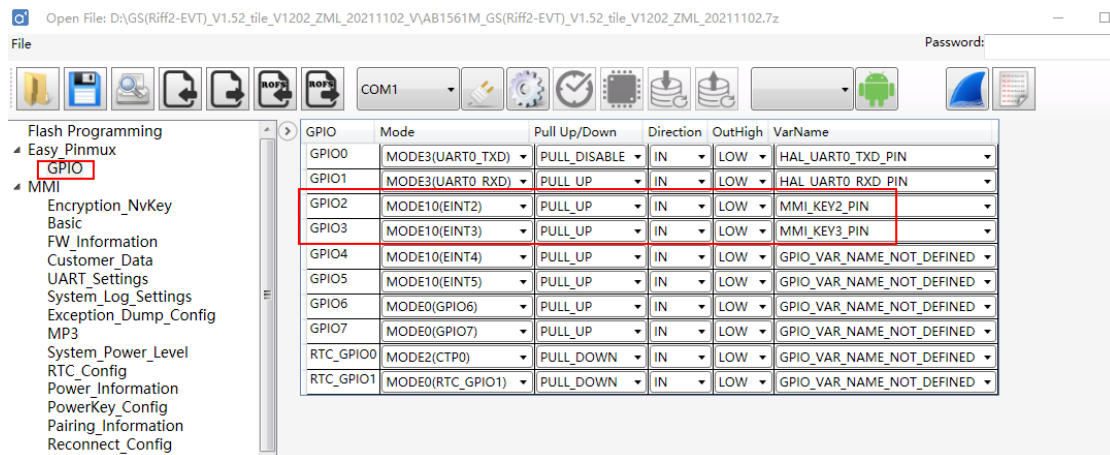




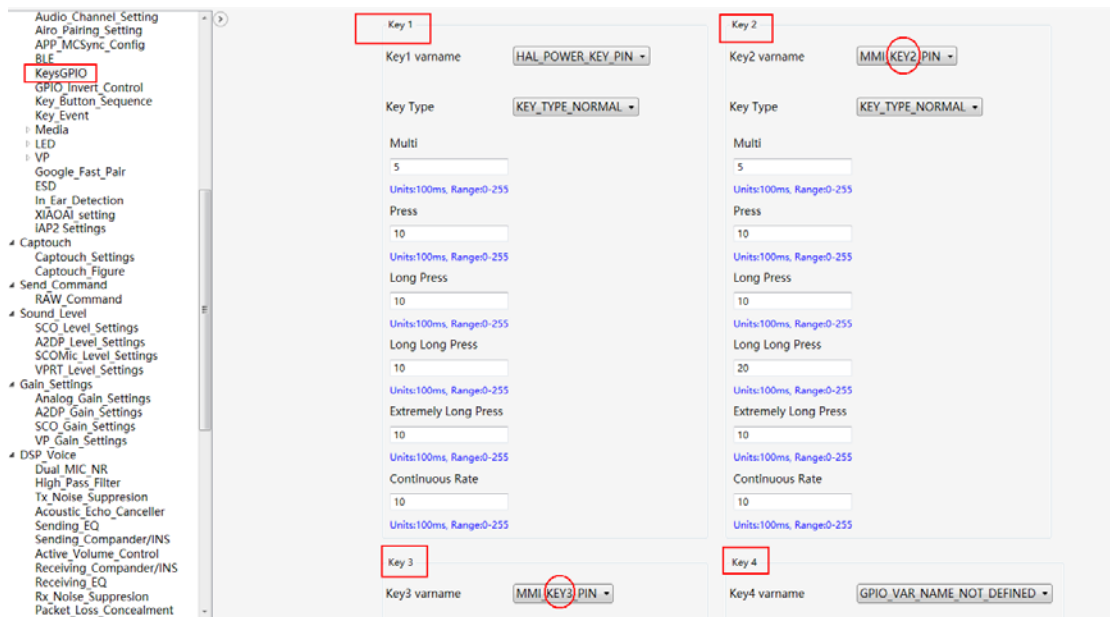
二十八、硬件 GPIO 与 EPT 工具配置对应关系

AB1562 的 GPIO 配置

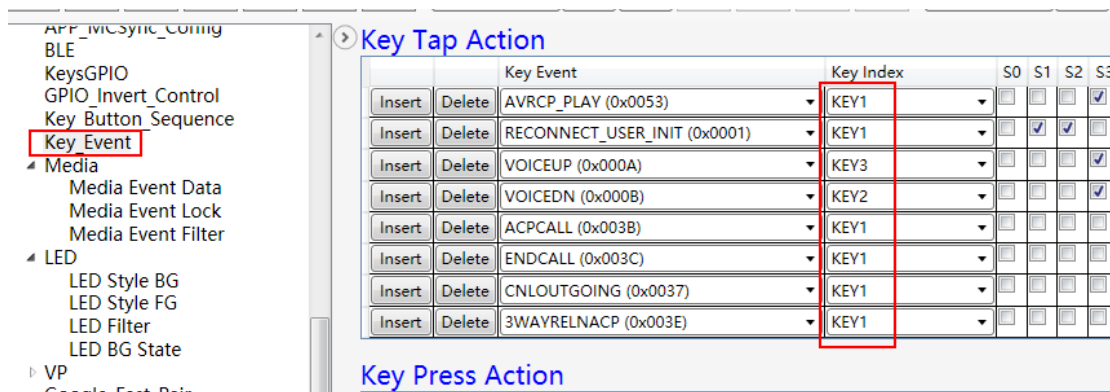
用 config tool 工具打开配置 7z 文件，在 Easy Pinmux > GPIO 可以看到被使用的 GPIO 被定义的名称，最多 10 个 GPIO 可自定义使用，Mode 是该 GPIO 能配置的所有模式（取决于硬件本身），VarName 是 GPIO 的管脚别名。



在 MMI > KeysGPIO 可以看到 Key 的相关配置（名称、类型、按下释放时间等）

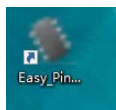


在 MMI > Key Event 中 Key Index 的 KEY1、KEY2 等对应 Key1、Key2 等

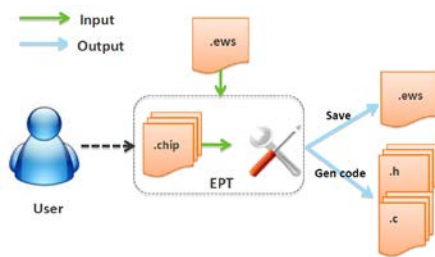


AB1568 的 GPIO 配置

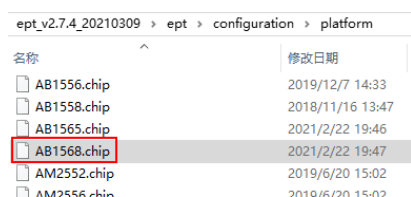
需要用 Easy_PinMux_Tool 工具打开 ews 文件



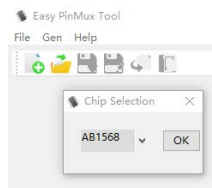
参考文件 Easy_PinMux_Tool_Users_Guide.pdf 熟悉工具的使用



chip 文件里是对应芯片的固定定义，不能更改



在 EPT 文件夹下的文件



打开工程下的 ews 文件（ept workspace，即 ept 工程）

IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset > mcu > project > ab1565_ab1568_evk > apps > headset_ref_design > ept_config > ab1568_evk

名称	修改日期	类型	大小
1568_IOT_SDK_05131031	2021/8/31 2:14	EWS 文件	17 KB
1568_IOT_SDK_headset_20201230	2021/11/9 9:47	EWS 文件	15 KB

GPIO 表（General Purpose Input Output，通用输入输出，实际物理管脚），文件 ept_gpio_drv.h 保存 GPIO 的相关设置

GPIO	EINT	Keypad	Voltage	Mode	Pull Up/Down	Direction	OutHigh	VarName	Comments
GPIO0			1.8V	NC	PD-150K_ohms				No User
GPIO1			1.8V	NC	PD-150K_ohms				No User
GPIO2			1.8V	NC	PD-150K_ohms				No User
GPIO3			1.8V	NC	PD-150K_ohms				No User
GPIO4			1.8V	NC	PD-150K_ohms				No User
GPIO5			1.8V	NC	PD-150K_ohms				No User
GPIO6			1.8V	NC	PD-150K_ohms				No User
GPIO7			1.8V	NC	PD-150K_ohms				No User
GPIO8			1.8V	NC	PD-150K_ohms				No User
GPIO9			1.8V	NC	PD-150K_ohms				No User
GPIO10			1.8V	NC	PD-150K_ohms				No User
GPIO11			1.8V	NC	PD-150K_ohms				No User
GPIO12			1.8V	NC	PD-150K_ohms				No User
GPIO13			1.8V	NC	PD-120K_ohms				No User
GPIO14			1.8V	NC	PD-120K_ohms				No User
GPIO15			1.8V	NC	PD-120K_ohms				No User
GPIO16			1.8V	NC	PD-120K_ohms				No User
GPIO17			1.8V	NC	PD-120K_ohms				No User
GPIO18			1.8V	NC	PD-120K_ohms				No User
GPIO19			1.8V	NC	PD-120K_ohms				No User
GPIO20			1.8V	NC	PD-150K_ohms				No User
GPIO21			1.8V	NC	PD-150K_ohms				No User
GPIO22			1.8V	NC	PD-150K_ohms				No User
GPIO23			1.8V	MODE_9	9:EINT23				Used for EINT23
GPIO24			1.8V	MODE_9	9:EINT24				Used for EINT24
GPIO25			1.8V	MODE_9	9:EINT25				Used for EINT25
GPIO26			1.8V	NC	PD-150K_ohms				No User

对应原理图



每个 GPIO 的可配置模式 Mode 是有限的（由硬件本身决定），对应 chip 文件的说明

GPIO	EINT	Keypad	Voltage	Mode	Pull Up/Down	Direction	OutHigh	VarName
GPIO0			1.8V	9:EINT0	PD-150K_ohms			
GPIO1			1.8V	NC	PD-150K_ohms			
GPIO2			1.8V	NC	PD-150K_ohms			
GPIO3			1.8V	NC	PD-150K_ohms			
GPIO4			1.8V	NC	PD-150K_ohms			
GPIO5			1.8V	NC	PD-150K_ohms			
GPIO6			1.8V	NC	PD-150K_ohms			
GPIO7			1.8V	NC	PD-150K_ohms			
GPIO8			1.8V	NC	PD-150K_ohms			
GPIO9			1.8V	NC	PD-150K_ohms			
GPIO10			1.8V	NC	PD-150K_ohms			
GPIO11			1.8V	NC	PD-150K_ohms			
GPIO12			1.8V	NC	PD-150K_ohms			
GPIO13			1.8V	NC	PD-120K_ohms			
GPIO14			1.8V	NC	PD-120K_ohms			
GPIO15			1.8V	NC	PD-120K_ohms			
GPIO16			1.8V	NC	PD-120K_ohms			
GPIO17			1.8V	NC	PD-120K_ohms			

HAL_UART0_DTE_DCD_PIN
HAL_UART0_DTE_DSR_PIN
HAL_UART0_DTE_DTR_PIN
HAL_UART0_DTE_RI_PIN
HAL_UART1_DTE_DCD_PIN
HAL_UART1_DTE_DSR_PIN
HAL_UART1_DTE_DTR_PIN
HAL_UART1_DTE_RI_PIN
HAL_UART2_DTE_DCD_PIN
HAL_UART2_DTE_DSR_PIN
HAL_UART2_DTE_DTR_PIN
HAL_UART2_DTE_RI_PIN
BSP_CTP_SW_I2C_SCL_PIN
BSP_CTP_SW_I2C_SDA_PIN
BSP_CTP_RST_PIN
BSP_CTP_EINT_PIN
BSP_SPEAKER_ENABLE_PIN
BSP_AUXADC_ENABLE_PIN
BSP_GNSS_POWER_PIN

GPIO的别名

同时在 chip 文件内找到

```
[GPIO_variables]
; The RD (EPT user) should set their own variable name.
HAL_UART0_DTE_DCD_PIN
HAL_UART0_DTE_DSR_PIN
HAL_UART0_DTE_DTR_PIN
HAL_UART0_DTE_RI_PIN
HAL_UART1_DTE_DCD_PIN
HAL_UART1_DTE_DSR_PIN
HAL_UART1_DTE_DTR_PIN
HAL_UART1_DTE_RI_PIN
HAL_UART2_DTE_DCD_PIN
HAL_UART2_DTE_DSR_PIN
HAL_UART2_DTE_DTR_PIN
HAL_UART2_DTE_RI_PIN
BSP_CTP_SW_I2C_SCL_PIN
BSP_CTP_SW_I2C_SDA_PIN
BSP_CTP_RST_PIN
BSP_CTP_EINT_PIN
BSP_SPEAKER_ENABLE_PIN
BSP_AUXADC_ENABLE_PIN
BSP_GNSS_POWER_PIN
BSP_BIO_SENSOR_PPG_VDRV_EN
BSP_BIO_SENSOR_32K
BSP_BIO_SENSOR_RST_PORT_PIN
BSP_BIO_SENSOR_AFE_PWD_PIN
BSP_WIFI_26M_PIN
BSP_WIFI_RESET_PIN
BSP_WIFI_ENABLE_PIN
BSP_HP_ANA_SW_PIN
UPDATE_HOST_STATUS_PIN
QUERY_MODEM_STATUS_PIN
NOTIFY_MODEM_EXCEPTION_PIN
TRIGGER_MODEM_RESET_PIN
NOTIFY_MODEM_WAKEUP_PIN
TRIGGER_MODEM_START_PIN
TRIGGER_MODEM_WAKEUP_PIN
EXT_NTC_POWER_SW_PIN
EXT_NTC_ADC_PIN
EXT_DSP_POWER_SW_PIN
EXT_DSP_CLK_SW_PIN
EXT_DSP_RTC_CLK_SW_PIN
EXT_DSP_32K_CLK_PIN
NOTIFY_EXT_DSP_IRQ_PIN
BSP_LED0_PIN
BSP_LED1_PIN
BSP_LED2_PIN
BSP_EINTKEY0_PIN
BSP_EINTKEY1_PIN
BSP_EINTKEY2_PIN
BSP_EINTKEY3_PIN
BSP_EINTKEY4_PIN
BSP_GSENSOR_KEY_PIN
BSP_LINE_IN_DET_PIN
BSP_DETACHABLE_MIC_DET_PIN
BSP_IRSENSOR_KEY_PIN
BSP_ROTARY_ENCODER0_PIN_A
BSP_ROTARY_ENCODER0_PIN_B
BSP_ROTARY_ENCODER1_PIN_A
BSP_ROTARY_ENCODER1_PIN_B
BSP_ROTARY_ENCODER2_PIN_A
BSP_ROTARY_ENCODER2_PIN_B
BSP_PSENSOR_KEY_EVENT_NOTIFY_PIN
BSP_PSENSOR_KEY_POWER_PIN
```

```
00070:
00071: extern const char HAL_UART0_DTE_DCD_PIN;
00072: extern const char HAL_UART0_DTE_DSR_PIN;
00073: extern const char HAL_UART0_DTE_DTR_PIN;
00074: extern const char HAL_UART0_DTE_RI_PIN;
00075: extern const char HAL_UART1_DTE_DCD_PIN;
00076: extern const char HAL_UART1_DTE_DSR_PIN;
00077: extern const char HAL_UART1_DTE_DTR_PIN;
00078: extern const char HAL_UART1_DTE_RI_PIN;
00079: extern const char HAL_UART2_DTE_DCD_PIN;
00080: extern const char HAL_UART2_DTE_DSR_PIN;
00081: extern const char HAL_UART2_DTE_DTR_PIN;
00082: extern const char HAL_UART2_DTE_RI_PIN;
00083: extern const char BSP_CTP_SW_I2C_SCL_PIN;
00084: extern const char BSP_CTP_SW_I2C_SDA_PIN;
00085: extern const char BSP_CTP_RST_PIN;
00086: extern const char BSP_CTP_EINT_PIN;
00087: extern const char BSP_SPEAKER_ENABLE_PIN;
00088: extern const char BSP_AUXADC_ENABLE_PIN;
00089: extern const char BSP_GNSS_POWER_PIN;
00090: extern const char BSP_BIO_SENSOR_PPG_VDRV_EN;
00091: extern const char BSP_BIO_SENSOR_32K;
00092: extern const char BSP_BIO_SENSOR_RST_PORT_PIN;
00093: extern const char BSP_BIO_SENSOR_AFE_PWD_PIN;
00094: extern const char BSP_WIFI_26M_PIN;
00095: extern const char BSP_WIFI_RESET_PIN;
00096: extern const char BSP_HP_ANA_SW_PIN;
00097: extern const char UPDATE_HOST_STATUS_PIN;
00098: extern const char QUERY_MODEM_STATUS_PIN;
00099: extern const char NOTIFY_MODEM_EXCEPTION_PIN;
00100: extern const char TRIGGER_MODEM_RESET_PIN;
00101: extern const char NOTIFY_MODEM_WAKEUP_PIN;
00102: extern const char TRIGGER_MODEM_START_PIN;
00103: extern const char TRIGGER_MODEM_WAKEUP_PIN;
00104: extern const char EXT_NTC_POWER_SW_PIN;
00105: extern const char EXT_NTC_ADC_PIN;
00106: extern const char EXT_DSP_POWER_SW_PIN;
00107: extern const char EXT_DSP_CLK_SW_PIN;
00108: extern const char EXT_DSP_RTC_CLK_SW_PIN;
00109: extern const char EXT_DSP_32K_CLK_PIN;
00110: extern const char NOTIFY_EXT_DSP_IRQ_PIN;
00111: extern const char BSP_LED0_PIN;
00112: extern const char BSP_LED1_PIN;
00113: extern const char BSP_LED2_PIN;
00114: extern const char BSP_EINTKEY0_PIN;
00115: extern const char BSP_EINTKEY1_PIN;
00116: extern const char BSP_EINTKEY2_PIN;
00117: extern const char BSP_EINTKEY3_PIN;
00118: extern const char BSP_EINTKEY4_PIN;
00119: extern const char BSP_GSENSOR_KEY_PIN;
00120: extern const char BSP_LINE_IN_DET_PIN;
00121: extern const char BSP_DETACHABLE_MIC_DET_PIN;
00122: extern const char BSP_IRSENSOR_KEY_PIN;
00123: extern const char BSP_ROTARY_ENCODER0_PIN_A;
00124: extern const char BSP_ROTARY_ENCODER0_PIN_B;
00125: extern const char BSP_ROTARY_ENCODER1_PIN_A;
00126: extern const char BSP_ROTARY_ENCODER1_PIN_B;
00127: extern const char BSP_ROTARY_ENCODER2_PIN_A;
00128: extern const char BSP_ROTARY_ENCODER2_PIN_B;
00129: extern const char BSP_PSENSOR_KEY_EVENT_NOTIFY_PIN;
00130: extern const char BSP_PSENSOR_KEY_POWER_PIN;
00131:
00132:
00133: extern const char HAL_I2C_MASTER_0_SCL_PIN;
00134: extern const char HAL_I2C_MASTER_0_SDA_PIN;
00135: extern const char HAL_I2C_MASTER_1_SCL_PIN;
00136: extern const char HAL_I2C_MASTER_1_SDA_PIN;
00137: extern const char HAL_I2C_MASTER_2_SCL_PIN;
00138: extern const char HAL_I2C_MASTER_2_SDA_PIN;
00139: extern const char HAL_I2C_A0_SCL_PIN;
00140: extern const char HAL_I2C_A0_SDA_PIN;
```

被设置来使用的 GPIO

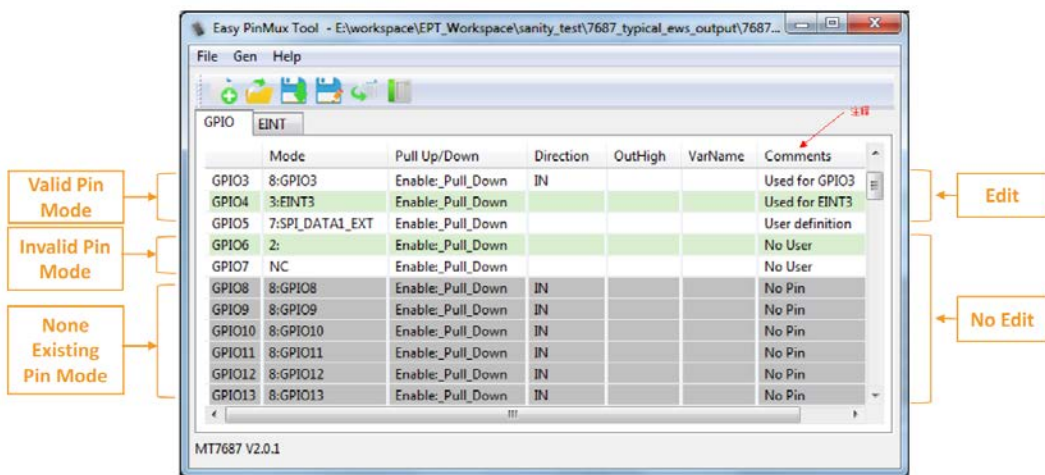
GPIO	EINT	Keypad	Voltage	Mode	Pull Up/Down	Direction	OutHigh	VarName	Comments
GPIO8			1.8V	NC	PD-150K_ohms				No User
GPIO9			1.8V	NC	PD-150K_ohms				No User
GPIO10			1.8V	NC	PD-150K_ohms				No User
GPIO11			1.8V	NC	PD-150K_ohms				No User
GPIO12			1.8V	NC	PD-150K_ohms				No User
GPIO13			1.8V	NC	PD-120K_ohms				No User
GPIO14			1.8V	NC	PD-120K_ohms				No User
GPIO15			1.8V	NC	PD-120K_ohms				No User
GPIO16			1.8V	NC	PD-120K_ohms				No User
GPIO17			1.8V	NC	PD-120K_ohms				No User
GPIO18			1.8V	NC	PD-120K_ohms				No User
GPIO19			1.8V	NC	PD-120K_ohms				No User
GPIO20			1.8V	NC	PD-150K_ohms				No User
GPIO21			1.8V	NC	PD-150K_ohms				No User
GPIO22			1.8V	NC	PD-150K_ohms				No User
GPIO23			1.8V	9:EINT23	PD-150K_ohms				Used for EINT23
GPIO24			1.8V	9:EINT24	PD-150K_ohms				Used for EINT24
GPIO25			1.8V	9:EINT25	PD-150K_ohms				Used for EINT25
GPIO26			1.8V	NC	PD-150K_ohms				No User
GPIO27			1.8V	0:GPIO27	PD-150K_ohms	IN			No Pin

在 ept_gpio_var.h 文件中会增加被设置的变量名

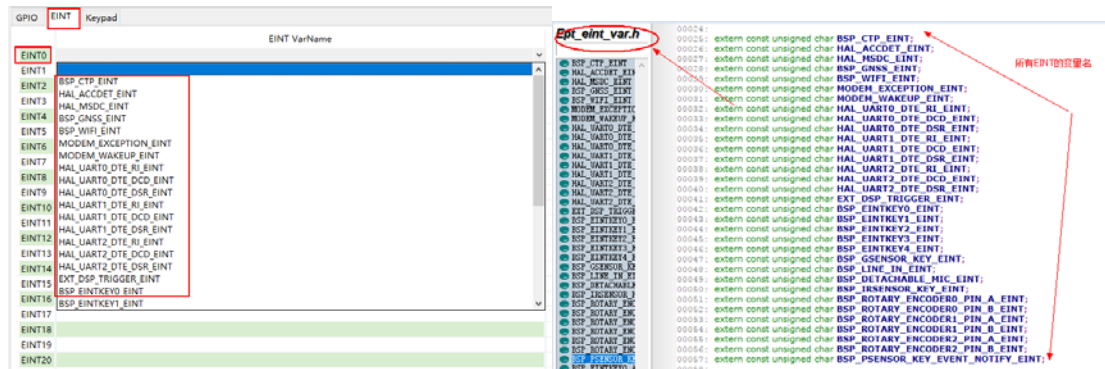

```

Ept_gpio_var.h
00022:
00023:
00024:
00025: extern const char BSP_ROTARY_ENCODER_PIN_B;
00026: extern const char BSP_ROTARY_ENCODER_PIN_B_M_GPIO;
00027: extern const char BSP_ROTARY_ENCODER_PIN_B_M_SPI_MST2_MOSI;
00028: extern const char BSP_ROTARY_ENCODER_PIN_B_M_I2S_MST2_WS;
00029: extern const char BSP_ROTARY_ENCODER_PIN_B_M_I2S_SLV2_WS;
00030: extern const char BSP_ROTARY_ENCODER_PIN_B_M_DMIC1_DAT1;
00031: extern const char BSP_ROTARY_ENCODER_PIN_B_M_UART1_RTS;
00032: extern const char BSP_ROTARY_ENCODER_PIN_B_M_ESC_MOSI;
00033: extern const char BSP_ROTARY_ENCODER_PIN_B_M_DSP_JTCK;
00034: extern const char BSP_ROTARY_ENCODER_PIN_B_M_AP_IRSTB;
00035: extern const char BSP_ROTARY_ENCODER_PIN_B_M_EINT;
00036: extern const char BSP_ROTARY_ENCODER_PIN_B_M_DEBUGMON7;
00037:
00038:
00039: extern const char BSP_ROTARY_ENCODER_PIN_A;
00040: extern const char BSP_ROTARY_ENCODER_PIN_A_M_GPIO;
00041: extern const char BSP_ROTARY_ENCODER_PIN_A_M_SPI_MST2_SIO2;
00042: extern const char BSP_ROTARY_ENCODER_PIN_A_M_I2S_MST2_CLK;
00043: extern const char BSP_ROTARY_ENCODER_PIN_A_M_I2S_SLV2_CLK;
00044: extern const char BSP_ROTARY_ENCODER_PIN_A_M_DMIC0_CLK;
00045: extern const char BSP_ROTARY_ENCODER_PIN_A_M_SPI_MST0_CS1;
00046: extern const char BSP_ROTARY_ENCODER_PIN_A_M_ESC_SIO2;
00047: extern const char BSP_ROTARY_ENCODER_PIN_A_M_DSP_JTDI;
00048: extern const char BSP_ROTARY_ENCODER_PIN_A_M_AP_JTDO;
00049: extern const char BSP_ROTARY_ENCODER_PIN_A_M_EINT;
00050: extern const char BSP_ROTARY_ENCODER_PIN_A_M_DEBUGMON8;
00051:
00052:
00053: extern const char BSP_EINTKEY3_PIN;
00054: extern const char BSP_EINTKEY3_PIN_M_GPIO;
00055: extern const char BSP_EINTKEY3_PIN_M_SPI_MST2_SIO3;
00056: extern const char BSP_EINTKEY3_PIN_M_I2S_MST2_CLK;
00057: extern const char BSP_EINTKEY3_PIN_M_I2S_SLV2_CLK;
00058: extern const char BSP_EINTKEY3_PIN_M_DMIC0_DAT1;
00059: extern const char BSP_EINTKEY3_PIN_M_SPI_MST0_CS2;
00060: extern const char BSP_EINTKEY3_PIN_M_ESC_SIO3;
00061: extern const char BSP_EINTKEY3_PIN_M_DSP_JKST;
00062: extern const char BSP_EINTKEY3_PIN_M_I2C2_SDA;
00063: extern const char BSP_EINTKEY3_PIN_M_EINT;
00064: extern const char BSP_EINTKEY3_PIN_M_DEBUGMON9;
00065:
00066:
00067:

```



EINT 表，（External Interrupt，外部中断），EINT VarName 是 GPIO 的 EINT 模式下的别名，灰色表示不能自定义，在 GPIO 表中的 GPIO 的 EINT 模式被打开，才能设置 EINT VarName



在 chip 文件找到对应说明

```

[EINT_variables]
; The sample "TEST_VAR_NAME1" is for reference only, and is not actually used during driver code.
; The RO (EPT user) should set their own variable name.
BSP_CTP_EINT
HAL_ACCDET_EINT
HAL_MSDC_EINT
BSP_GNSS_EINT
BSP_WIFI_EINT
MODEM_EXCEPTION_EINT
MODEM_WAKEUP_EINT
HAL_UART0_DTE_RL_EINT
HAL_UART0_DTE_DCD_EINT
HAL_UART0_DTE_DSR_EINT
HAL_UART1_DTE_RL_EINT
HAL_UART1_DTE_DCD_EINT
HAL_UART1_DTE_DSR_EINT
HAL_UART2_DTE_RL_EINT
HAL_UART2_DTE_DCD_EINT
HAL_UART2_DTE_DSR_EINT
EXT_DSP_TRIGGER_EINT
BSP_EINTKEY0_EINT
BSP_EINTKEY1_EINT
BSP_EINTKEY2_EINT
BSP_EINTKEY3_EINT
BSP_GSENSOR_KEY_EINT
BSP_LINE_IN_EINT
BSP_DETACHABLE_MIC_EINT
BSP_IRSENSOR_KEY_EINT
BSP_ROTARY_ENCODER0_PIN_A_EINT
BSP_ROTARY_ENCODER0_PIN_B_EINT
BSP_ROTARY_ENCODER1_PIN_A_EINT
BSP_ROTARY_ENCODER1_PIN_B_EINT
BSP_ROTARY_ENCODER2_PIN_A_EINT
BSP_ROTARY_ENCODER2_PIN_B_EINT
BSP_PSENSOR_KEY_EVENT_NOTIFY_EINT

```

配置 GPIO 参数后，EPT 可以生成（点击 Gen > GenCode）src 和 inc 代码文件到程序里

名称	修改日期	类型	大小
airo_key_config	2021/8/31 2:14	H 文件	12 KB
apps_customer_config	2021/8/31 2:14	H 文件	6 KB
ept_eint_drv	2021/10/27 15:57	H 文件	1 KB
ept_eint_var	2021/10/27 15:57	H 文件	2 KB
ept_gpio_drv	2021/10/27 15:57	H 文件	11 KB
ept_gpio_var	2021/10/27 15:57	H 文件	14 KB
ept_keypad_drv	2021/10/27 15:57	H 文件	1 KB
hal_feature_config	2021/8/31 2:14	H 文件	5 KB
mfi_apio_config	2021/8/31 2:14	H 文件	3 KB

被配置的 EINT 引脚在文件 ept_eint_var.c 中看到

GPIO	EINT	Keypad
EINT12		
EINT13		
EINT14		
EINT15		
EINT16		
EINT17		
EINT18		
EINT19		
EINT20		
EINT21		
EINT22		
EINT23	BSP_ROTARY_ENCODER0_PIN_B_EINT	
EINT24	BSP_ROTARY_ENCODER0_PIN_A_EINT	
EINT25	BSP_EINTKEY3_EINT	
EINT26		

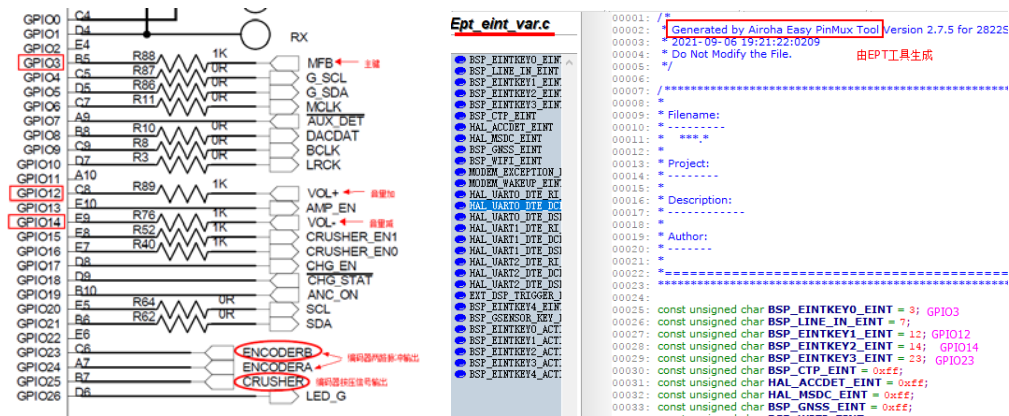
Ept_eint_var.c

00022: * 00023: 00024: 00025: const unsigned char BSP_ROTARY_ENCODER0_PIN_B_EINT = 23; 00026: const unsigned char BSP_ROTARY_ENCODER0_PIN_A_EINT = 24; 00027: const unsigned char BSP_EINTKEY3_EINT = 25; 00028: const unsigned char BSP_CTP_EINT = 0xff; 00029: const unsigned char HAL_ACCDET_EINT = 0xff; 00030: const unsigned char HAL_MSDC_EINT = 0xff; 00031: const unsigned char BSP_GNSS_EINT = 0xff;

打开原理图，查看音量键、主键、编码器、拨动开关、振动喇叭连接哪个 GPIO，可以在文件 ept_eint_var.c 中定义了 EINT 与实际管脚的映射关系，可以把 GPIO 管脚号映射互换过来。例如，管脚 GPIO3 只能连接到 EINT3，而 EINT3 可以有很多个变量名

GPIO	1.8V	9:EINT3	EINT3	BSP_EINTKEY0_EINT
GPIO4	1.8V	0:GPIO3	EINT4	HAL_UART1_DTE_DSR_EINT
GPIO5	1.8V	1:AP_JTMS	EINT5	HAL_UART2_DTE_RL_EINT
GPIO6	1.8V	2:UART0_CTS	EINT6	HAL_UART2_DTE_DCD_EINT
GPIO7	1.8V	3:DMIC0_DAT	EINT7	HAL_UART2_DTE_DSR_EINT
GPIO8	1.8V	4:UART2_RXD		EXT_DSP_TRIGGER_EINT
GPIO9	1.8V	5:SPI_MST2_CS0	EINT8	BSP_EINTKEY0_EINT
GPIO10	1.8V	6:PAD_DFUNK_SIN	EINT9	BSP_EINTKEY1_EINT
GPIO11	1.8V	7:AP_JTDI	EINT10	BSP_EINTKEY2_EINT
GPIO12	1.8V	8:CTP1	EINT11	BSP_EINTKEY3_EINT
		9:EINT3	EINT12	BSP_EINTKEY4_EINT
		10:DEBUGMON3	EINT13	BSP_GSENSOR_KEY_EINT
				BSP_LINE_IN_EINT
				BSP_DETACHABLE_MIC_EINT

ept_keypad_drv.h 文件宏定义 POWERKEY_POSITION 为 DEVICE_KEY_POWER



ept_gpio_var.c 文件是由 EPT 工具的以下配置生成的，定义变量名对应的具体管脚号

GPIO	EINT	Keypad	Voltage	Mode	Pull Up/Down	Direction	OutHigh	VarName	Comments
GPIO0			1.8V	1:UART0_TXD	PU-150K_ohms			HAL_UART0_TXD_PIN	Used for UART0_TXD
GPIO1			1.8V	1:UART0_RXD	PU-150K_ohms			HAL_UART0_RXD_PIN	Used for UART0_RXD
GPIO2			1.8V	NC	PD-150K_ohms				No User
GPIO3			1.8V	9:EINT3	PD-150K_ohms			BSP_EINTKEY0_PIN	Used for EINT3
GPIO4			1.8V	2:UART1_TXD	Disable			HAL_UART1_TXD_PIN	Used for UART1_TXD
GPIO5			1.8V	2:UART1_RXD	PU-150K_ohms			HAL_UART1_RXD_PIN	Used for UART1_RXD
GPIO6			1.8V	4:I2S_MST0_MCLK	Disable			HAL_I2S_MST0_MCLK_PIN	Used for I2S_MST0_MCLK
GPIO7			1.8V	0:GPIO7	Disable	IN		BSP_LINE_IN_DET_PIN	Used for GPIO7
GPIO8			1.8V	4:I2S_MST0_TX	Disable			HAL_I2S_MST0_TX_PIN	Used for I2S_MST0_TX
GPIO9			1.8V	4:I2S_MST0_CK	Disable			HAL_I2S_MST0_CK_PIN	Used for I2S_MST0_CK
GPIO10			1.8V	4:I2S_MST0_WS	Disable			HAL_I2S_MST0_WS_PIN	Used for I2S_MST0_WS
GPIO11			1.8V	4:I2S_MST0_RX	PD-150K_ohms			HAL_I2S_MST0_RX_PIN	Used for I2S_MST0_RX
GPIO12			1.8V	9:EINT12	PD-150K_ohms			BSP_EINTKEY1_PIN	Used for EINT12
GPIO13			1.8V	NC	PD-120K_ohms				No User
GPIO14			1.8V	9:EINT14	PD-120K_ohms			BSP_EINTKEY2_PIN	Used for EINT14
GPIO15			1.8V	NC	PD-120K_ohms				No User
GPIO16			1.8V	NC	PD-120K_ohms				No User
GPIO17			1.8V	NC	PD-120K_ohms				No User
GPIO18			1.8V	NC	PD-120K_ohms				No User
GPIO19			1.8V	NC	PD-120K_ohms				No User
GPIO20			1.8V	NC	PD-150K_ohms				No User
GPIO21			1.8V	NC	PD-150K_ohms				No User
GPIO22			1.8V	NC	PD-150K_ohms				No User
GPIO23			1.8V	9:EINT23	PD-150K_ohms			BSP_EINTKEY3_PIN	Used for EINT23
GPIO24			1.8V	NC	PD-150K_ohms				No User
GPIO25			1.8V	NC	PD-150K_ohms				No User
GPIO26			1.8V	NC	PD-150K_ohms				No User
GPIO27			1.8V	0:GPIO27	PD-150K_ohms	IN			No Pin

ept_eint_var.c 文件是由 EPT 工具的以下配置生成的 (EINT VarName 与 GPIO 的 VarName 是没有任何关联)，EINT3 绑定 GPIO3。

GPIO	EINT	Keypad	EINT VarName
EINT0			
EINT1			
EINT2			
EINT3			BSP_EINTKEY0_EINT
EINT4			
EINT5			
EINT6			
EINT7			
EINT8			
EINT9			
EINT10			
EINT11			
EINT12			BSP_EINTKEY1_EINT
EINT13			
EINT14			BSP_EINTKEY2_EINT
EINT15			
EINT16			
EINT17			
EINT18			
EINT19			
EINT20			
EINT21			
EINT22			
EINT23			BSP_EINTKEY3_EINT
EINT24			
EINT25			
EINT26			
EINT27			
EINT28			Used

```
const unsigned char BSP_EINTKEY0_EINT = 3;
const unsigned char BSP_EINTKEY1_EINT = 12;
const unsigned char BSP_EINTKEY2_EINT = 14;
const unsigned char BSP_EINTKEY3_EINT = 23;
```

GPIO3 – EINT3 – KEY2 主键

GPIO12 – EINT12 – KEY0 音量加

GPIO14 – EINT14 – KEY1 音量减

目前所有按键都是经过 EINT 方式执行的。

GPIO	EINT	Keypad		GPIO	EINT	Keypad
		Voltage	Mode			
GPIO0	1.8V	1:UART0_TXD	Disabl	EINT0		EINT VarName
GPIO1	1.8V	1:UART0_RXD	PU-15i	EINT1		
GPIO2	1.8V	NC	PD-15i	EINT2		
GPIO3	1.8V	2:UART1_TXD	Disabl	EINT3	BSP_EINTKEY0_EINT	
GPIO4	1.8V	2:UART1_RXD	PU-15i	EINT4		
GPIO5	1.8V	4:I2S_MST0_MCLK	Disabl	EINT5		
GPIO6	1.8V	0:GPIO7	Disabl	EINT6		
GPIO7	1.8V	4:I2S_MST0_TX	Disabl	EINT7		
GPIO8	1.8V	4:I2S_MST0_CK	Disabl	EINT8		
GPIO9	1.8V	4:I2S_MST0_WS	Disabl			
GPIO10	1.8V	4:I2S_MST0_DV	PD-15i			
GPIO11	1.8V					

提示：EINT3 与 BSP_EINTKEY0_EINT 中的数字是没有绝对关系的，可任意对应，EINTx 中的数字范围是 0-26，BSP_EINTKEYx_EINT 中的数字范围是 0-4（在 chip 文件中决定）

ab1568_evk 开发板 GPIO3 定义为音量加 KEY0，GPIO12 定义为音量减 KEY1，GPIO14 定义为主键 KEY2

但是现有的头戴耳机定义的 UI 是音量加键接 GPIO12，音量减键接 GPIO14，主键接 GPIO3 所以修改中断对应的别名即可

GPIO3 – EINT3 – KEY2 主键

GPIO12 – EINT12 – KEY0 音量加

GPIO14 – EINT14 – KEY1 音量减

GPIO	EINT	Keypad
EINT0		
EINT1		
EINT2		
EINT3	BSP_EINTKEY2_EINT	
EINT4		
EINT5		
EINT6		
EINT7		
EINT8		
EINT9		
EINT10		
EINT11		
EINT12	BSP_EINTKEY0_EINT	
EINT13		
EINT14	BSP_EINTKEY1_EINT	

```

const unsigned char BSP_EINTKEY2_EINT = 3;
const unsigned char BSP_EINTKEY0_EINT = 12;
const unsigned char BSP_EINTKEY1_EINT = 14;

```

接下来一步一步追踪 GPIO 与按键编号值如何关联起来的：

在文件 bsp_eint_key_custom.c 中调用函数 BSP_EINTKEY_DRIVER_INIT 检查配置

<pre> Bsp_eint_key_custom.c #include "bsp_eint_key_custom.h" #include "bsp_eint_key.h" #include "bsp_eint_key_normal.h" #include "bsp_eint_key_rtc.h" #include "hal_gpio.h" #include "hal_log.h" #include "assert.h" #ifdef AIR_BSP_EINT_KEY_AC BSP_EINTKEY_DECLARE_AC BSP_EINTKEY_SET_ACTIVE else BSP_EINTKEY_DECLARE_AC BSP_EINTKEY_SET_ACTIVE #endif BSP_EINTKEY_DECLARE_EXTN #ifdef AIR_BSP_EINT_KEY_RT </pre>	<pre> 00112: /* config the GPIO port, eint mode, eint number, key data, active level */ 00113: 00114: 00115: void bsp_eint_key_custom_init(void) 00116: { 00117: uint32_t available_key_nums = 0; 00118: 00119: BSP_EINTKEY_DRIVER_INIT(0, available_key_nums); 00120: BSP_EINTKEY_DRIVER_INIT(1, available_key_nums); 00121: BSP_EINTKEY_DRIVER_INIT(2, available_key_nums); 00122: BSP_EINTKEY_DRIVER_INIT(3, available_key_nums); 00123: BSP_EINTKEY_DRIVER_INIT(4, available_key_nums); 00124: 00125: if (0 == available_key_nums) { 00126: LOG_MSGID_E(eint_key, "[eint_key]eint key pin has not been config"); 00127: assert(0); 00128: } 00129: } </pre>
<pre> Bsp_eint_key_custom.c #include "bsp_eint_key_custom.h" #include "bsp_eint_key.h" #include "bsp_eint_key_normal.h" #include "bsp_eint_key_rtc.h" #include "hal_gpio.h" #include "hal_log.h" #include "assert.h" #ifdef AIR_BSP_EINT_KEY_AC BSP_EINTKEY_DECLARE_AC BSP_EINTKEY_SET_ACTIVE else BSP_EINTKEY_DECLARE_AC BSP_EINTKEY_SET_ACTIVE #endif BSP_EINTKEY_DECLARE_EXTN #ifdef AIR_BSP_EINT_KEY_RT </pre>	<pre> 00089: #define BSP_EINTKEY_DRIVER_INIT(key_index, available_key_nums) 00090: { 00091: if (BSP_EINTKEY_KEY_INDEX_PIN != 0x0) { BSP_EINTKEY0_PIN等是引脚配置 00092: if (BSP_EINTKEY_KEY_INDEX_EINT != 0x0) { BSP_EINTKEY0_EINT等是按键配置 00093: bsp_eint_key_normal_gpio_driver_init(new_driver = bsp_eint_key_driver_buf[key_index].normal); 00094: new_driver->gpio_port = BSP_EINTKEY_KEY_INDEX_PIN; 00095: new_driver->driver.key_data = BSP_EINT_KEY_DATA0; 00096: BSP_EINTKEY_SET_ACTIVE_LEVEL(new_driver, BSP_EINTKEY_KEY_INDEX_ACTIVE_LEVEL); 00097: } 00098: } 00099: BSP_EINTKEY_RTC_GPIO_DRIVER_INIT(key_index); 00100: available_key_nums++; 00101: } 00102: 00103: </pre>

上述的函数里 BSP_EINT_KEY_DATA0 与

new_driver->gpio_port = BSP_EINTKEY_KEY_INDEX_PIN;\n
new_driver->driver.key_data = BSP_EINT_KEY_DATA0;\n


```

Bsp_eint_key_custom.h
00067: // #define BSP_EINT_KEY_TIMING_ENABLE
00068:
00069: /*user could modify for your keydata*/
00070: #define BSP_EINT_KEY_NUMBER 5 /*default supported key numbers*/
00071: #define BSP_EINT_KEY_DATA0 126 /*keyad defined by user, must be 125<key_data<254*/
00072: #define BSP_EINT_KEY_DATA1 127 /*keyad defined by user, must be 125<key_data<254*/
00073: #define BSP_EINT_KEY_DATA2 128 /*keyad defined by user, must be 125<key_data<254*/
00074: #define BSP_EINT_KEY_DATA3 129 /*keyad defined by user, must be 125<key_data<254*/
00075: #define BSP_EINT_KEY_DATA4 130 /*keyad defined by user, must be 125<key_data<254*/
00076:

```

用户可修改按键值

```

Hal_keypad_table.h
00171: #define DEVICE_KEY_F12 122
00172: #define DEVICE_KEY_BACK 123
00173: #define DEVICE_KEY_HOME 124
00174: #define MAX_DEVICE_KEYS 125
00175:
00176: #define EINT_KEY_0 126 /*eint key 0*/
00177: #define EINT_KEY_1 127 /*eint key 1*/
00178: #define EINT_KEY_2 128 /*eint key 2*/
00179: #define EINT_KEY_3 129 /*eint key 3*/
00180: #define EINT_KEY_4 130 /*eint key 4*/
00181: #define EINT_KEY_5 131 /*eint key 5*/
00182: #define EINT_KEY_6 132 /*eint key 6*/
00183: #define EINT_KEY_7 133 /*eint key 7*/
00184:

```

用户定义的按键值

范围 126-253

```

Bsp_eint_key_normal_gpio_driver_new
00132:
00133: void bsp_eint_key_normal_gpio_driver_new(
00134:     bsp_eint_key_normal_gpio_driver_t *new_driver, const char gpio_port, const char eint_mode, unsigned char eint_number)
00135: {
00136:     new_driver->gpio_port = gpio_port; GPIO号
00137:     new_driver->eint_mode = eint_mode; 中断输入模式
00138:     new_driver->eint_number = eint_number; 中断输入号
00139:     new_driver->driver_init = bsp_eint_key_normal_gpio_init;
00140:     new_driver->driver_enable = bsp_eint_key_normal_gpio_enable;
00141:     new_driver->driver_is_active = bsp_eint_key_normal_gpio_driver_is_active;
00142:     new_driver->driver_set_debounce_time_msec = bsp_eint_key_normal_gpio_set_debounce_time_msec;
00143:     bsp_eint_key_register_gpio_driver((bsp_eint_key_gpio_driver_t*)new_driver);
00144: }

```

```

Bsp_eint_key_normal_gpio.h
00047: /** @brief Normal GPIO Interface. */
00048: typedef struct {
00049:     bsp_eint_key_gpio_driver_t driver; /**< The common driver interface. */
00050:     hal_gpio_pin_t gpio_port; /**< GPIO port*/
00051:     uint8_t eint_mode; /**< GPIO eint mode*/
00052:     hal_eint_number_t eint_number; /**< eint number*/
00053: } bsp_eint_key_normal_gpio_driver_t;

```

```

extern void bsp_eint_key_normal_gpio_driver_new(
    bsp_eint_key_normal_gpio_driver_t *new_driver, const char gpio_port, const char eint_mode, unsigned char eint_number);
#define BSP_EINTKEY_DRIVER_INIT(key_index, available_key_nums)
{
    if (BSP_EINTKEY_KEY_INDEX == PIN) != 0x00) {
        if (BSP_EINTKEY_KEY_INDEX == EINT) {
            bsp_eint_key_normal_gpio_driver_new(new_driver, BSP_EINTKEY_KEY_INDEX == PIN, BSP_EINTKEY_KEY_INDEX == PIN_M_EINT, BSP_EINTKEY_KEY_INDEX == EINT);
            new_driver->gpio_port = BSP_EINTKEY_KEY_INDEX == PIN;
            new_driver->driver_key_data = BSP_EINT_KEY_DATA == key_index;
            BSP_EINTKEY_SET_ACTIVE_LEVEL(new_driver, BSP_EINTKEY_KEY_INDEX == key_index == ACTIVE_LEVEL);
        }
        BSP_EINTKEY_RTC_GPIO_DRIVER_INIT(key_index);
    }
    available_key_nums++;
}

```

该函数把GPIO号与EINT号关联起来了

“##”符号具有连接字符串的作用，把两个宏参数贴在一起，执行完A##B后变为AB

BSP_EINTKEY0_PIN = 3; BSP_EINTKEY0_PIN_M_EINT = 9; BSP_EINTKEY0_EINT = 3;

BSP_EINTKEY1_PIN = 12; BSP_EINTKEY1_PIN_M_EINT = 9; BSP_EINTKEY1_EINT = 12;

BSP_EINTKEY2_PIN = 14; BSP_EINTKEY2_PIN_M_EINT = 9; BSP_EINTKEY2_EINT = 14;

BSP_EINTKEY3_PIN = 23; BSP_EINTKEY3_PIN_M_EINT = 9; BSP_EINTKEY3_EINT = 23;

Ept_gpio_var.c Ept_gpio_var.c Ept_eint_var.c

```

Apps_config_key_remapper.c
00071: /* The structure defines the item of key mapping table. */
00072: typedef struct {
00073:     uint8_t key_event_map; /* Size of the mapping table */
00074:     uint8_t key_event_map_list; /* Pointer to the apps_config_key_event_map_list */
00075: } apps_config_key_event_map_item;
00076: /* Initialize the mapping table, if this is an avrt project, the variable apps_config_key_event_map_list is const */
00077: void bsp_eint_key_normal_gpio_driver_new(bsp_eint_key_normal_gpio_driver_t *new_driver, const char gpio_port, const char eint_mode, unsigned char eint_number)
00078: {
00079:     new_driver->gpio_port = gpio_port;
00080:     new_driver->eint_mode = eint_mode;
00081:     new_driver->eint_number = eint_number;
00082:     new_driver->driver_init = bsp_eint_key_normal_gpio_init;
00083:     new_driver->driver_enable = bsp_eint_key_normal_gpio_enable;
00084:     new_driver->driver_is_active = bsp_eint_key_normal_gpio_driver_is_active;
00085:     new_driver->driver_set_debounce_time_msec = bsp_eint_key_normal_gpio_set_debounce_time_msec;
00086:     bsp_eint_key_register_gpio_driver((bsp_eint_key_gpio_driver_t*)new_driver);
00087: }

```

```

Customized_key_config.c
00048:
00049:
00050: const apps_config_key_event_map temp_key_short_click_config[] = {
00051:     {
00052:         DEVICE_KEY_POWER,
00053:         KEY_ACCEPT_CALL,
00054:         (1 <= APP_HFP_INCOMING)
00055:     },
00056:     {
00057:         DEVICE_KEY_POWER,
00058:         KEY_SWAY_HOLD_ACTIVE_ACCEPT_OTHER,
00059:         (1 <= APP_HFP_TWIC_INCOMING)
00060:     },
00061: }

```

```

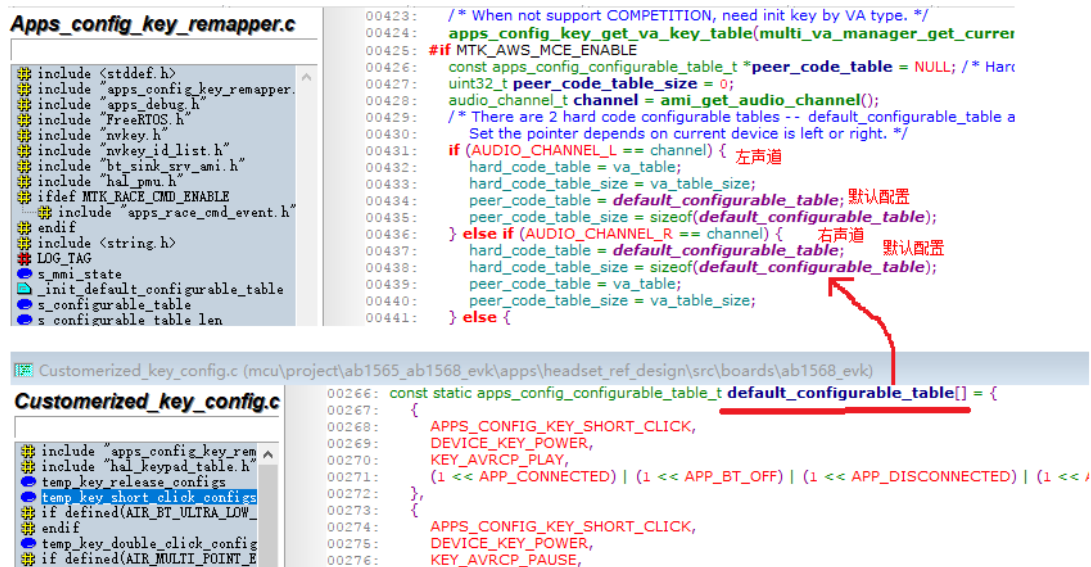
Customized_key_config.c
00048:
00049:
00050: const apps_config_key_event_map temp_key_short_click_config[] = {
00051:     {
00052:         DEVICE_KEY_POWER,
00053:         KEY_ACCEPT_CALL,
00054:         (1 <= APP_HFP_INCOMING)
00055:     },
00056:     {
00057:         DEVICE_KEY_POWER,
00058:         KEY_SWAY_HOLD_ACTIVE_ACCEPT_OTHER,
00059:         (1 <= APP_HFP_TWIC_INCOMING)
00060:     },
00061: }

```

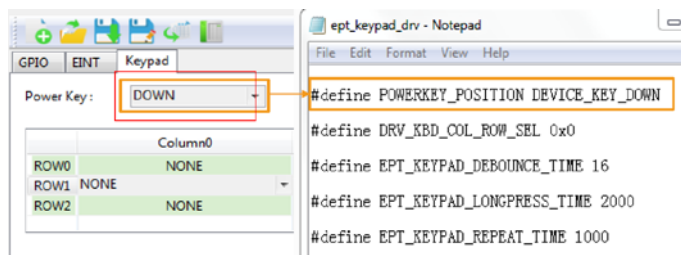
```

Apps_config_event_list.h
00052:
00053: KEY_DISCOVERABLE = 0x0000,
00054: KEY_CANCEL_DISCOVERABLE = 0x0001,
00055: KEY_VOICE_UP = 0x0002,
00056: KEY_VOICE_DOWN = 0x0003,
00057: KEY_BY_OFF = 0x0004,
00058: KEY_POWER_ON = 0x0005,
00059: KEY_POWER_OFF = 0x0006,
00060: KEY_SYSTEM_REBOOT = 0x0007,
00061: KEY_RESET_PAIRING_DEVICES = 0x0008,
00062: KEY_RECONNECT_LAST_DEVICE = 0x0009,

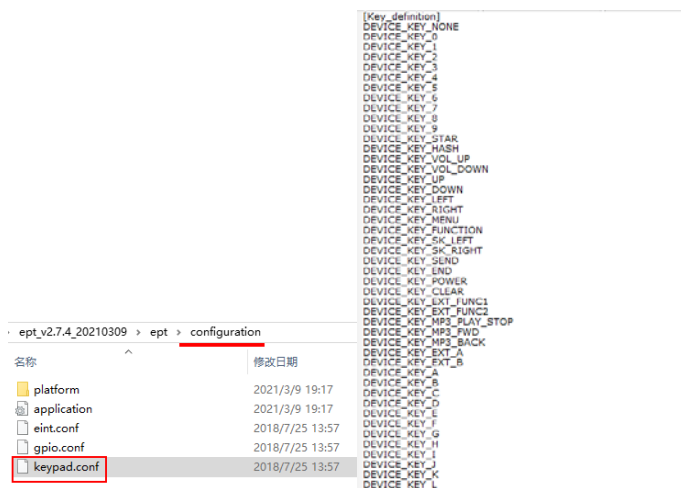
```



按键 Power Key（唯一固定存在的按键，其它添加的按键需要添加）的配置
Power Key 可以有多种别名



别名对应文件 keypad.conf 中找到



别名的编号在文件 hal_keypad_table.h 里定义，最下方是添加的按键别名
如 EINT_KEY_0、EINT_KEY_1、EINT_KEY_2、EINT_KEY_3、EINT_KEY_4、EINT_KEY_5

```

#ifdef __HAL_KEYPAD_TABLE_H_
#define __HAL_KEYPAD_TABLE_H_

#ifdef __cplusplus
extern "C" {
#endif

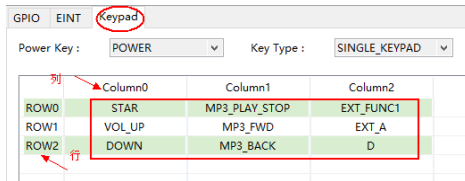
#define DEVICE_KEY_0 0
#define DEVICE_KEY_1 1
#define DEVICE_KEY_2 2
#define DEVICE_KEY_3 3
#define DEVICE_KEY_4 4
#define DEVICE_KEY_5 5
#define DEVICE_KEY_6 6
#define DEVICE_KEY_7 7
#define DEVICE_KEY_8 8
#define DEVICE_KEY_9 9
#define DEVICE_KEY_STAR 10
#define DEVICE_KEY_HASH 11
#define DEVICE_KEY_NONE 0xFF

#ifdef __cplusplus
}
#endif
}

#endif __HAL_KEYPAD_TABLE_H_

```

如果 key type 为 SINGLE_KEYPAD，那么可以设置为 3 行 3 列个键的键盘功能，每个键又有很多种别名



PowerKey 选为 POWER，则对应 DEVICE_KEY_POWER

```

Ept_keypad_drv.h      00029:      #define __EPT_CAPTOUCH_KEY__
00029:
00030:
00031:      #if defined(__EPT_CAPTOUCH_KEY__)
00032:      #define KEYPAD_MAPPING \
00033:      DEVICE_KEY_0, \
00034:      DEVICE_KEY_1, \
00035:      DEVICE_KEY_NONE, \
00036:      DEVICE_KEY_NONE
00037:      #endif
00038:
00039:      #define POWERKEY_POSITION DEVICE_KEY_POWER
00040:
00041:      #define DRV_KBD_CAPTOUCH_SEL 0x3
00042:
00043:
00044:      #endif /* __EPT_KEYPAD_DRV_H */

```

按键功能定制在文件 `customerized_key_config.c`

```

00274: #define temp_left_key_dlong_release_configs temp_key_dlong_release_configs
00275: // For right side or default configurable mapping table
00276: const static appcs_configurable_configuring_table_t default_configurable_table[] = {
00277:
00278:     {
00279:         APPS_CONFIG_KEY_SHORT_CLICK,
00280:         DEVICE_KEY_POWER,
00281:         KEY_AVRCP_PLAY,
00282:         1 // << APP_CONNECTED
00283:     },
00284:
00285:     {
00286:         APPS_CONFIG_KEY_SHORT_CLICK,
00287:         DEVICE_KEY_POWER,
00288:         KEY_AVRCP_PAUSE,
00289:         1 // << APP_A2DP_PLAYING | 1 // << APP_ULTRA_LOW_LATENCY_PLAYING | 1 // << APP_WIRED_HUS
00290:     },
00291:
00292:     {
00293:         APPS_CONFIG_KEY_DOUBLE_CLICK,
00294:         DEVICE_KEY_POWER,
00295:         KEY_AVRCP_FORWARD,
00296:         1 // << APP_A2DP_PLAYING | 1 // << APP_ULTRA_LOW_LATENCY_PLAYING | 1 // << APP_WIRED_HUS
00297:     },
00298:
00299:     {
00300:         APPS_CONFIG_KEY_TRIPLE_CLICK,
00301:         DEVICE_KEY_POWER,
00302:         KEY_AVRCP_BACKWARD,
00303:         1 // << APP_A2DP_PLAYING | 1 // << APP_ULTRA_LOW_LATENCY_PLAYING | 1 // << APP_WIRED_HUS
00304:     },
00305:
00306:     {
00307:         APPS_CONFIG_KEY_LONG_PRESS_RELEASE_2,
00308:         APPS_CONFIG_KEY_LONG_PRESS_RELEASE_2,
00309:         0
00310:     }
00311: };

```

按键行为被定义在文件 `apps_config_event_list.h`

```

000446: * This enum defines the key action.
000447: */
000448: typedef enum {
000449:     KEY_ACTION_INVALID = 0,           /*< invalid key action id. */
000500:
000501:     KEY_ACTION_BASE = 0x0001,        /*< Declare event base, all ac
000502:
000503:
000504:     KEY_DISCOVERABLE = 0x0002,       /*< Start BT discoverable. */
000505:     KEY_CANCEL_DISCOVERABLE = 0x0003, /*< Set BT to invisible. */
000506:     KEY_VOICE_UP = 0x0004,           /*< Volume up. */
000507:     KEY_VOICE_DN = 0x0005,           /*< Volume down. */
000508:     KEY_BT_OFF = 0x0006,             /*< BT off. */
000509:     KEY_POWER_ON = 0x0017,           /*< It's a SW power on, so pow
000510:     KEY_POWER_OFF = 0x0018,          /*< System power off, but if d
000511:     KEY_SYSTEM_REBOOT = 0x0019,      /*< Trigger system reboot. */
000512:     KEY_RESET_PAIRED_DEVICES = 0x001A, /*< Unpair all devices. */
000513:     KEY_RECONNECT_LAST_DEVICE = 0x001E, /*< Actively reconnect
000514:
000515:     KEY_WAKE_UP_VOICE_ASSISTANT = 0x0020, /*< Trigger voice ass
000516:     KEY_WAKE_UP_VOICE_ASSISTANT_CONFIRM = 0x0021, /*< Used with t
000517:                                     key long enough and releases key after the li
000518:                                     use the action to trigger voice assistant. */
000519:     KEY_WAKE_UP_VOICE_ASSISTANT_NOTIFY = 0x0022, /*< Notify user the
000520:
000521:     KEY_REDIAL_LAST_CALL = 0x0034,    /*< Redial last call. */
000522:     KEY_CANCEL_OUT_GOING_CALL = 0x0037, /*< Cancel outgoing call. */
000523:     KEY_REJCALL = 0x0038,             /*< Reject incoming call. */
000524:     KEY_REJCALL_SECOND_PHONE,         /*< Reject the new incoming call
000525:     KEY_ONHOLD_CALL = 0x003A,         /*< Hold current call or resume fro
000526:     KEY_ACCEPT_CALL = 0x003B,        /*< Accept incoming call. */
000527:     KEY_END_CALL = 0x003C,           /*< End current call. */
000528:
000529:
000530:
000531:
000532:
000533:
000534:
000535:
000536:
000537:
000538:
000539:
000540:
000541:
000542:
000543:
000544:
000545:
000546:
000547:
000548:
000549:
000550:
000551:
000552:
000553:
000554:
000555:
000556:
000557:
000558:
000559:
000560:
000561:
000562:
000563:
000564:
000565:
000566:
000567:
000568:
000569:
000570:
000571:
000572:
000573:
000574:
000575:
000576:
000577:
000578:
000579:
000580:
000581:
000582:
000583:
000584:
000585:
000586:
000587:
000588:
000589:
000590:
000591:
000592:
000593:
000594:
000595:
000596:
000597:
000598:
000599:
000600:
000601:
000602:
000603:
000604:
000605:
000606:
000607:
000608:
000609:
000610:
000611:
000612:
000613:
000614:
000615:
000616:
000617:
000618:
000619:
000620:
000621:
000622:
000623:
000624:
000625:
000626:
000627:
000628:
000629:
000630:
000631:
000632:
000633:
000634:
000635:
000636:
000637:
000638:
000639:
000640:
000641:
000642:
000643:
000644:
000645:
000646:
000647:
000648:
000649:
000650:
000651:
000652:
000653:
000654:
000655:
000656:
000657:
000658:
000659:
000660:
000661:
000662:
000663:
000664:
000665:
000666:
000667:
000668:
000669:
000670:
000671:
000672:
000673:
000674:
000675:
000676:
000677:
000678:
000679:
000680:
000681:
000682:
000683:
000684:
000685:
000686:
000687:
000688:
000689:
000690:
000691:
000692:
000693:
000694:
000695:
000696:
000697:
000698:
000699:
000700:
000701:
000702:
000703:
000704:
000705:
000706:
000707:
000708:
000709:
000710:
000711:
000712:
000713:
000714:
000715:
000716:
000717:
000718:
000719:
000720:
000721:
000722:
000723:
000724:
000725:
000726:
000727:
000728:
000729:
000730:
000731:
000732:
000733:
000734:
000735:
000736:
000737:
000738:
000739:
000740:
000741:
000742:
000743:
000744:
000745:
000746:
000747:
000748:
000749:
000750:
000751:
000752:
000753:
000754:
000755:
000756:
000757:
000758:
000759:
000760:
000761:
000762:
000763:
000764:
000765:
000766:
000767:
000768:
000769:
000770:
000771:
000772:
000773:
000774:
000775:
000776:
000777:
000778:
000779:
000780:
000781:
000782:
000783:
000784:
000785:
000786:
000787:
000788:
000789:
000790:
000791:
000792:
000793:
000794:
000795:
000796:
000797:
000798:
000799:
000800:
000801:
000802:
000803:
000804:
000805:
000806:
000807:
000808:
000809:
000810:
000811:
000812:
000813:
000814:
000815:
000816:
000817:
000818:
000819:
000820:
000821:
000822:
000823:
000824:
000825:
000826:
000827:
000828:
000829:
000830:
000831:
000832:
000833:
000834:
000835:
000836:
000837:
000838:
000839:
000840:
000841:
000842:
000843:
000844:
000845:
000846:
000847:
000848:
000849:
000850:
000851:
000852:
000853:
000854:
000855:
000856:
000857:
000858:
000859:
000860:
000861:
000862:
000863:
000864:
000865:
000866:
000867:
000868:
000869:
000870:
000871:
000872:
000873:
000874:
000875:
000876:
000877:
000878:
000879:
000880:
000881:
000882:
000883:
000884:
000885:
000886:
000887:
000888:
000889:
000890:
000891:
000892:
000893:
000894:
000895:
000896:
000897:
000898:
000899:
000900:
000901:
000902:
000903:
000904:
000905:
000906:
000907:
000908:
000909:
000910:
000911:
000912:
000913:
000914:
000915:
000916:
000917:
000918:
000919:
000920:
000921:
000922:
000923:
000924:
000925:
000926:
000927:
000928:
000929:
000930:
000931:
000932:
000933:
000934:
000935:
000936:
000
```

AB1568 EVK 的 SDK 默认的 key UI 功能在文档中有说明

AB1565 AB1568 Headset Reference Design User Guide.pdf

This section shows a mapping table of the keys, actions, LEDs, voice prompts, ring tones and any related comments for a specific function. For example, for the 'Power on' function, search for 'Power on' in the 'Functionality' column in Table 25. Key mapping table. The Key, Action, LED, Voice prompt, Ring tone, and comments that are associated with the 'Power on' function (i.e. Press for three seconds; LED rapidly flashes three seconds; Say "Power-On"; and "In the power off state") are shown in the adjacent cells on the same row.

Key	Functionality	Action	LED	Voice prompt	Comment
Power Key	Accept call	Press	LEDO is ON	NA	An incoming call is active.
	End call	Press	NA	"Call ended"	A call is active.
	Reject	Double-click	NA	"Call Rejected"	An incoming call is active.
	Hold	Double-click	LEDO blinks slowly	NA	During an active call.
	Hold and reject	Double-click	NA	"Call Rejected"	There is incoming call during an active call.
	Hold and accept	Press	NA	NA	There is another incoming when there is an active call, or there is an active call and another held call.
	End active and recover held	Press	NA	NA	An activity call and another hold call.
	End three-way call	Press	NA	NA	Three-way call is active.
	Music play	Press	NA	NA	In the connected state.
	Music pause	Press	NA	NA	In playing music state.
	Music forward	Double-click	NA	NA	In playing music state.
	Music backward	Triple-click	NA	NA	In playing music state.
	Power on	Long press for 3 seconds.	LEDO blink quickly; LED1 is OFF	"Power on"	When device is power off.
	Power off	Long press for 3 seconds.	LEDO blinks quickly; LED1 is OFF	"Power off"	When device is power on.
	Wake up voice assistant	Long press for 1 second and release before 3 seconds	NA	A short beep	When the device is connected to a smart phone.
EINT_KEY_0	Enter pairing mode	Double-click	LEDO and LED1 is on	"Enter pairing mode"	If AIR_MULTIPPOINT_ENABLE is set to "on", the key function works when the device is not connected to a smart phone or not in music playing or HFP status.
	Switch ANC and pass through	Double-click	NA	A short beep	When the device is connected to another side
EINT_KEY_1	Volume up	Press	NA	NA	In the connected, incoming/outgoing call, call active or playing music states.
	Switch audio path	Triple-click	NA	NA	Switch between A2DP and line in.
EINT_KEY_1	Volume down	Press	NA	NA	In the connected, incoming/outgoing call, call active, or playing music state.

测试时抓 log，发现按键对应的键值为

- 音量加 - 127
- 主键 - 126
- 音量减 - 128

在 keypad table 中对应

- 音量加 - 127 - EINT_KEY_1
- 主键 - 126 - EINT_KEY_0
- 音量减 - 128 - EINT_KEY_2

把音乐播放控制改到主键上


```

include "apps_config_key_remove"
include "air_multi_point_enable_k"
temp_key_release_configs
temp_key_short_click_configs
if defined(AIR_BT_ULTRA_LOW_L
endif
temp_key_double_click_configs
if defined(AIR_MULTI_POINT_EN
endif
endif
ifdef AIR_MCSYS_SHARE_ENABLE
else
endif
temp_key_triple_click_configs
if defined(OMTK_AWS_MCF_ENABLE
endif
if defined(SUPPORT_ROLE_HANDO
endif
ifdef APPS_LINE_IN_SUPPORT
endif
endif
temp_key_long_press1_configs
temp_key_long_press2_configs
temp_key_long_press3_configs
temp_key_key_bt_ultra_configs
if defined(AIR_BT_ULTRA_LOW_L
endif
endif
temp_key_dlong_configs
temp_key_repeat_configs
temp_key_press_configs
temp_key_long_press_release1_
temp_key_long_press_release2_
temp_key_long_press_release3_
temp_key_slong_release_config
temp_key_dlong_release_config
temp_left_key_release_configs
temp_left_key_short_click_co
temp_left_key_double_click_c
temp_left_key_triple_click_co
temp_left_key_long_press1_co
temp_left_key_long_press2_co
temp_left_key_long_press3_co
temp_left_key_dlong_configs
temp_left_key_dlong_configs
temp_left_key_repeat_configs
temp_left_key_press_configs
temp_left_key_long_press_rele
temp_left_key_long_press_rele
temp_left_key_long_press_rele
temp_left_key_dlong_release_c
temp_left_key_dlong_release_c
default configurable table
left_configurable table
ifdef AIR_MULTI_POINT_ENABLE
endif
ifdef MTK_WA_XIAOWEI_ENABLE

```

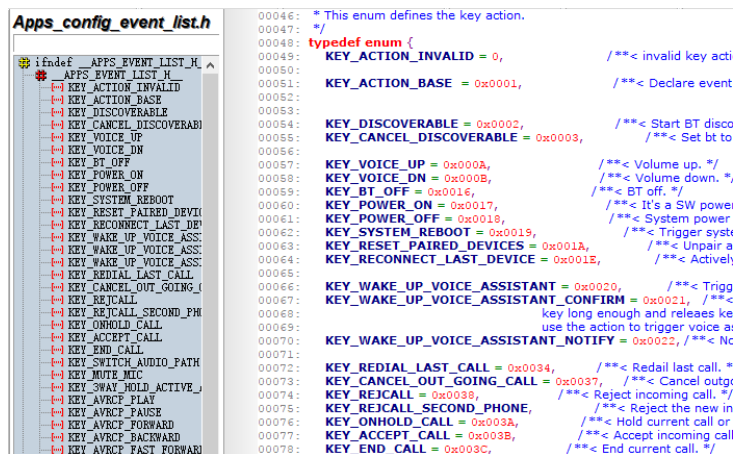
音量加、音量減

```

include "apps_config_key_remove"
include "hal_keypad_table.h"
temp_key_release_configs
if defined(AIR_BT_ULTRA_LOW_L)
endif
temp_key_double_click_configs
if defined(AIR_MULTI_POINT_EN)
endif
endif
ifdef AIR_MCSYSN_SHARE_ENABLE
else
temp_key_triple_click_configs
if defined(OTK_AWS_MCF_ENABLE)
endif
if defined(SUPPORT_ROLE_HANDO)
endif
ifdef AFPS_LINE_IN_SUPPORT
endif
temp_key_long_press1_configs
temp_key_long_press2_configs
temp_key_long_press3_configs
temp_key_long_press4_configs
if defined(AIR_BT_ULTRA_LOW_L)
else
endif
temp_key_down_configs
temp_key_repeat_configs
temp_key_press_configs
temp_key_long_press_release1_
temp_key_long_press_release2_
temp_key_long_press_release3_
temp_key_long_release_config
temp_key_double_release_config
temp_left_key_release_configs
temp_left_key_short_click_config
temp_left_key_double_click_co
temp_left_key_triple_click_co
temp_left_key_long_press1_con
temp_left_key_long_press2_con
temp_left_key_long_press3_con

```

按键事件在 apps config event list.h 文件中添加



二十九、Shell UI 消息分发管理机制

系统架构（应用层、中间层、驱动层）

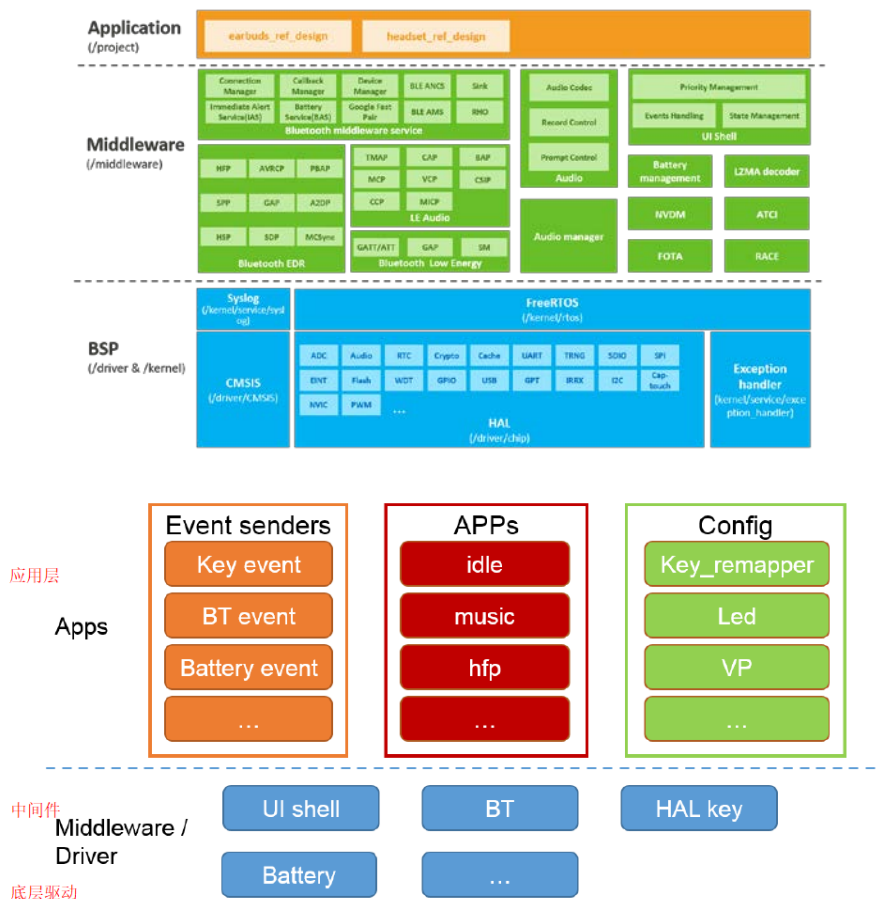


Figure 1. System architecture of 155X earbuds application layer

查阅文档 [Airoha_IoT_SDK_UI_Framework_Developers_Guide.pdf](#)

Shell UI 处理进程在文件 `apps_init.c` 中

路径是...\\mcu\\project\\ab1565_ab1568_evk\\apps\\headset_ref_design\\src\\apps

每个 UI 的 activity 都是由创建，销毁，暂停，恢复，更新等周期不同消息产生对应的处理方式在不同文件里（文件名包含 idle 字样的）

File Name
App_aws_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab156
App_aws_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab156
App_battery_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_battery_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_fast_pair_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_fm_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab156
App_fm_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab156
App_fota_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_fota_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_gsound_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_gsound_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_hfp_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_hfp_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_home_screen_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1
App_home_screen_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1
App_in_ear_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_in_ear_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_leakage_detection_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\proj
App_leakage_detection_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\proj
App_line_in_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_line_in_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_multi_vs_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab156
App_multi_vs_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab156
App_music_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_music_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_power_saving_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1
App_power_saving_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1
App_rhbc_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_rhbc_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565_ab1
App_smc_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_smc_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab1565
App_usb_audio_idle_activity.c (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab156
App_usb_audio_idle_activity.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\mcu\project\ab156

```
#include fota.h
#include "rwdm_config.h"
#include apps_debug.h
if defined(AIR_APP_SYSTEM_ON_BY_LINE_IN_ENABLE)
#include app_line_in_idle_activity.h
endif
// UI_SHELL_IDLE_BT_CONN_ACTIVITY
// POWER_OFF_TIMER_NAME
// WAIT_TIME_BEFORE_POWER_OFF
// RECONNECT_DEVICE_TIMES
// TIME_TO_RECONNECT_DEVICE
// TIME_TO_START_VISIBLE_AFTER_POWER_ON
// TIME_TO_STOP_RECONNECTION
// VISIBLE_TIMEOUT
// s_app_homescreen_context
// s_factory_reset_pending_event
// s_factory_reset_key_action
// s_factory_reset_doing
// s_ready_to_
// s_reconnect_device_idx
// RETRY_ENTER_RTC_MODE_TIMES
// app_home_screen_check_power_off_and_reboot
// app_home_screen_check_and_do_power_off
// trigger_reboot_flow
// ifdef MTK_AWC_ENABLE
// ifdef MTK_AWC_HOWLING_TURN_OFF_AWC
// app_home_screen_disable_anc_when_howling
// endif
// app_home_screen_process_anc_and_pass_through
// endif
// app_home_screen_fact_rst_rwdm_flag
// app_home_screen_fact_rst_link_key_rwdm_flag
// proc_ui_shell_group
// proc_key_event_group
// ifdef MTK_AWC_ENABLE
// homescreen_app_aws_data_proc
// homescreen_app_get_mmi_state
// homescreen_app_bt_connection_manager_event_proc
// ifdef AIR_BT_ULTRA_LOW_LATENCY_ENABLE
// homescreen_app_ull_event_proc
// endif
// homescreen_app_aws_event_proc
// app_interaction_event_proc
// homescreen_app_audio_anc_event_proc
// app_home_screen_idle_activity_proc
// app_home_screen_idle_activity_is_aws_connected

00130:
00131: void app_home_screen_check_power_off_and_reboot(void)
00132: {
00133:     if (!s_ready_to_off) {
00134:         if (APP_HOME_SCREEN_STATE_POWERING_OFF == s_app_homescreen_context.state) {
00135:             /* Enter RTC mode as power off action when no waiting and power_key released. */
00136:             uint32_t i;
00137:             /* Because some chip can display LED in RTC mode, must clear it before enter RTC mode. */
00138:             apps_config_led_disable_all();
00139:             for (i = 0; i++) {
00140:                 if (HAL_RTC_STATUS_ERROR == hal_rtc_enter_rtc_mode()) {
00141:                     APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", Enter RTC mode fail !!!", 0);
00142:                     if (i > RETRY_ENTER_RTC_MODE_TIMES) {
00143:                         configASSERT(0 && "Enter RTC mode fail !!!");
00144:                     } else {
00145:                         break;
00146:                     }
00147:                 }
00148:             }
00149:         } else if (APP_HOME_SCREEN_STATE_REBOOT == s_app_homescreen_context.state) {
00150:             /* Reset WDT as reboot action. */
00151:             hal_wdt_status_t ret;
00152:             APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", app_home_screen_check_and_do_powe
00153:             ret = hal_wdt_software_reset();
00154:             APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", app_home_screen_check_and_do_powe
00155:         }
00156:     } ? end if s_ready_to_off ?
00157: } ? end app_home_screen_check_power_off_and_reboot ?
00158:
00159: static void app_home_screen_check_and_do_power_off(homescreen_local_context_type_t *lo
00160: {
00161:     /* Must confirm BT off before system power off and reboot. */
00162:     if (local_context->bt_power_state == APP_HOME_SCREEN_BT_POWER_STATE_DISABLED) {
00163:         if (APP_HOME_SCREEN_STATE_POWERING_OFF == local_context->state
00164:             && !local_context->power_off_waiting_time_out
00165:             && local_context->power_off_waiting_release_key == DEVICE_KEY_NONE) {
00166:             APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", ready to system off", 0);
00167:             s_ready_to_off = true;
00168:         } else if (APP_HOME_SCREEN_STATE_REBOOT == local_context->state
00169:             && !local_context->power_off_waiting_time_out) {
00170:             APPS_LOG_MSGID_I(UI_SHELL_IDLE_BT_CONN_ACTIVITY", ready to reboot", 0);
00171:             s_ready_to_off = true;
00172:         }
00173:     }
}
```

UI 架构是在代码库里的一个叫 UI Shell 的中间件模块。它可以帮助应用开发者去设计和实现应用。要开发一个应用，开发者需要去处理该应用的状态机，同时根据预定义用户方案同步内部或应用之间。洛达 UI Shell 提供实现这些要求的功能：UI Shell 负责派发消息的

有优先级和回调函数的状态（活动）用于处理接收事件
系统架构有寄存器回调、发送事件、调度事件

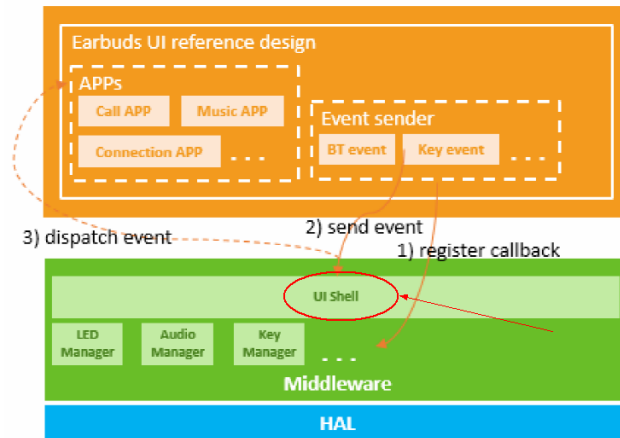


Figure 1. System architecture with UI framework

UI Shell 是中间件的一个模块

提供一个接口允许开发者去创建应用程序

不能直接调用其他中间件或 HAL 模块的 API

工程里的应用程序

寄存器 `proc_event` 函数在 UI Shell 启动时被使用

启动或停止 UI Shell 架构

创建活动或寄存器回调函数要运行其他中间件/HAL 模块或发送事件消息给 UI Shell

UI Shell 里的活动表示在 UI Shell 的有限状态机的一个状态，每个活动都有自己的回调函数去处理接收事件和决定做什么

应用程序可以调用函数去启用和停止一个新的活动。当一个活动被创建后，它就会被压入一个活动堆。



Figure 2. A new activity created

活动有多种类型

1、进程活动（每个工程都有且仅有一个，最高优先级）

2、空闲活动（每个应用程序可以有一个或更多空闲活动，空闲活动处理公共消息和开始短暂活动，生命周期活动是不能中断或失能的，除关机进程外）

3、短暂活动（在系统运行时由空闲活动或短暂活动创建，比空闲活动优先级高）

在 UI Shell 中有 7 种优先级：最高、高、中、低、最低、空闲顶、空闲低

活动堆：系统启动后，空闲活动 1 和 2 被创建，空闲活动 2 创建短暂活动 1，空闲活动 1 创建短暂活动 2，短暂活动 1 创建短暂活动 3

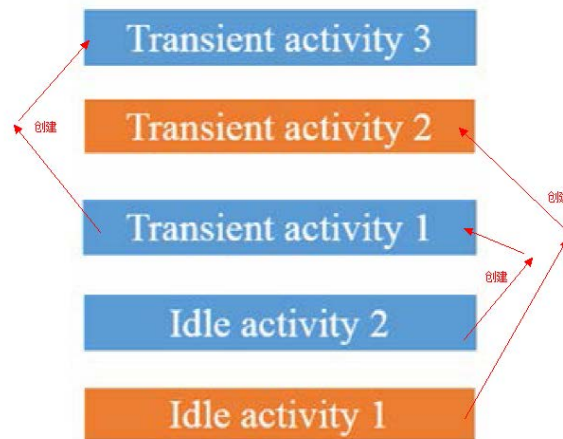


Figure 3. Stack example of two applications

活动分为主活动、空闲活动、短暂活动。开机后主活动和空闲活动被创建，空闲活动和短暂活动都可以创建短暂活动

UI Shell 采用事件消息进行同步。模块或应用程序可以发送事件给 UI Shell。UI Shell 会派送事件给已经在活动堆里的活动。事件可以分为内部事件和用户定义事件。

UI Shell 采用一个循环来处理事件，当一个事件被发送到 UI Shell,这个事件消息就会通过活动堆移动

当 UI Shell 任务启动后，它会检查在它内部序列里的事件消息，如果有事件消息，UI Shell 就会通过活动堆处理这个消息，如果序列里没有事件消息，UI Shell 任务进入等待，直到接收到新的消息

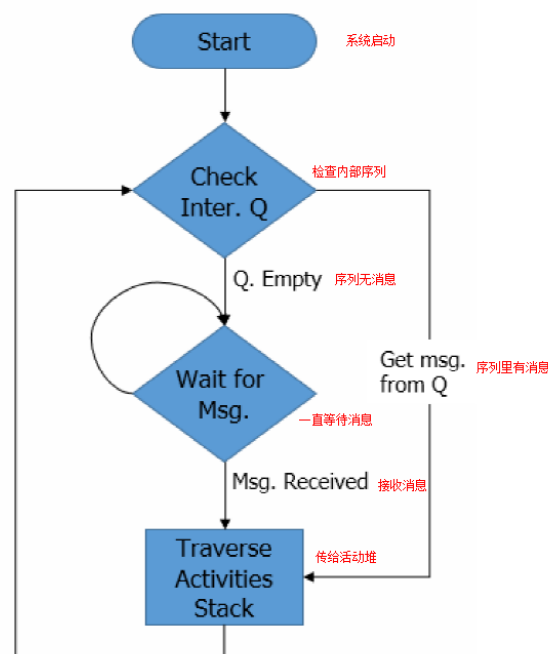


Figure 4. UI shell process event messages

消息来回移动

Pre-proc 活动最先处理事件，在活动堆里的短暂和空闲活动每次只被处理一个消息
一个事件消息是由在活动堆从顶到底的活动处理的，在活动里定义的回调函数在该活动处理

消息时被调用，回调函数的返回值（true 或 false）可以决定该事件消息是否必须派送到下一个活动。

内部事件：一个活动需要接收的事件（创建、暂停、恢复、销毁、更新、结果、请求允许、允许）

活动堆：每个活动被创建后都会被压入活动堆，优先级最高在堆顶，优先级最低在堆底。当模块或应用发消息给 UI Shell 时，UI Shell 会分配事件给存在活动堆里的活动进行处理

- EVENT_ID_SHELL_SYSTEM_ON_CREATE 创建
 - An activity receives this event when it is created.
- EVENT_ID_SHELL_SYSTEM_ON_PAUSE 暂停
 - When an activity is created at the top of the stack, the previous top activity receives this event.
- EVENT_ID_SHELL_SYSTEM_ON_RESUME 恢复
 - When a top activity is destroyed, the subsequent top activity becomes the top activity and receives this event.
- EVENT_ID_SHELL_SYSTEM_ON_DESTROY 销毁
 - When an activity must be destroyed, the activity receives this event before the activity is destroyed.
- EVENT_ID_SHELL_SYSTEM_ON_REFRESH 更新
 - When an activity is created at the top of stack or the second top activity becomes the top activity.
 - This event is received after EVENT_ID_SHELL_SYSTEM_ON_CREATE or when the EVENT_ID_SHELL_SYSTEM_ON_RESUME event processed.
- EVENT_ID_SHELL_SYSTEM_ON_RESULT 结果
 - When a branch activity sends data to its trunk activity, the trunk activity receives this event.
- EVENT_ID_SHELL_SYSTEM_ON_REQUEST_ALLOWANCE 请求恢复
 - When there is an activity request allowance, every activity receives the event. The activity can return true to allow the request.
- EVENT_ID_SHELL_SYSTEM_ON_ALLOW 允许
 - When an allowance request is allowed by all active activities, the activity which sends the request receives this event.

用户定义事件：在工程目录下的

```
...\mcu\project\ab1565_ab1568_evk\apps\headset_ref_design\inc\apps\events
apps_events_event_group.h
...\mcu\project\ab1565_ab1568_evk\apps\headset_ref_design\src\apps\events
apps_events_event_group.c
```

举例分析如何使用 UI Shell：

案例 1：有电话进来并按键接听电话的流程

- 1、BT 模块调用一个在事件发送器里定义好的 BT 回调函数
- 2、事件发送器通过调用 API 函数 ui_shell_send_event() 发送包含有来电信息的事件
- 3、UI Shell 派送该事件给所有的活动
- 4、当 pre-proc 功能里一个 CALL APP 空闲活动接收到事件，就会调用 API 函数 ui_shell_send_event() 来启动 CALL APP 短暂活动
- 5、UI Shell 开始调用该短暂活动
- 6、当用户按下按键接听电话，按键模块会调用一个定义在事件发送器里的按键回调函数
- 7、事件发送器通过调用 API 函数 ui_shell_send_event() 发送包含有按下按键信息的事件
- 8、UI Shell 派送该事件给所有的活动，因为电话短暂活动比其他活动的优先级高，所以它在其他活动之前接收事件

9、CALL APP 短暂活动接收事件和调用 BT 模块的 API 函数去接受来电。它会返回 true，因为按键事件不应该派送给下一个活动

① ≫ ② ≫ ≫ ⑨

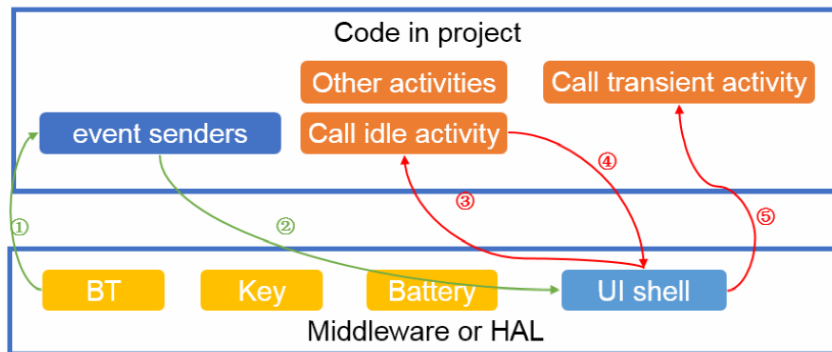


Figure 5. Receive incoming call event

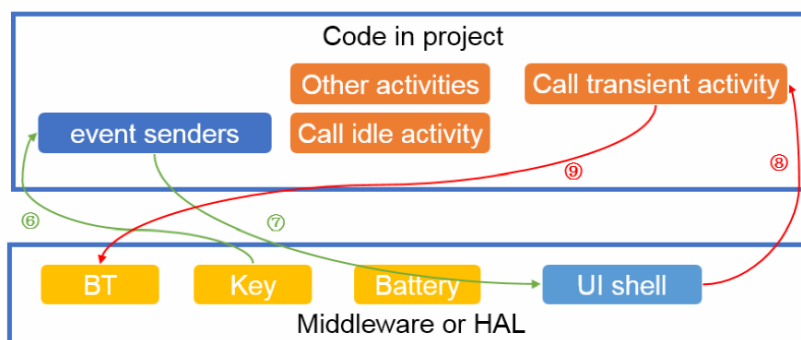


Figure 6. Receive key event to accept call

初始化各种事件发送器，初始化对应的回调函数 `callback`（在 HAL 模块中被注册），当回调函数被执行，事件被发送到 APPs。

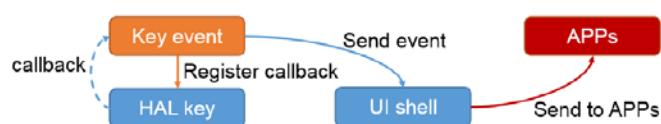
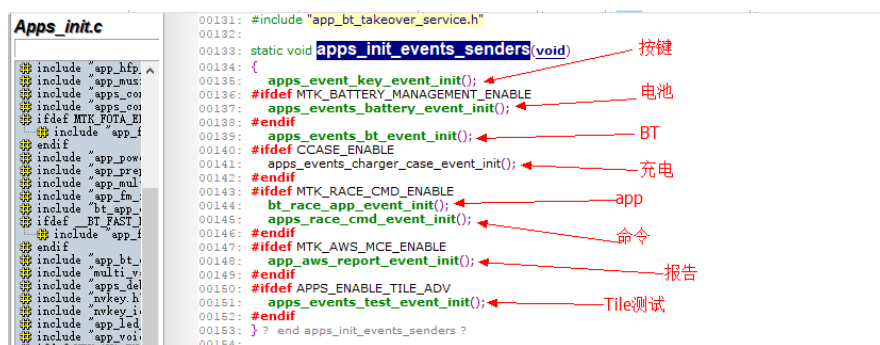


Figure 2. Key event sender

按键事件发送器工作过程



Event senders

- 2.1.1.  Battery event
- 2.1.2.  BT event
- 2.1.3.  Key event

接收 UI Shell 分发的消息的 activity，启动各活动程序

```
Apps_init.c
00159: static void apps_init_applications(void)
00160: {
00161:     ui_shell_set_pre_proc_func(app_preproc_activity_proc);
00162:     ui_shell_start_activity(NULL, app_event_state_report_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00163:     ui_shell_start_activity(NULL, app_multi_va_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00164:     #ifdef AIR_BT_ULTRA_LOW_LATENCY_ENABLE
00165:     ui_shell_start_activity(NULL, app_ull_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00166:     #endif
00167:     /* Register all idle activities here. The last registered activity is the foreground activity. */
00168:     #ifdef GSOUND_LIBRARY_ENABLE
00169:     ui_shell_start_activity(NULL, app_gsound_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00170:     #endif
00171:     #ifdef MTK_SMART_CHARGER_ENABLE
00172:     DrvCharger_SmartCase_Init();
00173:     ui_shell_start_activity(NULL, app_smcharger_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00174:     #endif
00175:     #ifdef MTK_BATTERY_MANAGEMENT_ENABLE
00176:     ui_shell_start_activity(NULL, app_battery_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00177:     #endif
00178:     #ifdef SUPPORT_ROLE_HANDOVER_SERVICE
00179:     ui_shell_start_activity(NULL, app_rho_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00180:     #endif
00181:     /* APPS_USB_AUDIO_SUPPORT for headset project */
00182:     #ifdef APPS_USB_AUDIO_SUPPORT
00183:     ui_shell_start_activity(NULL, app_usb_audio_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00184:     #endif
00185:     ui_shell_start_activity(NULL, app_hfp_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00186:     ui_shell_start_activity(NULL, app_music_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00187:     #ifdef MTK_FOTA_ENABLE
00188:     ui_shell_start_activity(NULL, app_fota_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00189:     #endif
00190:     #ifdef CCASE_ENABLE
00191:     ui_shell_start_activity(NULL, app_charger_case_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00192:     #endif
00193:     #ifdef RACE_FIND_ME_ENABLE
00194:     ui_shell_start_activity(NULL, app_find_me_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00195:     #endif
00196:     #ifdef BT_FAST_PAIR_ENABLE
00197:     ui_shell_start_activity(NULL, app_fast_pair_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00198:     #endif
00199:     #ifdef APPS_SLEEP_AFTER_NO_CONNECTION
00200:     ui_shell_start_activity(NULL, app_power_saving_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00201:     #endif
00202:     #ifdef MTK_IN_EAR_FEATURE_ENABLE
00203:     ui_shell_start_activity(NULL, app_in_ear_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00204:     #endif
00205:     ui_shell_start_activity(NULL, app_home_screen_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_TOP, NULL, 0);
00206:     #ifdef MTK_AMA_ENABLE
00207:     ui_shell_start_activity(NULL, app_ama_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00208:     #endif
00209:     /* APPS_LINE_IN_SUPPORT for headset project */
00210:     #ifdef APPS_LINE_IN_SUPPORT
00211:     ui_shell_start_activity(NULL, app_line_in_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00212:     #endif
00213:     #ifdef MTK_VA_XIAOWEI_ENABLE
00214:     ui_shell_start_activity(NULL, app_va_xiaowei_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00215:     #endif
00216:     #ifdef MTK_VA_XIAOAI_ENABLE
00217:     ui_shell_start_activity(NULL, app_va_xiaoai_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00218:     #endif
00219:     #ifdef MTK_LEAKAGE_DETECTION_ENABLE
00220:     ui_shell_start_activity(NULL, app_leakage_detection_idle_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00221:     #endif
00222:     #ifdef AIR_MULTI_POINT_ENABLE
00223:     ui_shell_start_activity(NULL, app_bt_emp_service_activity_proc, ACTIVITY_PRIORITY_IDLE_BACKGROUND, NULL, 0);
00224:     #endif
}
```

案例 2：电池电量太低而关机

- 1、电池模块调用一个在事件发送器里定义好的电池回调函数
- 2、事件发送器通过调用 API 函数 ui_shell_send_event()发送包含有电池电压信息的事件
- 3、UI Shell 派送该事件给所有的活动
- 4、当 BATTERY APP 空闲活动接收到事件，它会通过调用 ui_shell_send_event()发送关机请求给 UI Shell
- 5、完整的活动堆处理电池电压信息事件后，UI Shell 派送关机请求给每个活动
- 6、HOMESCREEN APP 空闲活动接收到事件，它调用在 HAL 的 API 函数去执行关机

①》②》.....》⑤

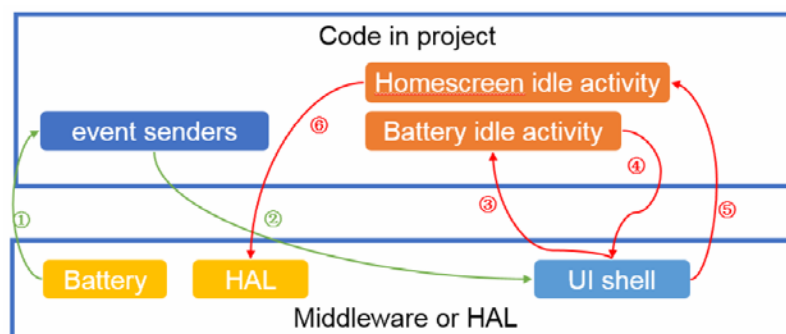


Figure 7. Power off when low battery

剖析 SDK 中各个文件的作用

CHP200_sdk_V2.7.1 > mcu

名称	修改日期	类型
config	2021/11/24 21:51	文件夹
doc	2021/11/24 21:51	文件夹
driver	2021/11/24 21:51	文件夹
kernel	2021/11/24 21:51	文件夹
middleware	2021/11/24 21:52	文件夹
out	2021/11/24 21:52	文件夹
prebuilt	2021/11/24 21:52	文件夹
project	2021/11/24 21:52	文件夹
tools	2021/11/24 21:53	文件夹
rule	2021/8/2 18:42	CONFIG: MK 文件

应用层与中间层和底层之间来回传递消息

CHP200_sdk_V2.7.1 > mcu > driver

名称	修改日期
board	2021/1
chip	2021/1
CMSIS	2021/1

底层（板载驱动、外设和内核驱动）

CHP200_sdk_V2.7.1 > mcu > middleware

名称	修改日期
MTK	2021/
third_party	2021/

中间层（UI Shell、Battery、Fota、audio、bluetooth、smart_charger）

CHP200_sdk_V2.7.1 > mcu > project > ab1565_ab1568_evk > apps

名称	修改日期	类型
bootloader	2021/11/24 21:52	文件夹
earbuds_ref_design	2021/11/24 21:52	文件夹
headset_ref_design	2021/11/24 21:52	文件夹

应用层

三十、打开或关闭 HAL 硬件模块

在文件 Hal_feature_config.h 里，注释掉表示关闭该模块

Hal_feature_config.h (d:\iot_sdk_for_bt_audio_v2.7.1\ull_tws_headset\dsp\project\ab1565_ab1568_evk\apps\dsp0_headset_ref_design\inc)

Hal_feature_config.h

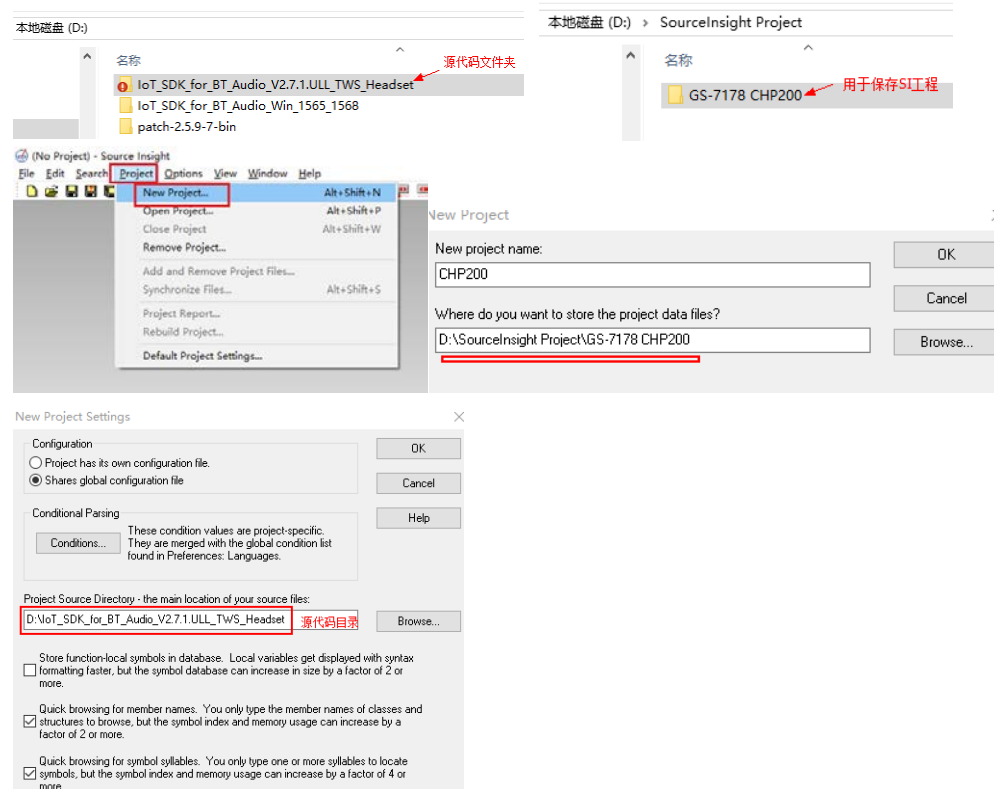
```
ifndef HAL_FEATURE_CONFIG_H_
#define HAL_FEATURE_CONFIG_H_
...
#endif
```

```
00041:
00042:
00043: /*****
00044:  * module ON or OFF feature option,only option in this temple
00045: *****/
00046:
00047: #define HAL_AUDIO_MODULE_ENABLED
00048: #define HAL_CACHE_MODULE_ENABLED
00049: #define HAL_CCNT_MODULE_ENABLED
00050: #define HAL_CLOCK_MODULE_ENABLED
00051: // #define HAL_EINT_MODULE_ENABLED
00052: // #define HAL_FLASH_MODULE_ENABLED
00053: #define HAL_SW_DMA_MODULE_ENABLED
00054: #define HAL_GPIO_MODULE_ENABLED
00055: #define HAL_GPT_MODULE_ENABLED
00056: #define HAL_HW_SEMAPHORE_MODULE_ENABLED
00057: // #define HAL_I2C_MASTER_MODULE_ENABLED
00058: // #define HAL_I2S_MODULE_ENABLED
00059: #define HAL_NVIC_MODULE_ENABLED
00060: // #define HAL_SPI_MASTER_MODULE_ENABLED
00061: #define HAL_UART_MODULE_ENABLED
00062: // #define HAL_WDT_MODULE_ENABLED
00063: #define HAL_TRNG_MODULE_ENABLED
00064: // #define HAL_PMU_MODULE_ENABLED
00065: // #define HAL_DCCO_MODULE_ENABLED
00066: #define HAL_SLEEP_MANAGER_ENABLED
00067: #define HAL_HW_RESOURCE_ASSIGNMENT_CHECK
00068: // #define HAL_USB_MODULE_ENABLED
00069: #define HAL_DWT_MODULE_ENABLED
00070: #define HAL_TIME_CHECK_ENABLED
00071: /*****
00072:  * customization module feature option
00073: *****/
00074:
```

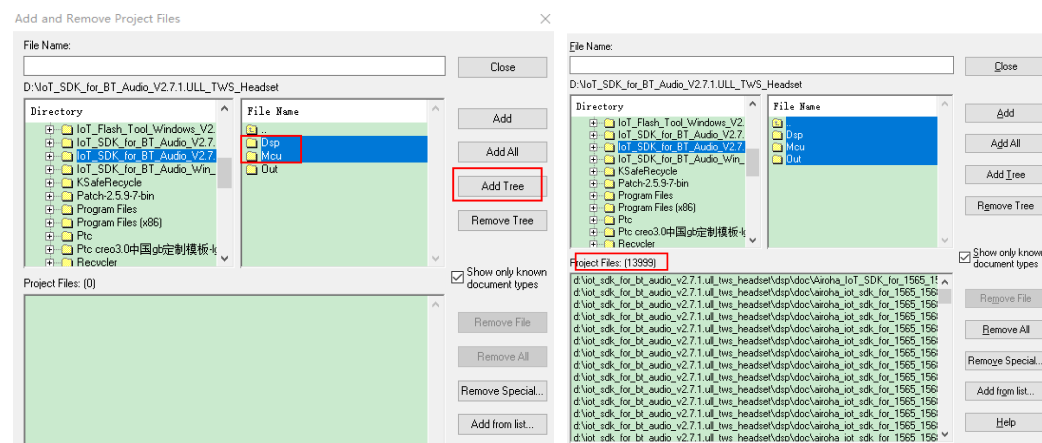
注释掉即可关闭该模块

三十一、SourceInsight 查找工程文件和代码

新建工程，输入工程名和选择保存路径（一般在源代码根目录下建一个文件夹存放 SI 工程）

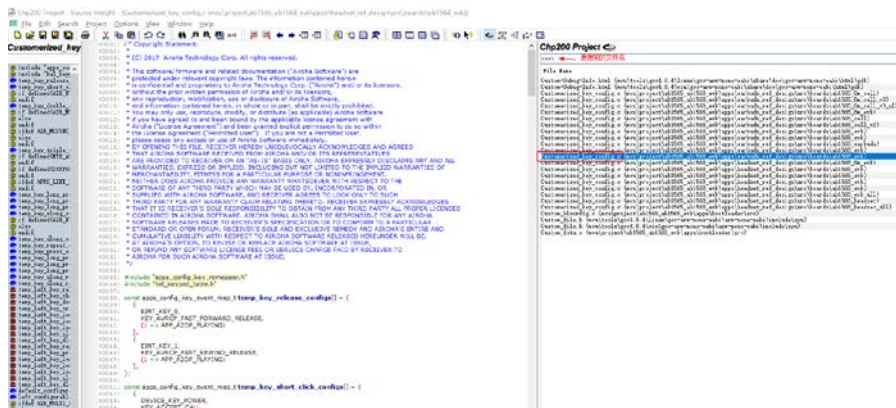


选定要添加哪些文件路径到工程里，点击 Add Tree 即可

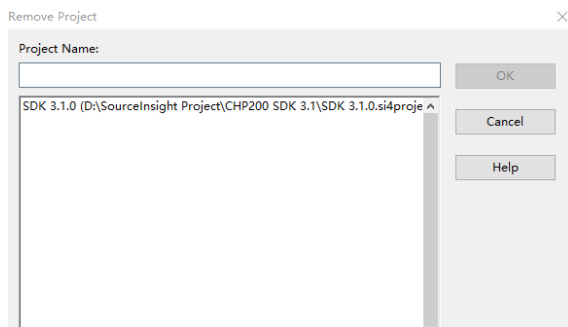


如果要删除或添加文件路径，点击 Add and Remove Project Files，再添加或删除路径
Remove All 是删除所有路径

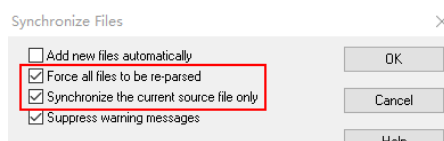
可以快速搜索到文件以及文件中的指定内容



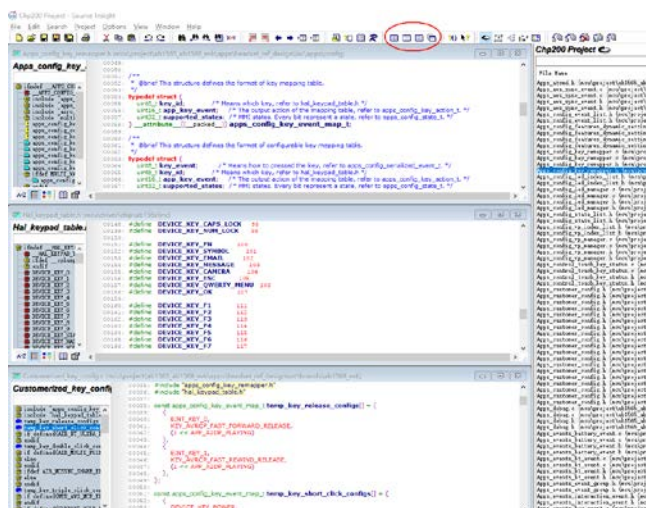
移除工程，点击 **Project > Remove Project**，选择工程，点击 **OK** 即可，同时可以查询到工程的所在目录



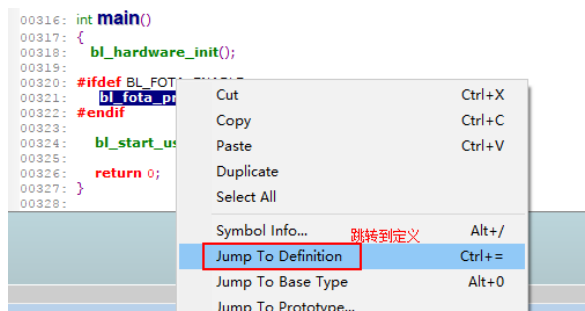
Project>Sync.....方便对源代码阅读



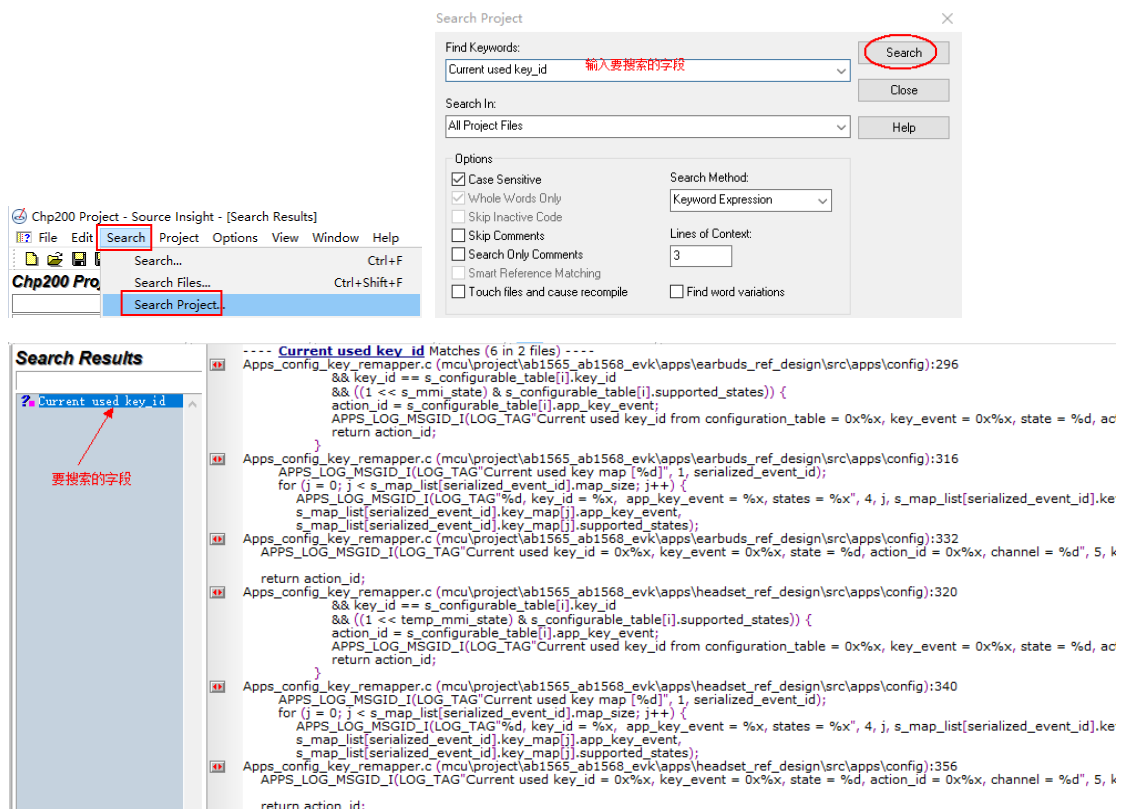
可以同时多个窗口查看不同文件



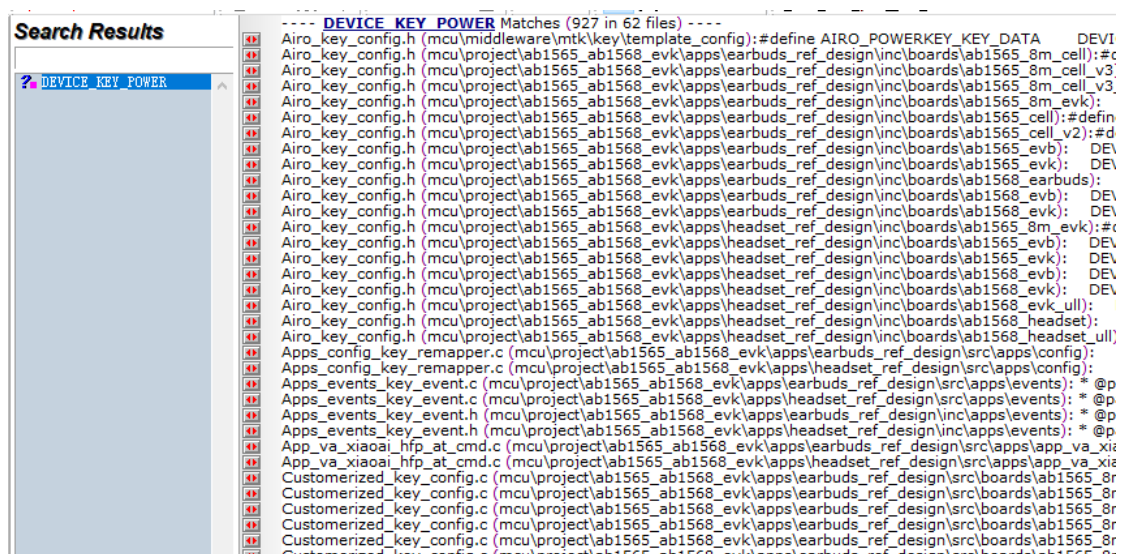
右击代码里的函数名可以选择跳转到定义处



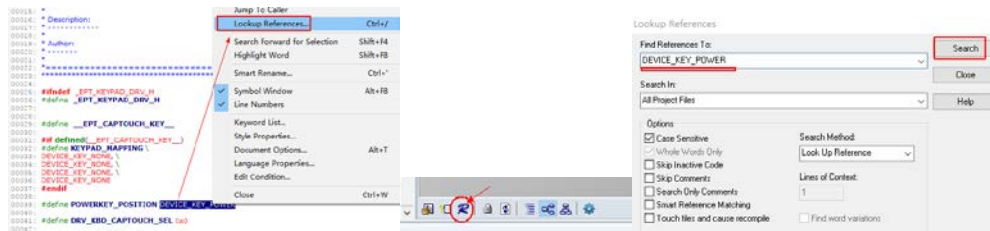
在一个工程目录下所有文件中搜索某个字段（这个查找方式可以把 log 所有信息找到对应的所在文件）



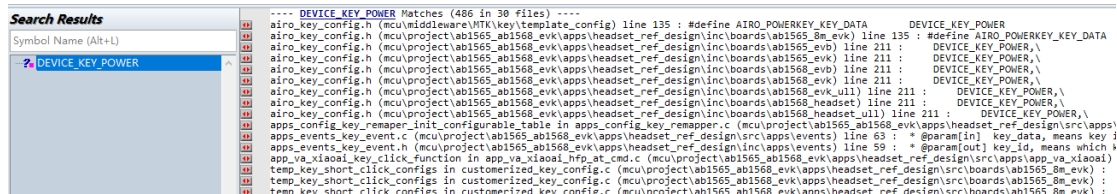
点击红色三角可以展开文件，点 ◀▶ 这个可以前进/返回，点 [F7] [F8] 可以打开调用的文件



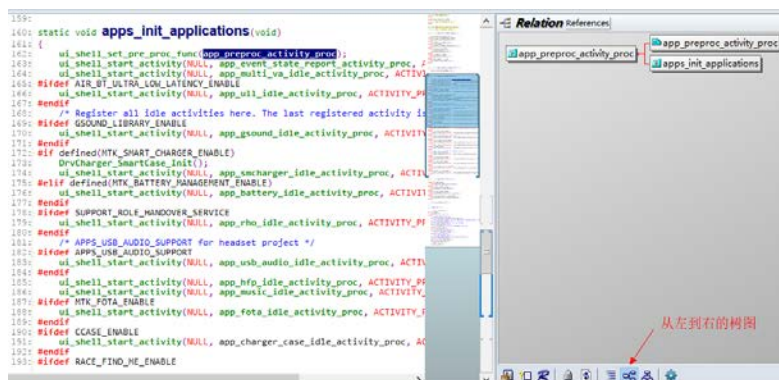
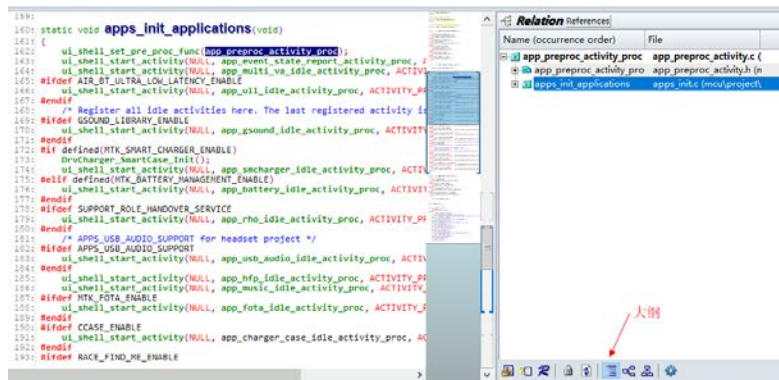
或者这种查找方式（在所有工程文件查找）

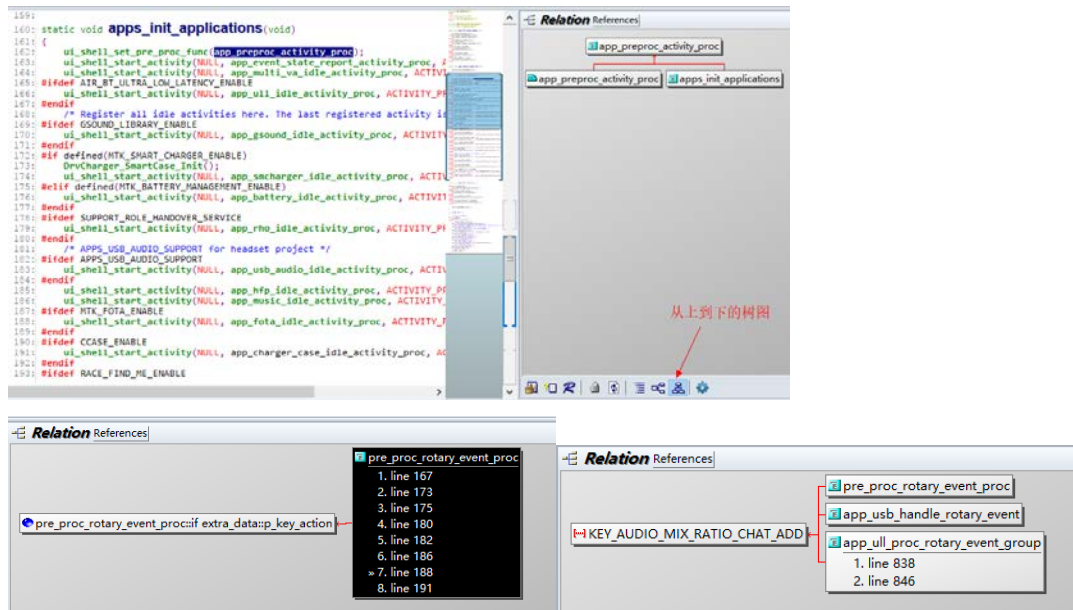


点击双三角展开/收起代码位置， 切换上一个或下一个代码位置

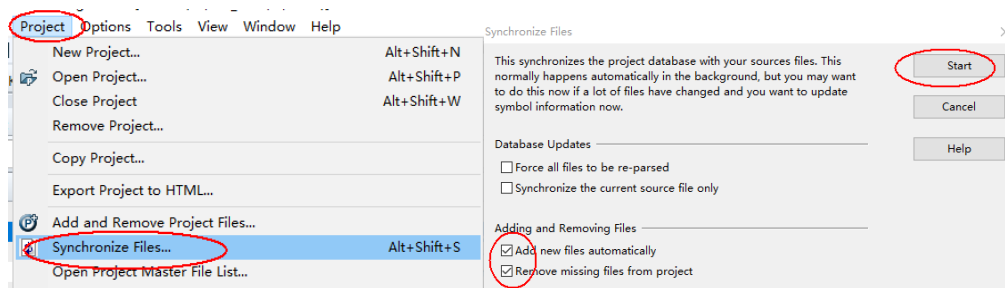


以树图形式查看关联某函数或某变量的所有文件（声明，定义，调用的地方都会列出来），比如 `app_preproc_activity_proc`，点击该函数，在树图窗口中可以以三种形式查看，双击树图中的显示就可以打开相应的代码位置

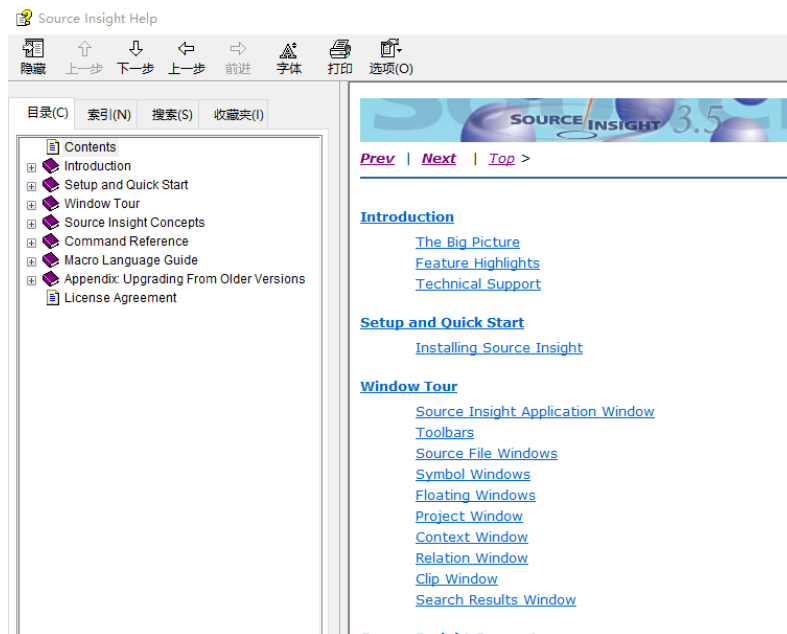




同步文件功能



更多的使用说明可以查阅软件菜单 Help > Help



三十二、AB1561M 的 EVK 板外设驱动文件

在 SDK 目录... \mcu\driver\board\component 下是外设驱动代码源文件，如音频接口、中断输入按键、灯、旋转编码器等等，可以参考该目录下的驱动添加其它新驱动文件

IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset > mcu > driver > board > component			
名称	修改日期	类型	大小
bsp_audio_ext_codec	2021/10/27 15:59	文件夹	
bsp_eint_key	2021/8/31 2:13	文件夹	
bsp_flash	2021/8/31 2:13	文件夹	
bsp_gsensor_key	2021/8/31 2:13	文件夹	
bsp_inear	2021/8/31 2:13	文件夹	
bsp_led	2021/8/31 2:13	文件夹	
bsp_psensor_key	2021/8/31 2:13	文件夹	
bsp_rotary_encoder	2021/8/31 2:13	文件夹	

比如板载 LED 驱动文件

IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset > mcu > driver > board > component > bsp_led			
名称	修改日期	类型	大小
inc	2021/8/31 2:13	文件夹	
src	2021/8/31 2:13	文件夹	
module	2021/8/31 2:13	MK 文件	5 KB

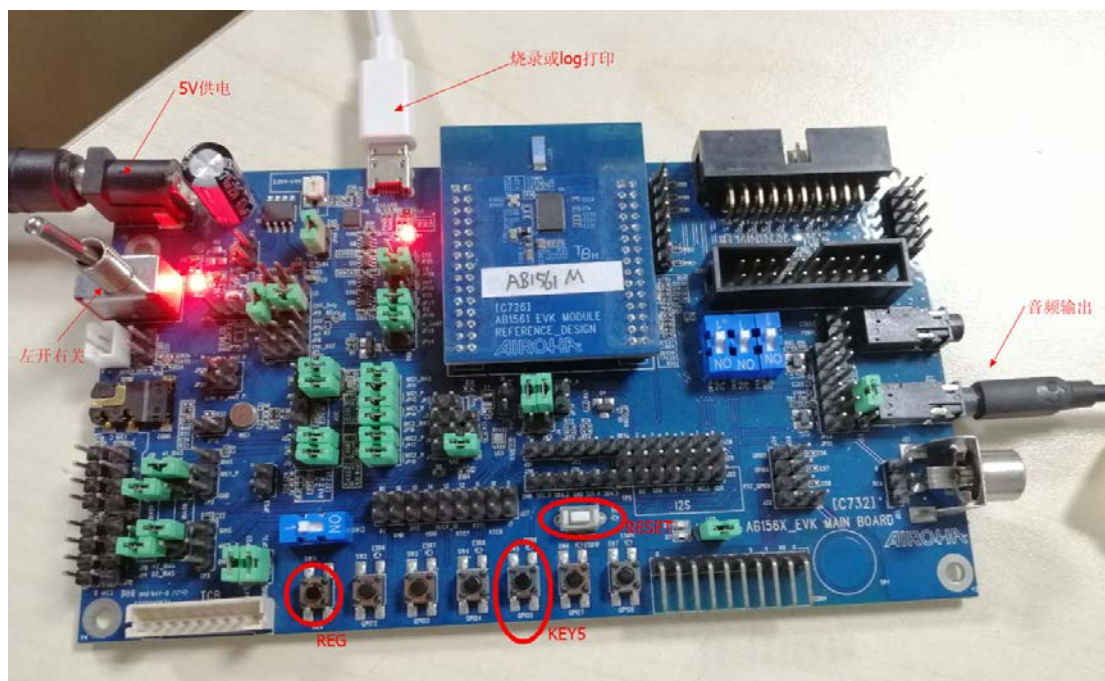
中间件 Middleware 文件夹下是联发科 MTK 和第三方的算法

IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset > mcu > middleware		
名称	修改日期	类型
MTK	2021/8/31 2:14	文件夹
third_party	2021/8/31 2:14	文件夹

编译规则在每个文件夹下的 mk 文件（说明了编译的头文件和源文件的路径）

component > bsp_audio_ext_codec		<pre>COMPONENT_SRC = driver/board/component #int key driver source C_FILES += \$(COMPONENT_SRC)/bsp_audio_ext_codec/src/bsp_audio_ext_codec.c C_FILES += \$(COMPONENT_SRC)/bsp_audio_ext_codec/src/CM9601.c ##### #include path CFLAGS += -Iinclude CFLAGS += -I\$(SOURCE_DIR)/kernel/service/inc CFLAGS += -I\$(SOURCE_DIR)/driver/chip/inc CFLAGS += -I\$(SOURCE_DIR)/driver/chip/\$(IC_CONFIG)/inc #external audio codec driver include path CFLAGS += -I\$(SOURCE_DIR)/\$(COMPONENT_SRC)/bsp_audio_ext_codec/inc CFLAGS += -DAIR_AUDIO_EXT_CODEC_ENABLE</pre>
名称		
inc		
src		
module		

烧录程序到 EVK 板：



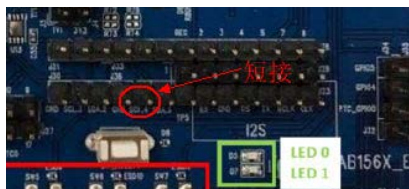
- 1、在 config tool 上点烧录，板上先短按 RESET 键，再短按 REG 键，开始进行下载
- 2、先短按 RESET 键，再长按 REG 键，EVK 板开机
- 3、长按 KEY5 进配对模式

其它参考文档..\AB1561_AB1562_AB1563_SDK\V1.5.1\doc\AB1562 目录下

“AB1561_AB1562_AB1563_Get_Started_Guide.pdf”

“AB1561_Headset_Reference_Design_User_Guide.pdf”

如果要在 EVK 板上测试 Riff2 软件，需要把 GPIO4 短接到 GND，才能正常开机（因为 GPIO4 为低时才能开机，Line-in 保护的原因）

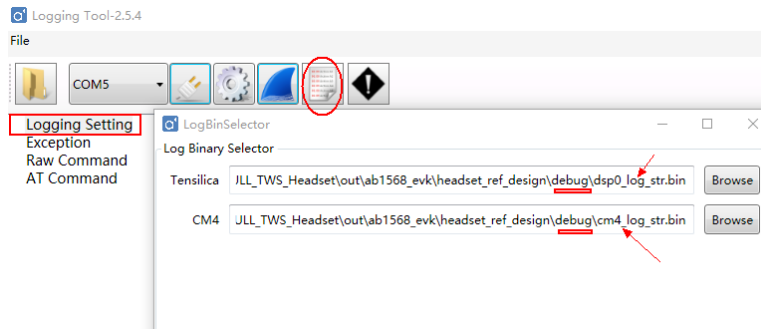


，短接后，长按 REG 键开机。

三十三、使用 Logging tool 监测系统运行事件信息

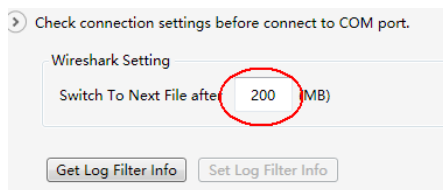
AB1568 的系统运行监测：两种方式（一是只用 ATK 里 logging tool 工具，二是 Config Tool 里面的 logging tool 工具），用于调试和验证修改代码的结果

打开 ATK 里的 logging tool 工具，首先选择 log 的 bin 文件（修改代码重新编译后，log.bin 文件就会被重新生成，在 SDK 的 out 目录下 Debug 下，所以对应工程要用对应 log.bin 文件）



dsp 和 cm4 抓 log 的 bin 文件

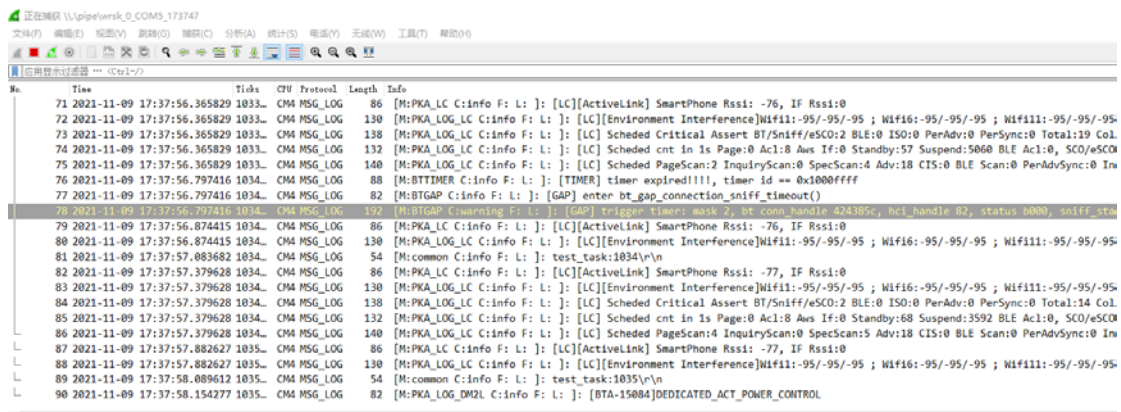
打开端口并点击启动网络分包分析器



设置存储 log 信息的文件大小



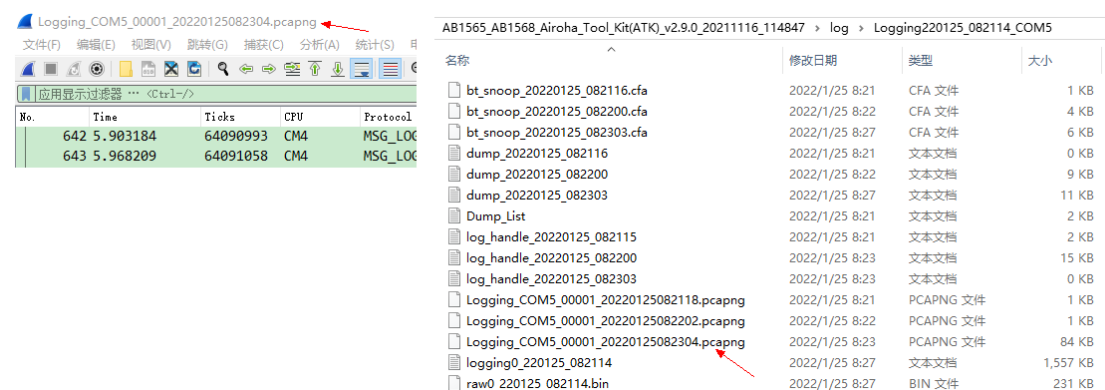
未过滤信息，则显示所有 log



Info 里的第一个中括号 M 表示 module 模块类型，eint_key 表示中断输入按键类型的 log，common 表示公共的 log，C 表示 log 类型（info 是用 LOG_MSGID_I 函数打印的）

```
#define LOG_MSGID_I(module, message, arg_cnt, ...) \
LOG_I(module, message, ##__VA_ARGS__)
```

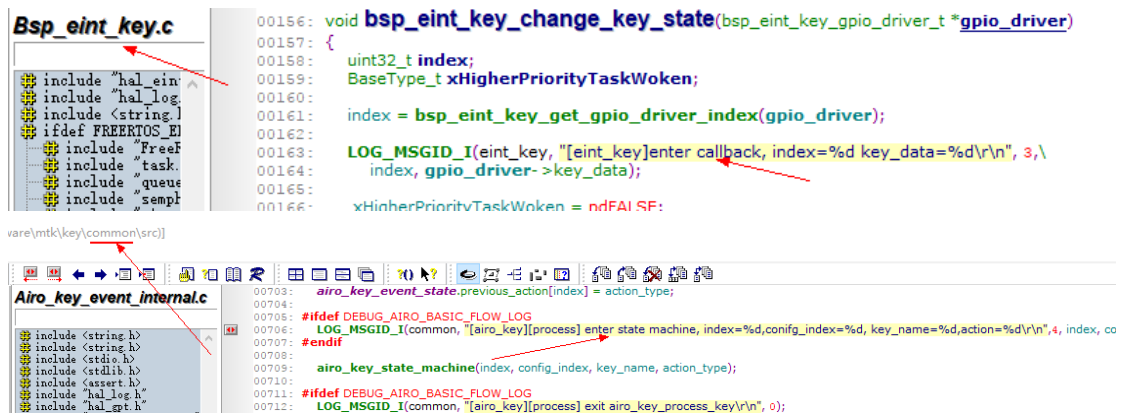
Log 信息会自动保存成 pcapng 文件



也可以另存为不同类型的文件（pcapng、txt 等），同样用 open 来打开



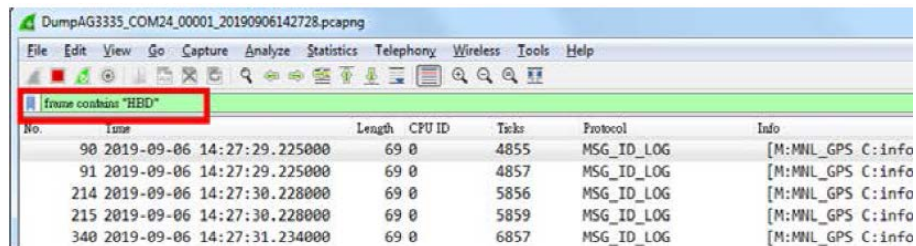
[M:eint_key C:info F: L:]: [eint_key]enter callback, index=0 key_data=
[M:common C:info F: L:]: [airo_key][process] enter state machine, inde



过滤关键字有两种

1. frame contains “string” – Filter the string by case sensitive.
2. frame matches “(?)string” – Filter the string by case insensitive.

过滤 HBD 字样



过滤 event 字样: frame contains “event”

No.	Time	Ticks	CPU	Protocol	Length	Info
1697	2021-11-09 17:44:54.808625	1452.	CM4	MSG_LOG	81	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x2, 0x407f, delay:0
1702	2021-11-09 17:44:54.808625	1452.	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x40, state = 3,
1720	2021-11-09 17:44:54.808625	1452.	CM4	MSG_LOG	84	[M:AudSrc C:info F: L:]: [VENDOR_SE]ami_execute_vendor_se: event = 6 \n
1722	2021-11-09 17:44:54.808625	1452.	CM4	MSG_LOG	99	[M:apps C:info F: L:]: [LINE_IN_MMI]am callback, event: 6, current path: 0, started: 0
1723	2021-11-09 17:44:54.808625	1452.	CM4	MSG_LOG	79	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x18, 0x3, delay:0
1801	2021-11-09 17:44:54.809077	1452.	CM4	MSG_LOG	96	[M:apps C:info F: L:]: [Music_APP]utils key_id=0x7f, key_event=0x40, key_action=0x0
1948	2021-11-09 17:44:54.834626	1452.	CM4	MSG_LOG	78	[M:VOICE_PROMPT_APP C:info F: L:]: voice prompt callback event=0\n
1984	2021-11-09 17:44:55.035503	1452.	CM4	MSG_LOG	84	[M:apps C:info F: L:]: [app_key_event]key event = 0x0, key_data = 0x7f
1985	2021-11-09 17:44:55.035503	1452.	CM4	MSG_LOG	79	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x2, 0x7f, delay:0
1992	2021-11-09 17:44:55.035503	1452.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x0, state = 3,
1993	2021-11-09 17:44:55.035503	1452.	CM4	MSG_LOG	95	[M:apps C:info F: L:]: [Music_APP]utils key_id=0x7f, key_event=0x0, key_action=0x0
2102	2021-11-09 17:44:55.035503	1452.	CM4	MSG_LOG	84	[M:AudSrc C:info F: L:]: [VENDOR_SE]ami_execute_vendor_se: event = 7 \n
2104	2021-11-09 17:44:55.035503	1452.	CM4	MSG_LOG	99	[M:apps C:info F: L:]: [LINE_IN_MMI]am callback, event: 7, current path: 0, started: 0
2105	2021-11-09 17:44:55.035503	1452.	CM4	MSG_LOG	79	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x18, 0x3, delay:0
2185	2021-11-09 17:44:55.059927	1452.	CM4	MSG_LOG	78	[M:VOICE_PROMPT_APP C:info F: L:]: voice prompt callback event=2\n
2197	2021-11-09 17:44:55.153048	1452.	CM4	MSG_LOG	84	[M:apps C:info F: L:]: [app_key_event]key event = 0x1, key_data = 0x7f
2198	2021-11-09 17:44:55.153048	1452.	CM4	MSG_LOG	80	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x2, 0x17f, delay:0
2202	2021-11-09 17:44:55.153048	1452.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x1, state = 3,
2203	2021-11-09 17:44:55.153048	1452.	CM4	MSG_LOG	95	[M:apps C:info F: L:]: [Music_APP]utils key_id=0x7f, key_event=0x1, key_action=0xb
2213	2021-11-09 17:44:55.153048	1452.	CM4	MSG_LOG	112	[M:sink_srv C:info F: L:]: [sink][avrcp]process_avrcp_action[s]-event: 0xf8150005, base: 0b

按下按键就可以打印出对应的事件信息，可以追踪运行信息

过滤 Key_remap 字样：frame contains “Key_remap”

No.	Time	Ticks	CPU	Protocol	Length	Info
5479	2021-11-10 08:22:18.741993	2182.	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x40, state = 0, action_id = 0x0, channel = 0
5717	2021-11-10 08:22:18.958833	2182.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
5941	2021-11-10 08:22:19.043345	2182.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x2, state = 0, action_id = 0x0, channel = 0
5971	2021-11-10 08:22:20.957313	2184.	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
6214	2021-11-10 08:22:21.174417	2185.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
6433	2021-11-10 08:22:21.278228	2185.	CM4	MSG_LOG	158	[M:apps C:info F: L:]: [Key_remap] Current used key_id from configuration_table = 0x18, key_event = 0x1, state = 0, action_id = 0x0, channel = 0
6505	2021-11-10 08:22:23.274807	2187.	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x40, state = 0, action_id = 0x0, channel = 0
6783	2021-11-10 08:22:23.467343	2187.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
6872	2021-11-10 08:22:23.467343	2187.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x40, state = 0, action_id = 0x0, channel = 0
7299	2021-11-10 08:22:23.614474	2187.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
7534	2021-11-10 08:22:23.814573	2187.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x2, state = 0, action_id = 0x0, channel = 0
7686	2021-11-10 08:22:26.739577	2190.	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x18, key_event = 0x40, state = 0, action_id = 0x0, channel = 0
7927	2021-11-10 08:22:26.962686	2190.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
8106	2021-11-10 08:22:27.118975	2191.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x1, state = 0, action_id = 0x0, channel = 0
8177	2021-11-10 08:22:29.398998	2193.	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x80, key_event = 0x40, state = 0, action_id = 0x0, channel = 0
8665	2021-11-10 08:22:29.662017	2193.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x80, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
8678	2021-11-10 08:22:29.862117	2193.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x80, key_event = 0x1, state = 0, action_id = 0x0, channel = 0
8757	2021-11-10 08:22:32.094934	2196.	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x40, state = 0, action_id = 0x0, channel = 0
9245	2021-11-10 08:22:32.441582	2196.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
9258	2021-11-10 08:22:32.641862	2196.	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x1, state = 0, action_id = 0x0, channel = 0

同时过滤几个字样：frame matches "Crusher volume|music volume"

按下按键就会打印信息处理，方便查看是哪个按键事件

要知道哪个按键按下：frame contains “[key]”

No.	Time	Ticks	CPU	Protocol	Length	Info
161189	2021-11-10 14:43:56.313766	358	CM4	MSG_LOG	89	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=3 eint=3 keydata=126 status=0
161190	2021-11-10 14:43:56.313766	358	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=12 eint=12 keydata=127 status=1
161191	2021-11-10 14:43:56.313766	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=14 eint=14 keydata=128 status=1
161192	2021-11-10 14:43:56.313766	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=19 eint=19 keydata=129 status=0
161193	2021-11-10 14:43:56.313766	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=25 eint=25 keydata=130 status=1
162438	2021-11-10 14:44:18.524119	358	CM4	MSG_LOG	89	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=3 eint=3 keydata=126 status=0
162439	2021-11-10 14:44:18.524119	358	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=12 eint=12 keydata=127 status=1
162440	2021-11-10 14:44:18.524119	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=14 eint=14 keydata=128 status=1
162441	2021-11-10 14:44:18.524119	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=19 eint=19 keydata=129 status=0
162442	2021-11-10 14:44:18.524119	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=25 eint=25 keydata=130 status=1
163687	2021-11-10 14:44:40.735534	358	CM4	MSG_LOG	89	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=3 eint=3 keydata=126 status=0
163688	2021-11-10 14:44:40.735534	358	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=12 eint=12 keydata=127 status=1
163689	2021-11-10 14:44:40.735534	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=14 eint=14 keydata=128 status=1
163690	2021-11-10 14:44:40.735534	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=19 eint=19 keydata=129 status=0
163691	2021-11-10 14:44:40.735534	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=25 eint=25 keydata=130 status=1
164936	2021-11-10 14:45:02.946469	358	CM4	MSG_LOG	89	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=3 eint=3 keydata=126 status=0
164937	2021-11-10 14:45:02.946469	358	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=12 eint=12 keydata=127 status=1
164938	2021-11-10 14:45:02.946469	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=14 eint=14 keydata=128 status=1
164939	2021-11-10 14:45:02.946469	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=19 eint=19 keydata=129 status=0
164940	2021-11-10 14:45:02.946469	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=25 eint=25 keydata=130 status=1

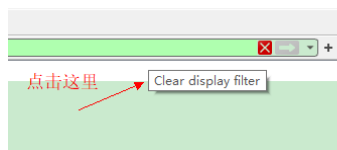
同时过滤多个关键字：frame contains "[key]" || frame contains "[Key_remap]"

No.	Time	Ticks	CPU	Protocol	Length	Info
167438	2021-11-10 14:45:47.366474	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=25 eint=25 keydata=130 status=1
167475	2021-11-10 14:45:47.366474	378	CM4	MSG_LOG	95	[M:apps C:info F: L:]: [Key_remap] apps_config_key_remapper_init_configurable_table
167476	2021-11-10 14:45:47.366474	378	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] apps_config_key_remapper_init_configurable_table success, s_configurable_table_len = 7
168683	2021-11-10 14:46:09.581864	358	CM4	MSG_LOG	89	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=3 eint=3 keydata=126 status=0
168684	2021-11-10 14:46:09.581864	358	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=12 eint=12 keydata=127 status=1
168685	2021-11-10 14:46:09.581864	358	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=14 eint=14 keydata=128 status=1
168686	2021-11-10 14:46:09.581864	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=19 eint=19 keydata=129 status=0
168687	2021-11-10 14:46:09.581864	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=25 eint=25 keydata=130 status=1
168724	2021-11-10 14:46:09.582866	377	CM4	MSG_LOG	95	[M:apps C:info F: L:]: [Key_remap] apps_config_key_remapper_init_configurable_table
168725	2021-11-10 14:46:09.582866	378	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] apps_config_key_remapper_init_configurable_table success, s_configurable_table_len = 7
169932	2021-11-10 14:46:31.798051	358	CM4	MSG_LOG	89	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=3 eint=3 keydata=126 status=0
169933	2021-11-10 14:46:31.798051	358	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=12 eint=12 keydata=127 status=1
169934	2021-11-10 14:46:31.798051	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=14 eint=14 keydata=128 status=1
169935	2021-11-10 14:46:31.798051	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=19 eint=19 keydata=129 status=0
169936	2021-11-10 14:46:31.798051	359	CM4	MSG_LOG	91	[M:eint_key C:info F: L:]: [eint_key][key]GPIO=25 eint=25 keydata=130 status=1
169973	2021-11-10 14:46:31.798051	378	CM4	MSG_LOG	95	[M:apps C:info F: L:]: [Key_remap] apps_config_key_remapper_init_configurable_table
169974	2021-11-10 14:46:31.798051	378	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] apps_config_key_remapper_init_configurable_table success, s_configurable_table_len = 7
170589	2021-11-10 14:46:32.710619	1271	CM4	MSG_LOG	133	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x40, state = 0, action_id = 0x0, channel = 0
170957	2021-11-10 14:46:32.001869	1430	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x0, state = 0, action_id = 0x0, channel = 0
171220	2021-11-10 14:46:33.054664	1626	CM4	MSG_LOG	132	[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x7f, key_event = 0x1, state = 0, action_id = 0x0, channel = 0

暂停 log 打印，定位 log 的前后信息。先停止，并选中要定位的 log

No.	Time	Ticks	CPU	Protocol	Length	Info
95	10.020392	47266	CM4	MSG_LOG	50	[M:race C:info F: L:]: GS DEVICE NAME
96	10.020392	47267	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 2
97	10.020392	47267	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 3
98	10.020392	47267	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 4
99	10.020392	47267	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 5
100	10.020392	47267	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 6
102	10.020392	47268	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 8
229	21.550791	58802	CM4	MSG_LOG	50	[M:race C:info F: L:]: GS DEVICE NAME
230	21.550791	58802	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 2
231	21.550791	58802	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 3
232	21.550791	58803	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 4
233	21.550791	58803	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 5
234	21.550791	58803	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 6
447	29.414393	66667	CM4	MSG_LOG	50	[M:race C:info F: L:]: GS DEVICE NAME
448	29.414393	66667	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 2
449	29.414393	66667	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 3
450	29.414393	66667	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 4
451	29.414393	66668	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 5
452	29.414393	66668	CM4	MSG_LOG	52	[M:race C:info F: L:]: GS DEVICE NAME 6

选中要定位的



可以看到定位的 log 的前后信息

```

133 [M:race C:info F: L: ]: RACE_ClaimPacket, race_type[0x5B], race_id[0xA01], dat_len[1], channel_1
50 [M:race C:info F: L: ]: GS DEVICE NAME
52 [M:race C:info F: L: ]: GS DEVICE NAME 2
52 [M:race C:info F: L: ]: GS DEVICE NAME 3
52 [M:race C:info F: L: ]: GS DEVICE NAME 4
52 [M:race C:info F: L: ]: GS DEVICE NAME 5
52 [M:race C:info F: L: ]: GS DEVICE NAME 6
103 [M:nvdm C:warning F:nvdm_log_warning L:311]: task: race write { AB15, F202 } with 11 bytes
52 [M:race C:info F: L: ]: GS DEVICE NAME 8
82 [M:hal C:info F: L: ]: [hal_flash_write] address=0x7f103d, length =1\r\n
94 [M:hal C:info F: L: ]: [hal_flash_write] address=0x7f103d, length = 1 result ok!\r\n
93 [M:hal C:info F: L: ]: [hal_flash_write] address=0x7f103e, length =19\r\n
95 [M:hal C:info F: L: ]: [hal_flash_write] address=0x7f103e, length = 19 result ok!\r\n
83 [M:hal C:info F: L: ]: [hal_flash_write] address=0x7f1051, length =10\r\n

```

定位的

Logging 工具里打印的是程序里 LOG_TAG 函数的执行结果

```

292307 2021-11-10 15:16:29.772918 353 CM4 MSG_LOG 91 [M:eint_key C:info F: L: ]: [eint_key][key]GPIO-12 eint=12 keydata=127 status=1
292308 2021-11-10 15:16:29.772918 354 CM4 MSG_LOG 91 [M:eint_key C:info F: L: ]: [eint_key][key]GPIO-14 eint=14 keydata=128 status=1
292309 2021-11-10 15:16:29.772918 354 CM4 MSG_LOG 91 [M:eint_key C:info F: L: ]: [eint_key][key]GPIO-19 eint=19 keydata=129 status=0
292310 2021-11-10 15:16:29.772918 354 CM4 MSG_LOG 91 [M:eint_key C:info F: L: ]: [eint_key][key]GPIO-25 eint=25 keydata=130 status=1
292347 2021-11-10 15:16:29.772918 373 CM4 MSG_LOG 95 [M:apps C:info F: L: ]: [Key_remap] apps_config_key_remapper_init_configurable_table
292348 2021-11-10 15:16:29.772918 373 CM4 MSG_LOG 133 [M:apps C:info F: L: ]: [Key_remap] apps_config_key_remapper_init_configurable_table success, s_configurable_table_len = 7

```

Apps_config_key_remapper.c

```

00008: } * _init_default_configurable_table ?
00009:
00009: void apps_config_key_remapper_init_configurable_table(void)
00010: {
00010:     uint32_t size = 0;
00010:     bool push_to_work = false;
00011:     uint32_t i;
00011:     nvkey_status_t ret = nvkey_data_item_length(NVKEYID_CONFIGURABLE_KEY_ACTION_TABLE, &size);
00012:
00012:     APPS_LOG_MSGID_E(LOG_TAG, "apps_config_key_remapper_init_configurable_table", i);
00013:     if (NVKEY_STATUS_ITEM_NOT_FOUND == ret || size == 0) {
00013:         /* when read nvkey failed, initialize configurable table from hardcode. */
00014:         _init_default_configurable_table();
00015:     } else if (NVKEY_STATUS_OK == ret) {
00015:         if (s_configurable_table)
00016:             s_configurable_table = (s_configurable_table_t *)vPortFree(s_configurable_table);
00017:         s_configurable_table = (s_configurable_table_t *)vPortMalloc(size);
00018:         if (s_configurable_table) {
00018:             s_configurable_table_len = 0;
00019:             if (s_configurable_table) {
00019:                 ret = nvkey_read_data(NVKEYID_CONFIGURABLE_KEY_ACTION_TABLE,
00020:                                     (uint32_t *)&s_configurable_table, &size);
00021:                 if (NVKEY_STATUS_OK == ret) {
00021:                     s_configurable_table_len = size / (sizeof(apps_config_configurable_table_t));
00022:                     APPS_LOG_MSGID_E(LOG_TAG, "apps_config_key_remapper_init_configurable_table success, s_configurable_table_len = %d",
00023:                                     s_configurable_table_len);
00023:                 } else {
00024:                     APPS_LOG_MSGID_E(LOG_TAG, "apps_config_key_remapper_init_configurable_table, read data fail: %d", i, ret);
00024:                     vPortFree(s_configurable_table);
00025:                     s_configurable_table = NULL;
00025:                 }
00026:             } else {
00026:                 APPS_LOG_MSGID_E(LOG_TAG, "apps_config_key_remapper_init_configurable_table, malloc fail", 0);
00027:             }
00027:         }
00028:         if (NVKEY_STATUS_OK == ret) {
00028:             APPS_LOG_MSGID_W(LOG_TAG, "apps_config_key_remapper_init_configurable_table, read size ret = %d", i, ret);
00029:             if (s_configurable_table) {
00029:                 vPortFree(s_configurable_table);
00030:                 s_configurable_table = NULL;
00030:             }
00031:         }
00031:     }
00032: }

```

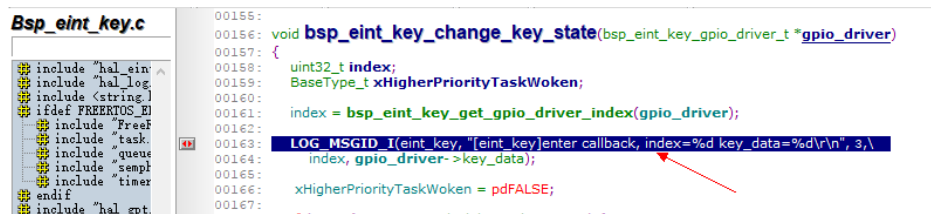
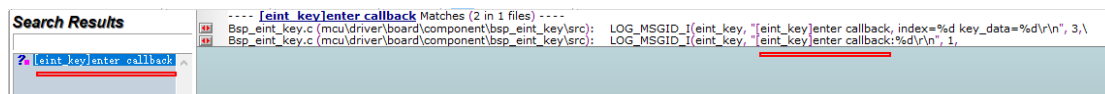
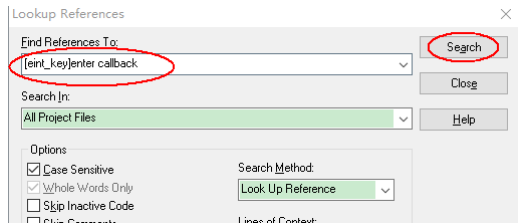
比如想知道 log 里的某条信息从哪个文件打印出来的


```

169 2021-11-10 20:43:37.757292 1012. CM4 MSG_LOG 97 [M:battery_management C:info F: L: ]: charging [soc_value: 100][batans :4343][step:2]
170 2021-11-10 20:43:37.757292 1012. CM4 MSG_LOG 87 [M:battery_management C:info F: L: ]: Charging smooth [soc: 100 Vbat :4342]
171 2021-11-10 20:43:37.757292 1012. CM4 MSG_LOG 110 [M:battery_management C:info F: L: ]: [BH]battery_linear_gauge_task VBUS on smooth_S0C:100 SOC100
172 2021-11-10 20:43:38.080390 1012. CM4 MSG_LOG 54 [M:common C:info F: L: ]: test_task:1011\r\n\r\n
173 2021-11-10 20:43:39.071818 1013. CM4 MSG_LOG 54 [M:common C:info F: L: ]: test_task:1012\r\n\r\n
174 2021-11-10 20:43:39.246717 1013. CM4 MSG_LOG 88 [M:eint_key C:info F: L: ]: [eint_key]enter callback, index=1 key_data=127\r\n\r\n
175 2021-11-10 20:43:39.246717 1013. CM4 MSG_LOG 126 [M:common C:info F: L: ]: [airo_key][process] enter state machine, index=6,config_index=2, key_nam
176 2021-11-10 20:43:39.246717 1013. CM4 MSG_LOG 90 [M:common C:info F: L: ]: [airo_key][state] current: row=0,col=0, state_id=2\r\n\r\n
177 2021-11-10 20:43:39.246717 1013. CM4 MSG_LOG 71 [M:common C:info F: L: ]: [airo_key][state] next: row = 1\r\n\r\n
178 2021-11-10 20:43:39.246717 1013. CM4 MSG_LOG 85 [M:apps C:info F: L: ]: [app_key_event]key event = 0x40, key_data = 0x7f

```

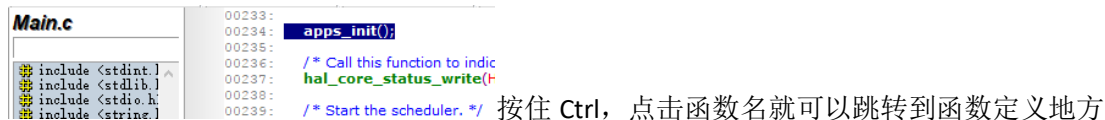
Search > Lookup References



由上述的 log 所在的函数的层层被调用一直往上顺藤摸瓜（右击函数名，选 Jump To Caller）找到 main 函数了。

...\mcu\project\ab1565_ab1568_evk\apps\headset_ref_design\src\main.c

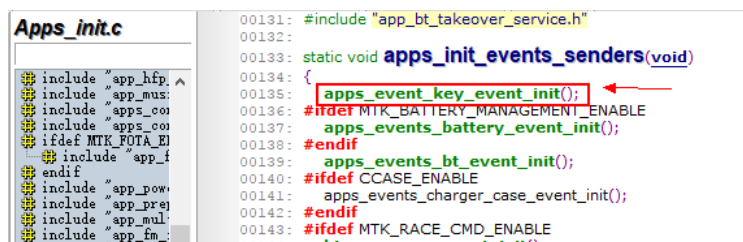
main 函数中调用 apps_init();应用程序初始化



apps_init()里调用 apps_init_events_senders();



apps_init_events_senders()里调用 apps_event_key_event_init();



apps_event_key_event_init()里调用 airo_key_register_callback(...);

```

Aapps_events_key 00270: void apps_event_key_event_init(void)
00271: {
00272:     bool status;
00273:     status = airo_key_event_init();
00274:     if (status != true) {
00275:         APPS_LOG_MSGID_E("sct key init fail, status:%d", 1, status);
00276:     }
00277:     #if !defined(AIR_SLIDE_POWER_KEY_ENABLE)
00278:     uint8_t reason = pmu_get_power_on_reason();
00279:     if (reason & 0x01) {
00280:         /* Power on reason is pressed power key, set the flag. */
00281:         s_press_from_power_on = pmu_get_pwrkey_state() == 1 ? true : false;
00282:         s_press_event_counter = s_press_from_power_on ? 2 : 0;
00283:     }
00284:     APPS_LOG_MSGID_I("inti key power on by reason : %x", 1, reason);
00285:     #endif
00286:     /* Register key callback. */
00287:     status = airo_key_register_callback(_apps_key_airo_key_callback, NULL);
00288:     if (status != true) {
00289:         APPS_LOG_MSGID_E("register callback fail, status:%d", 1, status);
00290:     }

```

airo_key_register_callback 调用了 airo_eint_key_init();

```

Airo_key_event.c 00141: bool airo_key_register_callback(airo_key_callback_t callback, void *user_data)
00142: {
00143:     if (callback == NULL) {
00144:         return false;
00145:     }
00146:     airo_key_event_context.callback = callback;
00147:     airo_key_event_context.user_data = user_data;
00148:     #if !defined(AIRO_KEY_FEATRURE_POWERKEY)
00149:     airo_powerkey_init();
00150:     #endif
00151:     #if !defined(AIRO_KEY_FEATRURE_POWERKEY_POWERON_PRESS_EVENT)
00152:     airo_powerkey_poweron_press_event_simulation();
00153:     #endif
00154:     #if !defined(MTK_KEYPAD_ENABLE)
00155:     airo_keypad_init();
00156:     #endif
00157:     #if !defined(MTK_EINT_KEY_ENABLE)
00158:     airo_eint_key_init();
00159:     #endif
00160:     #if !defined(AIRO_KEY_FEATRURE_EINTKEY_POWERON_PRESS_EVENT)
00161:     airo_eint_key_poweron_press_event_simulation();
00162:     #endif

```

airo_eint_key_init ()调用了 bsp_eint_key_init (...)

```

Airo_eint_key.c 00062: bool airo_eint_key_init(void)
00063: {
00064:     bsp_eint_key_config_t key_config;
00065:     uint32_t debounce_time;
00066:     key_config.longpress_time = 0;
00067:     key_config.repeat_time = 0;
00068:     /*set debouce as 5ms, more sensitive*/
00069:     debounce_time = 5;
00070:     bsp_eint_key_set_debounce_time(debounce_time);
00071:     /*register normal keypad callback*/
00072:     bsp_eint_key_register_callback(airo_eint_key_callback, NULL);
00073:     bsp_eint_key_enable();
00074:     return true;
00075: }

```

bsp_eint_key_init (...)调用了 bsp_eint_key_custom_init();

```

Bsp_eint_key.c 00285: bool bsp_eint_key_init(bsp_eint_key_config_t *config)
00286: {
00287:     uint8_t i;
00288:     uint8_t count = 0;
00289:     bool is_active;
00290:     bsp_eint_key_gpio_driver_t *gpio_driver;
00291:     #if !defined(BSP_EINT_KEY_TIMING_ENABLE)
00292:     if (config == NULL) {
00293:         LOG_MSGID_I(eint_key, "[eint_key]config pioter is null\r\n", 0);
00294:         return false;
00295:     }
00296:     #endif
00297:     if (eint_key_context.has_initiaized == true) {
00298:         LOG_MSGID_I(eint_key, "[eint_key]bsp eint key has initiaized\r\n", 0);
00299:         return false;
00300:     }
00301:     memset(&eint_key_context, 0, sizeof(bsp_eint_key_context_t));
00302:     bsp_eint_key_custom_init();
00303:     #if !defined(BSP_EINT_KEY_TIMING_ENABLE)
00304:     if (config->longpress_time == 0) {
00305:         eint_key_context.longpress_time = 2000; /*default 2s*/
00306:     }
00307:     #endif

```

bsp_eint_key_custom_init()调用了 BSP_EINTKEY_DRIVER_INIT(...);

```

Bsp_eint_key_custom.c 00112: /* config the GPIO port, eint mode, eint number, key data, active
00113: 00114: void bsp_eint_key_custom_init(void)
00115: {
00116:     uint32_t available_key_nums = 0;
00117:     BSP_EINTKEY_DRIVER_INIT(0, available_key_nums);
00118:     BSP_EINTKEY_DRIVER_INIT(1, available_key_nums);
00119:     BSP_EINTKEY_DRIVER_INIT(2, available_key_nums);
00120:     BSP_EINTKEY_DRIVER_INIT(3, available_key_nums);
00121:     BSP_EINTKEY_DRIVER_INIT(4, available_key_nums);
00122: }

```

```

Bsp_eint_key_custom.c 0009: #define BSP_EINTKEY_DRIVER_INIT(key_index, available_key_nums)\
00010: {\
00011:     if (BSP_EINTKEY##key_index##_PIN != 0x0ff) {\
00012:         if (BSP_EINTKEY##key_index##_EINT != 0x0ff) {\
00013:             bsp_eint_key_normal_gpio_driver_t*new_driver = &bsp_eint_key_gpio_driver_buf[key_index].normal\
00014:             bsp_eint_key_normal_gpio_driver_new(new_driver, BSP_EINTKEY##key_index##_PIN, BSP_EINTKE\
00015:             new_driver->gpio_port = BSP_EINTKEY##key_index##_PIN;\
00016:             new_driver->driver_key_data = BSP_EINT_KEY_DATA##key_index;\
00017:             BSP_EINTKEY_SET_ACTIVE_LEVEL(new_driver, BSP_EINTKEY##key_index##_ACTIVE_LEVEL);\
00018:         } else {\
00019:             BSP_EINTKEY_RTC_GPIO_DRIVER_INIT(key_index);\
00020:         }\
00021:         available_key_nums++;\
00022:     }\
00023: }
00024:
00025: #endif AIR_BSP_EINT_KEY_AC
00026: #if BSP_EINTKEY_DECLARE_AC
00027: BSP_EINTKEY_SET_ACTIVE
00028: else
00029: BSP_EINTKEY_DECLARE_AC
00030: BSP_EINTKEY_SET_ACTIVE
00031:
00032:
00033:
00034:

```

```

Bsp_eint_key_normal_gpio.c
#include "bsp_eint_key_custom.h"
#include "bsp_eint_key_normal_gpio.h"
#include "syslog.h"
#ifdef EINT_KEY_HAL_MASK
if (PRODUCT_VERSION == 7697)
    eint_ack_interrupt
endif
endif
bsp_eint_key_normal_gpio_callback
bsp_eint_key_normal_gpio_init
eint_config

00133: void bsp_eint_key_normal_gpio_driver_new(
00134:     bsp_eint_key_normal_gpio_driver_t *new_driver, const char gpio_port, c
00135: {
00136:     new_driver->gpio_port = gpio_port;
00137:     new_driver->eint_mode = eint_mode;
00138:     new_driver->eint_number = eint_number;
00139:     new_driver->driver_init = bsp_eint_key_normal_gpio_init;
00140:     new_driver->driver_enable = bsp_eint_key_normal_gpio_enable;
00141:     new_driver->driver_is_active = bsp_eint_key_normal_gpio_driver_is
00142:     new_driver->driver_set_debounce_time_msec = bsp_eint_key_normal_
00143:     bsp_eint_key_register_gpio_driver((bsp_eint_key_gpio_driver_t*)new
00144: }

```

```

00107:
00108: int bsp_eint_key_register_gpio_driver(bsp_eint_key_gpio_driver_t *new_driver)
00109: {
00110:     static int bsp_eint_key_registered_driver_num = 0;
00111:
00112:     if (bsp_eint_key_registered_driver_num < BSP_EINT_KEY_NUMBERS) {
00113:         eint_key_context_gpio_driver[bsp_eint_key_registered_driver_num] = new_driver;
00114:         bsp_eint_key_registered_driver_num++;
00115:         return bsp_eint_key_registered_driver_num;
00116:     }
00117:     return -1;
00118: }

```

```

00069:                                     /*user could modify for your keydata*/
00070: #define BSP_EINT_KEY_NUMBER      5 /*default supported key
00071:                                     /*key defined by use
00072: #define BSP_EINT_KEY_DATA0      126 /*key defined by use
00073: #define BSP_EINT_KEY_DATA1      127 /*key defined by use
00074: #define BSP_EINT_KEY_DATA2      128 /*key defined by use
00075: #define BSP_EINT_KEY_DATA3      129 /*key defined by use
00076: #define BSP_EINT_KEY_DATA4      130 /*key defined by use

```

```

Bsp_eint_key_normal_gpio.c      00045: static void bsp_eint_key_normal_gpio_callback(void *user_data)
                                00046: {
                                00047:     bsp_eint_key_normal_gpio_driver_t *normal_gpio_driver = (bsp_eint_key_normal_gpio_driver_t *)user_data;
                                00048:     bsp_eint_key_change_key_state(user_data);
                                00049:     #ifndef EINT_KEY_HAL_MASK
                                00050:         hal_eint_unmask(normal_gpio_driver->eint_number);
                                00051:     #endif
                                00052: }

```

```

Bsp_eint_key.c
#include "hal_eint.h"
#include "hal_log.h"
#include <string.h>
#ifdef FREERTOS_E
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"
#include "timer.h"
#endif
#include "hal_gpio.h"
#include <stdint.h>

00156: void bsp_eint_key_change_key_state(bsp_eint_key_gpio_driver_t *gpio_driver)
00157: {
00158:     uint32_t index;
00159:     BaseType_t xHigherPriorityTaskWoken;
00160:
00161:     index = bsp_eint_key_get_gpio_driver_index(gpio_driver);
00162:
00163:     LOG_MSGID_I(eint_key, "Eint_key]enter callback, index=%d key_data=%d\n", 3,
00164:         index, gpio_driver->key_data);
00165:
00166:     xHigherPriorityTaskWoken = pdFALSE;
00167:
00168:     if (eint_key_context.is_init_ready == true) {
00169:         gpio_driver->key_state = BSP_EINT_KEY_RELEASE;

```

```

0009e:
0009f: static uint32_t bsp_eint_key_get_gpio_driver_index(bsp_eint_key_gpio_driver_t *driver)
00099:
00099: int i;
00100: for (i = 0; i < BSP_EINT_KEY_NUMBER; i++) {
00101:     if (driver == eint_key_context_gpio_driver[i]) {
00102:         return i;
00103:     }
00104: }
00105: return -1;
00106: }

```

然后调用函数 `airo_key_process_key(...)`, 打印 log 信息:

```
[M:common C:info F: L: ]: [airo_key][process] enter state machine, index=5,config_index=1, key_name=126,action=0\r\n
```

```
#ifdef DEBUG_AIRO_BASIC_FLOW_LOG
LOG_MSGID_I(common, "[airo_key][process] enter state machine, index=%d,config_index=%d, key_name=%d,action=%d\r\n", 4, index, config_index, key_name, action_type);
#endif
```

接着调用函数 `airo_key_state_machine(...)`, 打印 log 信息:

```
[M:common C:info F: L: ]: [airo_key][state] current: row=0,col=0, state id=2\r\n
```

```
[M:common C:info F: L: ]: [airo_key][state] next: row = 1\r\n
```

```
#ifdef DEBUG_AIRO_BASIC_FLOW_LOG
LOG_MSGID_I(common, "[airo_key][state] current: row=%d,col=%d, state id=%d\r\n", 3, airo_key_event_state.airo_state[index], action, p_key_note->id);
#endif
```

```
#ifdef DEBUG_AIRO_BASIC_FLOW_LOG
LOG_MSGID_I(common, "[airo_key][state] next: row = %d\r\n", 1, airo_key_event_state.airo_state[index]);
#endif
```

接着调用函数 `_apps_key_airo_key_callback(...)`, 打印 log 信息:

```
[M:apps C:info F: L: ]: [app_key_event]key event = 0x40, key_data = 0x7e
```

```
Apps_events_key_event.c 00052:
00053: #define LOG_TAG "[app_key_event]" /* Log tag */
00054:
00055: static bool s_press_from_power_on = false; /* If user press the key from power on, need do spe
00056: static int32_t s_press_event_counter = 0; /* When power on reason is press key, count the time
00057:
00058: extern uint8_t pmu_get_power_on_reason();
00059:
00060: /**
00061:  * @brief The implementation of the key callback.
00062:  * @param[in] event, the key event type, refer to airo_key_event_t.
00063:  * @param[in] key_data, means key id, ex. DEVICE_KEY_POWER or EINT_KEY_0.
00064:  * @param[in] user_data, The same user_data input in airo_key_register_callback()
00065:  */
00066: static void _apps_key_airo_key_callback(airo_key_event_t event, uint8_t key_data, void
00067: {
00068:     APPS_LOG_MSGID_I(LOG_TAG"key event = 0x%x, key_data = 0x%x", 2, event, key_data);
00069:     uint16_t *p_key_action = (uint16_t *)pvPortMalloc(sizeof(uint16_t));
00070:     ui_shell_status_t status;
00071:     if (defined(AIRO_KEY_ENABLE))
```

接着调用函数 `ui_shell_send_event(...)`, 打印 log 信息:

```
[M:ui_shell C:info F: L: ]: ui_shell_send_event: 0x2, 0x407e, delay:0
```

```
Ui_shell_manager.c 00310: ui_shell_status_t ui_shell_send_event(
00311: bool from_isr,
00312: ui_shell_event_priority_t priority,
00313: uint32_t event_group,
00314: uint32_t event_id,
00315: void *data, size_t data_len,
00316: void (*special_free_extra_func)(void),
00317: uint32_t delay_ms)
00318: {
00319:     ui_shell_status_t ret = UI_SHELL_STATUS_OK;
00320:     if (!s_message_handler) {
00321:         UI_SHELL_LOG_MSGID_E("Should not call send event before ui shell running", 0);
00322:         ret = UI_SHELL_STATUS_INVALID_STATE;
00323:     } else if (priority > EVENT_PRIORITY_UI_SHELL_SYSTEM) {
00324:         ret = UI_SHELL_STATUS_INVALID_PARAMETER;
00325:     } else {
00326:         UI_SHELL_LOG_MSGID_I("ui_shell_send_event: 0x%x, 0x%x, delay:%d", 3, event_gro
00327:         if (delay_ms) {
00328:             ui_shell_delay_event_group(event_group, delay_ms);
00329:         }
00330:         ret = UI_SHELL_STATUS_OK;
00331:     }
00332:     return ret;
00333: }
```

按下主键 (GPIO3 映射到 EINT_KEY2), 打印 log 信息:

```
[M:eint_key C:info F: L: ]: [eint_key]enter callback, index=2 key_data=128\r\n
[M:common C:info F: L: ]: [airo_key][timer]stop time handle=0xaaaa0005,index=7\r\n
[M:common C:info F: L: ]: [airo_key][timer]free time handle=0xaaaa0005,index=7\r\n
[M:common C:info F: L: ]: [airo_key][process] enter state machine, index=7,config_index=3, key_name=128,action=0\r\n
[M:common C:info F: L: ]: [airo_key][state] current: row=5,col=0, state id=8\r\n
[M:common C:info F: L: ]: [airo_key][state] next: row = 6\r\n
[M:apps C:info F: L: ]: [app_key_event]key event = 0x40, key_data = 0x80
[M:ui_shell C:info F: L: ]: ui_shell_send_event: 0x2, 0x4080, delay:0
[M:common C:info F: L: ]: [airo_key][timer]allocate timer handle=0xaaaa0005,index=7\r\n
[M:common C:info F: L: ]: [airo_key][timer]start time handle=0xaaaa0005,index=7,time=500\r\n
[M:common C:info F: L: ]: [airo_key][process] exit airo_key_process_key\r\n
[M:ui_shell C:info F: L: ]: handle message group : 0x2, id : 0x4080, extra_data : 0x14213820, data_len : 0x2, free_func : 0x0
[M:apps C:info F: L: ]: [Key_remap] Current used key_id = 0x80, key_event = 0x40, state = 24, action_id = 0x0, channel = 0
[M:apps C:info F: L: ]: [vp_manager] set VP preempt index=12 sync=0 delay=0 level=2
```

```
#define EINT_KEY_0 126 /*eint key 0*/
#define EINT_KEY_1 127 /*eint key 1*/
#define EINT_KEY_2 128 /*eint key 2*/
#define EINT_KEY_3 129 /*eint key 3*/
#define EINT_KEY_4 130 /*eint key 4*/
#define EINT_KEY_5 131 /*eint key 5*/
#define EINT_KEY_6 132 /*eint key 6*/
```

这条 log 是由函数 apps_config_key_event_remapper_map_action 打印

[M:apps C:info F: L:]: [Key_remap] Current used key_id = 0x80, key_event = 0x40, state = 24, action_id = 0x0, channel = 0

```
Apps_config_key_remapper.c 00352:         break;
00353:     }
00354: }
00355: } end if serialized_event_id<...?
00356: APPS_LOG_MSGID_I(LOG_TAG"Current used key_id = 0x%x, key_event = 0x%x, state = %d, action_id = 0x%x, channel = %d", s, key_id, key_event, temp_mmi_s
00357: return action_id;
00358: } end apps_config_key_event_remapper_map_action_in_temp_state ?
00359: }
```

```
Apps_config_key_remapper.c 00216: apps_config_key_action_t apps_config_key_event_remapper_map_action(
00217: uint8_t key_id
00218: airo_key_event_t key_event)
00219: {
00220:     apps_config_key_action_t action_id = KEY_ACTION_INVALID;
00221:     size_t i;
00222:     apps_config_serialized_event_t serialized_event_id = APPS_CONFIG_KEY_INVALID;
00223:     const apps_config_key_event_map_list_t *mapped_list = NULL;
00224:     audio_channel_t channel = 0;
00225:     #ifdef MTK_AWS_MCE_ENABLE
00226:     channel = ami_get_audio_channel(); // 获取声道通道
00227:     #endif
00228:     /* Convert the key event from airo_key_event_t to apps_config_serialized_event_t. */
00229:     switch (key_event) {
00230:     case AIRO_KEY_RELEASE:
00231:         serialized_event_id = APPS_CONFIG_KEY_RELEASE;
00232:         break;
00233:     case AIRO_KEY_SHORT_CLICK:
00234:         break;
00235:     }
```

```
Apps_config_key_remapper.c 00303:     }
00304:     #if MTK_AWS_MCE_ENABLE
00305:     /* When device is left, use the left mapping tables */
00306:     if (AUDIO_CHANNEL_L == channel) {
00307:         mapped_list = &s_left_map_list(serialized_event_id);
00308:     } else
00309:     #endif
00310:     /* Use the default(right side) mapping tables */
00311:     mapped_list = &s_map_list(serialized_event_id);
00312:     /* For debug log
00313:     APPS_LOG_MSGID_I(LOG_TAG"Current used key map [%d]", 1, seri
00314:     for (j = 0; j < s_map_list(serialized_event_id).map_size; j++) {
```

上述的 log 中 channel = 0 说明是音频左声道

```
Bt_sink_srv_ami.h 00918: /** @brief Get Audio Channel in DSP */
00919: typedef enum {
00920:     AUDIO_CHANNEL_NONE = -1, /**< Audio channel none. */
00921:     AUDIO_CHANNEL_L = 0, /**< Audio channel L. */
00922:     AUDIO_CHANNEL_R = 1, /**< Audio channel R. */
00923:     AUDIO_CHANNEL_SWAP = 2, /**< Audio channel SWAP. */
00924: } audio_channel_t;
00925: #if defined(MTK_PEQ_ENABLE) || defined(MTK_LINEIN_PEQ_ENABLE)
00926: #define PEQ_DEFAULT_ENABLE (0xFF)
00927: #define PEQ_DEFAULT_SOUND_MODE (1)
```

按下左声道的 Power Key 电源键，打印 log 信息：

[M:apps C:info F: L:]: [Key_remap] Current used key_id from configuration_table = 0x18, key_event = 0x1, state = 3, action_id = 0x53, channel = 0

```
Apps_config_key_remapper.c 00285: } end switch key_event ?
00286: if (serialized_event_id < APPS_CONFIG_KEY_INVALID) {
00287:     /* If the configurable table have not been initialized from system on, initialize it. */
00288:     if (!s_configurable_table && !s_configurable_table_ready) {
00289:         _init_default_configurable_table();
00290:     }
00291:     /* The configurable table have higher priority than the hard code table. */
00292:     if (s_configurable_table) {
00293:         for (i = 0; i < s_configurable_table_len; i++) {
00294:             if (serialized_event_id == s_configurable_table[i].key_event
00295:                 && key_id == s_configurable_table[i].key_id
00296:                 && ((1 <= s_mmi_state) & s_configurable_table[i].supported_states)) {
00297:                 action_id = s_configurable_table[i].app_key_event;
00298:                 APPS_LOG_MSGID_I(LOG_TAG"Current used key_id from configuration_table = 0x%x, key_event = 0x%x, state = %d, action_id = 0x%x, char
00299:                 return action_id;
00300:             }
00301:         }
00302:     }
00303: }
```

为了验证上述的跟踪判断是否正确，修改程序里的 log 信息：

信息中加 888888 或 666


```

Apps_config_key_remapper.c 001002: /* Initialize the mapping table, if this is an AWS project, the variable is for right side. */
001003: const static apps_config_key_event_map_list_t const s_map_list[APPS_CONFIG_KEY_IN]
001004: {
001005:     { sizeof(temp_key_release_configs) / sizeof(apps_config_key_event_map_t),
001006:       temp_key_release_configs
001007:     },
001008:     { sizeof(temp_key_short_click_configs) / sizeof(apps_config_key_event_map_t),
001009:       temp_key_short_click_configs
001010:     },
001011:     {
001012:     },
001013:     { sizeof(temp_key_double_click_configs) / sizeof(apps_config_key_event_map_t),
001014:       temp_key_double_click_configs
001015:     },
001016:     {
001017:     },
001018:     { sizeof(temp_key_triple_click_configs) / sizeof(apps_config_key_event_map_t),
001019:       temp_key_triple_click_configs
001020:     },
001021:     {
001022:     },
001023:     { sizeof(temp_key_long_press1_configs) / sizeof(apps_config_key_event_map_t),
001024:       temp_key_long_press1_configs
001025:     },
001026:     {
001027:     },
001028:     { sizeof(temp_key_long_press2_configs) / sizeof(apps_config_key_event_map_t),
001029:       temp_key_long_press2_configs
001030:     },
001031:     {
001032:     },
001033:     { sizeof(temp_key_long_press3_configs) / sizeof(apps_config_key_event_map_t),
001034:       temp_key_long_press3_configs
001035:     },
001036:     {
001037:     },
001038:     {
001039:     },
001040:     {
001041:     },
001042:     {
001043:     },
001044:     {
001045:     },
001046:     {
001047:     },
001048:     {
001049:     },
001050:     {
001051:     },
001052:     {
001053:     },
001054:     {
001055:     },
001056:     {
001057:     },
001058:     {
001059:     },
001060:     {
001061:     },
001062:     {
001063:     },
001064:     {
001065:     },
001066:     {
001067:     },
001068:     {
001069:     },
001070:     {
001071:     },
001072:     {
001073:     },
001074:     {
001075:     },
001076:     {
001077:     },
001078:     {
001079:     },
001080:     {
001081:     },
001082:     {
001083:     },
001084:     {
001085:     },
001086:     {
001087:     },
001088:     {
001089:     },
001090:     {
001091:     },
001092:     {
001093:     },
001094:     {
001095:     },
001096:     {
001097:     },
001098:     {
001099:     },
001100:     {
001101:     },
001102:     {
001103:     },
001104:     {
001105:     },
001106:     {
001107:     },
001108:     {
001109:     },
001110:     {
001111:     },
001112:     {
001113:     },
001114:     {
001115:     },
001116:     {
001117:     },
001118:     {
001119:     },
001120:     {
001121:     },
001122:     {
001123:     },
001124:     {
001125:     },
001126:     {
001127:     },
001128:     {
001129:     },
001130:     {
001131:     },
001132:     {
001133:     },
001134:     {
001135:     },
001136:     {
001137:     },
001138:     {
001139:     },
001140:     {
001141:     },
001142:     {
001143:     },
001144:     {
001145:     },
001146:     {
001147:     },
001148:     {
001149:     },
001150:     {
001151:     },
001152:     {
001153:     },
001154:     {
001155:     },
001156:     {
001157:     },
001158:     {
001159:     },
001160:     {
001161:     },
001162:     {
001163:     },
001164:     {
001165:     },
001166:     {
001167:     },
001168:     {
001169:     },
001170:     {
001171:     },
001172:     {
001173:     },
001174:     {
001175:     },
001176:     {
001177:     },
001178:     {
001179:     },
001180:     {
001181:     },
001182:     {
001183:     },
001184:     {
001185:     },
001186:     {
001187:     },
001188:     {
001189:     },
001190:     {
001191:     },
001192:     {
001193:     },
001194:     {
001195:     },
001196:     {
001197:     },
001198:     {
001199:     },
001200:     {
001201:     },
001202:     {
001203:     },
001204:     {
001205:     },
001206:     {
001207:     },
001208:     {
001209:     },
001210:     {
001211:     },
001212:     {
001213:     },
001214:     {
001215:     },
001216:     {
001217:     },
001218:     {
001219:     },
001220:     {
001221:     },
001222:     {
001223:     },
001224:     {
001225:     },
001226:     {
001227:     },
001228:     {
001229:     },
001230:     {
001231:     },
001232:     {
001233:     },
001234:     {
001235:     },
001236:     {
001237:     },
001238:     {
001239:     },
001240:     {
001241:     },
001242:     {
001243:     },
001244:     {
001245:     },
001246:     {
001247:     },
001248:     {
001249:     },
001250:     {
001251:     },
001252:     {
001253:     },
001254:     {
001255:     },
001256:     {
001257:     },
001258:     {
001259:     },
001260:     {
001261:     },
001262:     {
001263:     },
001264:     {
001265:     },
001266:     {
001267:     },
001268:     {
001269:     },
001270:     {
001271:     },
001272:     {
001273:     },
001274:     {
001275:     },
001276:     {
001277:     },
001278:     {
001279:     },
001280:     {
001281:     },
001282:     {
001283:     },
001284:     {
001285:     },
001286:     {
001287:     },
001288:     {
001289:     },
001290:     {
001291:     },
001292:     {
001293:     },
001294:     {
001295:     },
001296:     {
001297:     },
001298:     {
001299:     },
001300:     {
001301:     },
001302:     {
001303:     },
001304:     {
001305:     },
001306:     {
001307:     },
001308:     {
001309:     },
001310:     {
001311:     },
001312:     {
001313:     },
001314:     {
001315:     },
001316:     {
001317:     },
001318:     {
001319:     },
001320:     {
001321:     },
001322:     {
001323:     },
001324:     {
001325:     },
001326:     {
001327:     },
001328:     {
001329:     },
001330:     {
001331:     },
001332:     {
001333:     },
001334:     {
001335:     },
001336:     {
001337:     },
001338:     {
001339:     },
001340:     {
001341:     },
001342:     {
001343:     },
001344:     {
001345:     },
001346:     {
001347:     },
001348:     {
001349:     },
001350:     {
001351:     },
001352:     {
001353:     },
001354:     {
001355:     },
001356:     {
001357:     },
001358:     {
001359:     },
001360:     {
001361:     },
001362:     {
001363:     },
001364:     {
001365:     },
001366:     {
001367:     },
001368:     {
001369:     },
001370:     {
001371:     },
001372:     {
001373:     },
001374:     {
001375:     },
001376:     {
001377:     },
001378:     {
001379:     },
001380:     {
001381:     },
001382:     {
001383:     },
001384:     {
001385:     },
001386:     {
001387:     },
001388:     {
001389:     },
001390:     {
001391:
```

```

Customized_key_config.c      00051: const apps_config_key_event_map_t temp_key_short_click_configs[] = {
                                00052: {
                                00053:     EINT_KEY_2, ←
                                00054:     KEY_ACCEPT_CALL,
                                00055:     (1 << APP_HFP_INCOMING)
                                00056: },
                                00057: {
                                00058:     EINT_KEY_2, ←
                                00059:     KEY_3WAY_HOLD_ACTIVE_ACCEPT_OTHER,
                                00060:     (1 << APP_HFP_TWC_INCOMING)
                                00061: },
                                00062: {
                                00063:     EINT_KEY_2, ←
                                00064:     KEY_END_CALL,
                                00065:     (1 << APP_HFP_CALLTACTIVE) | (1 << APP_HFP_CALLTACTIVE_WITHOUT_SCO) | (1 <
                                00066:     | (1 << APP_HFP_OUTGOING) | (1 << APP_STATE_HELD_ACTIVE) | (1 << APP_HFP_
                                00067: },
                                00068: {
                                00069:     EINT_KEY_0, ←
                                00070:     KEY_VOICE_UP, ←
                                00071:     (<< APP_CONNECTED) | (1 << APP_HFP_INCOMING) | (1 << APP_HFP_OUTGOING) |
                                00072:     | (1 << APP_HFP_TWC_INCOMING) | (1 << APP_HFP_TWC_OUTGOING) | (1 << APP_
                                00073:     | (1 << APP_STATE_HELD_ACTIVE) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (
                                00074: },
                                00075: {
                                00076:     EINT_KEY_1, ←
                                00077:     KEY_VOICE_DN, ←
                                00078:     (<< APP_CONNECTED) | (1 << APP_HFP_INCOMING) | (1 << APP_HFP_OUTGOING) |
                                00079:     | (1 << APP_HFP_TWC_INCOMING) | (1 << APP_HFP_TWC_OUTGOING) | (1 << APP_
                                00080:     | (1 << APP_STATE_HELD_ACTIVE) | (1 << APP_ULTRA_LOW_LATENCY_PLAYING) | (
                                00081: },
                                00082: #if defined(AIR_BT_ULTRA_LOW_LATENCY_ENABLE)

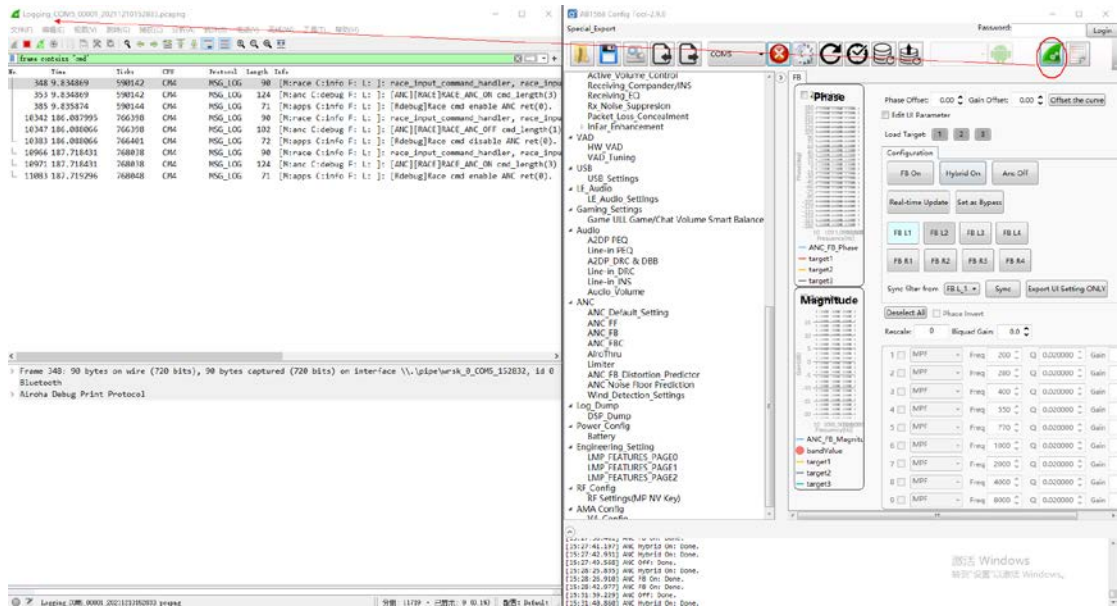
```

如果是左声道就把 default configurable table 赋给 peer code table

```

00415: */
00416: static void _init_default_configurable_table(void)
00417: {
00418:     const apps_config_configurable_table_t *hard_code_table = NULL; /* The pointer to hardcode configurable table.
00419:     uint32_t hard_code_table_size = 0;
00420:     const apps_config_configurable_table_t *va_table = NULL;
00421:     uint32_t va_table_size = 0;
00422:     nvkey_status_t nvkey_ret;
00423:     /* When not support COMPETITION, need init key by VA type. */
00424:     apps_config_key_get_va_key_table(multi_va_manager_get_current_va_type(), &va_table, &va_table_size
00425: #if MTK_ENABLE_MTK_SETTING
00426:     const apps_config_configurable_table_t *peer_code_table = NULL; /* Hard code table of peer side. */
00427:     uint32_t peer_code_table_size = 0;
00428:     audio_channel_t channel = ami_get_audio_channel();
00429:     /* There are 2 hard code configurable tables -- default_configurable_table and left_configurable_table.
00430:     Set the pointer depends on current device is left or right. */
00431:     if (AUDIO_CHANNEL_L == channel) {
00432:         hard_code_table = va_table;
00433:         hard_code_table_size = va_table_size;
00434:         peer_code_table = default_configurable_table;
00435:         peer_code_table_size = sizeof(default_configurable_table);
00436:     } else if (AUDIO_CHANNEL_R == channel) {
00437:         hard_code_table = default_configurable_table;
00438:         hard_code_table_size = sizeof(default_configurable_table);
00439:         peer_code_table = va_table;
00440:         peer_code_table_size = va_table_size;
00441:     } else {

```

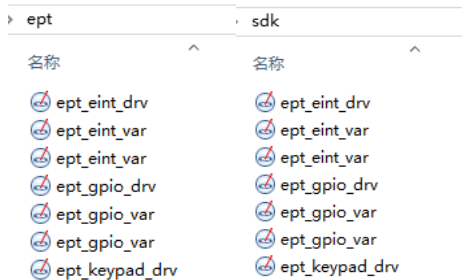



三十四、ANC 相关参数

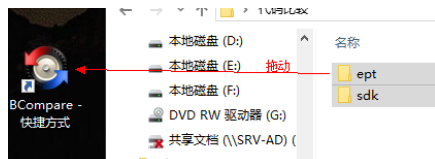
ANC 有四种模式（Hybrid 混合式(FF 前馈、FB 反馈)、PassThrough/StayAware/TalkThrough 通透、ANC Off、Crusher）
Gain 值，即增益

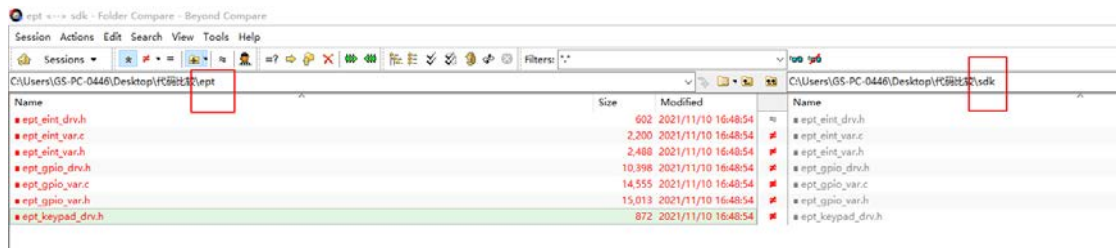
三十五、BCompare 比较文件代码差异

需要找出两个文件夹里的多个相同文件之间的代码差异

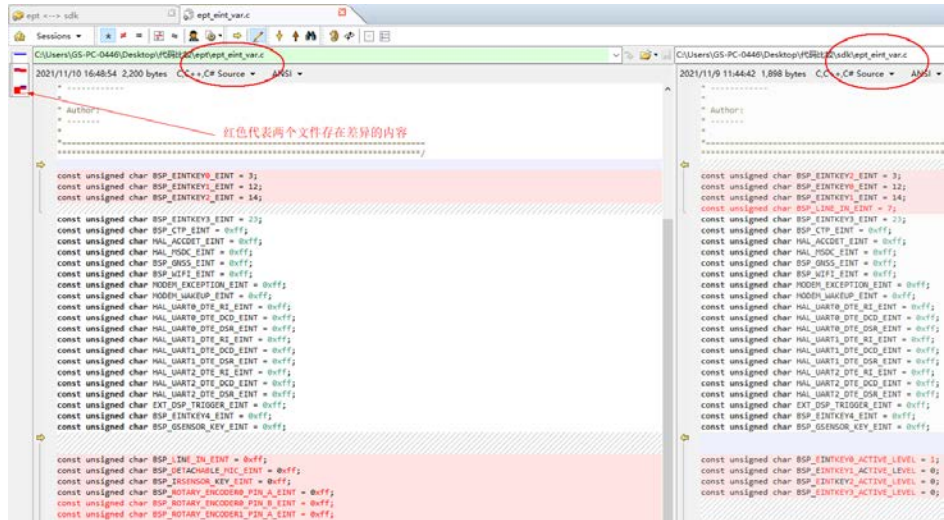


同时选中两个文件夹并拖到软件图标以打开文件





在左侧双击要比较的文件即可打开，一目了然地知道内容的差异之处



三十六、抓工程的 log 追踪旋转编码器执行

把第三方修改的可以通过旋转编码器调音量的工程下载到耳机，并抓其运行 log 来找到编码器事件

FW 文件夹

名称	修改日期	类
MACOSX	2021/11/18 19:41	文
ab1568_bootloader.bin	2021/11/12 23:47	BI
bos_headset_ref_design.bin	2021/11/12 23:47	BI
dsp0_headset_ref_design.bin	2021/11/12 23:47	BI
filesystem.bin	2021/11/12 23:47	BI
flash_download.cfg	2021/11/12 23:47	CF
nvdn.bin	2021/11/12 23:47	BI
partition_table.bin	2021/11/12 23:47	BI

COM Port

COM5

Start

onfig File

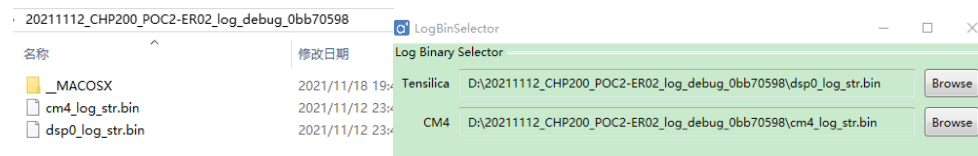
D:\20211112_CHP200_POC2-ER02_PCB1112_binaries_0bb70598\flash_download.cfg

Open

Download Information

☒	Name	Begin Address	End Address	Address Type	File Path
☒	BootLoader	0x08003000	0x0800c9a4	physical	D:\20211112_CHP200_POC2-ER02_PCB1112_binaries_0bb70598\ab1568_bootloader.bin
☒	PartitionTable	0x08000000	0x080001b0	physical	D:\20211112_CHP200_POC2-ER02_PCB1112_binaries_0bb70598\partition_table.bin
☒	DSP_FW	0x08013000	0x0808c510	physical	D:\20211112_CHP200_POC2-ER02_PCB1112_binaries_0bb70598\dsp0_headset_ref_design.bin
☒	MCU_FW	0x08133000	0x08261d1c	physical	D:\20211112_CHP200_POC2-ER02_PCB1112_binaries_0bb70598\bos_headset_ref_design.bin
☒	ROFS	0x0845f000	0x084a1000	physical	D:\20211112_CHP200_POC2-ER02_PCB1112_binaries_0bb70598\filesystem.bin
☒	NVDM	0x087e8000	0x08800000	physical	D:\20211112_CHP200_POC2-ER02_PCB1112_binaries_0bb70598\nvdn.bin

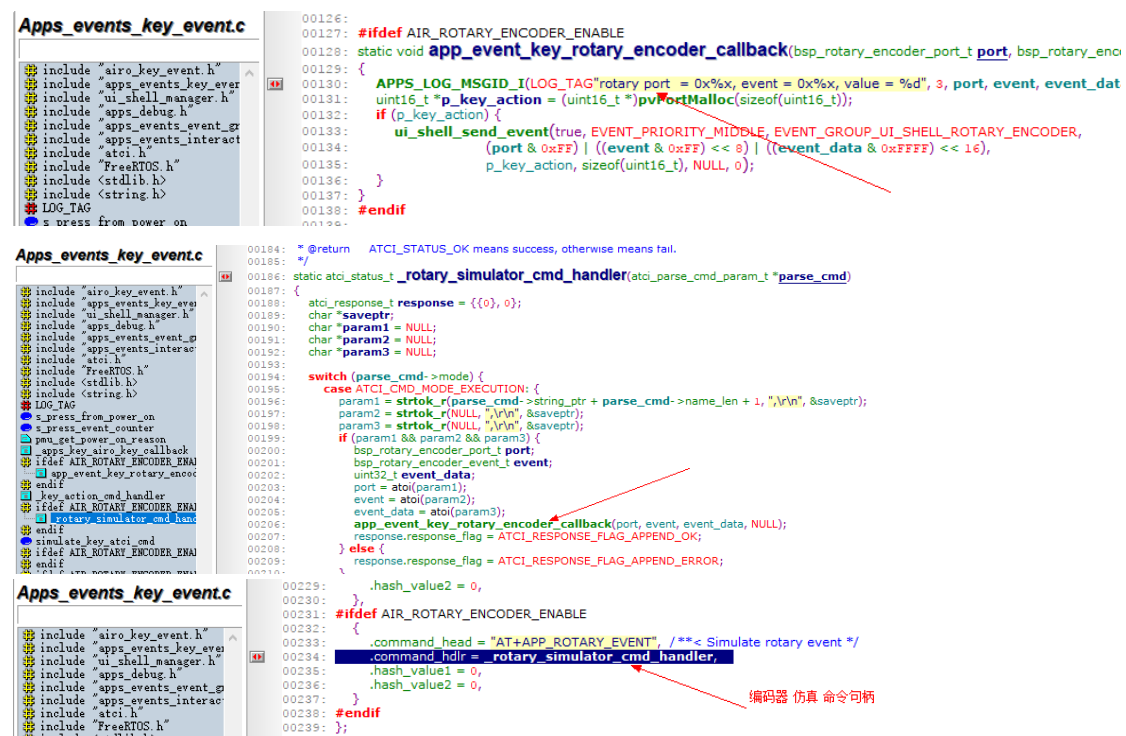
Log 文件夹



Log 信息打印：编码器上旋转时 event 为 0x1，下旋转时 event 为 0x0

No.	Time	Ticks	CPU	Protocol	Length	Info
141	2021-11-18 19:45:13.417902	45477	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x1, value = 1
373	2021-11-18 19:45:19.831389	51741	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x1, value = 1
1608	2021-11-18 19:46:37.928888	127...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1
1644	2021-11-18 19:46:38.569614	128...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1
1702	2021-11-18 19:46:40.349079	130...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x1, value = 1
1736	2021-11-18 19:46:41.989222	131...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x1, value = 1
1785	2021-11-18 19:46:43.540007	133...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x1, value = 1
1873	2021-11-18 19:46:48.657746	138...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x1, value = 1
1910	2021-11-18 19:46:49.216348	139...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1
6861	2021-11-18 19:52:10.047079	452...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x1, value = 1
6919	2021-11-18 19:52:12.330843	454...	CM4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1

Log 是在回调函数 app_event_key_rotary_encoder_callback 中打印的



旋转编码器是这样初始化的

Frame contains "event"									
No	Time	Ticks	CPU	Protocol	Length	Info			
1228	2021-11-18 20:52:45.131580	406	0M4	MSG_LOG	123	[M:apps C:info F: L:]: [Music_APP]activity app_music_proc_rotary_event_group, EX_CODEC_I2C Crusher volume : 5			
1228	2021-11-18 20:52:45.766216	406	0M4	MSG_LOG	85	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=742ms			
1228	2021-11-18 20:52:45.863146	406	0M4	MSG_LOG	84	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=96ms			
1228	2021-11-18 20:52:45.863653	406	0M4	MSG_LOG	91	[M:common C:info F: L:]: [bsp][rotary encoder] event_count=0 current_pos=32777			
1228	2021-11-18 20:52:45.976379	406	0M4	MSG_LOG	84	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=98ms			
1228	2021-11-18 20:52:45.976379	406	0M4	MSG_LOG	73	[M:common C:info F: L:]: [bsp][rotary encoder] event timeout			
1228	2021-11-18 20:52:45.976379	406	0M4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1			
1228	2021-11-18 20:52:45.976379	406	0M4	MSG_LOG	82	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x3, 0x10000, delay:0			
1228	2021-11-18 20:52:45.976379	406	0M4	MSG_LOG	123	[M:apps C:info F: L:]: [Music_APP]activity app_music_proc_rotary_event_group, EX_CODEC_I2C Crusher volume : 4			
1229	2021-11-18 20:52:46.092337	406	0M4	MSG_LOG	85	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=131ms			
1229	2021-11-18 20:52:46.092337	406	0M4	MSG_LOG	91	[M:common C:info F: L:]: [bsp][rotary encoder] event_count=0 current_pos=32778			
1229	2021-11-18 20:52:46.198415	406	0M4	MSG_LOG	73	[M:common C:info F: L:]: [bsp][rotary encoder] event timeout			
1229	2021-11-18 20:52:46.198415	406	0M4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1			
1229	2021-11-18 20:52:46.198415	406	0M4	MSG_LOG	82	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x3, 0x10000, delay:0			
1229	2021-11-18 20:52:46.198415	406	0M4	MSG_LOG	123	[M:apps C:info F: L:]: [Music_APP]activity app_music_proc_rotary_event_group, EX_CODEC_I2C Crusher volume : 3			
1229	2021-11-18 20:52:46.847566	406	0M4	MSG_LOG	85	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=753ms			
1229	2021-11-18 20:52:46.959057	406	0M4	MSG_LOG	85	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=111ms			
1229	2021-11-18 20:52:46.959057	406	0M4	MSG_LOG	91	[M:common C:info F: L:]: [bsp][rotary encoder] event_count=0 current_pos=32779			
1229	2021-11-18 20:52:47.067398	406	0M4	MSG_LOG	73	[M:common C:info F: L:]: [bsp][rotary encoder] event timeout			
1229	2021-11-18 20:52:47.067398	406	0M4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1			
1229	2021-11-18 20:52:47.067398	406	0M4	MSG_LOG	82	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x3, 0x10000, delay:0			
1229	2021-11-18 20:52:47.067398	406	0M4	MSG_LOG	123	[M:apps C:info F: L:]: [Music_APP]activity app_music_proc_rotary_event_group, EX_CODEC_I2C Crusher volume : 2			
1230	2021-11-18 20:52:47.085197	406	0M4	MSG_LOG	85	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=128ms			
1230	2021-11-18 20:52:47.187291	406	0M4	MSG_LOG	84	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=99ms			
1230	2021-11-18 20:52:47.187291	406	0M4	MSG_LOG	91	[M:common C:info F: L:]: [bsp][rotary encoder] event_count=0 current_pos=32780			
1230	2021-11-18 20:52:47.291541	406	0M4	MSG_LOG	73	[M:common C:info F: L:]: [bsp][rotary encoder] event timeout			
1230	2021-11-18 20:52:47.291541	406	0M4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1			
1230	2021-11-18 20:52:47.291541	406	0M4	MSG_LOG	82	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x3, 0x10000, delay:0			
1230	2021-11-18 20:52:47.291541	406	0M4	MSG_LOG	123	[M:apps C:info F: L:]: [Music_APP]activity app_music_proc_rotary_event_group, EX_CODEC_I2C Crusher volume : 1			
1230	2021-11-18 20:52:47.810770	406	0M4	MSG_LOG	85	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=625ms			
1230	2021-11-18 20:52:47.912305	406	0M4	MSG_LOG	85	[M:common C:info F: L:]: [bsp][rotary encoder] event interval time=100ms			
1230	2021-11-18 20:52:47.912305	406	0M4	MSG_LOG	91	[M:common C:info F: L:]: [bsp][rotary encoder] event_count=0 current_pos=32781			
1230	2021-11-18 20:52:48.019060	406	0M4	MSG_LOG	73	[M:common C:info F: L:]: [bsp][rotary encoder] event timeout			
1230	2021-11-18 20:52:48.019060	406	0M4	MSG_LOG	93	[M:apps C:info F: L:]: [app_key_event]rotary port = 0x0, event = 0x0, value = 1			
1230	2021-11-18 20:52:48.019060	406	0M4	MSG_LOG	82	[M:ui_shell C:info F: L:]: ui_shell_send_event: 0x3, 0x10000, delay:0			
1230	2021-11-18 20:52:48.019060	406	0M4	MSG_LOG	123	[M:apps C:info F: L:]: [Music_APP]activity app_music_proc_rotary_event_group, EX_CODEC_I2C Crusher volume : 0			

旋转编码器首先触发外部中断的回调函数打印 "[bsp][rotary encoder] event interval time=...
bsp_rotary_encoder.c 文件中定义了旋转编码器所有驱动函数：

Bsp_rotary_encoder.c

```

#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include <assert.h>
#include "hal.h"
#include "hal_gpio.h"
#include "hal_gpt.h"
#include "bsp_rotary_encoder.h"
#include "bsp_rotary_encoder_custom.h"
UPDATE_PIN_STATE
#define
update_pin_state
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN
BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN
BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN
bsp_rotary_encoder_last_trigger_pin_t
bsp_rotary_encoder_t
bsp_rotary_encoder_driver
bsp_rotary_encoder_driver_new
bsp_rotary_encoder_get_position
bsp_rotary_encoder_process_event
bsp_rotary_encoder_timeout_callback
bsp_rotary_encoder_eint_callback
bsp_rotary_encoder_pina_eint_callback
bsp_rotary_encoder_pinb_eint_callback
bsp_rotary_encoder_init
bsp_rotary_encoder_deinit
__attribute__
__attribute__
__attribute__
__attribute__

```

```

00204: }
00205:
00206: static void bsp_rotary_encoder_eint_callback(bsp_rotary_encoder_t *re, uint8_t current_state, bsp_rotary_encoder_pin
00207: {
00208:     hal_gpt_status_t gpt_status;
00209:     bsp_rotary_encoder_config_t *rec = &re->user_config;
00210:
00211:     #ifdef BSP_ROTARY_ENCODER_SINGLE_EDGE_TRIGGER
00212:         if (pin_config->pin_state == HAL_GPIO_DATA_LOW) {
00213:             pin_config->trigger_mode = HAL_EINT_EDGE_RISING;
00214:             hal_eint_set_trigger_mode(pin_config->eint_number, HAL_EINT_EDGE_RISING);
00215:         } else {
00216:             pin_config->trigger_mode = HAL_EINT_EDGE_FALLING;
00217:             hal_eint_set_trigger_mode(pin_config->eint_number, HAL_EINT_EDGE_FALLING);
00218:         }
00219:     }
00220:
00221:     LOG_MSGSID_I(common, "[bsp][rotary encoder] EINT%d trigger, pin_state=%d", 2, (int)pin_config->eint_number, (int)pin_co
00222:
00223:     #ifdef BSP_ROTARY_ENCODER_TIMING_DEBUG_ENABLE
00224:         bsp_rotary_encoder_port_t port = re->driver->port;
00225:         uint32_t current_timer_count(BSP_ROTARY_ENCODER_MAX);
00226:         static uint32_t last_timer_count(BSP_ROTARY_ENCODER_MAX) = 0;
00227:
00228:         hal_gpt_get_free_run_count(HAL_GPT_CLOCK_SOURCE_1M, &current_timer_count[port]);
00229:         if (last_timer_count[port] != 0) {
00230:             LOG_MSGSID_I(common, "[bsp][rotary encoder] event interval time=%dms", 1, (int)(current_timer_count[port] - last_timer_co
00231:             last_timer_count[port] = current_timer_count[port];
00232:         }
00233:     }
00234:
00235:     if (bsp_rotary_encoder_get_position(re, current_state) < 0) {
00236:         hal_eint_unmask(pin_config->eint_number);
00237:         return;
00238:     }
00239:
00240:     LOG_MSGSID_I(common, "[bsp][rotary encoder] event_count=%d current_pos=%d", 2, (int)re->event_count, (int)re->current_p
00241:     if (rec->event_threshold > 0) {
00242:         hal_gpt_sw_stop_timer_ms(rec->timer_handle);
00243:         re->event_count++;
00244:         if (re->event_count > rec->event_threshold) {

```

回调函数 bsp_rotary_encoder_eint_callback 被
函数 bsp_rotary_encoder_pina_eint_callback 和 bsp_rotary_encoder_pinb_eint_callback 调用
(回调函数是被其它函数在其参数里通过函数指针来调用的)

Bsp_rotary_encoder.c

```

include <stdint.h>
include <stdlib.h>
include <stdio.h>
include <stdbool.h>
include <string.h>
include <assert.h>
include "hal.h"
include "hal_gpio.h"
include "hal_gpt.h"
include "bsp_rotary_encoder.h"
include "bsp_rotary_encoder_custom.h"
UPDATE_PIN_STATE
#define
update_pin_state
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR
BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN
BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN
BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN
bsp_rotary_encoder_last_trigger_pin_t
bsp_rotary_encoder_t
bsp_rotary_encoder_driver
bsp_rotary_encoder_driver_new
bsp_rotary_encoder_get_position
bsp_rotary_encoder_process_event
bsp_rotary_encoder_timeout_callback
bsp_rotary_encoder_eint_callback
bsp_rotary_encoder_pina_eint_callback
bsp_rotary_encoder_pinb_eint_callback
bsp_rotary_encoder_init
bsp_rotary_encoder_deinit
__attribute__
__attribute__
__attribute__
__attribute__

```

```

00332:
00333: bsp_rotary_encoder_status_t bsp_rotary_encoder_init(bsp_rotary_encoder_port_t port, bsp_rotary_encoder_config_t *config)
00334: {
00335:     uint32_t i = 0;
00336:     hal_eint_config_t pina_eint_config;
00337:     hal_eint_config_t pinb_eint_config;
00338:     hal_nnin nin_t nnin_nort a;
00339:
00340: }
00341:
00342: if (HAL_EINT_STATUS_OK != hal_eint_register_callback(eint_number_a, bsp_rotary_encoder_pina_eint_callback, (void*)driver)) ||
00343:     HAL_EINT_STATUS_OK != hal_eint_register_callback(eint_number_b, bsp_rotary_encoder_pinb_eint_callback, (void*)driver)) {
00344:     LOG_MSGSID_E(common, "[bsp][rotary encoder] EINT register callback fail", 0);
00345:     return BSP_ROTARY_ENCODER_STATUS_ERROR;
00346: }
00347: LOG_MSGSID_I(common, "[bsp][rotary_encoder]idx=%d: PINA = %d, eint=%d, mode = %d; PINB = %d, eint=%d, mode = %d", 7,

```

app_event_key_rotary_encoder_callback 回调函数中调用消息发送器函数

```

29: static void app_event_key_rotary_encoder_callback(bsp_rotary_encoder_port_t port, bsp_r
29: {
30:     APPS_LOG_MSGSID_I(LOG_TAG"rotary port = 0x%x, event = 0x%x, value = %d", 3, port, event, e
31:     uint16_t *p_key_action = (uint16_t *)pvPortMalloc(sizeof(uint16_t));
32:     if (p_key_action) {
33:         ui_shell_send_event(true, EVENT_PRIORITY_MIDDLE, EVENT_GROUP_UI_SHELL_ROTARY_ENCOI
34:         (port & 0xFF) | ((event & 0xFF) << 8) | ((event_data & 0xFFFF) << 16),
35:         消息发送器
36:         p_key_action, sizeof(uint16_t), NULL, 0);
37:     }
38: }
39: #endif

```

查找 EVENT_GROUP_UI_SHELL_ROTARY_ENCODER

```
Search Results
---- EVENT_GROUP_UI_SHELL_ROTARY_ENCODER Matches (13 in 13 files) ----
441 Apps_events_event_group.h (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/inc/apps/events): EVENT_GROUP_UI_SHELL_ROTARY_ENCODER, /**<
442 Apps_events_key_event.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/events): ui_shell_send_event(true, EVENT_PRIORITY_MIDDLE, E
443 App_hfp_activity.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/app_hfp): case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
444 App_hfp_idle_activity.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/app_hfp): case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
445 App_line_in_idle_activity.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/app_line_in): case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
446 App_music_activity.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/app_music): case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
447 App_music_preproc_activity.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/app_music): case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
448 App_ul_idle_activity.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/app_ull): case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
449 App_usb_audio_idle_activity.c (mcu/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/app_usb_audio): case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
```

```
typedef enum {
    EVENT_GROUP_UI_SHELL_SYSTEM = 0, /**<
    EVENT_GROUP_UI_SHELL_APP_BASE, /**<
} ui_shell_event_group_t;

typedef enum {
    EVENT_GROUP_UI_SHELL_APP_INTERACTION = EVENT_GROUP_UI_SHELL_APP_BASE, /**<
    EVENT_GROUP_UI_SHELL_KEY, /**< group for key events */
#ifdef AIR_ROTARY_ENCODER_ENABLE
    EVENT_GROUP_UI_SHELL_ROTARY_ENCODER, /**< group for rotary encoder events */
#endif
} ui_shell_event_group_t;
```

音乐播放情况下，会遍历 music 的活动进程，比如按下按键、旋动编码器等

```
App_music_activity.c
00425:
00426: bool app_music_activity_proc(
00427:     struct ui_shell_activity *self,
00428:     uint32_t event_group,
00429:     uint32_t event_id,
00430:     void *extra_data,
00431:     size_t data_len)
00432: {
00433:     bool ret = false;
00434:     switch (event_group) {
00435:     case EVENT_GROUP_UI_SHELL_SYSTEM: {
00436:         /* UI Shell internal events, please refer to doc/Airoha_IoT_SDK_UI_Fr
00437:         ret = _proc_ui_shell_group(self, event_id, extra_data, data_len)
00438:         break;
00439:     case EVENT_GROUP_UI_SHELL_KEY: {
00440:         ret = app_music_proc_key_event_group(self, event_id, extra_data, data_len)
00441:         break;
00442:     case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
00443:         /* Rotary encoder event */
00444:         ret = app_music_proc_rotary_event_group(self, event_id, extra_data, data_len)
00445:         break;
00446:     default:
00447:         ret = false;
00448:     }
00449:     return ret;
}
```

进入音乐进程中旋转编码器事件组，改变音量大小

```
App_music_activity.c
00149:
00150: #ifdef AIR_ROTARY_ENCODER_ENABLE
00151: static bool app_music_proc_rotary_event_group(
00152:     struct ui_shell_activity *self,
00153:     uint32_t event_id,
00154:     void *extra_data,
00155:     size_t data_len)
00156: {
00157:     bool ret = false;
00158:     bsp_rotary_encoder_port_t port;
00159:     bsp_rotary_encoder_event_t event;
00160:     uint32_t rotary_data;
00161:     if (!extra_data) {
00162:         return ret;
00163:     }
00164:     apps_config_key_action_t key_action = *(uint16_t *)extra_data;
00165:     app_event_rotary_event_decode(&port, &event, &rotary_data, event_id);
00166:     switch (key_action) {
00167:     case KEY_VOICE_UP:
00168:     case KEY_VOICE_DN: {
00169:         #if defined(AIR_BT_ULTRA_LOW_LATENCY_ENABLE)
00170:         bt_ull_streaming_info_t info = {0};
00171:         bt_ull_streaming_t streaming = {
00172:             .streaming_interface = BT_ULL_STREAMING_INTERFACE_SPEAKER,
00173:             .port = 0,
00174:         };
00175:         if ((BT_STATUS_SUCCESS == bt_ull_get_streaming_info(streaming, &info)) && info.is_playing) {
00176:             bt_status_t bt_status;
00177:             bt_ull_volume_t volume_param;
00178:             volume_param.streaming_interface = BT_ULL_STREAMING_INTERFACE_SPEAKER;
00179:             volume_param.streaming_port = 0;
00180:             volume_param.channel = BT_ULL_VOLUME_CHANNEL_DUAL;
00181:             volume_param.volume = rotary_data; /* The delta value. */
00182:             if (KEY_VOICE_UP == key_action) {
00183:                 volume_param.action = BT_ULL_VOLUME_ACTION_SET_UP;
00184:             } else {
00185:                 volume_param.action = BT_ULL_VOLUME_ACTION_SET_DOWN;
00186:             }
00187:             bt_status = bt_ull_action(BT_ULL_ACTION_SET_STREAMING_VOLUME, &volume_param, sizeof(volume_param));
00188:             APPS_LOG_MSGID_I(APP_MUSIC_ACTI" app_music_proc_rotary_event_group, ull delat volume : %d, result: %d",
00189:                 2, rotary_data, bt_status);
00190:         }
00191:     }
00192:     #endif
00193:     #endif
00194:     {
00195:         uint8_t volume = bt_sink_srv_get_volume(NULL, BT_SINK_SRV_VOLUME_A2DP);
00196:         if (KEY_VOICE_UP == key_action) {
00197:             if (volume + rotary_data < bt_sink_srv_ami_get_a2dp_max_volume_level()) {
00198:                 volume += rotary_data;
00199:             } else {
00200:                 volume = bt_sink_srv_ami_get_a2dp_max_volume_level();
00201:             }
00202:         } else if (volume > rotary_data) {
00203:             volume -= rotary_data;
00204:         } else {
00205:             volume = 0;
00206:         }
00207:         APPS_LOG_MSGID_I(APP_MUSIC_ACTI" app_music_proc_rotary_event_group, music volume : %d", 1, volume);
00208:         bt_sink_srv_send_action(BT_SINK_SRV_ACTION_SET_VOLUME, &volume);
00209:     }
00210:     ret = true;
00211:     break;
00212: default:
00213:     break;
00214: }
00215: }
00216: }
00217: } ? end switch key_action ?
```

音量控制函数

```
App_music_idle_activity.c
#include "FreeRTOS.h"
#include "app_music_idle_activ
#include "app_music_utils.h"
#include "app_music_activity.h"
#include "apps_events_interact
#include "bt_sink_srv_mmi.h"
#include "stdlib.h"
#include "apps_debug.h"
#include "apps_events_event_gr
#include "bt_sink_srv.h"
#include "bt_device_manager.h"
#include "apps_config_vp_in"
#include "apps_config_vp_mu"
#include "rsrc_end.h"
#include "rsrc_mmi_set_peq_group_id
endif
#include "apps_config_key_rane
#include "MTK_IN_EAR_FEATURE_ENAB
#include "app_in_ear_utils.h"
endif
#include "bt_role_handler.h"
s_app_music_context
proc_ui_shell_group
proc_key_event_group
ifdef MTK_IN_EAR_FEATURE_ENAB
ifdef SUPPORT_ROLE_HANDOVER
app_music_role_data_t
app_music_role_data_t
role_handler_get_data
role_handler_update_da
role_handler_service_s
endif
endif
proc_ui_shell_group
app_music_idle_activity_handl
proc_key_event_group
app_music_idle_activity_proc
ifdef MTK_IN_EAR_FEATURE_ENAB
music_in_ear_atci_update_c
music_volume_down_set
app_music_in_ear_atci_cmd
app_music_in_ear_atci_debu
endif

00443: atci_send_response(&response);
00444: return ATCI_STATUS_OK;
00445: } end _music_in_ear_atci_update_cfg ?
00446:
00447: static atci_status_t _music_volume_up_set(atci_parse_cmd_param_t *parse
00448: {
00449:     atci_response_t response = {{0}, 0};
00450:     uint32_t value = 0;
00451:     bool ret = FALSE;
00452:     switch (parse_cmd->mode) {
00453:     case ATCI_CMD_MODE_EXECUTION: {
00454:         value = atoi(parse_cmd->string_ptr + parse_cmd->name_len + 1);
00455:         APPS_LOG_MSGID_I(APP_MUSIC_TAG) [MUSIC ATCI] _music_volume_up_set
00456:         ret = app_music_set_volume_value(true, value);
00457:         if (ret == TRUE) {
00458:             response.response_flag = ATCI_RESPONSE_FLAG_APPEND_OK;
00459:         } else {
00460:             response.response_flag = ATCI_RESPONSE_FLAG_APPEND_ERROR;
00461:         }
00462:         break;
00463:     }
00464:     default:
00465:         response.response_flag = ATCI_RESPONSE_FLAG_APPEND_ERROR;
00466:         break;
00467:     }
00468:     response.response_len = strlen((char *)response.response_buf);
00469:     atci_send_response(&response);
00470:     return ATCI_STATUS_OK;
00471: } end _music_volume_up_set ?
00472:
00473: static atci_status_t _music_volume_down_set(atci_parse_cmd_param_t *pa
00474: {
00475:     atci_response_t response = {{0}, 0};
00476:     uint32_t value = 0;
00477:     bool ret = FALSE;
00478:     switch (parse_cmd->mode) {
00479:     case ATCI_CMD_MODE_EXECUTION: {
00480:         value = atoi(parse_cmd->string_ptr + parse_cmd->name_len + 1);
00481:         APPS_LOG_MSGID_I(APP_MUSIC_TAG) [MUSIC ATCI] _music_volume_up_set
00482:         ret = app_music_set_volume_value(false, value);
00483:         if (ret == TRUE) {
00484:             response.response_flag = ATCI_RESPONSE_FLAG_APPEND_OK;
00485:         } else {
00486:             response.response_flag = ATCI_RESPONSE_FLAG_APPEND_ERROR;
00487:         }
00488:         break;
00489:     }
00490:     default:
00491:         response.response_flag = ATCI_RESPONSE_FLAG_APPEND_ERROR;
00492:         break;
00493:     }
00494:     response.response_len = strlen((char *)response.response_buf);
00495:     atci_send_response(&response);
00496:     return ATCI_STATUS_OK;
00497: }
```

编码器用户定制设置在文件 bsp_rotary_encoder_custom.c

```
Bsp_rotary_encoder_custom.c
#include "bsp_rotary_encoder_custom
#include "hal_gpio.h"
#include "hal_log.h"
#include "assert.h"
BSP_ROTARY_ENCODER_DECLARE_EXTERN
bsp_rotary_encoder_driver_new
bsp_rotary_encoder_driver_init
BSP_ROTARY_ENCODER_DECLARE_EXTERN
BSP_ROTARY_ENCODER_DECLARE_EXTERN
BSP_ROTARY_ENCODER_DECLARE_EXTERN
bsp_rotary_encoder_custom_init

00040: #define BSP_ROTARY_ENCODER_DECLARE_EXTERN_VAR(num)\
00041: extern const char BSP_ROTARY_ENCODER_##num##_PIN_A;\
00042: extern const char BSP_ROTARY_ENCODER_##num##_PIN_B;\
00043: extern const char BSP_ROTARY_ENCODER_##num##_PIN_A_M_EINT;\
00044: extern const char BSP_ROTARY_ENCODER_##num##_PIN_B_M_EINT;\
00045: extern const unsigned char BSP_ROTARY_ENCODER_##num##_PIN_A_EINT;\
00046: extern const unsigned char BSP_ROTARY_ENCODER_##num##_PIN_B_EINT;\
00047:
00048: extern bsp_rotary_encoder_driver_t * bsp_rotary_encoder_driver_new(bsp_rotary_encoder
00049: #define BSP_ROTARY_ENCODER_DRIVER_INIT(index, available_device_nums)\
00050: {\
00051:     if ((BSP_ROTARY_ENCODER##index##_PIN_A != 0x0) && (BSP_ROTARY_ENCODER##index##
00052:         &&BSP_ROTARY_ENCODER##index##_PIN_A_EINT != 0x0) && (BSP_ROTARY_ENCODER##ind
00053:         bsp_rotary_encoder_driver_t *new_driver = bsp_rotary_encoder_driver_new(index);\
00054:         if (!new_driver) {\
00055:             LOG_MSGID_E(common, "[bsp][rotary_encoder]rotary encoder new driver fail(%d)", 0);\
00056:             return 0;\
00057:         }\
00058:         new_driver->pin_config_gpio_port = BSP_ROTARY_ENCODER##index##_PIN_A;\
00059:         new_driver->pin_config_gpio_port = BSP_ROTARY_ENCODER##index##_PIN_B;\
00060:         new_driver->pin_config_eint_mode = BSP_ROTARY_ENCODER##index##_PIN_A_M_EINT;\
00061:         new_driver->pin_config_eint_mode = BSP_ROTARY_ENCODER##index##_PIN_B_M_EINT;\
00062:         new_driver->pin_config_eint_number = BSP_ROTARY_ENCODER##index##_PIN_A_EINT;\
00063:         new_driver->pin_config_eint_number = BSP_ROTARY_ENCODER##index##_PIN_B_EINT;\
00064:         new_driver->port = index;\
00065:         available_device_nums++;\
00066:     }\
00067: }\
00068:
00069: BSP_ROTARY_ENCODER_DECLARE_EXTERN_VAR(0);
00070: BSP_ROTARY_ENCODER_DECLARE_EXTERN_VAR(1);
00071: BSP_ROTARY_ENCODER_DECLARE_EXTERN_VAR(2);
00072:
00073: void bsp_rotary_encoder_custom_init(void)
00074: {
00075:     uint32_t available_device_nums = 0;
00076:
00077:     BSP_ROTARY_ENCODER_DRIVER_INIT(0, available_device_nums);
00078:     BSP_ROTARY_ENCODER_DRIVER_INIT(1, available_device_nums);
00079:     BSP_ROTARY_ENCODER_DRIVER_INIT(2, available_device_nums);
00080:
00081:     if (0 == available_device_nums) {
00082:         LOG_MSGID_E(common, "[bsp][rotary_encoder]rotary encoder pin or eint has not been configur
00083:     }
00084: }
```

bsp_rotary_encoder.c 文件中调用 bsp_rotary_encoder_custom_init();函数

```
Bsp_rotary_encoder.c
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include <assert.h>
#include "hal.h"
#include "hal_gpio.h"
#include "hal_eint.h"
#include "hal_gpt.h"
#include "bsp_rotary_enco
#include "bsp_rotary_enco
UPDATE_PIN_STATE
REABS
update_pin_state
BSP_ROTARY_ENCODER_DEFIN
BSP_ROTARY_ENCODER_DEFIN
BSP_ROTARY_ENCODER_DEFIN
BSP_ROTARY_ENCODER_DEFIN
BSP_ROTARY_ENCODER_LA
BSP_ROTARY_ENCODER_LA
BSP_ROTARY_ENCODER_LA
bsp_rotary_encoder_last
bsp_rotary_encoder_t
rotary_encoder
bsp_rotary_encoder_drive
bsp_rotary_encoder_drive
bsp_rotary_encoder_get_p
bsp_rotary_encoder_proce

00332:
00333: bsp_rotary_encoder_status_t bsp_rotary_encoder_init(bsp_rotary_enco
00334: {
00335:     uint32_t i = 0;
00336:     hal_eint_config_t pina_eint_config;
00337:     hal_eint_config_t pinb_eint_config;
00338:     hal_gpio_pin_t gpio_port_a;
00339:     hal_gpio_pin_t gpio_port_b;
00340:     hal_eint_number_t eint_number_a;
00341:     hal_eint_number_t eint_number_b;
00342:     bsp_rotary_encoder_t *re;
00343:     bsp_rotary_encoder_driver_t *driver;
00344:
00345:     if (port > BSP_ROTARY_ENCODER_MAX) {
00346:         return BSP_ROTARY_ENCODER_STATUS_ERROR_INVALID_PARAMETER;
00347:     }
00348:
00349:     /*param check*/
00350:     if (!config->callback) {
00351:         LOG_MSGID_E(common, "[bsp][rotary encoder] GPIO init param error",
00352:         return BSP_ROTARY_ENCODER_STATUS_ERROR_INVALID_PARAMETER;
00353:     }
00354:
00355:     re = &rotary_encoder[port];
00356:     memset(re, 0, sizeof(bsp_rotary_encoder_t));
00357:
00358:     bsp_rotary_encoder_custom_init();
00359:
00360:     driver = re->driver;
00361: }
```


bsp_rotary_encoder_init 函数在 apps_event_rotary_init 函数中被调用

```
Apps_events_key_event.c
00245: };
00246:
00247: static void apps_event_rotary_init(void)
00248: {
00249:     uint32_t i;
00250:     bsp_rotary_encoder_status_t status;
00251:     bsp_rotary_encoder_port_t port;
00252:     bsp_rotary_encoder_config_t config = {
00253:         .eint_debounce_time_ms = 10,
00254:         .event_threshold = 1,
00255:         .event_timeout_ms = 100,
00256:         .callback = app_event_key_rotary_encoder_callback,
00257:         .user_data = NULL,
00258:     };
00259:
00260:     for (i = 0; i < sizeof(rotary_array) / sizeof(uint8_t); i++) {
00261:         port = rotary_array[i];
00262:         status = bsp_rotary_encoder_init(port, &config);
00263:         if (status != BSP_ROTARY_ENCODER_STATUS_OK) {
00264:             APPS_LOG_MSGID_E("Init port:%d fail: %d", 2, port, status);
00265:         }
00266:     }
00267: } ? end apps_event_rotary_init ?
00268: #endif
```

bsp_rotary_encoder_custom.c 的函数调用到 bsp_rotary_encoder.c 文件中的驱动函数

bsp_rotary_encoder_driver_new

```
Bsp_rotary_encoder.c
00111:
00112: bsp_rotary_encoder_driver_t * bsp_rotary_encoder_driver_new(bsp_rotary_encoder_port_t port)
00113: {
00114:     bsp_rotary_encoder_driver_t *new_driver;
00115:
00116:     if (port >= BSP_ROTARY_ENCODER_MAX) {
00117:         return NULL;
00118:     }
00119:     new_driver = &bsp_rotary_encoder_driver[port];
00120:     rotary_encoder[port].driver = new_driver;
00121:     return new_driver;
00122: }
00123:
00124: int32_t bsp_rotary_encoder_get_position(bsp_rotary_encoder_t *re, uint8_t current_state)
00125: {
00126:     uint32_t position = re->current_position;
00127:
00128:     re->pc_state <= 2;
00129:     re->pc_state |= (current_state & 0x3);
00130:     re->pc_state &= 0xF;
00131:
00132:     LOG_MSGID_I(common, "[bsp][rotary encoder] prev_curr_state=0x%x current_state=0x%x", 2, re,
00133:         0x00 0x01 0x02 0x03 0x04 0x05 0x06 0x07 0x08 0x09 0x0A 0x0B 0x0C 0x0D 0x0E 0x0F);
00134: }
```

```
static bsp_rotary_encoder_driver_t bsp_rotary_encoder_driver[BSP_ROTARY_ENCODER_MAX];

typedef struct
{
    bsp_rotary_encoder_pin_config_t pina_config;
    bsp_rotary_encoder_pin_config_t pinb_config;
    bsp_rotary_encoder_port_t port;
} bsp_rotary_encoder_driver_t;

typedef struct
{
    hal_gpio_pin_t gpio_port; /* GPIO port */
    hal_eint_number_t eint_number; /* eint number */
    uint8_t eint_mode; /* GPIO eint mode */
    hal_gpio_data_t pin_state;
    hal_eint_trigger_mode_t trigger_mode;
} bsp_rotary_encoder_pin_config_t;
```

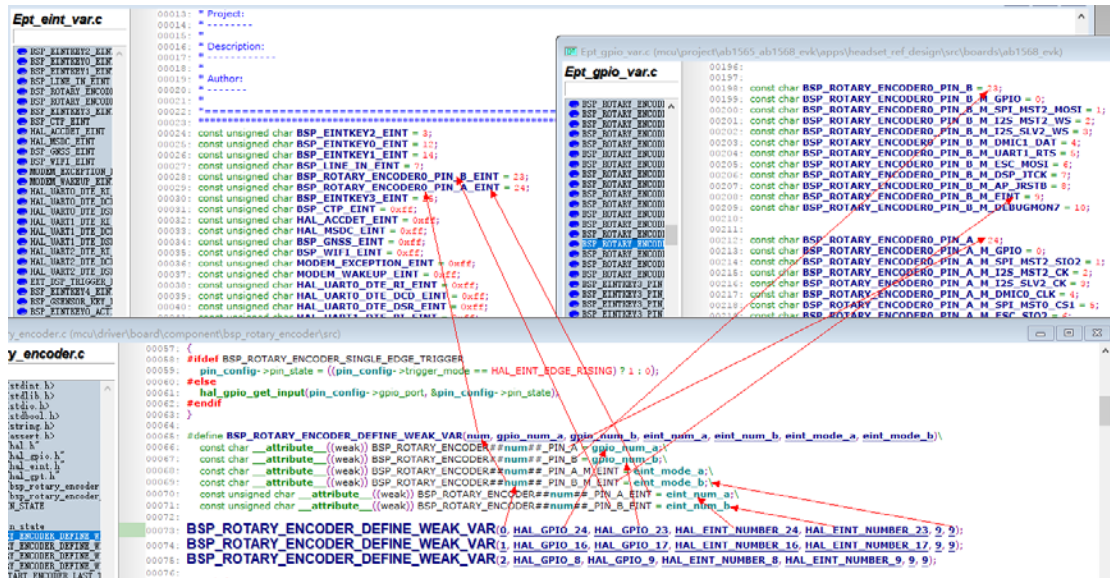
```
Hal_platform.h
00427: /*
00428:
00429: typedef enum {
00430:     HAL_GPIO_0 = 0, /* GPIO pin0 */
00431:     HAL_GPIO_1 = 1, /* GPIO pin1 */
00432:     HAL_GPIO_2 = 2, /* GPIO pin2 */
00433:     HAL_GPIO_3 = 3, /* GPIO pin3 */
00434:     HAL_GPIO_4 = 4, /* GPIO pin4 */
00435:     HAL_GPIO_5 = 5, /* GPIO pin5 */
00436:     HAL_GPIO_6 = 6, /* GPIO pin6 */
00437:     HAL_GPIO_7 = 7, /* GPIO pin7 */
00438:     HAL_GPIO_8 = 8, /* GPIO pin8 */
00439:     HAL_GPIO_9 = 9, /* GPIO pin9 */
00440:     HAL_GPIO_10 = 10, /* GPIO pin10 */
00441:     HAL_GPIO_11 = 11, /* GPIO pin11 */
00442:     HAL_GPIO_12 = 12, /* GPIO pin12 */
00443:     HAL_GPIO_13 = 13, /* GPIO pin13 */
00444:     HAL_GPIO_14 = 14, /* GPIO pin14 */
00445:     HAL_GPIO_15 = 15, /* GPIO pin15 */
00446:     HAL_GPIO_16 = 16, /* GPIO pin16 */
00447:     HAL_GPIO_17 = 17, /* GPIO pin17 */
00448:     HAL_GPIO_18 = 18, /* GPIO pin18 */
00449:     HAL_GPIO_19 = 19, /* GPIO pin19 */
00450:     #if (defined(MT2822)) || defined(MT2822A)
00451:     HAL_GPIO_20 = 20, /* GPIO pin20 */
00452:     HAL_GPIO_21 = 21, /* GPIO pin21 */
00453:     HAL_GPIO_22 = 22, /* GPIO pin22 */
00454:     HAL_GPIO_23 = 23, /* GPIO pin23 */
00455:     HAL_GPIO_24 = 24, /* GPIO pin24 */
00456:     HAL_GPIO_25 = 25, /* GPIO pin25 */
00457:     HAL_GPIO_26 = 26, /* GPIO pin26 */
00458:     #endif
00459: } hal_gpio_pin_t;
```

默认编码器配置管脚是 10 和 11，修改编码器连接的管脚对应上原理图

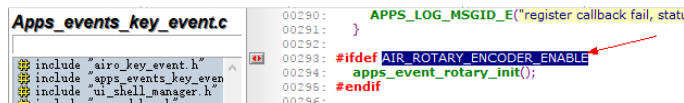
```
Bsp_rotary_encoder.c
00063: }
00064:
00065: #define BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR(num, gpio_num_a, gpio_num_b, eint_num_a, eint_num_b, eint_mode_a, eint_mode_b)
00066: const char __attribute__((weak)) BSP_ROTARY_ENCODER_##num##_PIN_A = gpio_num_a;
00067: const char __attribute__((weak)) BSP_ROTARY_ENCODER_##num##_PIN_B = gpio_num_b;
00068: const char __attribute__((weak)) BSP_ROTARY_ENCODER_##num##_PIN_A_M_EINT = eint_mode_a;
00069: const char __attribute__((weak)) BSP_ROTARY_ENCODER_##num##_PIN_B_M_EINT = eint_mode_b;
00070: const unsigned char __attribute__((weak)) BSP_ROTARY_ENCODER_##num##_PIN_A_EINT = eint_num_a;
00071: const unsigned char __attribute__((weak)) BSP_ROTARY_ENCODER_##num##_PIN_B_EINT = eint_num_b;
00072:
00073: BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR(0, HAL_GPIO_10, HAL_GPIO_11, HAL_EINT_NUMBER_10, HAL_EINT_NUMBER_11, 9, 9);
00074: BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR(1, HAL_GPIO_16, HAL_GPIO_17, HAL_EINT_NUMBER_16, HAL_EINT_NUMBER_17, 9, 9);
00075: BSP_ROTARY_ENCODER_DEFINE_WEAK_VAR(2, HAL_GPIO_8, HAL_GPIO_9, HAL_EINT_NUMBER_8, HAL_EINT_NUMBER_9, 9, 9);
00076:
00077: typedef enum
00078: {
00079:     BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN_NONE = 0,
00080:     BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN_A = 1,
00081:     BSP_ROTARY_ENCODER_LAST_TRIGGER_PIN_B = 2,
00082: } bsp_rotary_encoder_last_trigger_pin_t;
00083: //02310231 CW
```



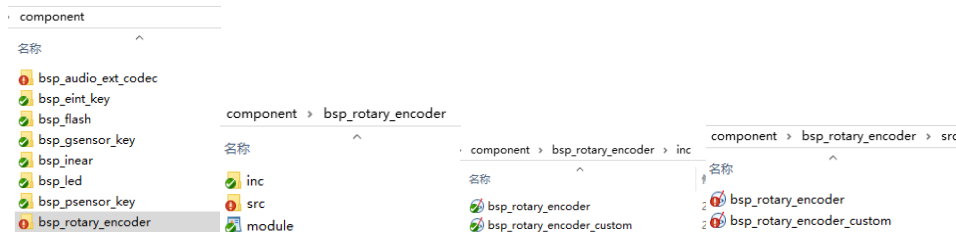
与 GPIO 配置以及 EINT 配置一一对应



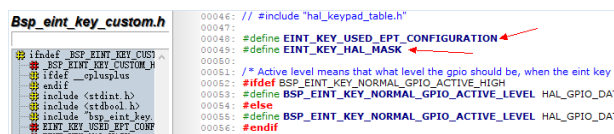
从函数调用追踪代码，发现必须有宏定义 AIR_ROTARY_ENCODER_ENABLE，所有旋转编码器驱动代码才能启用



只需要在添加外设文件夹 mcu\driver\board\component 的对应文件做修改即可



比如 EINT KEY



前面的理解有错误的地方，就是宏定义到底在那里开启，需要回溯到工程下的 mk 文件，工程编译过程会编译 GCC 文件夹下的 mk 文件（编译哪个 mk 文件取决于命令行的参数，mk 文件定义了不同的宏 y 和 n，即开还是关，不用硬件选择不同的 mk 进行编译）

```
GS-35514@GSPC22002102 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset
$ ./build.sh ab1568_evk headset_ref_design
```

```
ab1568_evk
headset_ref_design
board_share: ab1565_ab1568_evk
cm4: headset_ref_design
dsp0: dsp0_headset_ref_design
fm: feature_68_evk.mk
fd0: feature_68_evb.mk
```

project > ab1565_ab1568_evk > apps > headset_ref_design > GCC				
名称	修改日期	类型	大小	
syscalls	2021/8/31 2:14	C 文件	6 KB	
ab156x_evb.cmm	2021/8/31 2:14	CMM 文件	2 KB	
ab156x_flash.ld	2021/8/31 2:14	LD 文件	17 KB	
ab156x_flash_8m.ld	2021/8/31 2:14	LD 文件	17 KB	
ab156x_flash_8m_fota_test.ld	2021/8/31 2:14	LD 文件	18 KB	
ab156x_sysram.ld	2021/8/31 2:14	LD 文件	11 KB	
feature	2021/8/31 2:14	MK 文件	4 KB	
feature_65_8m_evk	2021/8/31 2:14	MK 文件	12 KB	
feature_65_evb	2021/8/31 2:14	MK 文件	11 KB	
feature_65_evb_emp	2021/8/31 2:14	MK 文件	11 KB	
feature_65_evk	2021/8/31 2:14	MK 文件	11 KB	
feature_65_evk_emp	2021/8/31 2:14	MK 文件	11 KB	
feature_68_evb_ull	2021/8/31 2:14	MK 文件	12 KB	
feature_68_evk	2021/10/27 15:57	MK 文件	11 KB	
feature_68_evk_emp	2021/8/31 2:14	MK 文件	11 KB	
feature_68_evk_ull	2021/8/31 2:14	MK 文件	12 KB	
feature_68_evk_ull_a2dp	2021/8/31 2:14	MK 文件	12 KB	
feature_68_headset	2021/8/31 2:14	MK 文件	13 KB	
feature_68_headset_ull	2021/8/31 2:14	MK 文件	13 KB	
test_65_evk_vendor_1	2021/8/31 2:14	MK 文件	3 KB	
feature_68_evk.mk.rej	2021/10/27 15:59	REJ 文件	1 KB	
Makefile.rej	2021/10/27 15:59	REJ 文件	1 KB	
startup_ab156x.s	2021/8/31 2:14	S 文件	11 KB	
Makefile	2021/10/27 15:57	文件	21 KB	

打开 mk 文件 feature_68_evk.mk, y 表示打开宏, n 表示关闭宏, 搜索此文件发现没有旋转编码器的宏

AIR_ROTARY_ENCODER_ENABLE

```

IC_CONFIG                = ab156x
BOARD_CONFIG             = ab156x_evb
IC_TYPE                  = ab1568
BOARD_TYPE               = ab1568_evk
BL_FEATURE                = feature_68_evb.mk

FLASH_SIZE_8M            = y
# debug level: none, error, warning, info and debug
MTK_DEBUG_LEVEL          = info
MTK_NVDM_ENABLE          = y
MTK_USB_DEMO_ENABLED     = y
MTK_USB_AUDIO_V1_ENABLE  = y
MTK_USB_AUDIO_V2_ENABLE  = n
MTK_USB_AUDIO_MICROPHONE = n
MTK_HAI_EXT_32K_ENABLE   = n
MTK_NO_PSRAM_ENABLE      = y
# heap dump
MTK_SUPPORT_HEAP_DEBUG   = y
MTK_SUPPORT_HEAP_DEBUG_ADVANCED = n
# heap peak profiling
MTK_HEAP_SIZE_GUARD_ENABLE = n
#SWLA
MTK_SWLA_ENABLE          = y
#system service at cmd
MTK_SYSTEM_AT_COMMAND_ENABLE = y
MTK_OS_CPU_UTILIZATION_ENABLE = y
# system hang debug: none, y, o1, o2 and mp
MTK_SYSTEM_HANG_TRACER_ENABLE = y
#####
MTK_BT_AT_COMMAND_ENABLE = y

# CAPID store to NVDM
MTK_CAPID_IN_NVDM_AT_COMMAND_ENABLE = y

# port service
MTK_PORT_SERVICE_ENABLE = y

# bt codec enable
MTK_BT_CODEC_ENABLED = y
# bt codec ble enable
MTK_BT_CODEC_BLE_ENABLED = y

```

但是在文件 feature_68_headset.mk 里有宏 AIR_ROTARY_ENCODER_ENABLE

```

AIR_SLIDE_POWER_KEY_ENABLE = n

#Fix afe rate to 48KHZ
FIXED_SAMPLING_RATE_TO_48KHZ = y

# Use audio transmitter
MTK_AUDIO_TRANSMITTER_ENABLE = y

# Ultra low latency
ULL_LIB = $(strip $(SOURCE_DIR))/middleware/MTK/bt_ultra_low_latenc
ifeq ($(ULL_LIB), $(wildcard $(ULL_LIB)))
AIR_BT_ULTRA_LOW_LATENCY_ENABLE = n
endif

AIR_ROTARY_ENCODER_ENABLE = y

# Do not enter RTC mode when Line in or USB in.
AIR_APP_SYSTEM_ON_BY_LINE_IN_ENABLE = y

#Voice detachable MIC support
AIR_AUDIO_DETACHABLE_MIC_ENABLE = y

#Wired audio support
AIR_WIRED_AUDIO_ENABLE = y
# Wired audio game chat volume smart balance enable
AIR_GAME_CHAT_VOLUME_SMART_BALANCE_ENABLE = y

```

所以编译的命令行应该改为下图命令参数

```

ab1568_headset
headset_ref_design
board_share: ab1565_ab1568_evk
cm4: headset_ref_design
dsp0: dsp0_headset_ref_design
fm: feature_68_headset.mk
fd0: feature_65_8m_evb_headset.mk

GS-355148GSPC22002102 MSYS /d/IoT_SDK_for_BT_Audio_V2.7.1.ULL_TWS_Headset
$ ./build.sh ab1568_headset headset_ref_design

```

或者添加该宏 AIR_ROTARY_ENCODER_ENABLE 到 feature_68_evk.mk 后编译

手动添加

#	Time	Tids	CPU	Protocol	Length	Info
2033	2021-11-23 14:01:02.917315	22387	CMA	MSG_LOG	102	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 debounce counter=0 trigger_mode=0
2034	2021-11-23 14:01:02.917315	22387	CMA	MSG_LOG	88	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] EINT23 trigger, pin_state=0
2035	2021-11-23 14:01:02.917315	22387	CMA	MSG_LOG	97	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] prev_curr_state=0xd current_state=0x1
2036	2021-11-23 14:01:02.917315	22387	CMA	MSG_LOG	80	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] MID_dpc_state=0xd
2045	2021-11-23 14:01:05.543968	25011	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=2624ms
2046	2021-11-23 14:01:05.543968	25012	CMA	MSG_LOG	102	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 debounce counter=0 trigger_mode=0
2047	2021-11-23 14:01:05.543968	25012	CMA	MSG_LOG	88	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] EINT23 trigger, pin_state=1
2048	2021-11-23 14:01:05.543968	25012	CMA	MSG_LOG	86	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] prev_curr_state=2625ms
2049	2021-11-23 14:01:05.543968	25012	CMA	MSG_LOG	97	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] prev_curr_state=0x7 current_state=0x3
2050	2021-11-23 14:01:05.543968	25012	CMA	MSG_LOG	81	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] MID_dpc_state=0xd7
2155	2021-11-23 14:01:25.756727	45230	CMA	MSG_LOG	90	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=20218ms
2156	2021-11-23 14:01:25.756727	45230	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=0
2155	2021-11-23 14:01:27.549659	47029	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=1799ms
2156	2021-11-23 14:01:27.549659	47029	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=1
2175	2021-11-23 14:01:28.992282	48461	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=1431ms
2176	2021-11-23 14:01:28.992282	48461	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=0
2186	2021-11-23 14:01:31.855492	51334	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=2872ms
2187	2021-11-23 14:01:31.855492	51334	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=1
2215	2021-11-23 14:01:35.163583	54635	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=3301ms
2216	2021-11-23 14:01:35.163583	54635	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=0
2221	2021-11-23 14:01:36.695486	56177	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=1541ms
2222	2021-11-23 14:01:36.695486	56177	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=1
2224	2021-11-23 14:01:37.081828	56499	CMA	MSG_LOG	88	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=322ms
2225	2021-11-23 14:01:37.081828	56499	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=0
2229	2021-11-23 14:01:37.312966	56781	CMA	MSG_LOG	88	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=282ms
2230	2021-11-23 14:01:37.312966	56781	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=1
2239	2021-11-23 14:01:39.418412	58899	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=2117ms
2240	2021-11-23 14:01:39.418412	58899	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=0
2246	2021-11-23 14:01:41.165146	60638	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=1739ms
2247	2021-11-23 14:01:41.165146	60638	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=1
2299	2021-11-23 14:01:48.655901	68134	CMA	MSG_LOG	89	[M:common:Cinfo F: L:]: [bsp:[rotary encoder] B EINT23 interval time=7495ms
2300	2021-11-23 14:01:48.655901	68134	CMA	MSG_LOG	92	[M:common:Cwarning F: L:]: [bsp:[rotary encoder] B EINT23 debounce pin_state=0

```

Bsp_rotary_encoder_custom.c
#include "bsp_rotary_encoder_custom.h"
#include "hal_gpio.h"
#include "hal_led.h"
#include "uart.h"

BSP_ROTARY_ENCODER_DECLARE_EXTERN
bsp_rotary_encoder_driver_gpio
BSP_ROTARY_ENCODER_DRIVER_INIT
BSP_ROTARY_ENCODER_DECLARE_EXTERN
BSP_ROTARY_ENCODER_DECLARE_EXTERN
BSP_ROTARY_ENCODER_DECLARE_EXTERN
bsp_rotary_encoder_custom_init

00066: }
00067:
00068:
00069: BSP_ROTARY_ENCODER_DECLARE_EXTERN_VAR(0);
00070: BSP_ROTARY_ENCODER_DECLARE_EXTERN_VAR(1);
00071: BSP_ROTARY_ENCODER_DECLARE_EXTERN_VAR(2);
00072:
00073: void bsp_rotary_encoder_custom_init(void)
00074: {
00075:     uint32_t available_device_nums = 0;
00076:
00077:     BSP_ROTARY_ENCODER_DRIVER_INIT(0, available_device_nums);
00078:     BSP_ROTARY_ENCODER_DRIVER_INIT(1, available_device_nums);
00079:     BSP_ROTARY_ENCODER_DRIVER_INIT(2, available_device_nums);
00080:
00081:     if (0 == available_device_nums)
00082:     {
00083:         LOG_MSGID_E(common, "[bsp]rotary_encoder/rotary encoder pin or eint has not been configured by ept tool on GPIO page/n/m, 0);
00084:
00085:
00086:         LOG_MSGID_I(common, "[bsp]rotary_encoder/rotary encoder pin or eint has been configured by ept tool on GPIO page/n/m, 0);
00087:

```

No.	Time	Index	CPU	Protocol	Length	Info
523	2021-11-23 14:22:14.465659	1054	CMA	MSDC_I2C	148	[M:common C:info F: L:]: [bsp][rotary encoder]????????rotary encoder pin or eint has been configured by ept tool on GPIO page/r/n
525	2021-11-23 14:22:14.465659	1055	CMA	MSDC_I2C	148	[M:common C:info F: L:]: [bsp][rotary encoder]I2C=0 PINS = 24, eint=24, mode = 9; PINS = 23, eint=23, mode = 9
528	2021-11-23 14:22:14.465659	1055	CMA	MSDC_I2C	148	[M:common C:info F: L:]: [bsp][rotary encoder]pin or eint has been configured by ept tool on GPIO page/r/n
528	2021-11-23 14:22:14.465659	1056	CMA	MSDC_I2C	124	[M:common C:info F: L:]: [bsp][rotary encoder]I2C=1 PINS = 16, eint=16, mode = 9; PINS = 17, eint=17, mode = 9
4043	2021-11-23 14:25:00.939271	167537	CMA	MSDC_I2C	102	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce counter=0 trigger_mode=0
4044	2021-11-23 14:25:00.939271	167537	CMA	MSDC_I2C	88	[M:common C:info F: L:]: [bsp][rotary encoder] EINT23 trigger, pin_state=1
4045	2021-11-23 14:25:00.939271	167537	CMA	MSDC_I2C	97	[M:common C:info F: L:]: [bsp][rotary encoder] prev_curr_state=0x2 current_state=0x2
4054	2021-11-23 14:25:02.336032	168932	CMA	MSDC_I2C	89	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=1395ms
4055	2021-11-23 14:25:02.336032	168932	CMA	MSDC_I2C	102	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce counter=0 trigger_mode=0
4056	2021-11-23 14:25:02.336032	168933	CMA	MSDC_I2C	88	[M:common C:info F: L:]: [bsp][rotary encoder] EINT23 trigger, pin_state=1
4057	2021-11-23 14:25:02.336032	168933	CMA	MSDC_I2C	86	[M:common C:info F: L:]: [bsp][rotary encoder] event curr. time=1395ms
4058	2021-11-23 14:25:02.336032	168933	CMA	MSDC_I2C	97	[M:common C:info F: L:]: [bsp][rotary encoder] prev_curr_state=0x0 current_state=0x0
4065	2021-11-23 14:25:06.606333	172126	CMA	MSDC_I2C	89	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=4285ms
4086	2021-11-23 14:25:08.100333	173126	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1
4094	2021-11-23 14:25:08.324857	174749	CMA	MSDC_I2C	89	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=1712ms
4095	2021-11-23 14:25:08.324857	174749	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1
4101	2021-11-23 14:25:09.903729	176509	CMA	MSDC_I2C	89	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=1508ms
4102	2021-11-23 14:25:09.903729	176509	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1
4107	2021-11-23 14:25:10.794518	177403	CMA	MSDC_I2C	88	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=893ms
4108	2021-11-23 14:25:10.794518	177403	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1
4113	2021-11-23 14:25:11.820954	178430	CMA	MSDC_I2C	89	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=1027ms
4114	2021-11-23 14:25:11.820954	178430	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1
4126	2021-11-23 14:25:13.404786	180010	CMA	MSDC_I2C	89	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=1508ms
4127	2021-11-23 14:25:13.404786	180010	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1
4132	2021-11-23 14:25:14.425847	181037	CMA	MSDC_I2C	89	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=1026ms
4133	2021-11-23 14:25:14.425847	181037	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1
4138	2021-11-23 14:25:15.326351	181927	CMA	MSDC_I2C	88	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 interval time=890ms
4139	2021-11-23 14:25:15.326351	181927	CMA	MSDC_I2C	92	[M:common C:info F: L:]: [bsp][rotary encoder] B EINT23 debounce pin state=1

```
[bsp][rotary_encoder]99999999rotary encoder pin or eint has been configured by ept tool on GPIO page\r\n[bsp][rotary_encoder]idx=0: PINA = 24, eint=24, mode = 9; PINB = 23, eint=23, mode = 9\r\n[bsp][rotary_encoder]99999999rotary encoder pin or eint has been configured by ept tool on GPIO page\r\n[bsp][rotary_encoder]idx=1: PINA = 16, eint=16, mode = 9; PINB = 17, eint=17, mode = 9
```

```

Bsp_rotary_encoder.c
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
#include <sys/types.h>
#include "hal.h"
#include "hal_gpio.h"
#include "hal_i2c.h"
#include "hal_mcp23017.h"
03332:
03333: bsp_rotary_encoder_status_t bsp_rotary_encoder_init(bsp_rotary_encode
03334: {
03335:     uint32_t i = 0;
03336:     hal_eint_config_t pin_a_eint_config;
03337:     hal_eint_config_t pin_b_eint_config;
03338:     hal_gpio_pin_t gpio_port_a;
03339:     hal_gpio_pin_t gpio_port_b;
03340:     hal_eint_number_t eint_number_a;
03341:     hal_eint_number_t eint_number_b;
03342:     bsp_rotary_encoder_t *re;
03343:     bsp_rotary_encoder_driver_t *driver;

```

```

00440: .....
00441:
00442:
00443: if (HAL_EINT_STATUS_OK != hal_eint_register_callback(eint_number_a, bsp_rotary_encoder_pina_eint_callback, (void*)
00444: HAL_EINT_STATUS_OK) != hal_eint_register_callback(eint_number_b, bsp_rotary_encoder_pinb_eint_callback, (void*)
00445: LOG_MSGID_E(common, "[bsp][rotary_encoder] EINT register callback fail", 0);
00446: return BSP_ROTARY_ENCODER_STATUS_ERROR;
00447: }
00448: LOG_MSGID_I(common, "[bsp][rotary_encoder] idx=x: PINA = %d, eint=%d, mode = %d; PINB = %d, eint=%d, mode = %d", 7,
00449: \
00450: driver->pina_config.gpio_port, \
00451: driver->pina_config.eint_number, \
00452: driver->pina_config.eint_mode, \
00453: driver->pinb_config.gpio_port, \
00454: driver->pinb_config.eint_number, \
00455: driver->pinb_config.eint_mode);

```

4113	2021-11-23	14:25:11.829954	178430	CM4	MSG_LOG	89	[M:common C:info F: L:]	[bsp[rotary encoder] B EINT23 interval time=1027ms
4114	2021-11-23	14:25:11.829954	178430	CM4	MSG_LOG	92	[M:common C:warning F: L:]	[bsp[rotary encoder] B EINT23 debounce pin state=1
4126	2021-11-23	14:25:13.404786	180010	CM4	MSG_LOG	89	[M:common C:info F: L:]	[bsp[rotary encoder] B EINT23 interval time=1580ms
4127	2021-11-23	14:25:13.404786	180010	CM4	MSG_LOG	92	[M:common C:warning F: L:]	[bsp[rotary encoder] B EINT23 debounce pin state=0
4132	2021-11-23	14:25:14.425847	181037	CM4	MSG_LOG	89	[M:common C:info F: L:]	[bsp[rotary encoder] B EINT23 interval time=1026ms
4133	2021-11-23	14:25:14.425847	181037	CM4	MSG_LOG	92	[M:common C:warning F: L:]	[bsp[rotary encoder] B EINT23 debounce pin state=1
4138	2021-11-23	14:25:15.326351	181927	CM4	MSG_LOG	88	[M:common C:info F: L:]	[bsp[rotary encoder] B EINT23 interval time=890ms
4139	2021-11-23	14:25:15.326351	181927	CM4	MSG_LOG	92	[M:common C:warning F: L:]	[bsp[rotary encoder] B EINT23 debounce pin state=0

经过反复检查，发现问题出在引脚的 Mode 不对（在 Ept_gpio_drv.h 文件），在 EPT 工具上配置编码器引脚，生成 inc 和 src 文件，结果我只把改动代码复制到 SDK 文件里，却忽略了文件 Ept_gpio_drv.h 的配置内容的含义，对应 EPT 的 Mode

[illegible]


```
App_usb_audio_idle_activity.c
00349:
00350: static bool app_usb_handle_rotary_event(
00351:     ui_shell_activity_t *self,
00352:     uint32_t event_id,
00353:     void *extra_data,
00354:     size_t data_len)
00355: {
00356:     bool ret = false;
00357:     #if defined(AIR_USB_AUDIO_MIX_ENABLE) || defined(AIR_USB_AUDIO_OUT_ENABLE)
00358:     static uint8_t s_ratio = ULL_MIX_RATIO_BALANCED_LEVEL;
00359:     #endif
00360:     bsp_rotary_encoder_port_t port;
00361:     bsp_rotary_encoder_event_t event;
00362:     uint32_t rotary_data;
00363:     if (!extra_data) {
00364:         return ret;
00365:     }
00366:     apps_config_key_action_t key_action = *(uint16_t *)extra_data;
00367:     app_event_rotary_event_decode(&port, &event, &rotary_data, event_id);
00368:
00369:     switch (key_action) {
00370:     #if defined(AIR_USB_AUDIO_MIX_ENABLE) || defined(AIR_USB_AUDIO_OUT_ENABLE)
00371:     case KEY_AUDIO_MIX_RATIO_GAME_ADD: {
00372:         if (ULL_MIX_RATIO_GAME_MAX_LEVEL + rotary_data <= s_ratio) {
00373:             s_ratio += rotary_data;
00374:             app_usb_audio_mix_rotatry_change(s_ratio);
00375:         }
00376:         break;
00377:     }
00378:     case KEY_AUDIO_MIX_RATIO_CHAT_ADD: {
00379:         if (s_ratio + rotary_data <= ULL_MIX_RATIO_CHAT_MAX_LEVEL) {
00380:             s_ratio += rotary_data;
00381:             app_usb_audio_mix_rotatry_change(s_ratio);
00382:         }
00383:         break;
00384:     }
00385:     #endif
00386:     case KEY_VOICE_UP: {
00387:     #if defined(MTK_USB_AUDIO_HID_ENABLE)
00388:         USB_Audio_HID_VolumeUp(rotary_data);
00389:     #endif
00390:     #if defined(USB_AUDIO_LOG_I)
00391:         USB_AUDIO_LOG_I("rotary volume up:%d", 1, rotary_data);
00392:     #endif
00393:     }
00394:     case KEY_VOICE_DN: {
00395:     #if defined(MTK_USB_AUDIO_HID_ENABLE)
00396:         USB_Audio_HID_VolumeDown(rotary_data);
00397:     #endif
00398:     #if defined(USB_AUDIO_LOG_I)
00399:         USB_AUDIO_LOG_I("rotary volume dn:%d", 1, rotary_data);
00400:     #endif
00401:     }
00402:     }
```

播放音乐时转编码器

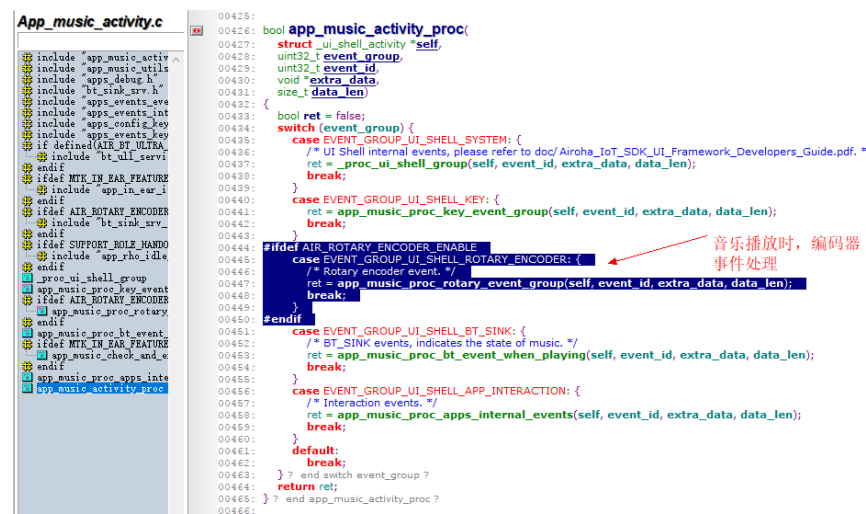
```
App_music_activity.c
00149:
00150: #ifndef AIR_ROTARY_ENCODER_ENABLE
00151: static bool app_music_proc_rotary_event_group(
00152:     struct_ui_shell_activity *self,
00153:     uint32_t event_id,
00154:     void *extra_data,
00155:     size_t data_len)
00156: {
00157:     bool ret = false;
00158:     bsp_rotary_encoder_port_t port;
00159:     bsp_rotary_encoder_event_t event;
00160:     uint32_t rotary_data;
00161:     if (!extra_data) {
00162:         return ret;
00163:     }
00164:     apps_config_key_action_t key_action = *(uint16_t *)extra_data;
00165:     app_event_rotary_event_decode(&port, &event, &rotary_data, event_id);
00166:     switch (key_action) {
00167:     case KEY_VOICE_UP:
00168:     case KEY_VOICE_DN: {
00169:     #if defined(AIR_BT_ULTRA_LOW_LATENCY_ENABLE)
00170:         bt_ull_streaming_info_t info = {0};
00171:         bt_ull_streaming_t streaming = {
00172:             .streaming_interface = BT_ULL_STREAMING_INTERFACE_SPEAKER,
00173:             .port = 0,
00174:         };
00175:
00176:         if ((BT_STATUS_SUCCESS == bt_ull_get_streaming_info(streaming, &info)) && info.is_playing) {
00177:             bt_status_t bt_status;
00178:             bt_volume_t volume_param;
00179:             volume_param.streaming.streaming_interface = BT_ULL_STREAMING_INTERFACE_SPEAKER;
00180:             volume_param.streaming.port = 0;
00181:             volume_param.channel = BT_ULL_VOLUME_CHANNEL_DUEL;
00182:             volume_param.volume = rotary_data; /* The delta value. */
00183:             if (KEY_VOICE_UP == key_action) {
00184:                 volume_param.action = BT_ULL_VOLUME_ACTION_SET_UP;
00185:             } else {
00186:                 volume_param.action = BT_ULL_VOLUME_ACTION_SET_DOWN;
00187:             }
00188:             bt_status = bt_ull_action(BT_ULL_ACTION_SET_STREAMING_VOLUME, &volume_param, sizeof(volume_param));
00189:             APPS_LOG_MSGID_I(APP_MUSIC_ACTI"app_music_proc_rotary_event_group, ull delat volume : %d, result: %d",
00190:                 2, rotary_data, bt_status);
00191:         }
00192:     }
00193:     #endif
00194:     {
00195:         uint8_t volume = bt_sink_srv_get_volume(NULL, BT_SINK_SRV_VOLUME_A2DP);
00196:         if (KEY_VOICE_UP == key_action) {
00197:             if (volume + rotary_data < bt_sink_srv_ami_get_a2dp_max_volume_level()) {
00198:                 volume += rotary_data;
00199:             } else {
00200:                 volume = bt_sink_srv_ami_get_a2dp_max_volume_level();
00201:             }
00202:         } else {
00203:             if (volume > rotary_data) {
00204:                 volume -= rotary_data;
00205:             } else {
00206:                 volume = 0;
00207:             }
00208:         }
00209:         APPS_LOG_MSGID_I(APP_MUSIC_ACTI"app_music_proc_rotary_event_group, music volume : %d", 1, volume);
00210:         bt_sink_srv_send_action(BT_SINK_SRV_ACTION_SET_VOLUME, &volume);
00211:     }
00212:     ret = true;
00213:     break;
00214: }
```

```

: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 15
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 14
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 13
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 12
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 11
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 10
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 9
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 8
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 7
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 6
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 5
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 4
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 3
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 2
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 3
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 4
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 5
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 6
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 7
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 8
: L: ]: [Music_APP]activity app_music_proc_rotary_event_group, music volume : 9

```

app_music_proc_rotary_event_group 被 app_music_activity_proc 调用



```

00425:
00426: bool app_music_activity_proc(
00427:     ui_shell_activity_t *self,
00428:     uint32_t event_id,
00429:     void *extra_data,
00430:     size_t data_len)
00431: {
00432:     bool ret = false;
00433:     switch (event_group) {
00434:     case EVENT_GROUP_UI_SHELL_SYSTEM: {
00435:         /* UI Shell internal events, please refer to doc/Airoha_IoT_SDK_UI_Framework_Developers_Guide.pdf. */
00436:         ret = _proc_ui_shell_group(self, event_id, extra_data, data_len);
00437:         break;
00438:     }
00439:     case EVENT_GROUP_UI_SHELL_KEY: {
00440:         ret = app_music_proc_key_event_group(self, event_id, extra_data, data_len);
00441:         break;
00442:     }
00443:     #ifdef AIR_ROTARY_ENCODER_ENABLE
00444:     case EVENT_GROUP_UI_SHELL_ROTARY_ENCODER: {
00445:         /* Rotary encoder event. */
00446:         ret = app_music_proc_rotary_event_group(self, event_id, extra_data, data_len);
00447:         break;
00448:     }
00449:     #endif
00450:     case EVENT_GROUP_UI_SHELL_BT_SINK: {
00451:         /* BT_SINK events, indicates the state of music. */
00452:         ret = app_music_proc_bt_event_when_playing(self, event_id, extra_data, data_len);
00453:         break;
00454:     }
00455:     case EVENT_GROUP_UI_SHELL_APP_INTERACTION: {
00456:         /* Interaction events. */
00457:         ret = app_music_proc_apps_internal_events(self, event_id, extra_data, data_len);
00458:         break;
00459:     }
00460:     default: {
00461:         break;
00462:     }
00463:     }
00464:     return ret;
00465: }
00466:

```

音乐活动程序



```

00336: }
00337:
00338: bool app_music_idle_proc_bt_sink_events(ui_shell_activity_t *self, uint32_t event_id,
00339:     void *extra_data, size_t data_len)
00340: {
00341:     bool ret = false;
00342:     app_music_local_context_t *local_context = (app_music_local_context_t *)self->local_context;
00343:     if (local_context == NULL) {
00344:         return ret;
00345:     }
00346:     if (event_id == BT_SINK_SRV_EVENT_STATE_CHANGE) {
00347:         /* BT_SINK_SRV_EVENT_STATE_CHANGE: param->param->state_change */
00348:         APPS_LOG_MSGID_I(APP_MUSIC_UTILS) "app_music_proc_bt_sink_events param->pre=0x%x, param->now=0x%x, is_playing
00349:         3, param->previous, param->current, local_context->music_playing);
00350:         /* Try to start app_music_activity when the music starts playing. */
00351:         if ((param->previous != BT_SINK_SRV_STATE_STREAMING) && (param->current == BT_SINK_SRV_STATE_STREAMING)) {
00352:             /* If music playing is true, it indicates that the app_music_activity already exists. */
00353:             if (!local_context->music_playing) {
00354:                 local_context->isAutoPaused = false;
00355:                 ui_shell_start_activity(self, app_music_activity_proc, ACTIVITY_PRIORITY_MIDDLE, local_context, 0);
00356:                 APPS_LOG_MSGID_I(APP_MUSIC_UTILS) "app_music_proc_bt_sink_events ui_shell_start_activity, current activity: %s, is
00357:                 2, (uint32_t)self, local_context->isAutoPaused);
00358:                 local_context->music_playing = true;
00359:             }
00360:         }
00361:     }
00362: }
00363:
00364: bool app_music_idle_activity_proc(
00365:     ui_shell_activity_t *self,
00366:     uint32_t event_id,
00367:     void *extra_data,
00368:     size_t data_len)
00369: {
00370:     bool ret = false;
00371:     switch (event_group) {
00372:     case EVENT_GROUP_UI_SHELL_SYSTEM: {
00373:         /* UI Shell internal events, please refer to doc/Airoha_IoT_SDK_UI_Framework_Developers_Guide.pdf. */
00374:         ret = _proc_ui_shell_group(self, event_id, extra_data, data_len);
00375:         break;
00376:     }
00377:     case EVENT_GROUP_UI_SHELL_KEY: {
00378:         /* Key event. */
00379:         ret = _proc_key_event_group(self, event_id, extra_data, data_len);
00380:         break;
00381:     }
00382:     case EVENT_GROUP_UI_SHELL_BT_SINK: {
00383:         /* BT_SINK events, indicates the state of music. */
00384:         ret = app_music_idle_proc_bt_sink_events(self, event_id, extra_data, data_len);
00385:         break;
00386:     }
00387:     case EVENT_GROUP_UI_SHELL_BT_CONN_MANAGER: {
00388:         /* The event come from bt connection manager. */
00389:         ret = app_music_idle_proc_bt_cm_events(self, event_id, extra_data, data_len);
00390:         break;
00391:     }
00392:     case EVENT_GROUP_UI_SHELL_APP_INTERACTION: {
00393:         /* Interaction events. */
00394:         ret = app_music_idle_proc_apps_internal_events(self, event_id, extra_data, data_len);
00395:         break;
00396:     }
00397:     #if defined(MTK_AWS_MCE_ENABLE)
00398:     case EVENT_GROUP_UI_SHELL_AWS_MCE: {
00399:         /* AWS MCE events. */
00400:         ret = app_music_idle_proc_aws_mce_events(self, event_id, extra_data, data_len);
00401:         break;
00402:     }
00403:     #endif
00404:     default: {
00405:         break;
00406:     }
00407:     }
00408:     return ret;
00409: }
00410:

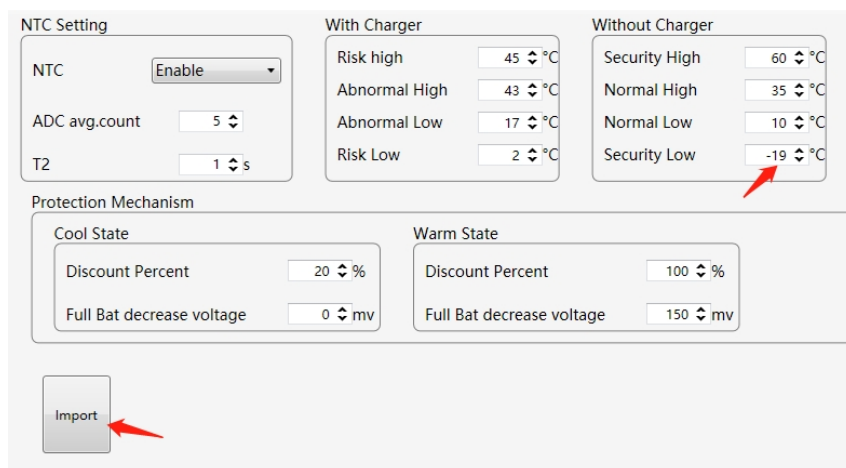
```

启动各种空闲活动 (ULL、gound、SMcharger、battery、rho、usb_audio、HFP、music、fota、power、home_screen、line_in 等等)



三十七、修改 NTC 参数

客户反馈耳机（AB1561M）在-15 度时，耳机无法工作，将 NTC 温度再调低了 2 度。



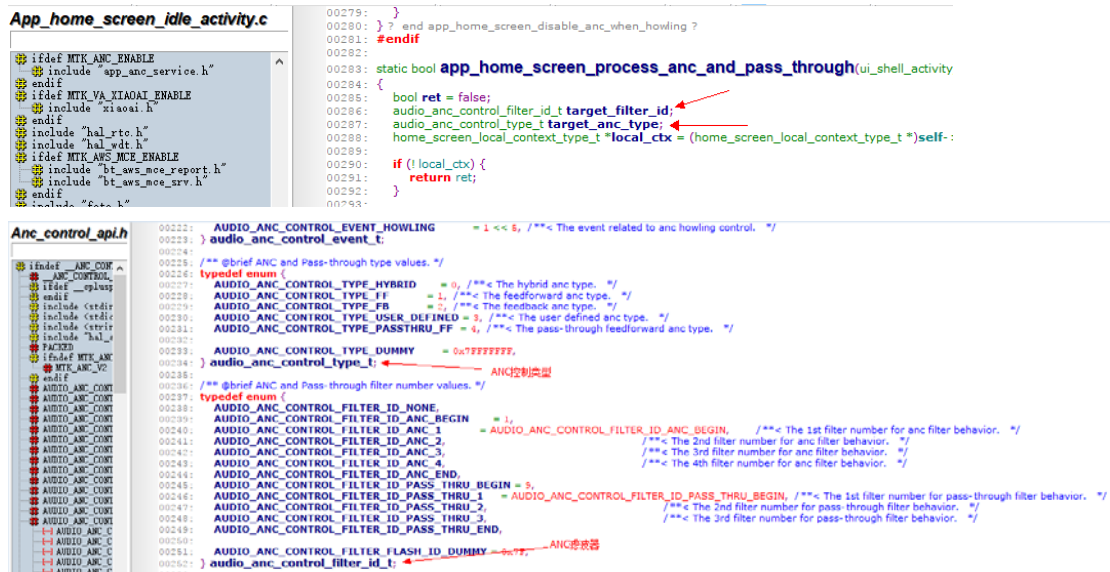
三十八、查看 ANC 模式和切换方法

首先工程的 mk 文件中开启了 ANC 模块

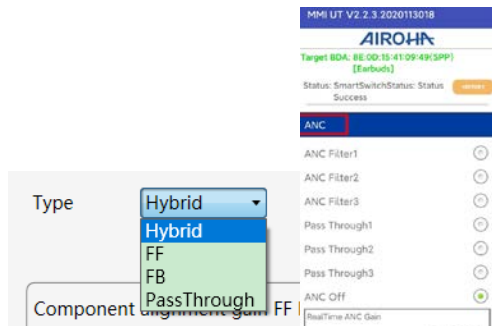
```
Feature_68_evk.mk (d:\chp200_sdk_v2.7.1\mcu\project\ab1561)
MTK_MINIDUMP_ENABLE = y
MTK_FULLDUMP_ENABLE = y

# ANC module
MTK_ANC_ENABLE = y
ifeq ($(MTK_ANC_ENABLE), y)
  ANC_SOURCE_CODE = $(strip $(SOURCE_DIR))/../mcu/m
  ifeq ($(ANC_SOURCE_CODE), $(wildcard $(ANC_SOURCE_CODE)))
    MTK_ANC_LIB_ENABLE = n
  else
    MTK_ANC_LIB_ENABLE = y
  endif
endif
```

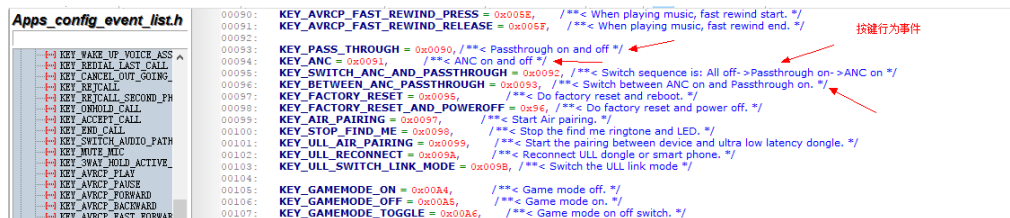
在源码文件中搜索 MTK_ANC_ENABLE



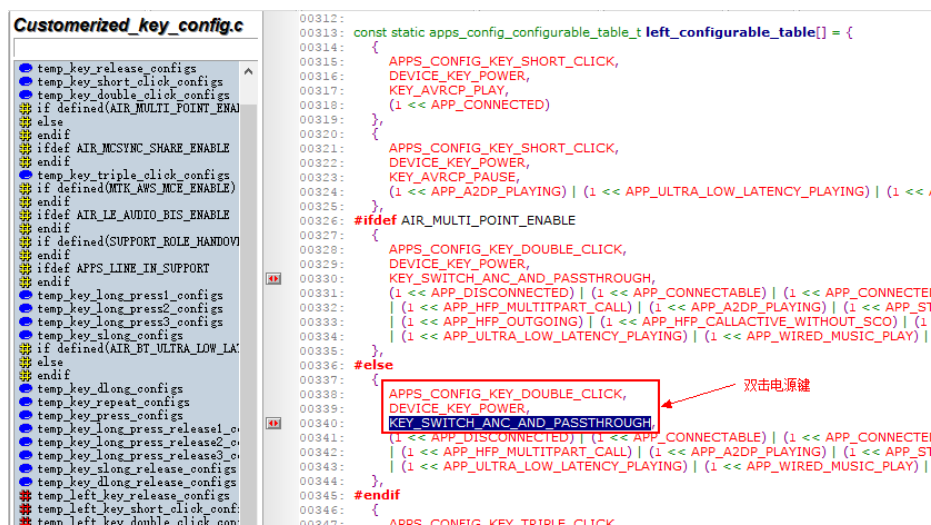
对应 Config tool 的选项和 APP 上的 ANC 选项



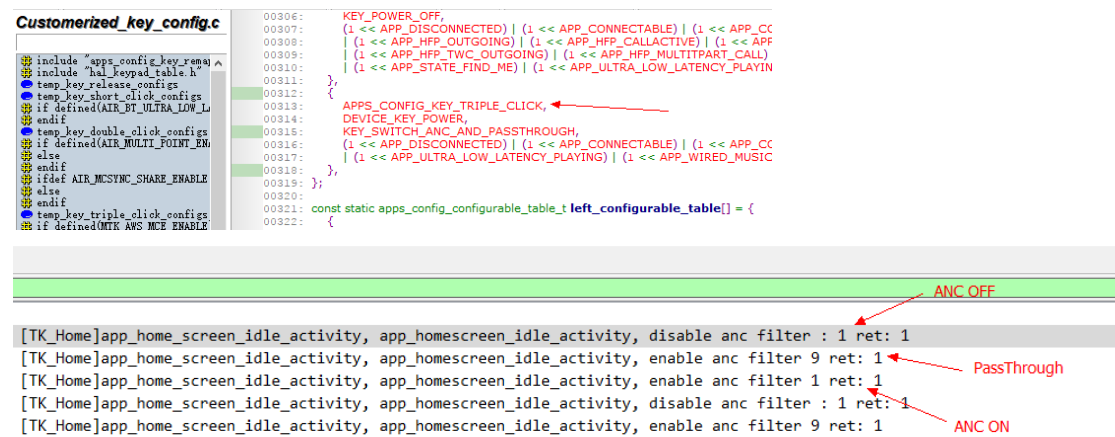
按键行为定义



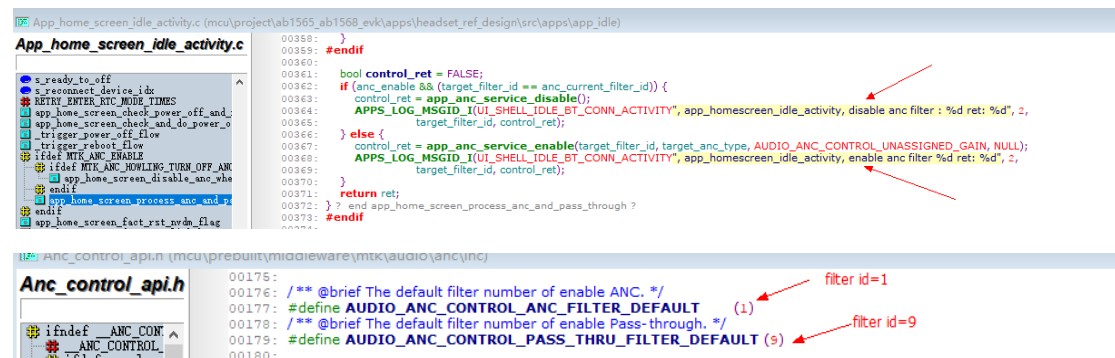
再追踪不同的行为产生于哪个按键，在 customized_key_config.c 文件



改成三击电源键

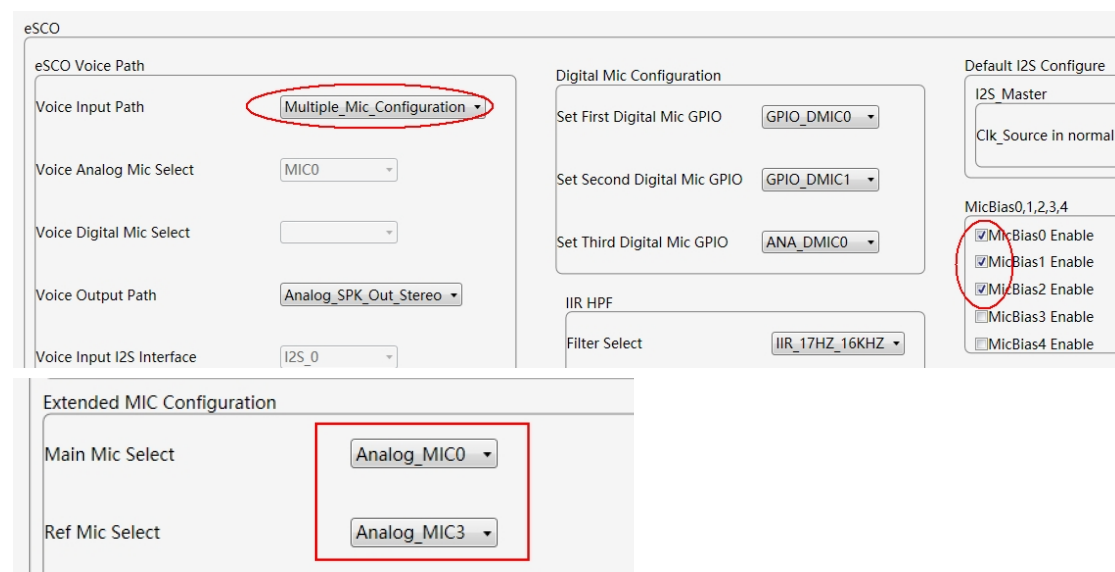


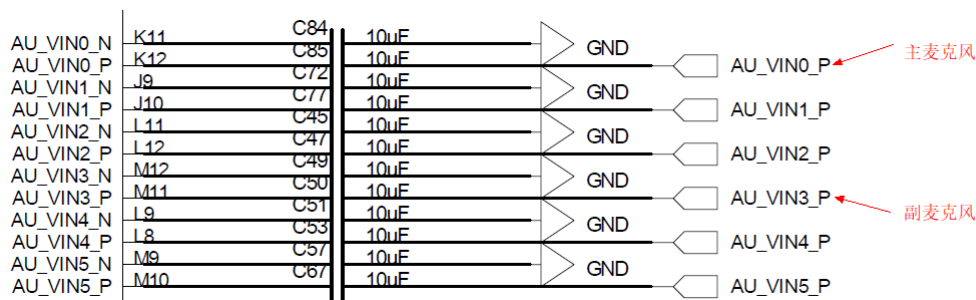
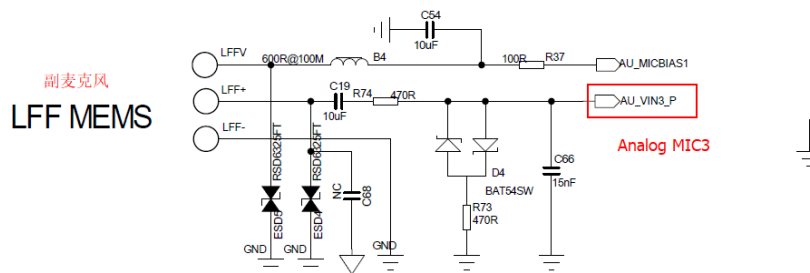
对程序里的打印



三十九、开启双麦通话功能

使用 ATK v2.9.0 工具设置 mic，micbias 是给 mic 提供偏置电压的





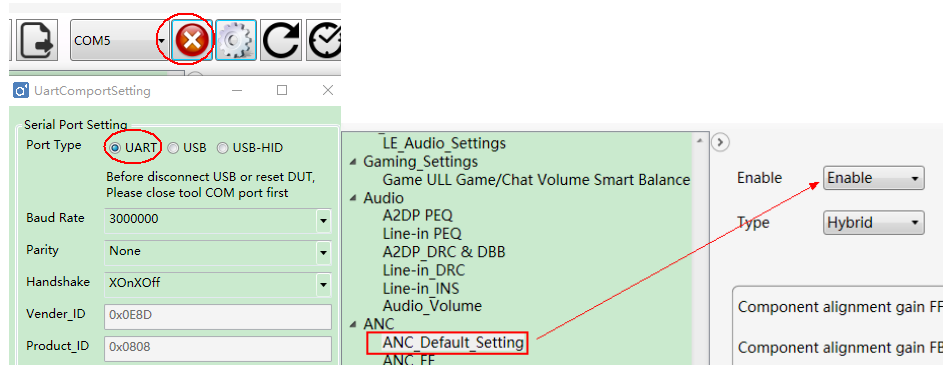
SideTone 是通话时会开启，用来收自己讲话声音回耳机端，让自己在戴着耳机通话时，也可以听到自己说话声音，不至于因为耳套盖住耳朵，只能听到很小的说话声音

四十、在线测试 ANC 功能同时抓 log

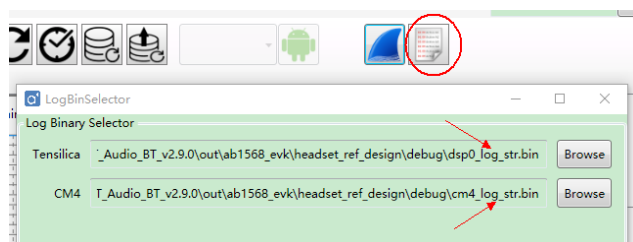
先将有 ANC 调整值的 download 到耳机

连接耳机 UART，在 config tool 打开 download 配置和串口

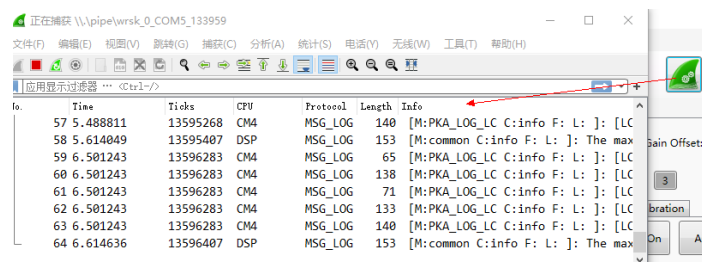
在 ANC 默认设置中开启 ANC，点击 Write NV



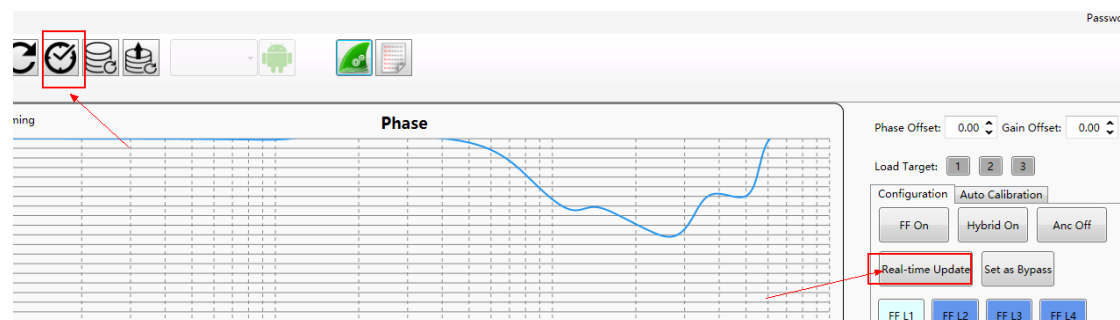
设置工程对应的 debug 的 bin 文件



点击启动调试分析器查看 log

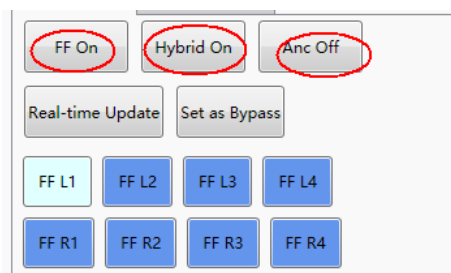


测试 ANC_FF, 先点击 Real-time Update (此功能只是实时生效, 查看效果, 并不会写入固件), 再点击 Write NV 写入



```
[13:42:58.241] Update ANC coefficient: Done.
[13:43:06.054] Write NV to device: Start...
[13:43:06.076] Write NV to device: No modified NV needs to update.
```

分别测试 FF On、Hybrid On、Anc Off, 戴着耳机听 ANC 效果



查看 log 特定信息

No.	Time	Ticks	CPU	Protocol	Length	Info
2536	363.304050	13953059	CM4	MSG_LOG	108	[M:hal C:info F: L:]: [hal_audio_status_set_running_flag] typr=12, dsp_enable=12288, enable=1\r\n
2627	363.305053	13953095	DSP	MSG_LOG	86	[M:hal C:info F: L:]: hal_audio_device_set_agent 64 on/off 0 user_conut 1
2631	363.305053	13953095	DSP	MSG_LOG	85	[M:hal C:info F: L:]: hal_audio_device_set_agent 3 on/off 1 user_conut 0
2649	363.342249	13953097	DSP	MSG_LOG	71	[M:hal C:info F: L:]: [DSP DEBUG] hal_audio_ul_set_iir = 4
2655	363.342249	13953098	DSP	MSG_LOG	86	[M:hal C:info F: L:]: hal_audio_device_set_agent 64 on/off 1 user_conut 1
2730	363.342249	13953115	CM4	MSG_LOG	107	[M:hal C:info F: L:]: [hal_audio_status_set_running_flag] typr=7, dsp_enable=12288, enable=1\r\n
2783	363.397120	13953136	CM4	MSG_LOG	108	[M:hal C:info F: L:]: [hal_audio_status_set_running_flag] typr=13, dsp_enable=12416, enable=1\r\n
2814	363.397120	13953139	DSP	MSG_LOG	87	[M:hal C:info F: L:]: hal_audio_device_set_agent 768 on/off 1 user_conut 1
2884	363.397120	13953150	CM4	MSG_LOG	108	[M:hal C:info F: L:]: [hal_audio_status_set_running_flag] typr=13, dsp_enable=12416, enable=1\r\n
2990	363.827139	13953537	DSP	MSG_LOG	85	[M:hal C:info F: L:]: hal_audio_device_set_agent 3 on/off 0 user_conut 1
2992	363.827139	13953537	DSP	MSG_LOG	71	[M:hal C:info F: L:]: [DSP DEBUG] hal_audio_ul_set_iir = 4
3167	363.866132	13953633	DSP	MSG_LOG	87	[M:hal C:info F: L:]: hal_audio_device_set_agent 768 on/off 0 user_conut 2
3209	363.866132	13953640	CM4	MSG_LOG	107	[M:hal C:info F: L:]: [hal_audio_status_set_running_flag] typr=7, dsp_enable=12416, enable=0\r\n

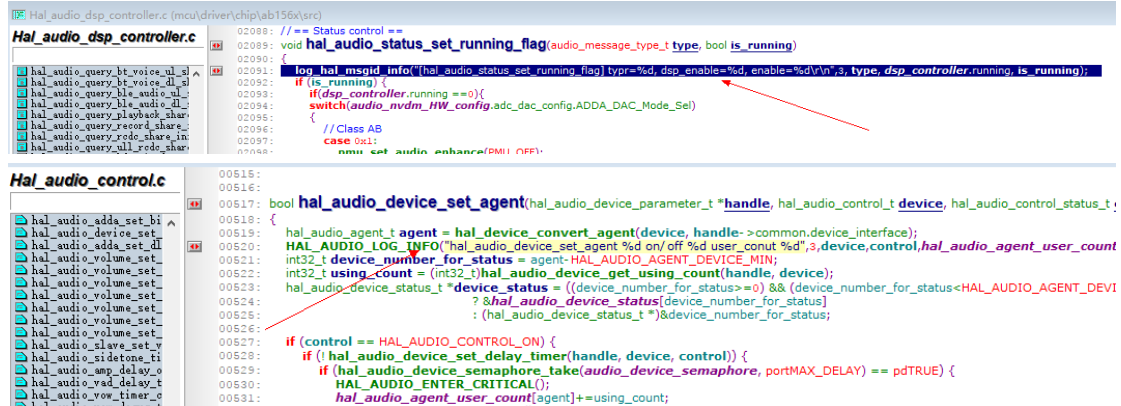
```
ifo F: L: ]: [hal_audio_status_set_running_flag] typr=12, dsp_enable=12288, enable=1\r\n
ifo F: L: ]: hal_audio_device_set_agent 3 on/off 1 user_conut 0
ifo F: L: ]: [DSP DEBUG] hal_audio_ul_set_iir = 4
ifo F: L: ]: hal_audio_device_set_agent 64 on/off 1 user_conut 1
ifo F: L: ]: [hal_audio_status_set_running_flag] typr=7, dsp_enable=12288, enable=1\r\n
ifo F: L: ]: [hal_audio_status_set_running_flag] typr=13, dsp_enable=12416, enable=1\r\n
ifo F: L: ]: hal_audio_device_set_agent 768 on/off 1 user_conut 1
ifo F: L: ]: [hal_audio_status_set_running_flag] typr=13, dsp_enable=12416, enable=1\r\n
ifo F: L: ]: hal_audio_device_set_agent 768 on/off 0 user_conut 2
ifo F: L: ]: [hal_audio_status_set_running_flag] typr=7, dsp_enable=12416, enable=0\r\n
```

Hybrid On

```

F: L: ]: hal_audio_device_set_agent 64 on/off 0 user_conut 1
F: L: ]: [hal_audio_status_set_running_flag] typr=7, dsp_enable=12288, enable=1\r\n
F: L: ]: [hal_audio_status_set_running_flag] typr=13, dsp_enable=12416, enable=1\r\n
F: L: ]: hal_audio_device_set_agent 768 on/off 1 user_conut 1
F: L: ]: [hal_audio_status_set_running_flag] typr=13, dsp_enable=12416, enable=1\r\n
F: L: ]: hal_audio_device_set_agent 64 on/off 0 user_conut 1
F: L: ]: hal_audio_device_set_agent 768 on/off 0 user_conut 2
F: L: ]: hal_audio_device_set_agent 3 on/off 0 user_conut 1
F: L: ]: [DSP DEBUG] hal_audio_ul_set_iir = 4 ANC Off
F: L: ]: hal_audio_device_set_agent 3 on/off 0 user_conut 1
F: L: ]: [DSP DEBUG] hal_audio_ul_set_iir = 4
F: L: ]: [hal_audio_status_set_running_flag] typr=12, dsp_enable=12416, enable=0\r\n
F: L: ]: hal_audio_device_set_agent 768 on/off 0 user_conut 1
F: L: ]: [hal_audio_status_set_running_flag] typr=7, dsp_enable=8320, enable=0\r\n
F: L: ]: hal_audio_device_set_agent 768 on/off 0 user_conut 1
F: L: ]: [hal_audio_status_set_running_flag] typr=13, dsp_enable=8192, enable=0\r\n

```



其他 ANC_FB、ANC_FBC 和 AiroThru 测试方法相同

保存 log 文件的方式

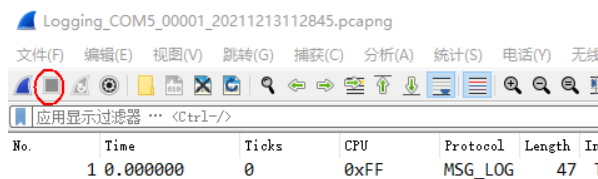
先点击停止



文件 » 另存为，会保存成 pcapng 文件类型

打开 log 文件的方式

进入分析器后点击停止



然后 文件 » open，打开对应的 pcapng 文件即可

1301_log.pcapng

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(T) 帮助(H)

应用显示过滤器 ... <Ctrl-/>

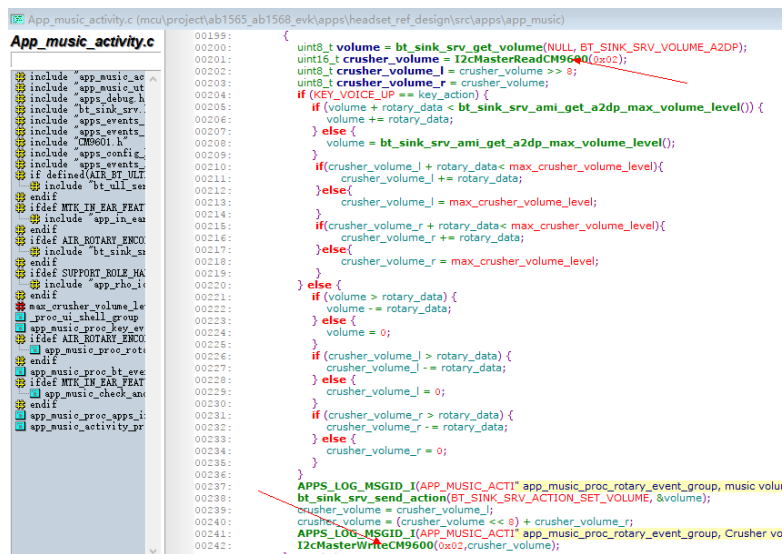
No.	Time	Ticks	CPU	Protocol	Length	Info
1	0.000000	0	0xFF	MSG_LOG	47	Tool Version: Logging Tool-1.5.11.2
2	0.027076	201990	DSP	MSG_LOG	118	[M:common C:info F: L:]: Wake Up From Sle
3	0.027076	201990	DSP	MSG_LOG	76	[M:common C:info F: L:]: tickCompensate
4	0.027076	201990	DSP	MSG_LOG	140	[M:common C:info F: L:]: Wake Up From Sle
5	0.027076	201992	DSP	MSG_LOG	49	[M:LC C:info F: L:]: [LC]Latency:608
6	0.027076	201992	DSP	MSG_LOG	54	[M:LC C:info F: L:]: [LC]CurrMaxBuf:36224
7	0.027076	201992	DSP	MSG_LOG	58	[M:LC C:info F: L:]: [LC]PartnerInitFail
8	0.027076	201992	DSP	MSG_LOG	72	[M:LC C:info F: L:]: [LC][ActiveLink]SP F
9	0.224427	202183	DSP	MSG_LOG	118	[M:common C:info F: L:]: Wake Up From Sle
10	0.224427	202184	DSP	MSG_LOG	76	[M:common C:info F: L:]: tickCompensate
11	0.224427	202184	DSP	MSG_LOG	142	[M:common C:info F: L:]: Wake Up From Sle
12	0.423331	202380	DSP	MSG_LOG	118	[M:common C:info F: L:]: Wake Up From Sle
13	0.423331	202380	DSP	MSG_LOG	76	[M:common C:info F: L:]: tickCompensate
14	0.423331	202380	DSP	MSG_LOG	142	[M:common C:info F: L:]: Wake Up From Sle
15	0.423331			LMP	6	
16	0.423331	202394	DSP	MSG_LOG	61	[M:LC_LINK_PRI C:info F: L:]: [LC]RxFull
17	0.530636	202486	DSP	MSG_LOG	118	[M:common C:info F: L:]: Wake Up From Sle
18	0.530636	202487	DSP	MSG_LOG	76	[M:common C:info F: L:]: tickCompensate

四十一、用编码器调节振动大小

现有工程可以实现编码器调节音乐音量大小，添加关联振动大小调节的代码在 CM9601 驱动文件添加函数 I2cMasterReadCM9600



在 apps 层的 app_music_activity.c 文件中调用



编译提示 CM9601 的函数未定义的错误

```
make[1]: 警告: jobservr 不可用, 正使用 -j1. 添加 "+" 到父 make 的规则. In file included from
.../project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/music/app_music/activity.c:155:0:
h:\mcu\driver\board\component\bsp_audio_ext_codec\inc\cm9601.h:183:17: warning: '120MasterWriteCM9600' used but never defined [enabled by default]
static uint16_t 120MasterWriteCM9600(uint8_t reg_addr, uint16_t write_data);
h:\mcu\driver\board\component\bsp_audio_ext_codec\inc\cm9601.h:184:17: warning: '120MasterReadCM9600' used but never defined [enabled by default]
static uint16_t 120MasterReadCM9600(uint8_t reg_addr);
.../out/ab1565_ab1568_evk/headset_ref_design/feature_68_evk/obj/project/ab1565_ab1568_evk/apps/headset_ref_design/src/apps/music/app_music/activity.o: In function 'app_music_proc_rotary_event_group':
h:\mcu\project\ab1565_ab1568_evk/apps/headset_ref_design/src/apps/music/app_music/activity.c:201: undefined reference to '120MasterWriteCM9600'
h:\mcu\project\ab1565_ab1568_evk/apps/headset_ref_design/src/apps/music/app_music/activity.c:242: undefined reference to '120MasterReadCM9600'
collect2.exe: error: ld returned 1 exit status
make: *** [Makefile:568: h:\mcu\out\ab1565_ab1568_evk/headset_ref_design/feature_68_evk/headset_ref_design.oif] 错误 1
```

在调用的文件里也包含了头文件了，还是同样的编译错误
查看了 CM9601 的其它驱动函数调用方法，在 driver 层调用的

```
Bsp_audio_ext_codec.c (mcu\driver\board\component\bsp_audio_ext_codec\src)
Bsp_audio_ext_codec.c
00092: bsp_audio_ext_codec_status_t bsp_audio_ext_codec_dac_enable(bsp_audio_ext_codec_status_t)
00093: {
00094:     switch (codec_type) {
00095:         case BSP_AUDIO_EXT_CODEC_TYPE_DA7212:
00096:             DA7212_dac_enable();
00097:             break;
00098:         case BSP_AUDIO_EXT_CODEC_TYPE_CM9601:
00099:             CM9601_dac_enable();
00100:             break;
00101:         default:
00102:             log_hal_msgid_info("[BSP_AUDIO_EXT_CODEC] codec type error", 0);
00103:             break;
00104:     }
```

bsp_audio_ext_codec_dac_enable 函数在 middleware 层的 scenario_wired_audio.c 文件的函数
audio_transmitter_wired_audio_open_playback 调用

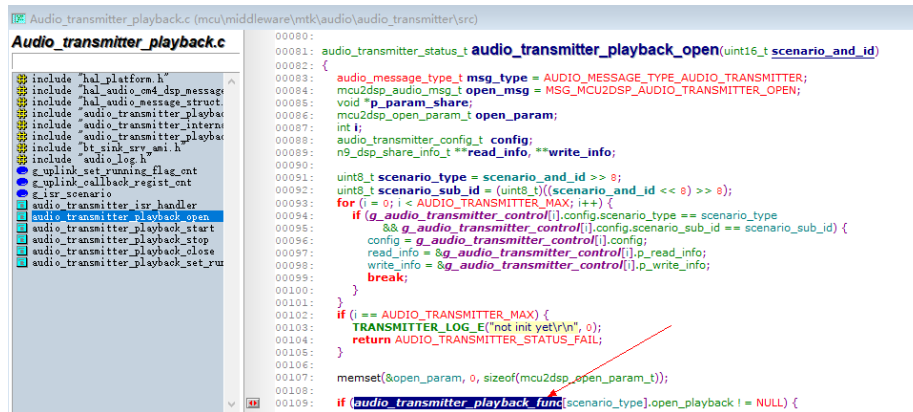
```
Scenario_wired_audio.c (mcu\middleware\mtk\audio\audio_transmitter\src\audio_transmitter_scenario_port)
Scenario_wired_audio.c
00478: open_param->stream_out = STREAM_OUT_AFE;
00479:
00480: #if 0
00481: open_param->stream_out_param.afc.audio_device = HAL_AUDIO_DEVICE_DAC_DUAL; // use DAC DUAL fi
00482: open_param->stream_out_param.afc.audio_interface = HAL_AUDIO_INTERFACE_1;
00483: #else
00484: open_param->stream_out_param.afc.audio_device = HAL_AUDIO_DEVICE_I2S_MASTER; // use I2S MAST
00485: open_param->stream_out_param.afc.audio_interface = HAL_AUDIO_INTERFACE_1; // I2S 0
00486:
00487: bsp_audio_ext_codec_dac_init(BSP_AUDIO_EXT_CODEC_TYPE_CM9601, NULL);
00488: bsp_audio_ext_codec_dac_enable(BSP_AUDIO_EXT_CODEC_TYPE_CM9601);
00489: is_ext_dac_initialized = true;
00490: #endif
00491: open_param->stream_out_param.afc.stream_channel = HAL_AUDIO_DIRECT;
00492: open_param->stream_out_param.afc.memory = HAL_AUDIO_MFM1;
00493:
```

audio_transmitter_wired_audio_open_playback 函数被 middleware 层的
audio_transmitter_playback_port.c 文件的数组 audio_transmitter_playback_func 调用

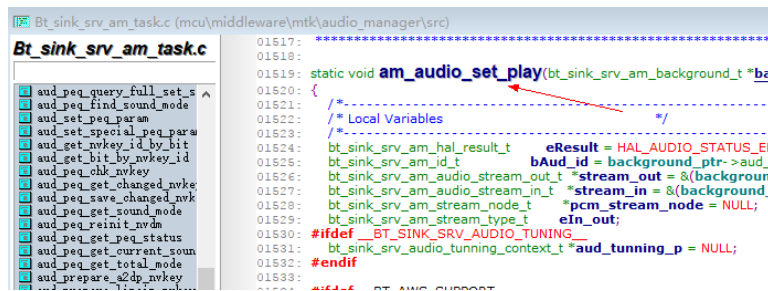
```
Audio_transmitter_playback_port.c (mcu\middleware\mtk\audio\audio_transmitter\src)
Audio_transmitter_playback_port.c
00087: {audio_transmitter_tdm_open_playback, audio_transmitter_tdm_start_playback, NULL},
00088: #else
00089: {NULL, NULL, NULL}, //AUDIO_TRANSMITTER_TDM
00090: #endif
00091: #if defined(AIR_WIRED_AUDIO_ENABLE)
00092: {audio_transmitter_wired_audio_open_playback, audio_transmitter_wired_audio_start_p
00093: #else
00094: {NULL, NULL, NULL}, //AUDIO_TRANSMITTER_ANC_MONITOR_STREAM
00095: #endif
00096: #if defined(AIR_ADVANCED_PASSTHROUGH_ENABLE)
00097: {advanced_passthrough_open_playback, advanced_passthrough_start_playback, advanc
00098: #else
00099: {NULL, NULL, NULL},
00100: #endif /* AIR_ADVANCED_PASSTHROUGH_ENABLE */
00101: }

Audio_transmitter_playback_port.c (mcu\middleware\mtk\audio\audio_transmitter\src)
Audio_transmitter_playback_port.c
00059: }
00060:
00061: /*-----
00062: audio_transmitter_playback_func_t audio_transmitter_playback_func(AUDIO_TRANSMITTER_SCENARIO_TYPI
00063: // open_playback_t, start_playback_t, set_runtime_config_playback
00064: {NULL, NULL, NULL}, //AUDIO_TRANSMITTER_AZDP_SOURCE
00065: #if defined(MTK_SENSOR_SOURCE_ENABLE)
00066: {audio_transmitter_gsensor_open_playback, audio_transmitter_gsensor_start_playback, NULL}
00067: #else
00068: {NULL, NULL, NULL}, //AUDIO_TRANSMITTER_GSENSOR
00069: #endif
00070: #if defined(MTK_MULTI_MIC_STREAM_ENABLE)
00071: {audio_transmitter_multimic_open_playback, audio_transmitter_multimic_start_playback, NULL}
```

audio_transmitter_playback_func 在 middleware 层的 audio_transmitter_playback.c 中被使用



audio_transmitter_playback_open 在 middleware 层的 bt_sink_srv_am_task.c 的函数 am_audio_set_play 调用



四十二、sh 文件与 makefile 文件与 MK 文件的关系

dsp	2021/10/28 11:13	文件夹	
mcu	2021/10/28 11:14	文件夹	
build	2021/8/2 18:42	SH 文件	26 KB
mapping_disk	2021/10/27 17:13	文件	1 KB

SDK 根目录下的 sh 脚本文件

mcu > project > ab1565_ab1568_evk > apps > headset_ref_design > GCC				
名称	修改日期	类型	大小	
ab156x_evb.cmm	2021/8/2 18:41	CMM 文件	2 KB	
ab156x_flash.ld	2021/8/2 18:41	LD 文件	17 KB	
ab156x_flash_8m.ld	2021/8/2 18:41	LD 文件	17 KB	
ab156x_flash_8m_fota_test.ld	2021/8/2 18:41	LD 文件	18 KB	
ab156x_sysram.ld	2021/8/2 18:41	LD 文件	11 KB	
feature	2021/8/2 18:41	MK 文件	4 KB	
feature_65_8m_evk	2021/8/2 18:41	MK 文件	12 KB	
feature_65_evb	2021/8/2 18:41	MK 文件	11 KB	
feature_65_evb_emp	2021/8/2 18:41	MK 文件	11 KB	
feature_65_evk	2021/8/2 18:41	MK 文件	11 KB	
feature_65_evk_emp	2021/8/2 18:41	MK 文件	11 KB	
feature_68_evb_ull	2021/8/2 18:41	MK 文件	12 KB	
feature_68_evk	2021/8/2 18:41	MK 文件	11 KB	
feature_68_evk_emp	2021/8/2 18:41	MK 文件	11 KB	
feature_68_evk_ull	2021/8/2 18:41	MK 文件	12 KB	
feature_68_evk_ull_a2dp	2021/8/2 18:41	MK 文件	12 KB	
feature_68_headset	2021/8/2 18:41	MK 文件	13 KB	
feature_68_headset_ull	2021/8/2 18:41	MK 文件	13 KB	
Makefile	2021/8/2 18:41	文件	21 KB	
startup_ab156x.s	2021/8/2 18:41	S 文件	11 KB	
syscalls	2021/8/2 18:41	C 文件	6 KB	
test_65_evk_vendor_1	2021/8/2 18:41	MK 文件	3 KB	

工程的 GCC 下的 Makefile 文件

mcu > project > ab1565_ab1568_evk > apps > headset_ref_design > GCC			
名称	修改日期	类型	大小
ab156x_evb.cmm	2021/8/2 18:41	CMM 文件	2 KB
ab156x_flash.ld	2021/8/2 18:41	LD 文件	17 KB
ab156x_flash_8m.ld	2021/8/2 18:41	LD 文件	17 KB
ab156x_flash_8m_fota_test.ld	2021/8/2 18:41	LD 文件	18 KB
ab156x_sysram.ld	2021/8/2 18:41	LD 文件	11 KB
feature	2021/8/2 18:41	MK 文件	4 KB
feature_65_8m_evk	2021/8/2 18:41	MK 文件	12 KB
feature_65_evb	2021/8/2 18:41	MK 文件	11 KB
feature_65_evb_emp	2021/8/2 18:41	MK 文件	11 KB
feature_65_evk	2021/8/2 18:41	MK 文件	11 KB
feature_65_evk_emp	2021/8/2 18:41	MK 文件	11 KB
feature_68_evb_ull	2021/8/2 18:41	MK 文件	12 KB
feature_68_evk	2021/8/2 18:41	MK 文件	11 KB
feature_68_evk_emp	2021/8/2 18:41	MK 文件	11 KB
feature_68_evk_ull	2021/8/2 18:41	MK 文件	12 KB
feature_68_evk_ull_a2dp	2021/8/2 18:41	MK 文件	12 KB
feature_68_headset	2021/8/2 18:41	MK 文件	13 KB
feature_68_headset_ull	2021/8/2 18:41	MK 文件	13 KB
Makefile	2021/8/2 18:41	文件	21 KB
startup_ab156x.s	2021/8/2 18:41	S 文件	11 KB
syscalls	2021/8/2 18:41	C 文件	6 KB
test_65_evk_vendor_1	2021/8/2 18:41	MK 文件	3 KB

工程的 GCC 下的 MK 文件

mcu > driver > board > component > bsp_audio_ext_codec			
名称	修改日期	类型	大小
inc	2021/8/2 18:41	文件夹	
src	2021/8/2 18:41	文件夹	
module	2021/8/2 18:41	MK 文件	

外设模块驱动文件目录下的 MK 文件

CHP200_sdk_V2.7.1 > mcu > middleware > MTK > battery_management			
名称	修改日期	类型	大小
inc	2021/12/3 10:31	文件夹	
port	2021/11/24 21:51	文件夹	
src	2021/11/24 21:51	文件夹	
module	2021/8/2 18:41	MK 文件	6 KB
readme	2021/8/2 18:41	文本文档	

中间件驱动文件也有 MK 文件

四十三、电量和音量等级设置

默认音量等级 101 级，这是不对的。

RAW Command

Sound_Level

SCO_Level_Settings

A2DP_Level_Settings

Line-In_Level_Settings

VP_Level_Settings

USB_Audio_Level_Settings

USB_IN_Level_Settings

USB/Line_OUT_Level_Settings

Gain_Settings

Analog_Gain_Settings

A2DP_Gain_Settings

Line-in_Gain_Settings

SCO_Gain_Settings

Record/WWE_Gain_Settings

VP_Gain_Settings

USB_Audio_Gain_Settings

Detach_Mic_Gain_Settings

General_Purpose_SW_Gain_Settings

DSP_Voice

Dual_MIC_NR

High_Pass_Filter

A2DP Digital Output Gain

Total Index

101

Auto Setting

000	-58.5	dB
001	-57.5	dB
002	-57.5	dB
003	-56.5	dB
004	-56.5	dB
005	-55.5	dB
006	-55.5	dB
007	-54.5	dB
008	-54.5	dB
009	-54.5	dB
010	-53.5	dB
011	-53.5	dB
012	-52.5	dB

A2DP Digital Output Gain

Total Index

101

Auto Setting

088	-20.5	dB
089	-19.5	dB
090	-19.5	dB
091	-18.5	dB
092	-18.5	dB
093	-18.5	dB
094	-17.5	dB
095	-17.5	dB
096	-16.5	dB
097	-16.5	dB
098	-15.5	dB
099	-15.5	dB
100	-15.5	dB

音量等级改成了 17，3dB 一档，最大音量为-1。可以按这个参数进行调整，但要注意最大音量是否太大。太大就需要整体往下调。

0	mute
1	-46
2	-43
3	-40
4	-37
5	-34
6	-31
7	-28
8	-25
9	-22
10	-19
11	-16
12	-13
13	-10
14	-7
15	-4
16	-1

使用的是 4.35V 的电池，电量等级的电压可适当调高一点。

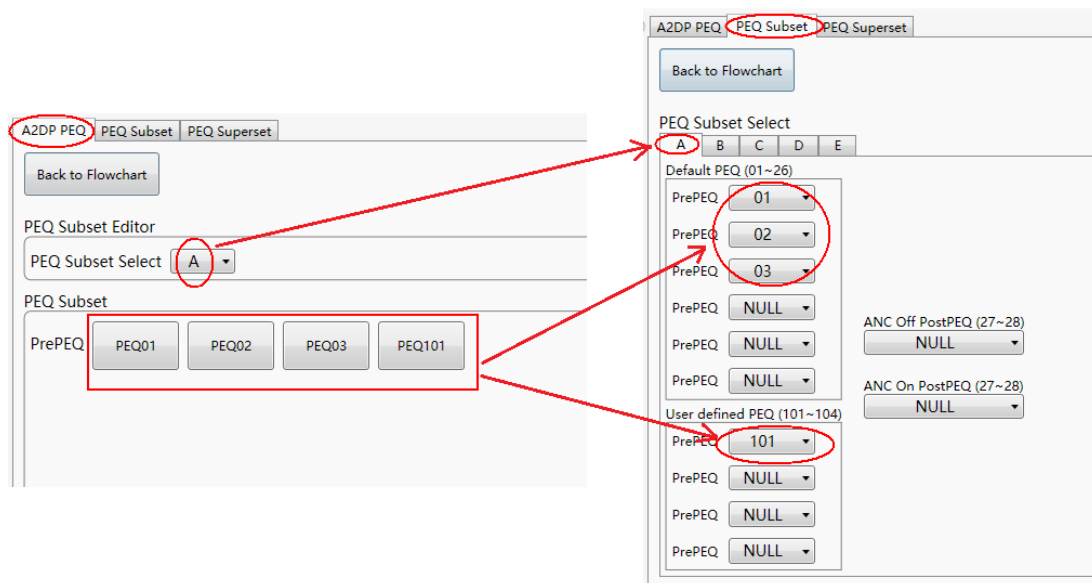
Battery Life Percentage		
Battery Level 10%	3.662 V	3.58
Battery Level 20%	3.707 V	3.7
Battery Level 30%	3.744 V	3.76
Battery Level 40%	3.774 V	3.8
Battery Level 50%	3.806 V	3.84
Battery Level 60%	3.848 V	3.92
Battery Level 70%	3.911 V	4
Battery Level 80%	3.984 V	4.1
Battery Level 90%	4.064 V	4.2

四十四、修改 custom EQ 的设置

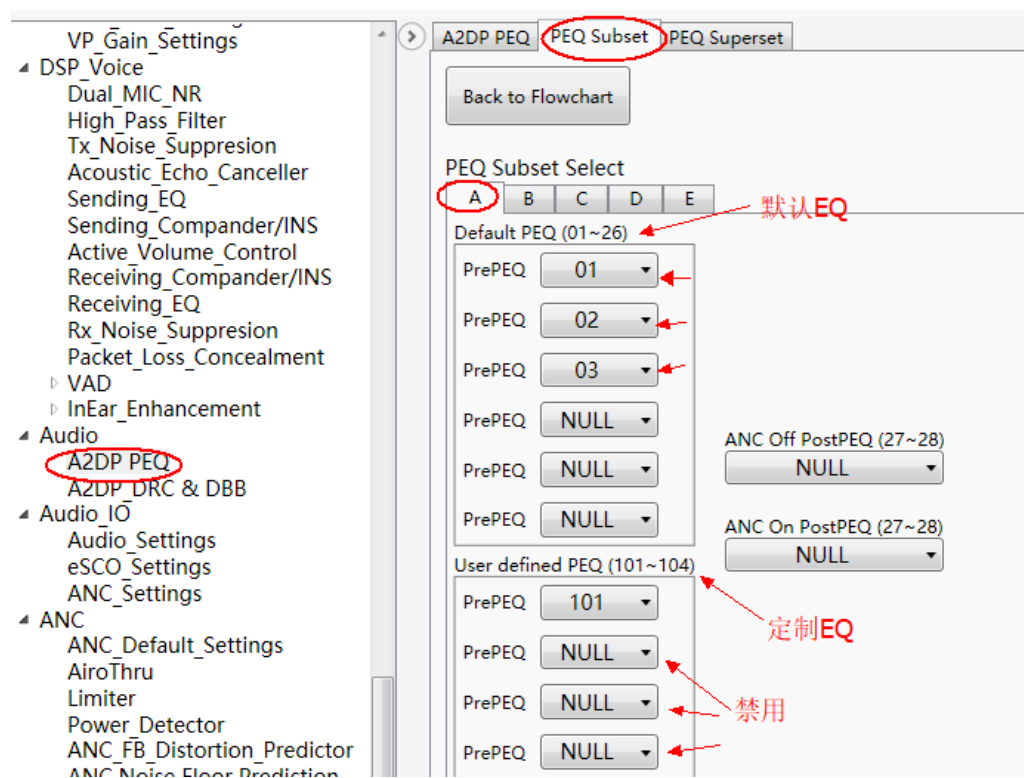
禁用 EQ102 - EQ104，设置 EQ101，再用小绿人通过手机更新 EQ 到耳机



上图中看出 PreEQ 和 PostEQ 是串联的，所有的效果都是在 PostEQ 上叠加 PreEQ，而 PostEQ 都是基于 music EQ 的，musicEQ 就是 normalEQ



27 和 28 是 PostEQ

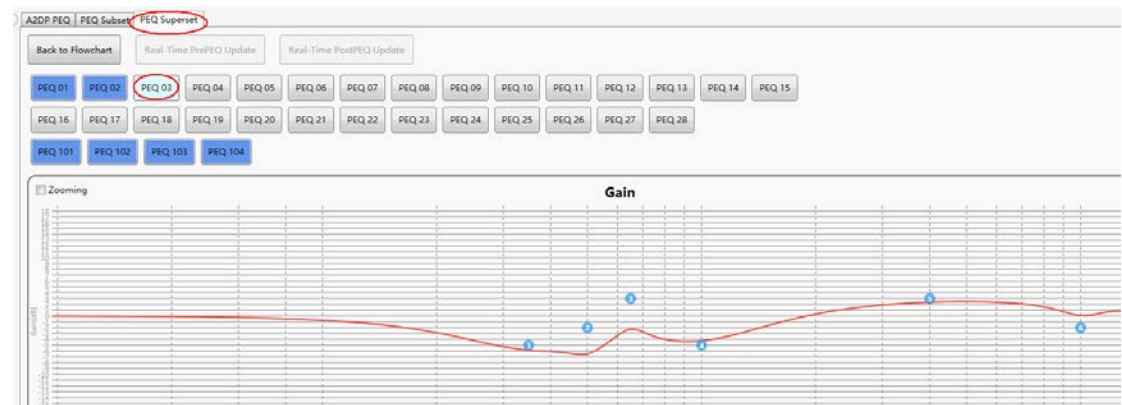


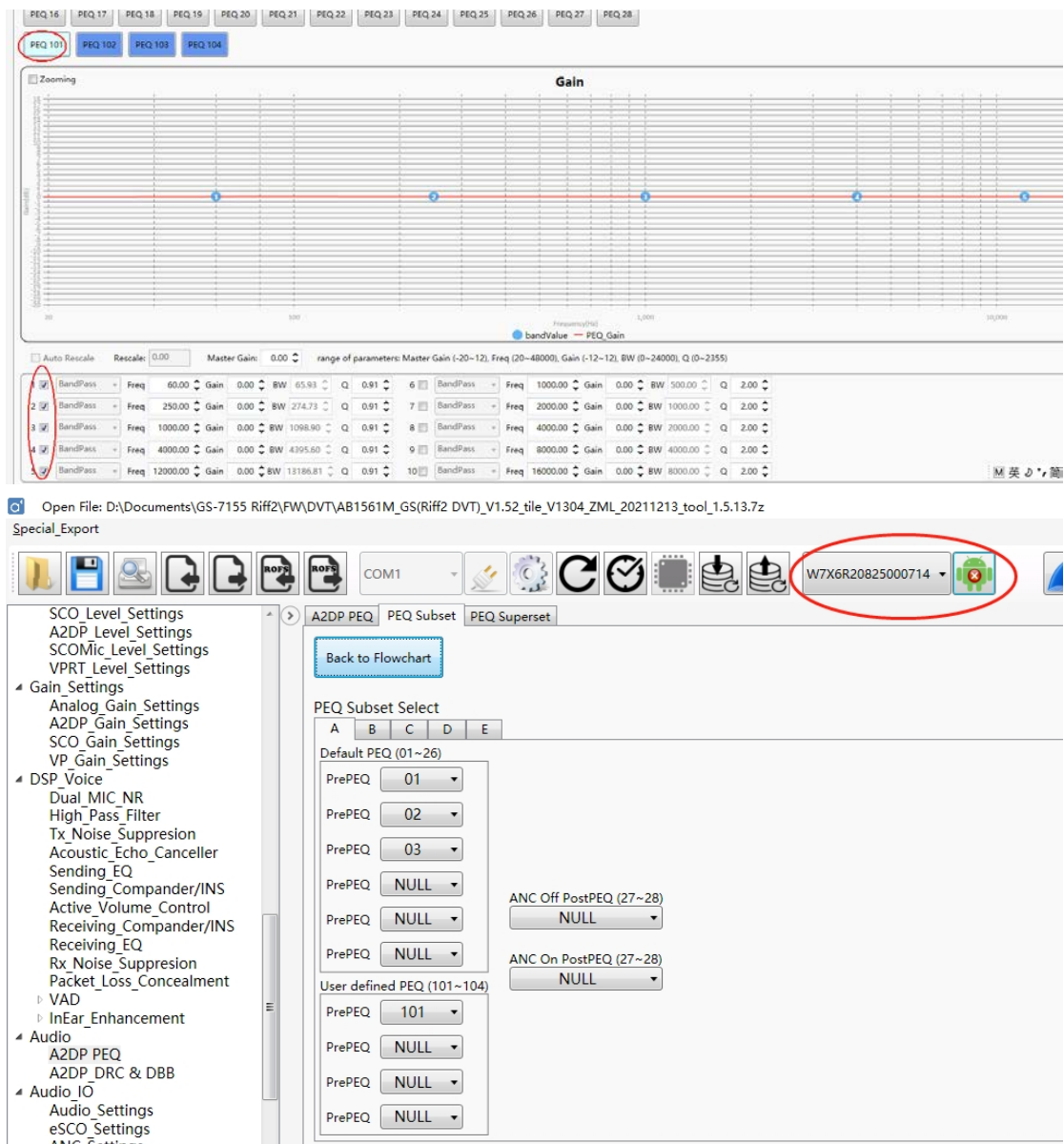


设置的频点

Auto Rescale Rescale: 0.00 Master Gain: 0.00 range of parameters: Master Gain (-20~12), Freq (20~48000), Gain (-100~50), BW (0~24000), Q (0~2355)

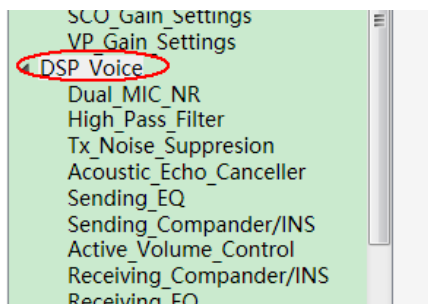
		Freq	Gain	BW	Q		Freq	Gain	BW	Q
1	HighPass	130.00	0.00	173.33	0.75	6	BandPass	1000.00	-5.00	1250.00
2	HighPass	130.00	0.00	173.33	0.75	7	BandPass	10000.00	-2.00	5000.00
3	BandPass	350.00	-5.00	437.50	0.80	8	BandPass	1800.00	-2.00	1800.00
4	BandPass	500.00	-2.00	151.52	3.30	9	BandPass	8000.00	0.00	4000.00
5	BandPass	650.00	3.00	185.71	3.50	10	BandPass	16000.00	0.00	8000.00

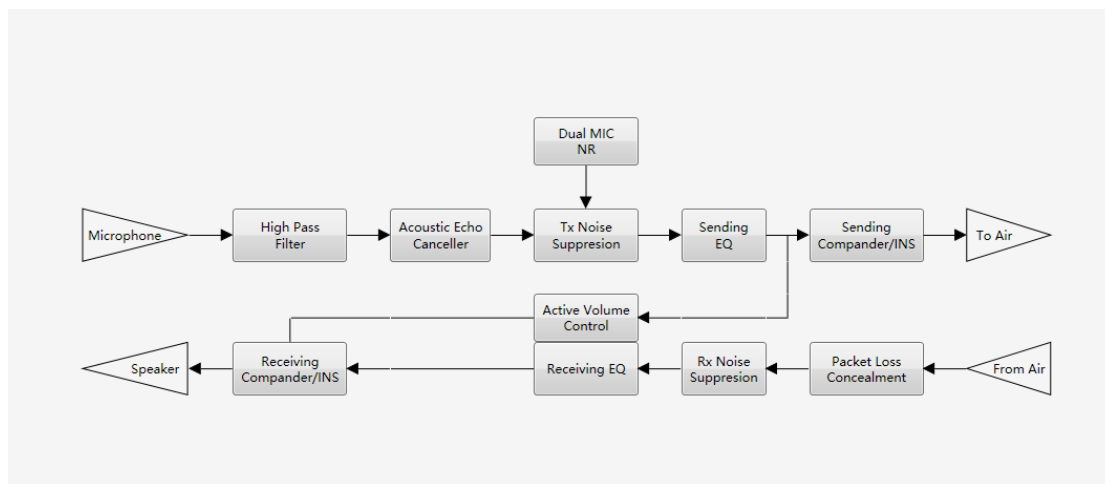




也可以保存成 FOTA 升级的 bin 文件，再用手机 app 升级 FW

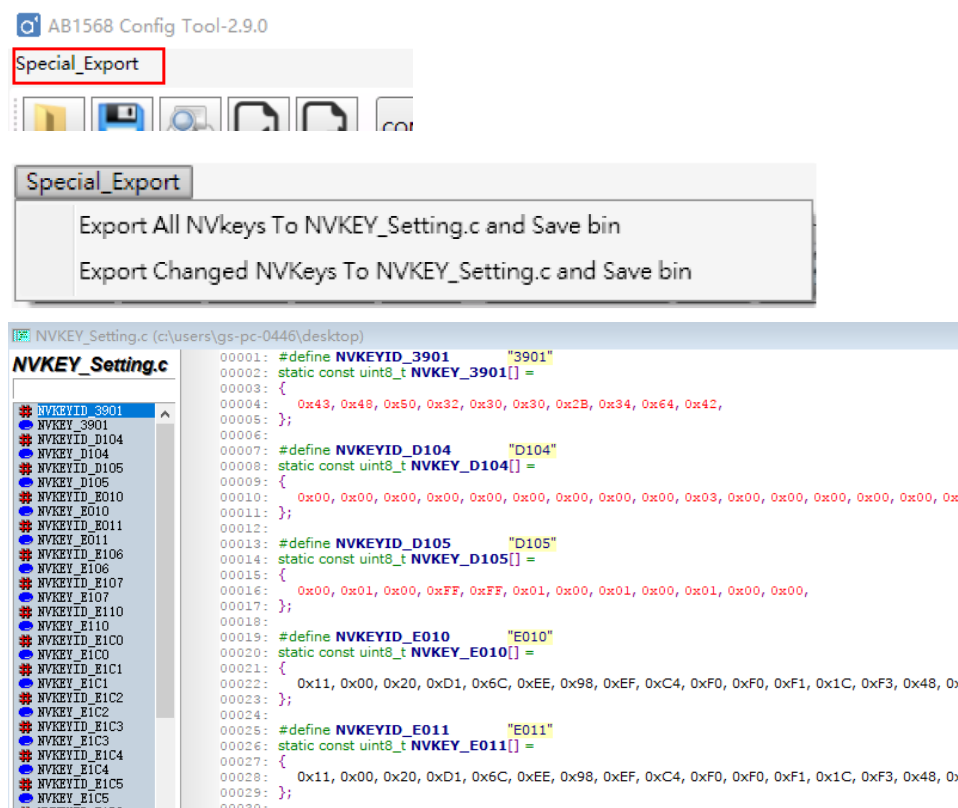
四十五、DSP 声音设置





四十六、找出修改的 NVkey 参数

Config Tool 软件提供了将 NVkey 设置导出到 C 文件中，即以 C 语言格式保存

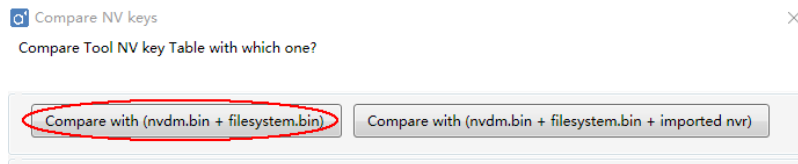


快速找出修改的 NVkey 值

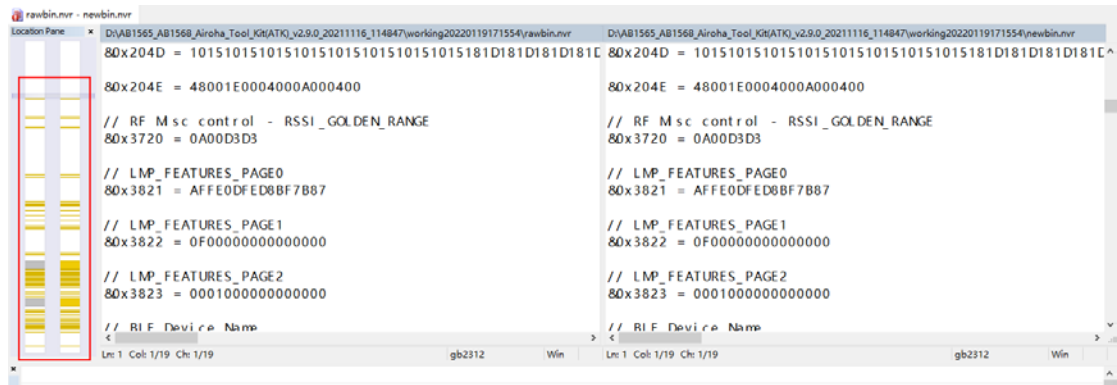
方法一：在 config tool 上打开 20211203_CHP200_POC2-ER03_PCBA_binaries_eb2ab222 工程，修改一些设置，保存成 CHP200_WS2_Filter_PCBA+4dB_PulseEQ_20220105_Dump_All.nvr

如果只把修改的设置写入代码，那么需要把 NVkey 的值提取出来，步骤如下：

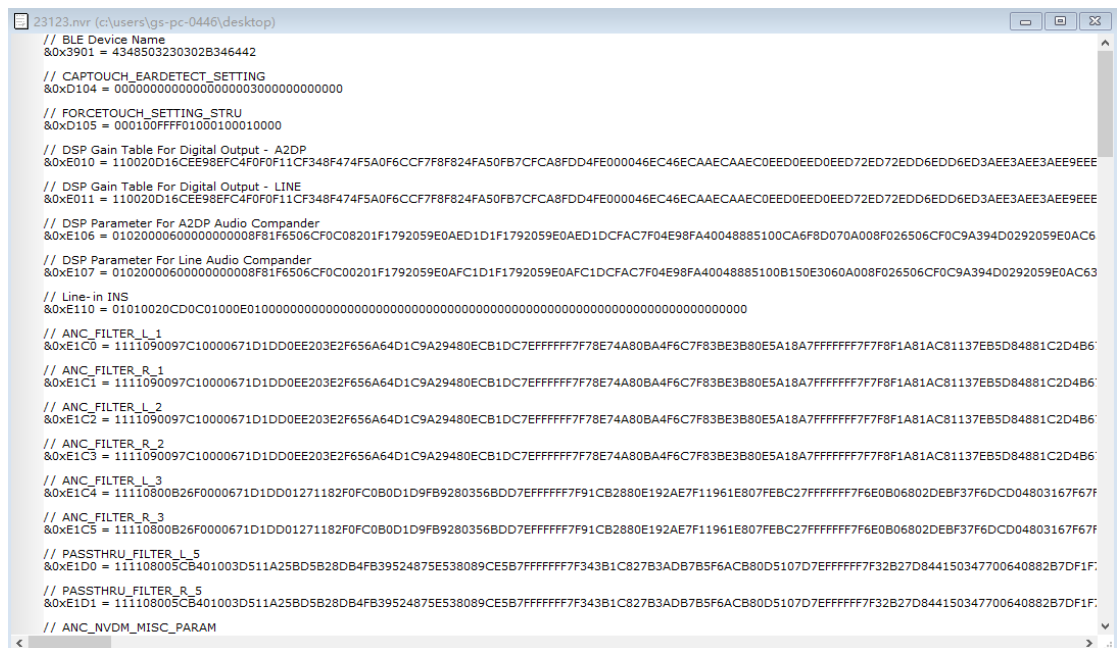
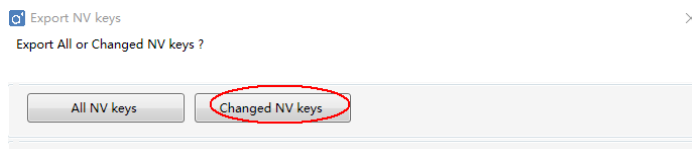
在 config tool 上打开 20211203_CHP200_POC2-ER03_PCBA_binaries_eb2ab222 工程，用 Import NV 功能把 nvr 文件导入，用



黄色底纹内容是改变的



或用 export NV 功能只把改变的 NVkey 保存出来



四十七、苹果手机上 SDK DUT 软件更新方法

在 TestFlight 上更新即可

